Alessandro Barenghi
Davide Martinenghi
Matteo Pradella
Matteo Rossi

ALGORITMI E PRINCIPI DELL'INFORMATICA:

esercizi risolti e commentati



Società Editrice ESCULAPIO

Alessandro Barenghi Davide Martinenghi
Matteo Pradella Matteo Rossi

ALGORITMI E PRINCIPI DELL'INFORMATICA:
esercizi risolti e commentati

Società Editrice ESCULAPIO

ISBN 978-88-9385-347-7

© Copyright 2023
Società Editrice Esculapio s.r.l.
Via Terracini, 30 - 40131 Bologna
www.editrice-esculapio.com - info@editrice-esculapio.it

Impaginazione: Carlotta Lenzi
Layout Copertina: Carlotta Lenzi
Stampato da: Digital Team - Fano (PU)

Printed in Italy

Le fotocopie per uso personale (cioè privato e individuale, con esclusione quindi di strumenti di uso collettivo) possono essere effettuate, nei limiti del 15% di ciascun volume, dietro pagamento alla S.I.A.E del compenso previsto dall'art. 68, commi 4 e 5, della legge 22 aprile 1941 n. 633. Tali fotocopie possono essere effettuate negli esercizi commerciali convenzionati S.I.A.E. o con altre modalità indicate da S.I.A.E. Per le riproduzioni ad uso non personale (ad esempio: professionale, economico o commerciale, strumenti di studio collettivi, come dispense e simili) l'editore potrà concedere a pagamento l'autorizzazione a riprodurre un numero di pagine non superiore al 15% delle pagine del volume.
CLEARedi - Centro Licenze e Autorizzazioni per le Riproduzioni Editoriali
Corso di Porta Romana, n. 108 - 20122 Milano
e-mail: autorizzazioni@clearedi.org - sito: http://www.clearedi.org.

Società Editrice ESCULAPIO

Prefazione di D. Mandrioli

Quando gli autori di questo nuovo eserciziario di "API" mi hanno chiesto se volessi associarmi alla sua costruzione, con non poco rammarico ho declinato l'invito: i primi anni di vita da pensionato ma anche, lo confesso, le frustrazioni patite negli ultimi anni della mia vita da accademico e da docente del Politecnico mi hanno tolto le motivazioni e l'impegno necessari per questo compito.

Mi hanno fatto molto piacere però l'invito ricevuto e la successiva proposta di presentare il lavoro dei miei (ex) colleghi, proposta che questa volta ho accettato volentieri, soprattutto dopo aver constatato che i principi ispiratori del corso, la sua organizzazione e la relativa offerta del materiale didattico manifestano una forte continuità con l'impostazione del passato, pur con la naturale e necessaria evoluzione.

Questo nuovo eserciziario è destinato a rimpiazzare il precedente "Esercizi di informatica teorica" i cui autori erano stati docenti dell'omonimo corso così come gli attuali autori sono docenti del corso di API. I due testi quindi sono in primo luogo la conseguenza del fatto che l'attuale corso API affianca agli argomenti tradizionali dell'informatica teorica –automi, linguaggi formali, computabilità e logica applicata all'informatica– quelli altrettanto classici dell'analisi e progettazione di algoritmi e strutture dati. Storicamente i due "programmi d'esame" sono stati altrettanto efficacemente sviluppati nell'ambito di corsi distinti e in maniera fortemente integrata come nel caso di API, a mio parere con maggiori benefici da questa seconda alternativa.

Seguendo una prassi naturale e abbastanza comune entrambi gli eserciziari sono stati costruiti a partire da una raccolta di precedenti "temi d'esame"; ovviamente poi gli autori hanno avuto cura di presentare questi temi d'esame e le relative soluzioni con ulteriori spiegazioni e commenti rispetto alle soluzioni "schematiche" offerte al momento dell'appello d'esame.

I temi d'esame sono il "biglietto da visita" di un corso universitario (e non solo) e nel nostro caso questo biglietto da visita gli è valsa l'etichetta di "esame anomalo" rispetto alla gran parte di altri esami del Politecnico, etichetta spesso riferita nel corso degli anni da numerosi studenti, in molti casi a mo' di critica e lamento, in alcuni altri casi invece come segno di soddisfazione e approvazione, talvolta ... postuma, ossia ricevuta da "studenti pentiti" che hanno ammesso di aver apprezzato questa "anomalia" solo dopo le prime esperienze nel mondo del lavoro. Personalmente mi piace riferire di questa etichetta con una punta di orgoglio –nonostante le critiche di molti– e mi fa piacere constatare che essa sia stata mantenuta dai miei "eredi" (sperando che essi mi perdonino il termine).

Ma in che cosa consiste questa anomalia? Consiste in un certo livello di "imprevedibilità", certamente variabile da esercizio a esercizio, che costringe lo studente a dedicare molta attenzione principalmente a capire l'essenza del quesito, talvolta non evidente da una lettura superficiale, e quindi ad elaborare una soluzione che raramente è la semplice ri-applicazione della stessa tecnica utilizzata

in esercizi simili. Non a caso infatti, che io sappia, questo corso è rimasto uno dei pochi durante l'esame del quale sia permessa la consultazione del testo e di altro materiale didattico.

Concludo quindi questa prefazione con qualche raccomandazione ai futuri utenti di questo eserciziario, chiedendo scusa agli autori per questa piccola invasione di campo:

- Non confondete tra loro le diverse funzioni dei vari tipi di materiale didattico: c'è una certa tendenza nell'affrontare in maniera utilitaristica la preparazione di un esame alla ricerca delle scorciatoie, spesso identificate nei cosiddetti "bigini" (termine tipico dei miei tempi) ossia strumenti tendenti a ridurre "in pillole" i contenuti di una conoscenza; ma pillole non è sinonimo di essenza! Un eserciziario –e tanto meno i lucidi delle lezioni– non possono sostituire, ma certo completano o rinfrescano l'apprendimento derivante dallo studio approfondito di un libro di testo.

- Non sostituite la qualità, in questo caso sinonimo di approfondimento e riflessione critica, con la quantità; in altri termini, non conviene mandare più o meno a memoria un gran numero di esercizi e relative soluzioni, perché intanto gli esercizi –che rappresentano i problemi reali che dovrete affrontare nel lavoro– che poi vi saranno proposti avranno diversità significative rispetto a quelli che trovate già svolti. Piuttosto impegnatevi a fondo su un buon campione di esercizi proposti, cercatene la soluzione prima di leggerla sull'eserciziario e confrontate la vostra con quella che trovate: un'eventuale diversità tra le due non significa necessariamente errore ma cercatene attentamente la ragione; nel caso coinvolgetene compagni di studio e/o i docenti stessi che saranno ben lieti di confrontarsi con voi. Chissà perché, nel mio caso un tale confronto avveniva più spesso dopo un esame non ben riuscito invece di prima, durante la sua preparazione.

- Come amava ripetere un collega "lo scopo dello studio di ingegneria al Politecnico è imparare ad imparare", non riempirsi la bocca di termini di moda, o *buzzword*, per dirla alla anglosassone, per far bella figura nelle conversazioni e magari anche nelle riunioni di lavoro. Sarà ancora vero?

Dino Mandrioli

Prefazione degli autori

Presentiamo qui una raccolta di esercizi sui temi tipici dell'informatica teorica, affrontati nella prima parte di questo libro, e sugli argomenti più comunemente trattati nei corsi di algoritmi e strutture dati, inclusi nella seconda parte.

La prima parte comprende quindi i modelli formali, raccolti nel primo capitolo, che useremo prevalentemente per caratterizzare i linguaggi formali, e che svilupperemo tramite macchine a stati (automi a stati finiti, automi a pila, macchine di Turing) e grammatiche formali (grammatiche regolari, non contestuali e generali). Per non trascurare la logica – strumento di capitale importanza nell'informatica teorica – abbiamo voluto aggiungere anche un modello che coniugasse il rigore formale della logica con la capacità di caratterizzare linguaggi formali, e quindi di agevolarne il confronto con gli altri modelli. Nonostante non faccia solitamente parte del programma dei corsi di base, per i motivi suddetti, alcuni esercizi faranno riferimento alla cosiddetta logica monadica del prim'ordine sulle stringhe, che fornisce un esempio significativo di formalismo espressivo strettamente contenuto nei linguaggi regolari.

Il secondo capitolo riguarda invece la computabilità. Gli esercizi proposti fanno riferimento ai classici concetti di decidibilità (ricorsività) e semidecidibilità applicati a problemi, insiemi di numeri naturali e linguaggi, nonché alla nozione di computabilità di una funzione. Nello svolgimento degli stessi si ricorrerà spesso a risultati e tecniche note, quali il teorema di Rice, le enumerazioni diagonali, la riduzione di problemi e l'indecidibilità di problemi quali l'arresto, la totalità di una funzione ed altri.

Nel terzo capitolo, all'interno della seconda parte, ci dedicheremo alla notazione asintotica ($\mathcal{O}$, Θ e Ω) e all'analisi di ricorrenze, mediante le consuete tecniche dell'albero di ricorsione, del metodo di sostituzione e del "teorema dell'esperto" (che è utile per risolvere le cosiddette ricorrenze lineari "con partizione bilanciata"); in qualche caso ci rifaremo a tecniche specializzate, ad esempio per risolvere le ricorrenze lineari "di ordine costante".

Il quarto capitolo affronta la complessità di problemi con la macchina di Turing e con la macchina RAM, con i vari criteri di costo applicabili. In questo capitolo, come nel precedente, si farà talvolta ricorso a maggiorazioni mediante l'applicazione di formule note, come la formula di Gauss e alcune serie notevoli.

L'ultimo capitolo offre un ampio spettro di problemi algoritmici che richiedono l'uso delle strutture dati più frequentemente trattate nei corsi di base: vettori, liste, matrici, pile, code, tabelle hash, alberi (tra cui gli alberi binari di ricerca) e grafi. Per ciascuno dei problemi presentati, spesso descritti mediante pseudocodice, si valuterà la complessità asintotica.

Gli esercizi presentati in questo volume sono liberamente tratti dai temi d'esame del corso di "Algoritmi e principi dell'informatica", di cui gli autori sono attualmente titolari, offerto nel corso di studi della laurea triennale in Ingegneria

Informatica del Politecnico di Milano dal 2010.

Nel seguito, faremo riferimento ai seguenti testi e dispense per richiami a risultati noti in informatica teorica e algoritmi:

- "Informatica Teorica" di Dino Mandrioli e Paola Spoletini [5].
- "Algoritmi e strutture di dati" di Alan Bertossi e Alberto Montresor [1].
- "Introduction to Algorithms" di Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest e Clifford Stein [2] (traduzione in italiano [3]).
- "Lecture Notes on Monadic First- and Second-Order Logic on Strings" di Dino Mandrioli, Davide Martinenghi, Angelo Morzenti, Matteo Pradella e Matteo Rossi [4].

Per quanto riguarda la notazione, di seguito viene fornito un breve sommario dei principali simboli utilizzati nel testo, corredati della loro semantica.

Simbolo	**Semantica**
$\mathbb{N}$	insieme dei numeri naturali
$\mathbb{R}$	insieme dei numeri reali
$\mathcal{E}$	enumerazione algoritmica delle Macchine di Turing
f_i	funzione calcolata dalla i-esima Macchina di Turing nella enumerazione $\mathcal{E}$
$\textvisiblespace$	cella vuota (*blank)* in una Macchina di Turing
ε	stringa vuota
$:=$	assegnamento di un valore ad una variabile

Talvolta adotteremo sigle comunemente utilizzate nei testi di riferimento, quali MT per la Macchina di Turing e MFO per la logica monadica del prim'ordine sulle stringhe (*Monadic First-Order logic on strings*), più altre che saranno definite nel contesto dei vari esercizi.

La notazione utilizzata per la descrizione di algoritmi è quella dello pseudocodice del testo [2], che usa il paradigma imperativo. Le allocazioni di memoria per i tipi di dato scalari sono implicite, mentre l'allocazione di una struttura dati S composta di n elementi scalari è indicata da $\textsc{Alloca}(S, n)$. Nel caso dell'allocazione di una matrice M di n righe e m colonne, verrà utilizzata la notazione $\textsc{Alloca}(M, n \times m)$; assumeremo in tal caso che siano disponibili gli attributi $M.rows$ e $M.cols$ per riferirsi a n ed m rispettivamente. Per evitare inutile ingombro sintattico, l'allocazione di tipi strutturati sarà indicata come per gli scalari (ad esempio, $\textsc{Alloca}(x, 1)$ può indicare l'allocazione di un nodo x di un albero).

Ricordiamo inoltre che, come da prassi del testo [2], l'accesso ad un campo di una struttura dati utilizza l'operatore . ed il nome della struttura dati rappresenta un riferimento alla stessa: l'operatore . effettua dunque anche una dereferenziazione (in maniera analoga all'operatore -> del linguaggio C). Per quanto riguarda i vettori, ricordiamo che la notazione $V[i..j]$ denota la porzione di vettore compresa tra il suo i-esimo elemento ed il suo j-esimo elemento, estremi

inclusi. Salvo avviso contrario, gli indici dei vettori iniziano a partire da 1, come in [2].

Alessandro Barenghi
Davide Martinenghi
Matteo Pradella
Matteo Rossi

Indice

Prefazione di D. Mandrioli **iii**

Prefazione degli autori **v**

I Informatica teorica 1

1 Modelli formali 3

1.1 Macchine a stati 3
1.2 Grammatiche formali 14
1.3 Specifiche mediante formule logiche 22
1.4 Linguaggi formali 26

2 Computabilità 33

2.1 Ricorsività di insiemi 33
2.2 Computabilità di funzioni 43
2.3 Decidibilità e semidecidibilità 52
2.4 Riduzioni 71

II Algoritmi e strutture dati 75

3 Ricorrenze, analisi e notazione asintotica 77

4 Complessità delle macchine astratte 97

5 Sintesi di algoritmi 121

5.1 Vettori, liste e matrici 121
5.2 Ordinamento 132
5.3 Tabelle Hash 136
5.4 Alberi 141
5.5 Grafi 157
5.6 Altre strutture dati 159

Bibliografia **166**

Parte I

Informatica teorica

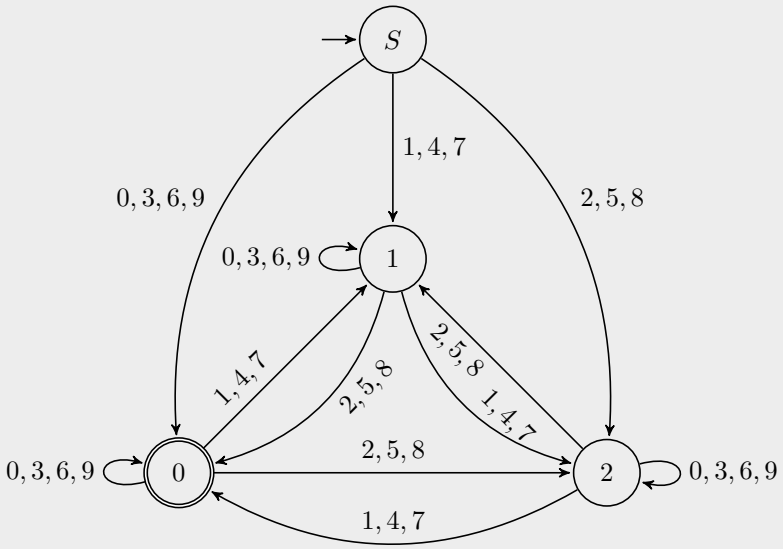
1
Modelli formali

1.1 Macchine a stati

Esercizio 1.1. Si scriva un automa a potenza minima (tra automa a stati finiti, a pila, macchina di Turing) che riconosca il linguaggio L fatto di tutte e sole le stringhe su $\{0,1,2,3,4,5,6,7,8,9\}^*$ che rappresentano numeri in base 10 divisibili per 3.

SOLUZIONE

Basta un automa a stati finiti che tenga conto della somma delle cifre modulo 3 (più uno stato iniziale per evitare di accettare la stringa vuota).

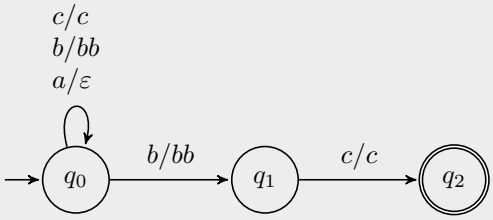


Esercizio 1.2. Si consideri il linguaggio $L \subseteq \{a,b,c\}^*$ composto dalle stringhe di lunghezza maggiore o uguale a 2 che finiscono con bc, e una funzione di traduzione τ che, per ogni stringa $x \in L$, cancella i simboli a, raddoppia le b, e mantiene inalterate le c. Per esempio $\tau(abc) = bbc$, $\tau(cbabc) = cbbbbc$, $\tau(c) = \bot$, $\tau(cbab) = \bot$.

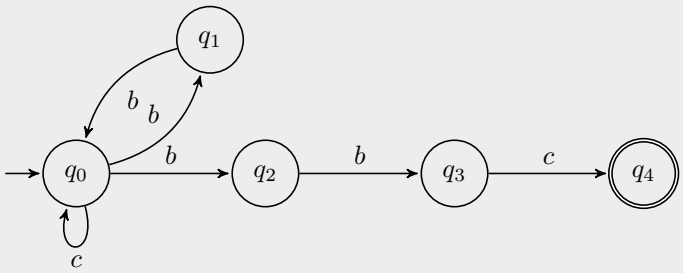
- Si scriva il trasduttore finito che calcoli la traduzione descritta.
- Si scriva l'automa a stati finiti che accetti il linguaggio $\tau(L)$; se possibile si mostri come l'automa che accetta $\tau(L)$ si possa derivare dal trasduttore definito precedentemente.
- Basandosi sulla domanda precedente, si delinei una procedura sistematica per ottenere da un trasduttore a stati finiti, l'automa che accetti il linguaggio di uscita del trasduttore stesso.

SOLUZIONE

Trasduttore a stati finiti per $\tau(L)$:



Automa (accettore) a stati finiti per $\tau(L)$:



Costruzione: in pratica basta "buttar via" la parte di ingresso, mentre si trasformano in transizioni le stringhe di uscita: se una stringa è vuota, si perde la transizione; se è fatta da più di un carattere si mette una catena di stati con una transizione per ogni carattere.

Esercizio 1.3. Si definisca *riconoscitore ad automi gemelli* (RAG) un meccanismo di calcolo ottenuto facendo operare due automi sullo stesso nastro di ingresso. Gli automi operano indipendentemente sul nastro di ingresso, ognuno di essi accede al nastro attraverso una propria testina. La parola presente sul nastro di ingresso si considera riconosciuta se e solo se i due automi la riconoscono (non necessariamente nello stesso numero di mosse). Si consideri il caso dei RAG costruiti come segue e si dica se il loro potere riconoscitore è maggiore, minore o uguale a quello delle loro componenti:

1. RAG costituito da un automa a stati finiti deterministico e uno non deterministico;
2. RAG costituito da un automa a stati finiti deterministico ed un automa a pila deterministico;
3. RAG costituito da due automi a pila non deterministici.

SOLUZIONE

Detti A_1 e A_2 gli automi che compongono un dato riconoscitore ad automi gemelli A, si osserva che il riconoscitore ad automi gemelli riconosce il linguaggio $L(A) = L(A_1) \cap L(A_2)$. È infatti sufficiente osservare che i due automi che lo compongono operano senza influenzarsi e la condizione di accettazione richiede che entrambi accettino la parola sul nastro d'ingresso. Si ha quindi che

1. Il riconoscitore ad automi gemelli ha la stessa potenza riconoscitiva delle sue componenti: i linguaggi regolari sono chiusi rispetto all'intersezione.
2. Il riconoscitore ad automi gemelli ha la stessa potenza riconoscitiva di un automa a pila deterministico. È possibile derivare questo risultato dal fatto che i linguaggi liberi dal contesto sono chiusi rispetto all'intersezione con i linguaggi regolari. Alternativamente, è possibile costruire un riconoscitore a pila deterministico equivalente al riconoscitore ad automi gemelli modificando l'organo di controllo dell'APD per incorporare anche il comportamento dell'automa a stati finiti.
3. Il riconoscitore ad automi gemelli ha potenza riconoscitiva superiore ad un automa a pila deterministico. È infatti noto che i linguaggi liberi dal contesto non sono chiusi rispetto all'intersezione. Un esempio di quanto detto è il riconoscitore ad automi gemelli con $L(A_1) = a^n b^n c^*$ e $L(A_2) = a^* b^n c^n$, con $n \in \mathbb{N}$.

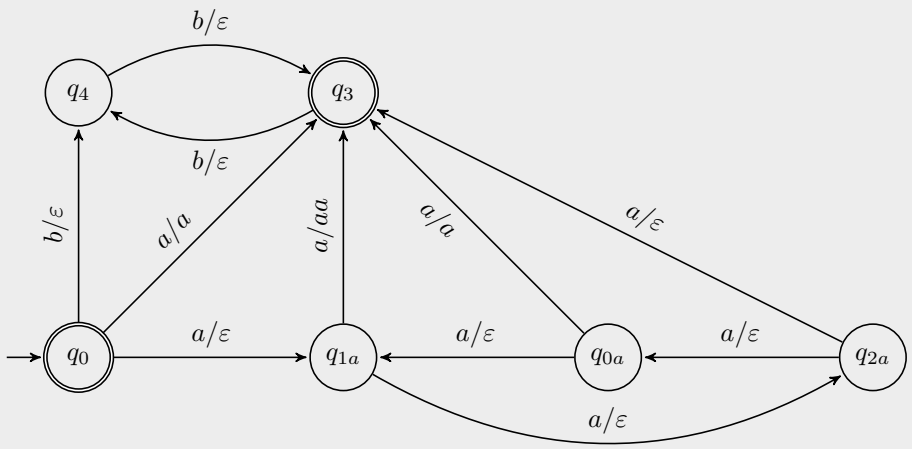
Esercizio 1.4. Si considerino i linguaggi $L_s = \{a^n b^{2m} \mid n, m \geq 0\}$ e $L_d = \{\varepsilon, a, aa\}$.
Si realizzi un trasduttore a potenza minima che calcoli la seguente traduzione da L_s a L_d:

$$\tau(a^n b^{2m}) = a^{n \bmod 3},$$

dove $x \bmod y$ indica il resto della divisione intera x/y.

SOLUZIONE

Per risolvere l'esercizio è sufficiente un trasduttore a stati finiti (TSF) non deterministico.



Osserviamo che il non determinismo in un formalismo di traduzione consente, in teoria, di associare a una stessa stringa (accettata) più di una traduzione; è comunque possibile, come nel trasduttore qui mostrato, avere una singola traduzione per ogni stringa accettata. Inoltre, a differenza degli automi accettori, per i quali il non determinismo non aggiunge potere espressivo, i TSF non deterministici sono strettamente più potenti di quelli deterministici. In questo caso, ad esempio, non si possono distinguere le a che costituiscono il suffisso lungo quanto il resto della divisione da quelle che costituiscono la parte relativa al quoziente; lo si potrebbe fare con un terminatore di stringa diverso da a, ma nel linguaggio L_s le b possono essere assenti.

Esercizio 1.5. Dire se esiste un automa a stati finiti che riconosce il seguente linguaggio: $L = \{a^n b^n c^n \mid n > 2\} \cup \{a^* b^* c^*\}$. Spiegare brevemente la risposta.

SOLUZIONE

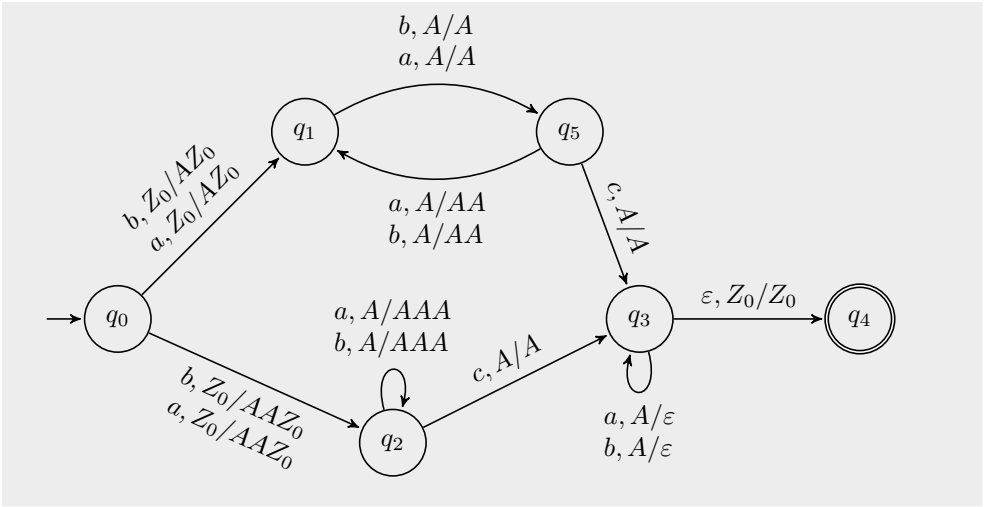
Sì, il linguaggio è regolare. Lo possiamo notare dal fatto che $\{a^n b^n c^n \mid n > 2\} \subset \{a^* b^* c^*\}$, quindi $L = \{a^* b^* c^*\}$.

Esercizio 1.6. Si definisca un automa a potenza minima per il linguaggio

$$\{xcy \mid x, y \in \{a, b\}^+, |x| = 2|y| \text{ oppure } 2|x| = |y|\}.$$

SOLUZIONE

Si tratta di un automa a pila non-deterministico. Occorre infatti la capacità di contare degli automi a pila per confrontare $|x|$ con $|y|$ (quindi un automa a stati finiti non basta), ma non è possibile prevedere deterministicamente di quale dei due casi di stringhe del linguaggio si tratti.



Esercizio 1.7. Si dica, giustificando brevemente le risposte, se le seguenti affermazioni sono vere o false:

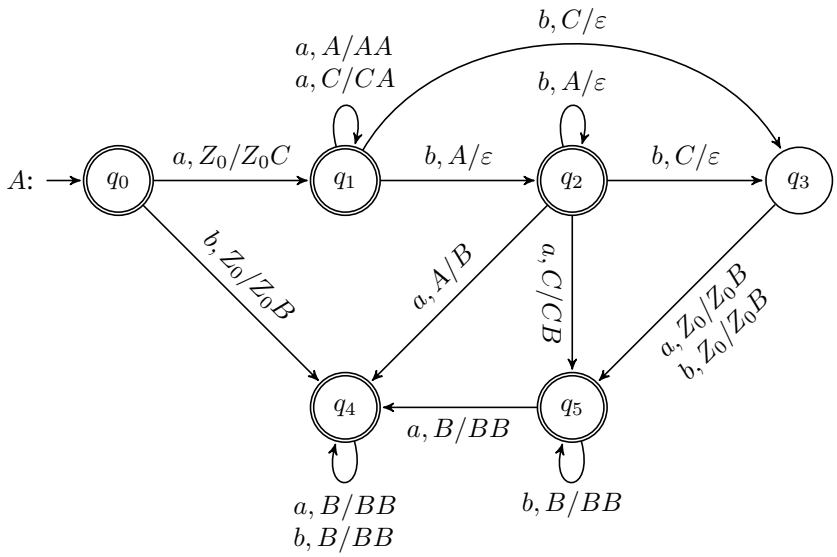
1. Dato un qualsiasi automa a pila A, sia deterministico che nondeterministico, esiste sempre un automa a pila, sia deterministico che nondeterministico, che riconosce il complemento di $L(A)$.
2. Dato un qualsiasi automa a pila A, sia deterministico che nondeterministico, esiste sempre una macchina di Turing, che riconosce il complemento di $L(A)$.

SOLUZIONE

1. No: la classe dei linguaggi riconosciuti da automi a pila nondeterministici non è chiusa rispetto al complemento, essendo chiusa rispetto all'unione ma non rispetto all'intersezione.
2. Sì: i linguaggi riconosciuti da automi a pila (deterministici o nondeterministici) sono ricorsivi, quindi lo sono anche i loro complementi, dal momento che la classe dei linguaggi ricorsivi è chiusa rispetto alla complementazione. Infine, per ogni linguaggio ricorsivo esiste una macchina di Turing che lo riconosce, dal momento che la famiglia dei linguaggi ricorsivi è un sottoinsieme della famiglia dei linguaggi ricorsivamente enumerabili.

Esercizio 1.8. Si considerino la grammatica G e l'automa a pila A seguenti, e i linguaggi $L(G)$ e $L(A)$ definiti da essi sull'alfabeto $\{a, b\}$:

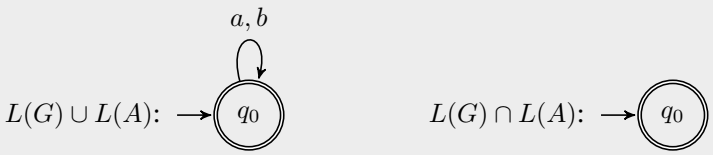
$$G: \quad S \to \varepsilon \mid aSb$$



Si costruiscano automi potenza minima che riconoscano rispettivamente $L(G) \cup L(A)$ e $L(G) \cap L(A)$.

SOLUZIONE

$L(G)$ è chiaramente il linguaggio $\{a^n b^n \mid n \geq 0\}$, mentre l'automa a pila A accetta tutte le stringhe che non sono in $L(G)$, oltre alla stringa nulla. Quindi $L(G) \cup L(A) = \{a, b\}^*$ e $L(G) \cap L(A) = \{\varepsilon\}$, entrambi linguaggi regolari. Quindi gli automi richiesti sono rispettivamente i seguenti:



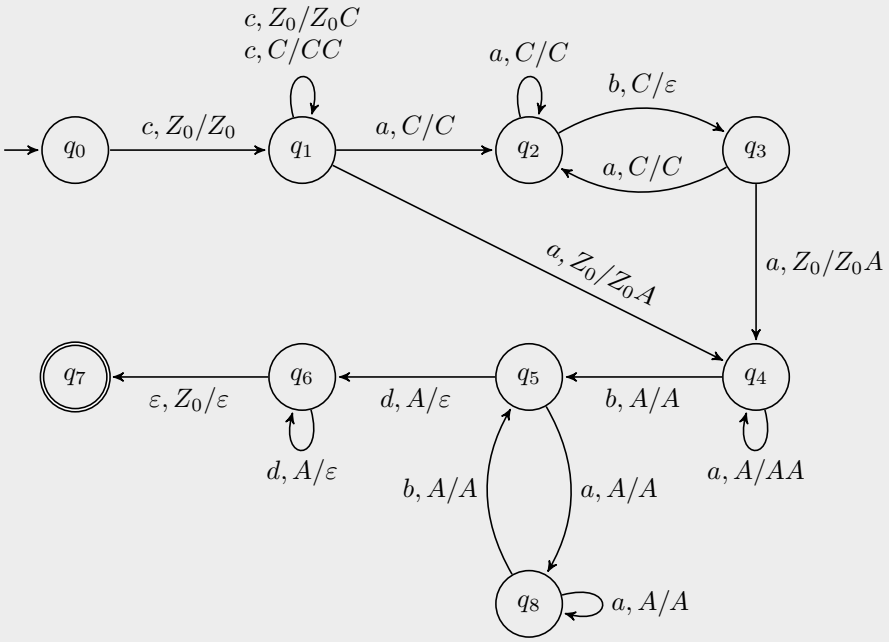
Esercizio 1.9. Si consideri il linguaggio

$$L = \{c^m a^{l_1} b a^{l_2} b \dots a^{l_m} b \dots a^{l_z} b d^n \mid n, m, l_i, z > 0, l_m = n\}.$$

Per esempio $ccaaabaaaababababdddd \in L$. Si definisca un automa A a potenza minima tale che $L(A) = L$.

SOLUZIONE

Basta un automa a pila deterministico. L'automa deve contare le c in ingresso, e lo fa mettendo $m-1$ simboli C sulla pila (la prima c viene contata con il simbolo Z_0). Alla prima a, si contano i gruppi di a separati da b, fino all'arrivo dell'm-esimo, eliminando una C per ogni gruppo dalla pila. A questo punto si impila A per ogni a trovata, fino alla prima b. Si ignorano le successive a e b, controllando solo che la sequenza finisca con b e che non ci siano due b consecutive; all'arrivo delle d si rimuovono le A dalla pila, controllando che siano in numero pari alle d.



Esercizio 1.10. Si considerino i seguenti linguaggi:

1. $L_1 = \{a^n b^n \mid n \geq 1\} \cup \{a^n b^{2n} \mid n \geq 1\}$
2. $L_2 = \{1a^n b^n \mid n \geq 1\} \cup \{2a^n b^{2n} \mid n \geq 1\}$
3. $L_3 = \{a^n 1 b^n \mid n \geq 1\} \cup \{a^n 2 b^{2n} \mid n \geq 1\}$
4. $L_4 = \{a^n b^n 1 \mid n \geq 1\} \cup \{a^n b^{2n} 2 \mid n \geq 1\}$

e le seguenti famiglie di macchine astratte, usate come riconoscitori di linguaggi:

- AF: automi a stati finiti;
- APD: automi a pila determinstici;
- APN: automi a pila nondeterministici.

Si riempia la seguente tabella indicando in ogni casella se la famiglia indicata nella riga è in grado di riconoscere il linguaggio indicato nella corrispondente colonna; giustificare brevemente le risposte.

	L_1	L_2	L_3	L_4
AF				
APD				
APN				

SOLUZIONE

	L_1	L_2	L_3	L_4
AF	NO	NO	NO	NO
APD	NO	SI	SI	NO
APN	SI	SI	SI	SI

AF: essendo le capacità di conteggio limitate dal numero degli stati, non sono in grado di riconoscere alcuno dei linguaggi proposti.

APD: gli APD riescono a discriminare tra le due componenti dell'unione grazie ai simboli 1 e 2 solo nei casi di L_2 ed L_3. In L_1 questi simboli non sono presenti, quindi alla lettura delle prime b non siamo in grado di capire come gestire la pila. Il caso L_4 è come il caso L_1: i simboli 1 e 2 sono presenti, ma si trovano troppo avanti per poter essere sfruttati.

APN: gli APN non hanno bisogno delle informazioni del caso precedente, in quanto possono sfruttare il non-determinismo per discriminare i due casi, adottando due politiche di gestione della pila diverse.

Esercizio 1.11. Si consideri il seguente linguaggio:

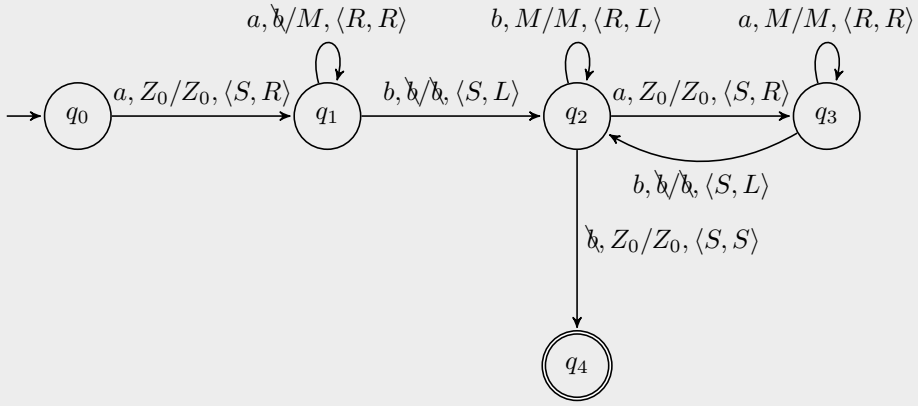
$$L = \{ab, abab, \ldots, aabb, aabbaabb, \ldots, a^n b^n, a^n b^n a^n b^n, \ldots\}.$$

Si sintetizzi un automa a *potenza minima* in grado di riconoscere L.

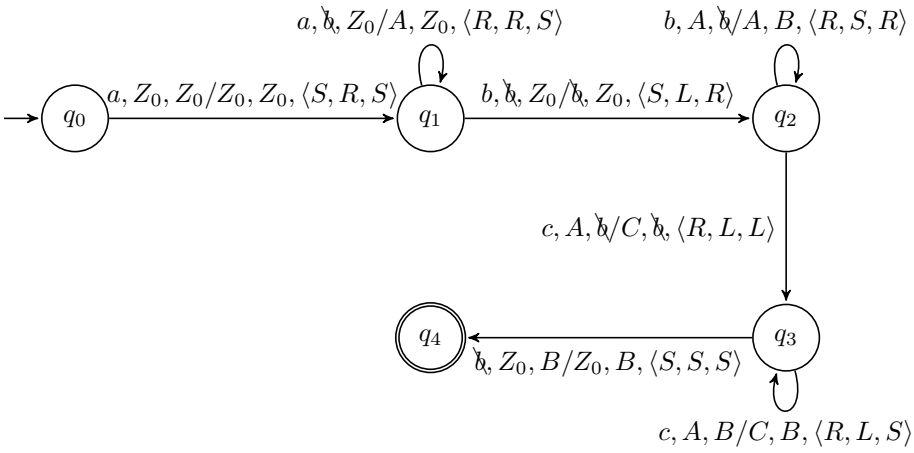
SOLUZIONE

Il linguaggio non può essere riconosciuto da un automa a pila, in quanto richiede la capacità di ricontare lo stesso numero più volte.

Una macchina di Turing che riconosce L è definita come segue:



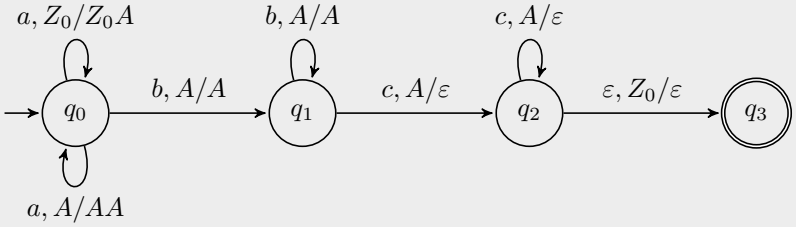
Esercizio 1.12. Si consideri la seguente macchina di Turing M, dove q_0 è stato iniziale e q_4 finale:



Si definisca un automa a potenza minima che accetti lo stesso linguaggio di M.

SOLUZIONE

La macchina usa il primo nastro per memorizzare il numero di a lette con dei simboli A; in seguito legge le b, mettendo simboli B sul secondo nastro. Alla lettura di ogni c, la macchina sposta la testina del primo nastro, controllando di arrivare a Z_0 dopo la lettura completa delle c. Il linguaggio riconosciuto dalla MT è quindi $\{a^n b^+ c^n \mid n > 0\}$. Esso si può riconoscere con il seguente automa a pila deterministico:



Esercizio 1.13.

1. Si definisca formalmente il seguente modello di automa, chiamato *macchina di Turing monodirezionale ad un nastro* (MTM1). Una MTM1 è un accettore nondeterministico (non effettua quindi traduzioni) dotato di un nastro di ingresso e un nastro di memoria. Il funzionamento è analogo a quello di una MT tradizionale, a parte il movimento delle due testine dei nastri: esso può essere solamente S (stop) o R (right).

2. Si confronti la capacità espressiva delle MTM1 con i modelli classici: automi a stati finiti, a pila e macchine di Turing.

3. Si consideri una variante "a nastro singolo" e deterministica della MTM1 e se ne valuti l'eventuale differenza di capacità espressiva.

SOLUZIONE

1. $A = (Q, \Sigma, \Gamma, q_0, F, \delta, Z_0)$, stato iniziale $q_0 \in Q$, stati finali $F \subseteq Q$. Funzione di transizione: $\delta : (Q - F) \times \Sigma \times \Gamma \to \wp(Q \times \Gamma \times \{S, R\}^2)$.

   Configurazione: per comodità usiamo come seconda componente la stringa in ingresso ancora da leggere (cioè la parte destra della stringa, testina inclusa); per la terza componente, che rappresenta il nastro, indichiamo la parte scritta e visitata dalla macchina (cioè la parte sinistra, testina inclusa, che può trovarsi su un simbolo $\not b$ nel caso di spostamento a destra).
   $(q, a.x, y.b) \vdash (q', a'.x, y.b'.c)$ se $(q', b', m_1, m_2) \in \delta(q, a, b)$ dove
   $a' = \varepsilon$ se $m_1 = R$, altrimenti $a' = a$;
   $c = \varepsilon$ se $m_2 = S$, altrimenti $c = \not b$.
   $x \in L(A)$ se e solo se $(q_0, z, Z_0) \vdash^* (q_F, x', y)$, con $q_F \in F$.

2. Una MTM1 accede solo ad una cella di memoria (quella corrente) e non può in alcun modo ritornare sulla parte sinistra del nastro di memoria. Questa informazione è finita ($\in \Gamma$) e può essere quindi codificata nello stato dell'organo di controllo. Per questo motivo le MTM1 definiscono solo linguaggi regolari. Definiscono anche tutti i linguaggi regolari in quanto le mosse della MTM1 catturano tutte le possibili mosse di un automa a stati finiti.

3. L'uso di un unico nastro e del determinismo non cambia nulla: chiaramente non c'è aumento di potere espressivo, ma non c'è nemmeno diminuzione, in quanto si possono comunque catturare tutte le possibili mosse di un automa a stati finiti deterministico. I linguaggi riconosciuti rimangono quindi regolari.

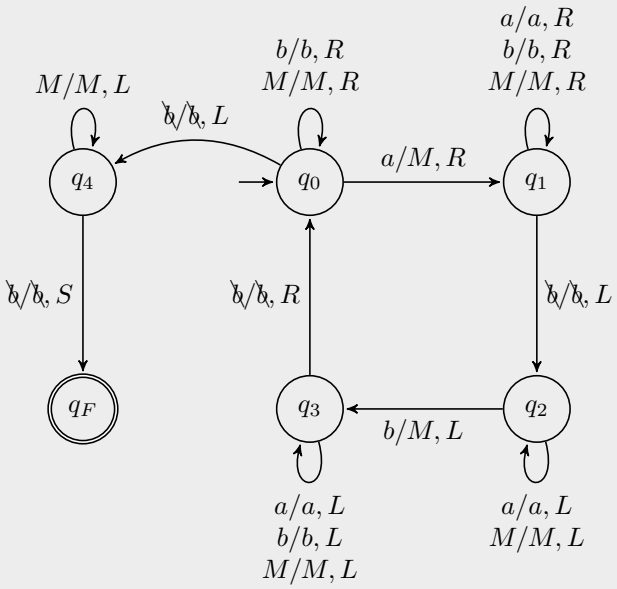
Esercizio 1.14.

1. Si definisca una macchina di Turing M *a nastro singolo* che riconosca il linguaggio $L \subseteq \{a, b\}^*$ consistente di tutte e sole le stringhe con un numero di occorrenze di a pari al numero di occorrenze di b.

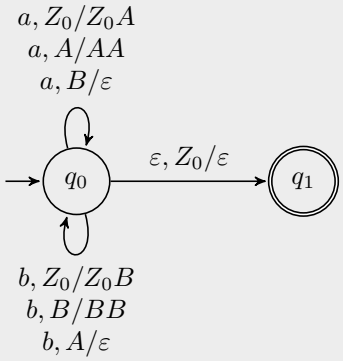
2. La macchina costruita è a potenza minima? In caso positivo, motivare la risposta; in caso negativo, si costruisca un automa a potenza minima per L.

SOLUZIONE

L'idea fondamentale è quella di compiere passate da sinistra a destra in cui si trasforma la prima a trovata in M, alternate a passate da destra a sinistra in cui si fa la stessa cosa con i simboli b. Una volta finite le a si deve controllare che il nastro contenga esclusivamente delle M. Assumiamo, come di consueto, che il nastro sia infinito in entrambe le direzioni (e quindi a sinistra del primo carattere e a destra dell'ultimo carattere si trova $\flat$).



È possibile riconoscere lo stesso linguaggio con un automa a pila non-deterministico; l'idea è usare simboli di pila A per tener traccia della lettura di un numero maggiore di a, oppure rispettivamente B per le b:



1.2 Grammatiche formali

Esercizio 1.15. Si consideri l'alfabeto $A = \{d, s, t\}$ e il linguaggio L di tutte e sole le stringhe su A che soddisfano i requisiti seguenti:

1. ogni stringa inizia con un prefisso che può essere la stringa vuota, la stringa d o la stringa ds;
2. dopo il prefisso, c'è la sequenza tds ripetuta un numero arbitrario di volte (anche nessuna volta);
3. infine, la stringa si chiude con la sequenza ts;
4. la sequenza dtd non appare mai all'interno della stringa;
5. la stringa contiene tutti e tre i simboli alfabetici.

Esempi di stringhe appartenenti al linguaggio sono: dts, $dsts$, $tdsts$, $dstdsts$, $tdstdsts$. Esempi di stringhe non appartenenti al linguaggio sono: $ddts$, $dtdsts$, ts, d.

Scrivere una grammatica a potenza generativa minima che generi L.

SOLUZIONE

Basta una grammatica regolare. Il vincolo 5 impone che, in caso di prefisso vuoto, la sequenza tds sia ripetuta almeno una volta. Il vincolo 4 significa che, in caso di prefisso d, la sequenza tds sia ripetuta zero volte. L'espressione regolare corrispondente a L è la seguente: $dts|((tds|ds)(tds)^*ts)$. Una grammatica regolare è come segue:

$S \to A \mid B \mid C$
$A \to tB$
$B \to dD$
$D \to sA \mid sL$
$C \to dL$
$L \to tM$
$M \to s$

Un modo equivalente per risolvere l'esercizio è di iniziare con una grammatica non regolare che genera il linguaggio:

$S \to dts \mid tdsR \mid dsR$
$R \to tdsR \mid ts$

e di trasformarla in una grammatica regolare evitando le sequenze di terminali:

$S \to tS_1 \mid dS_6$
$S_1 \to dS_2$
$S_2 \to sS_3$
$S_3 \to tS_4 \mid tS_7$

$S_4 \to dS_5$
$S_5 \to sS_3$
$S_6 \to sS_3 \mid tS_7$
$S_7 \to s$

Esercizio 1.16. Si consideri il linguaggio L delle stringhe a^nb^n, $a^nb^na^nb^n$, ... dove a^nb^n può essere ripetuto 1 o più volte, con $0 < n < 4$. Esempi di stringhe in L sono: $aabb$, $abab$. Esempi di stringhe non in L sono invece: abb, $aabbab$. Si scriva una grammatica a potenza minima per L e per $L_1 = L \cap \{(a^mb^m)^+ \mid m > 0\}$.

SOLUZIONE

L è un linguaggio regolare perché n può assumere solo i valori 1, 2, 3 e poi può seguire un'altra stringa con le stesse caratteristiche; non è quindi necessaria una capacità di conteggio illimitata. Una grammatica regolare che genera L è la seguente, facilmente convertibile in un automa a stati finiti:

$S \to aA_1 \mid aB_1 \mid aC1$
$A_1 \to bA_2 \mid b$
$A_2 \to aA_1$
$B_1 \to aB_2$
$B_2 \to bB_3$
$B_3 \to bB_4 \mid b$
$B_4 \to aB_1$
$C_1 \to aC_2$
$C_2 \to aC_3$
$C_3 \to bC_4$
$C_4 \to bC_5$
$C_5 \to bC_6 \mid b$
$C_6 \to aC_1$

L_1 coincide con L perché $L \subset \{(a^mb^m)^+ \mid m > 0\}$, quindi $L \cap \{(a^mb^m)^+ \mid m > 0\} = L$. La grammatica è dunque la stessa.

Esercizio 1.17.

1. Scrivere una grammatica a potenza generativa minima che generi il linguaggio L_1 di tutte le stringhe, sull'alfabeto $A = \{0, 1\}$, aventi la seguente proprietà: la stringa è non vuota e, se letta da sinistra a destra, è il complemento della stringa letta da destra a sinistra. Per complemento di una stringa si intende la stringa ottenuta scambiando gli 0 con gli 1.

2. Scrivere una grammatica a potenza generativa minima che generi il linguaggio L_2 di tutte le stringhe, sull'alfabeto $A = \{0, 1\}$, che non sono palindrome. Una stringa è palindroma se non cambia leggendola da sinistra a destra e da destra a sinistra.

3. Con riferimento alle proprietà di chiusura dei vari tipi di linguaggi rispetto alle operazioni insiemistiche, stabilire di che tipo è la grammatica a potenza generativa minima che genera l'intersezione di L_1 e L_2.

SOLUZIONE

1. Per L_1: $S \to 0S1 \mid 1S0 \mid 01 \mid 10$
2. Per L_2: $\begin{array}{l} S \to 1X0 \mid 0X1 \mid 0S0 \mid 1S1 \\ X \to 1X \mid 0X \mid \epsilon \end{array}$
3. L'intersezione tra L_1 e L_2 è ancora L_1 (nessuna stringa di L_1 è palindroma), quindi basta una grammatica non contestuale. Naturalmente questo è compatibile con il fatto che non ci sia chiusura rispetto all'intersezione tra linguaggi non contestuali, il che indica semplicemente che esistono due linguaggi non contestuali la cui intersezione non è non contestuale, ma non vuol dire che l'intersezione di due linguaggi non contestuali non possa essere non contestuale.

Esercizio 1.18. Si consideri il seguente linguaggio L definito sull'alfabeto $\Sigma = \{a, b, c, d\}$:

$$L = \{a^n b^i c^j d^k \mid n \geq 0, i \geq 0, j \geq 0, k \geq 0, n = i + j + k\}$$

Si specifichi una grammatica a potenza minima che generi L.

SOLUZIONE

È possibile generare il linguaggio tramite una grammatica non contestuale. In particolare, la grammatica mette in corrispondenza ogni a con la rispettiva b, c, o d a seconda della posizione della a stessa nella stringa.

$$G = \begin{cases} S \to aSd \mid A \\ A \to aAc \mid B \\ B \to aBb \mid \varepsilon \end{cases}$$

Esercizio 1.19. Si consideri la seguente grammatica G_1 con produzioni

$$T \quad \to \quad aTbT \quad \mid \quad bTaT \quad \mid \quad \varepsilon$$

e simbolo iniziale T. G_1 genera tutte e sole le stringhe in cui le a e le b sono in ugual numero.

Si consideri ora la seguente grammatica G_2 con produzioni

$$\begin{array}{lcl} S & \to & TaT \\ T & \to & aTbT \quad \mid \quad bTaT \quad \mid \quad \varepsilon \end{array}$$

e simbolo iniziale S.

1. Qual è il linguaggio generato da G_2? Motivare la risposta.
2. Fornire una dimostrazione della risposta data al punto 1.

SOLUZIONE

1. Il linguaggio generato da G_2 è il linguaggio di tutte e sole le stringhe in cui il numero di a è pari al numero di b più uno, ossia

$$L(G_2) = \{x \mid x \in \{a,b\}^* \wedge \#a(x) = \#b(x) + 1\}.$$

   Come affermato nel testo, le stringhe generate a partire da T sono quelle del linguaggio $L(G_1)$ di tutte e sole le stringhe in cui le a e le b sono in ugual numero. Le stringhe di $L(G_2)$, tramite la produzione $S \to TaT$, sono generate dalla giustapposizione di una a tra due stringhe di $L(G_1)$.

2. Da quanto scritto sopra, le stringhe generate da G_2 hanno la forma $x \cdot a \cdot y$, dove $x, y \in L(G_1)$. È immediato concludere che G_2 genera *soltanto* stringhe in cui vi sia una a in più rispetto alle b, poichè sia x sia y sono già bilanciate, appartenendo a $L(G_1)$. Per mostrare che G_2 genera *tutte* le stringhe in cui vi sia una a in più rispetto alle b, si consideri una generica stringa w tale per cui $\#a(w) = \#b(w) + 1$. Poiché w contiene più a che b, scandendo w da sinistra a destra, esiste necessariamente un carattere di w che è la prima a in soprannumero rispetto alle b incontrate (incluso il caso in cui non si siano incontrate b e cioè che la a sia il primo carattere di w). Sia x la sottostringa composta da tutti i caratteri a sinistra di quella a, e y quella composta da tutti i caratteri a destra della a. Per quanto scritto, la a che separa x da y è la prima a in soprannumero, quindi in x le a e le b sono in ugual numero. A questo punto segue necessariamente che anche nella y le a e le b sono in ugual numero, altrimenti la stringa $w = x \cdot a \cdot y$ non soddisferebbe più il vincolo $\#a(w) = \#b(w) + 1$.

Esercizio 1.20.

1. Scrivere una grammatica a potenza minima che generi il linguaggio seguente:

   $L_1 = \{(bc)^m \mid m \geq 2\}$

2. Sfruttando la risposta al punto precedente, scrivere una grammatica a potenza minima che generi il linguaggio seguente:

   $L_2 = \{b^k a^\ell (bc)^m a^n b^p \mid k > p, 2\ell = n, m \geq 2\}$

SOLUZIONE

1. È sufficiente una grammatica regolare, dove S_0 è l'assioma.

$$\begin{array}{lcl} S_0 & \to & bS_1 \\ S_1 & \to & cS_2 \\ S_2 & \to & bS_3 \\ S_3 & \to & c \mid cS_2 \end{array}$$

2. È sufficiente una grammatica non contestuale. Il linguaggio può infatti anche essere pensato nel modo seguente:
   $L_2 = \{b^s b^p a^\ell (bc)^m (aa)^\ell b^p \mid s \geq 1, p \geq 0, \ell \geq 0, m \geq 2\}$

$$\begin{array}{lcll} S & \to & UV & \\ U & \to & bU \mid b & \text{1 o più } b \\ V & \to & bVb \mid W & \text{generiamo } p \text{ lettere } b \text{ da ambo i lati} \\ W & \to & aWaa \mid S_0 & \text{generiamo } \ell \text{ lettere } a \text{ a sx e } 2\ell \text{ a dx} \end{array}$$

   dove S_0 è il simbolo iniziale della grammatica per L_1.

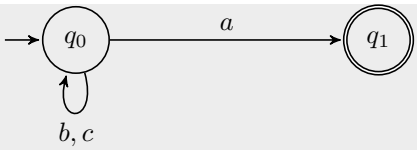
Esercizio 1.21. Si consideri la seguente grammatica:

$$G = \begin{cases} S & \to CBS \mid CS \mid BS \mid a \\ CB & \to BC \\ CC & \to c \\ BB & \to b \end{cases}$$

1. Di che tipo è la grammatica G?
2. Che linguaggio genera G?
3. Si scriva un automa a potenza minima che accetti il linguaggio generato da G.

SOLUZIONE

1. La grammatica è di tipo non ristretto, a causa della produzione $CB \to BC$, nella quale non è possibile isolare un contesto che non cambi tra la parte sinistra e la parte destra della produzione stessa.
2. Il linguaggio generato da G è regolare; in particolare è quello rappresentato dall'espressione regolare $(b|c)^*a$.
3. L'automa a potenza minima che riconosce $L(G)$ è il seguente automa a stati finiti:



Esercizio 1.22. Si consideri il linguaggio sull'alfabeto $\Sigma = \{a, b, c, d\}$ composto da tutte e sole le stringhe in cui:

1. compare al più una sola c o una sola d, mai entrambe;
2. se compare una c, il numero di a a sinistra della c è uguale a quello di b a sinistra della c; idem a destra della c;
3. se compare una d, il numero di a a sinistra della d è uguale al doppio di quello delle b a sinistra della d; idem a destra della d;
4. Nel caso in cui non compaiano c o d, il numero di a e b è arbitrario.

Si fornisca una grammatica a potenza minima che generi il linguaggio descritto.

SOLUZIONE

Una grammatica a potenza minima (libera dal contesto) che genera il linguaggio è la seguente:

$S \to$	$A \mid B \mid C$	tre casi: 1) né c né d, 2) c'è la c, 3) c'è la d
$A \to$	$aA \mid bA \mid \varepsilon$	il numero di a e b è arbitrario
$B \to$	$B'cB'$	compare la c
$B' \to$	$aB'bB' \mid$	a e b in ugual numero
	$bB'aB' \mid$	
	ε	
$C \to$	$C'dC'$	compare la d
$C' \to$	$aC'aC'bC' \mid$	a due volte più numerose delle b
	$aC'bC'aC' \mid$	
	$bC'aC'aC' \mid$	
	ε	

Esercizio 1.23. Si considerino le seguenti definizioni, dove siano $x, y \in V_T^*$ (cioè stringhe di simboli terminali), $A, B \in V_N$:

- Una grammatica si dice *lineare a destra* se le sue produzioni sono del tipo $A \to Bx$, oppure $A \to x$.
- Una grammatica si dice *lineare a sinistra* se le sue produzioni sono del tipo $A \to xB$, oppure $A \to x$.

- Una grammatica si dice *lineare a destra o a sinistra* se le sue produzioni sono del tipo $A \to xB$, oppure $A \to Bx$, oppure $A \to x$.
- Una grammatica si dice *lineare* se le sue produzioni sono del tipo $A \to xBy$, oppure $A \to x$.

Si confrontino tra loro le potenze generative (le famiglie di linguaggi generati) delle suddette classi di grammatiche tra loro e con quelle dei seguenti formalismi:

1. automi a stati finiti;
2. automi a pila nondeterministici;
3. automi a pila deterministici.

SOLUZIONE

1. Le grammatiche lineari a sinistra (GLS) sono una sorta di abbreviazione delle grammatiche regolari: infatti, ad esempio la produzione $A \to acB$ può essere trasformata nella coppia di produzioni regolari equivalenti $A \to aA_1$; $A_1 \to cB$ con A_1 simbolo nuovo. Quindi le GLS sono equipotenti agli automi a stati finiti.

   Gli automi a stati finiti sono equivalenti non solo alle grammatiche regolari le cui produzioni sono del tipo $A \to aB$, ma anche a quelle con produzioni del tipo $A \to Ba$ (che a loro volta sono equivalenti alle lineari a destra con ragionamento analogo al punto 1), purché esse non contengano contemporaneamente produzioni dei due tipi: basta infatti associare ad una transizione del tipo $\delta(A, a) = B$ una produzione del tipo $B \to Aa$, invece che $A \to aB$ e "invertire" stati finali con assioma: la derivazione parte dalla fine della stringa invece che dall'inizio. Quindi GLS, GLD, grammatiche regolari e automi a stati finiti sono tutti equipotenti.

2. Le grammatiche lineari a destra o a sinistra sono equipotenti alle grammatiche lineari: infatti esse ne sono un caso particolare (quindi non sono più potenti); inoltre ogni produzione del tipo $A \to xBy$ può essere trasformata nella coppia $A \to xA_1$, $A_1 \to By$, con A_1 simbolo nuovo.

   La grammatica lineare $S \to aSb \mid ab$ genera il linguaggio $a^n b^n$ che non è regolare. Quindi le grammatiche lineari e le lineari a destra o a sinistra sono più potenti di quelle lineari solo a sinistra o solo a destra.

   Le grammatiche lineari sono un caso particolare di quelle non contestuali. Tuttavia esse non possono generare linguaggi come $\{a^n b^n c^m d^m \mid n, m > 0\}$. Diamo qui una motivazione intuitiva: la caratteristica fondamentale di una grammatica lineare è che in ogni passo di derivazione vi è al più un solo simbolo nonterminale, e la ricorsione su tali simboli è l'unico meccanismo che le grammatiche hanno per "contare", ad esempio nella generazione di $a^n b^n$. Per questo motivo le grammatiche

lineari non possono generare nella stessa stringa sia $a^n b^n$ che $c^m d^m$, con n e m non limitati. Di conseguenza esse sono anche strettamente meno potenti degli automa a pila nondeterministici.

3. La seguente grammatica lineare
   $S \to aAb \mid aBbb$
   $A \to aAb \mid ab$
   $B \to aBbb \mid abb$
   invece genera il linguaggio $\{a^n b^n\} \cup \{a^n b^{2n}\}$ che è non-deterministico, intuitivamente in quanto non è possibile decidere quale gestione della pila adottare durante la lettura della prima parte delle b (cancello un simbolo in pila ogni due b o ogni singolo b?). Siccome il precedente linguaggio $\{a^n b^n c^m d^m\}$ è deterministico, se ne deduce che le potenze delle grammatiche lineari e degli automi a pila deterministici non sono confrontabili.

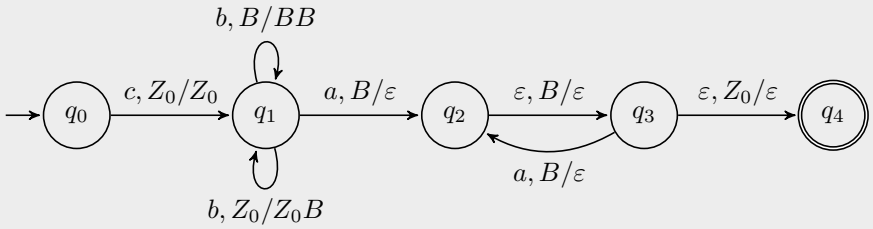
Esercizio 1.24. Sia data la seguente grammatica G:

$$\begin{array}{ll} S & \to BSa \mid C \\ BC & \to Cbb \\ C & \to c \end{array}$$

Si dica quale è il linguaggio generato da G. Si scriva inoltre un automa che riconosce il linguaggio generato da G. L'automa deve appartenere alla classe di potenza riconoscitiva minima tra quelle che riconoscono il linguaggio generato da G.

SOLUZIONE

Il linguaggio generato dalla grammatica è $L(G) = \{cb^{2n}a^n \mid n \geq 0\}$, riconosciuto dal seguente automa a pila deterministico:



Esercizio 1.25. Si scriva una grammatica che generi il linguaggio L_1 composto da stringhe sull'alfabeto $\Sigma = \{(,),]\}$, tali che le parentesi tonde siano bilanciate –ossia ben parentetizzate–, con la variante che la parentesi quadra può bilanciare un numero arbitrario (non nullo) di parentesi tonde aperte. Ad esempio, le stringhe $(()())$, $((()))$, $(((()](()]$ appartengono al linguaggio, mentre le stringhe $(()$, $((((](,])()()$ non vi appartengono. Si scriva una grammatica, preferibilmente a potenza minima che produca il linguaggio L_1.

SOLUZIONE

Una grammatica noncontestuale è necessaria e sufficiente per generare L_1, ad esempio la seguente:

		Casi di stringhe ben parentetizzate:
$S \to$	ε	vuota
$S \to$	BS	stringa con tonde bilanciate seguita da ben parentetizzata
$S \to$	$N]S$	non bilanciata seguita da] e da ben parentetizzata
		Casi di stringhe con tonde bilanciate:
$B \to$	ε	vuota
$B \to$	(B)	bilanciata racchiusa da tonde
$B \to$	BB	bilanciata seguita da bilanciata
		Casi di stringhe non bilanciate:
$N \to$	$(B$	una aperta in eccesso
$N \to$	$(N$	ulteriore aperta in eccesso
$N \to$	NN	concatenazione di non bilanciate

1.3 Specifiche mediante formule logiche

Esercizio 1.26. Si consideri il linguaggio $L_1 = \neg L_2$, dove $L_2 = (ab)^+$.

a) Si definisca L_1 nella logica MFO, se possibile, altrimenti se ne motivi l'impossibilità.

b) Con riferimento alle proprietà dei linguaggi *star-free*, si chiarisca se è possibile fornire un'espressione insiemistica per L_1 facendo uso del simbolo dell'insieme vuoto ($\emptyset$), degli insiemi di una singola lettera ($\{a\}$ e $\{b\}$), degli operatori di unione ($\cup$), intersezione ($\cap$), complemento ($\neg$) e concatenamento ($\cdot$), senza usare gli operatori di Kleene (* e $^+$).
Suggerimento: si noti che $\neg\emptyset = \{a, b\}^*$.

SOLUZIONE

a) Si può definire L_1 in MFO mediante la formula seguente:

$$
\begin{aligned}
&\neg(\\
&\exists x(x = 0 \wedge a(x)) && \wedge \\
&\forall x(a(x) \to \exists y(succ(x, y) \wedge b(y))) && \wedge \\
&\forall x((b(x) \wedge \neg last(x)) \to \exists y(succ(x, y) \wedge a(y))) && \wedge \\
&\exists x(last(x) \wedge b(x)) \\
&)
\end{aligned}
$$

b) Un'espressione per L_1 che non usi gli operatori di Kleene è la seguente:

$$\begin{array}{rl} (\neg\emptyset \cdot \{a\} \cdot \{a\} \cdot \neg\emptyset) & \text{(parole che hanno due } a \text{ consecutive)} \\ \cup & \\ (\neg\emptyset \cdot \{b\} \cdot \{b\} \cdot \neg\emptyset) & \text{(che hanno due } b \text{ consecutive)} \\ \cup & \\ \neg(\{a\} \cdot \neg\emptyset) & \text{(parole che non iniziano con } a) \\ \cup & \\ \neg(\neg\emptyset \cdot \{b\}) & \text{(parole che non finiscono con } b) \end{array}$$

Esercizio 1.27. Si costruisca una grammatica o automa o formula logica a potenza minima per il seguente linguaggio:

$$\{a^{2n}\Sigma^+b^{3n} \mid n > 0\} \cup \{\Sigma^+b^{3n} \mid n > 0\},$$

con $\Sigma = \{a, b\}$.

SOLUZIONE

Il linguaggio è aperiodico (o "star-free"), in quanto $\{a^{2n}\Sigma^+b^{3n} \mid n > 0\} \subset \{\Sigma^+b^{3n} \mid n > 0\} = \Sigma^+.b^3$. In pratica si tratta di stringhe con i seguenti vincoli: sono lunghe almeno 4 caratteri e finiscono con 3 b. Il linguaggio richiesto si può dunque esprimere con la seguente formula MFO:

$$\exists y(y > 0 \wedge b(y) \wedge b(y+1) \wedge b(y+2) \wedge last(y+2)).$$

Esercizio 1.28. Si definisca mediante un'opportuna formula MFO il linguaggio L fatto di tutte e sole le parole costruite sull'alfabeto $A = \{a, b, c, d\}$ che rispettano le seguenti regole:

- Ogni coppia di simboli d consecutivi (cioè tra i quali non ci sono altri simboli d, ma ci possono essere altri simboli) è separata da esattamente 3 simboli.
- Se la parola è della forma $x.ccccc.y$ (dove x e y sono stringhe di A^* tali che x non termina per c e y non inizia per c), allora la parola inizia e termina con un simbolo b.

SOLUZIONE

La formula desiderata è la seguente, i cui due congiunti più esterni catturano, rispettivamente, la prima e la seconda condizione del testo:

$$\forall x, y(x < y \wedge d(x) \wedge d(y) \wedge \forall z(x < z \wedge z < y \rightarrow \neg d(z)) \rightarrow y = x + 4)$$

$$\wedge$$

$$\exists x, y \left(\begin{array}{l} y = x + 4 \wedge \forall p(x \leq p \wedge p \leq y \rightarrow c(p)) \wedge \\ (last(y) \vee \exists z(z = y + 1 \wedge \neg c(z))) \wedge \\ (x = 0 \vee \exists k(k = x - 1 \wedge \neg c(k))) \\ \quad \rightarrow \forall u(u = 0 \vee last(u) \rightarrow b(u)) \end{array} \right)$$

Esercizio 1.29. Si vuole modellare il comportamento di una macchina che vende bottiglie di latte. La macchina accetta 2 tipi di monete: da 50 centesimi e da 1 euro. Nel momento in cui la macchina ha almeno 1 euro di credito, essa produce automaticamente una bottiglia di latte. Dopo che l'utente ha preso il latte, si può ricominciare a inserire monete nella macchina. Inoltre, l'utente può chiedere di avere indietro il resto che è presente in quel momento nella macchina.

Il comportamento della macchina viene modellato descrivendo sequenze di "eventi" rilevanti. Più precisamente, gli eventi di interesse sono i seguenti:

- $in50c$, e $in1$, che corrispondono al fatto che viene inserita una moneta da 50¢ o 1€, rispettivamente;
- $cred0$, e $cred50$, che corrispondono al fatto che il credito presente nella macchina è di 0 o 50, centesimi, rispettivamente (si noti che nel momento in cui si inserisce una moneta che farebbe aumentare il credito fino a essere uguale a o più di 1€, viene erogato il latte, quindi il credito è subito consumato);
- $latte$, che corrisponde al fatto che viene erogato il latte;
- $resto$, che corrisponde al fatto che si richiede il resto presente in quel momento.

Le sequenze di simboli che descrivono il comportamento della macchina sono alternanze di simboli corrispondenti a comandi dati alla macchina ($in50c$, $in1$, $resto$) e simboli che rappresentano il credito residuo ($cred0, cred50$). Se però l'inserimento di una nuova moneta porta all'erogazione del latte (perché si raggiungerebbe l'ammontare di 1€), allora il comando di inserimento è subito seguito dal simbolo $latte$ che indica l'erogazione del latte, che a sua volta è seguito dall'indicazione del credito residuo. Le sequenze di simboli iniziano e terminano con credito uguale a 0.

Immaginiamo, per esempio, una sequenza in cui l'utente inserisce prima una moneta da 50¢, poi un'altra da 50¢ (a questo punto la macchina avrebbe raggiunto 1€, quindi deve essere erogato il latte e aggiornato il credito), poi una da 50¢, poi una da 1€ (nel qual caso, di nuovo viene subito erogato il latte), poi viene chiesto il resto, poi viene inserita una moneta da 50¢, poi di nuovo viene chiesto il resto, e a questo punto la sequenza termina. Tale sequenza corrisponde alla seguente sequenza di simboli:
$cred0, in50c, cred50, in50c, latte,\ cred0, in50c, cred50, in1, latte,\ cred50, resto, cred0,$
$in50c, cred50, resto, cred0$.

Si formalizzino in logica i seguenti vincoli e proprietà:

1. All'inizio e alla fine il credito è 0.
2. Un simbolo rappresentante un credito, se non è l'ultimo della sequenza, deve essere subito seguito da un simbolo rappresentante un comando.
3. L'effetto dell'inserimento di una moneta sul credito, cioè come cambia il credito in seguito all'inserimento di una moneta, ed eventualmente l'emissione del latte (si noti che, in caso di emissione del latte, prima viene erogato il latte e poi viene segnalato il nuovo credito).
4. L'effetto di un comando di resto.

Il punteggio sarà tanto più alto, quanto meno potente è il formalismo usato per definire le sequenze.

SOLUZIONE

Le sequenze desiderate si possono definire mediante formule MFO, usando i predicati $in50c(x)$, $credo0(x)$, $latte(x)$, ecc., per esempio nel modo seguente:

1. Inizialmente e alla fine il credito è 0:

$$\exists x(x = 0 \wedge cred0(x)) \wedge \forall x(last(x) \rightarrow cred0(x))$$

2. Un simbolo di credito deve essere seguito da un simbolo di comando:

$$\forall x \forall y(y = x + 1 \wedge (cred0(x) \vee cred50(x)) \rightarrow (in50c(y) \vee in1(y) \vee resto(y)))$$

3. Effetto di un inserimento di moneta:

$$\forall x \forall y \left(\begin{array}{l} y = x + 1 \rightarrow \\ \left(\begin{array}{l} (cred0(x) \wedge in50c(y) \rightarrow \exists z(z = y + 1 \wedge cred50(z))) \\ \left(cred0(x) \wedge in1(y) \rightarrow \exists z, w \left(\begin{array}{l} z = y + 1 \wedge w = z + 1 \wedge \\ latte(z) \wedge cred0(w) \end{array} \right) \right) \wedge \\ \left(cred50(x) \wedge in1(y) \rightarrow \exists z, w \left(\begin{array}{l} z = y + 1 \wedge w = z + 1 \wedge \\ latte(z) \wedge cred50(w) \end{array} \right) \right) \wedge \\ \left(cred50(x) \wedge in50c(y) \rightarrow \exists z, w \left(\begin{array}{l} z = y + 1 \wedge w = z + 1 \wedge \\ latte(z) \wedge cred0(w) \end{array} \right) \right) \end{array} \right) \end{array} \right)$$

4. Effetto di una richiesta di resto:

$$\forall x \forall y(resto(x) \rightarrow \exists y(y = x + 1 \wedge cred0(y))).$$

1.4 Linguaggi formali

Esercizio 1.30. Sia dato un alfabeto Σ. Date due stringhe qualunque $x, y \in \Sigma^*$, diciamo che y è una *sottostringa* di x se e solo se y può essere ottenuta da x rimuovendo alcuni caratteri (anche nessuno o tutti). Dunque, ad esempio, tutte e sole le sottostringhe di $x = abaa$ sono: ε, a, b, ab, ba, aa, aba, aaa, baa, $abaa$.

Dato un qualunque linguaggio L di alfabeto Σ, ossia $L \subseteq \Sigma^*$, definiamo $SUBSEQ(L)$ come il linguaggio formato da tutte le sottostringhe di tutte le stringhe in L, ossia:

$$SUBSEQ(L) = \{y \in \Sigma^* \mid y \text{ è una sottostringa di } x, \text{ per qualche } x \in L\}.$$

Sia ora Σ un alfabeto unario, ossia $|\Sigma| = 1$. Si dimostri, o si confuti, la seguente affermazione:

Per ogni linguaggio L su alfabeto unario, $SUBSEQ(L)$ è un linguaggio regolare.

SOLUZIONE

L'affermazione è vera. Si consideri infatti un qualunque linguaggio L su alfabeto unario (per fissare le idee: $\Sigma = \{a\}$). Ora si hanno due casi: o L è infinito o è finito.

Se L è finito, significa che è definita la lunghezza massima di una stringa in L, cioè $\max_{x \in L} |x| = n$ per qualche $n \in \mathbb{N}$. Essendo l'alfabeto unario, $a^n \in L$. Ma allora è facile capire che tutte e sole le sottostringhe di a^n sono l'insieme $S = \{a^k \mid 0 \leq k \leq n\}$, e per di più non possono esisterne altre in $SUBSEQ(L)$, ossia $S = SUBSEQ(L)$. Essendo anche $SUBSEQ(L)$ di dimensione finita, è per forza regolare.

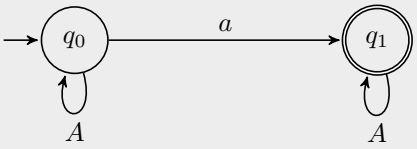
Se invece L è infinito, significa che esistono in L stringhe arbitrariamente grandi, ossia per ogni $n \in \mathbb{N}$, esiste un $n' > n$ tale che $a^{n'} \in L$. Perciò, per ogni $n \in \mathbb{N}$ esiste un $n' > n$ tale che l'insieme $\{a^k \mid 0 \leq k \leq n'\}$ è un sottoinsieme di $SUBSEQ(L)$. Pertanto, in pratica, $SUBSEQ(L) = \{a^k \mid k \in N\}$, e dunque $SUBSEQ(L)$ è il linguaggio universo su Σ, che è evidentemente regolare.

In verità si può addirittura dimostrare che per ogni linguaggio L su qualunque alfabeto finito Σ (di qualunque cardinalità), $SUBSEQ(L)$ è regolare. La dimostrazione, piuttosto impegnativa, è nota come Lemma di Higman; si veda A. G. Higman: Ordering by divisibility in abstract algebra. Proceedings of the London Mathematical Society, 3:326-336, 1952.

Esercizio 1.31. Sia L un qualunque linguaggio regolare costruito su un alfabeto A e sia a un simbolo di A. Il linguaggio L' che consiste di tutte le stringhe di L che contengono la a è regolare? Motivare opportunamente la risposta.

SOLUZIONE

Sia L'' il linguaggio di tutte le stringhe contenenti la a. Tale linguaggio è chiaramente regolare, essendo ad esempio riconosciuto dal seguente automa nondeterministico a stati finiti (l'etichetta A indica un qualunque simbolo di A, incluso a).



Il linguaggio L' si può ottenere facendo l'intersezione di L e L'', che sono entrambi regolari. Poiché la famiglia dei linguaggi regolari è chiusa rispetto all'intersezione, anche L' è regolare.

Esercizio 1.32. Siano $A = \{0, 1, 2\}$ e $B = \{0, 1\}$. Si considerino i linguaggi $L_1 = \{x1x^R \mid x \in A^+\}$ e $L_2 = B^+212B^+$.

1. Si dica quali sono i modelli a *potenza minima* per L_1 ed L_2, motivando adeguatamente la risposta.
2. Si descriva il funzionamento di un automa (o si definisca una grammatica) a potenza minima per l'intersezione tra L_1 e L_2.

SOLUZIONE

1. Rispettivamente automa a pila non deterministico e automa a stati finiti: per L_1 si sfrutta la pila per inserire i simboli di x e poi controllare x^R rimuovendo simboli dalla pila (il non-determinismo è necessario per poter discriminare il simbolo 1 centrale, visto che potrebbe far parte di x); L_2 è chiaramente un linguaggio regolare.
2. Per prima cosa notiamo: $L_1 \cap L_2 = \{x212x^R \mid x \in B^+\}$. Quindi in questo caso l'automa a pila può essere *deterministico*, in quanto x non contiene il simbolo 2, quindi la sottostringa 212 identifica univocamente la parte centrale della stringa. L'automa può procedere in maniera usuale, impilando i simboli di x fino a raggiungere 212; a questo punto rimuoverà i simboli dalla pila, controllando che la parte restante sia x^R.

Esercizio 1.33. Si considerino i seguenti linguaggi sull'alfabeto $A = \{a, b, c\}$:

$$L_1 = A^* \cdot \{b\} - (A^* \cdot (A - \{a\}) \cdot A^* \cdot \{b\})$$
$$L_2 = A^* - (A^* \cdot (A - \{b\}) \cdot A^*)$$
$$L_3 = L_1 \cdot L_2$$

dove * è la stella di Kleene, $-$ è la differenza insiemistica e $\cdot$ è la concatenazione.

Utilizzare un formalismo a potenza minima (tra automi, grammatiche e logica MFO) che caratterizzi il linguaggio L_3.

SOLUZIONE

Si può facilmente verificare che $L_1 = a^* \cdot b$ e $L_2 = b^*$. Pertanto $L_3 = a^* \cdot b^+$. È quindi un linguaggio regolare (caratterizzato, per l'appunto, dall'espressione regolare data). Tuttavia si tratta anche di un linguaggio di tipo *star-free*, poiché, come si vede dalle espressioni originali, la stella di Kleene è applicata solo sull'intero alfabeto A. Si può quindi formalizzare il linguaggio L_3 mediante una formula MFO, come segue:

$$
\begin{array}{rl}
\exists x(x = 0 \wedge (a(x) \vee b(x))) & \text{all'inizio c'è } a \text{ o } b \\
\wedge & \\
\forall x(a(x) \rightarrow \exists y(y = x + 1 \wedge (a(y) \vee b(y)))) & \text{dopo } a \text{ c'è } a \text{ o } b \\
\wedge & \\
\forall x(b(x) \rightarrow (last(x) \vee \exists y(y = x + 1 \wedge b(y)))) & \text{dopo } b \text{ c'è } b \text{ o è l'ultimo carattere} \\
\wedge & \\
\exists x(b(x) \wedge last(x)) & \text{l'ultimo carattere è } b \\
 & \text{(congiunto ridondante)}
\end{array}
$$

Esercizio 1.34.

1. È vero che se un linguaggio L non è regolare allora neanche il suo complemento L^c è regolare? Perché?
2. È vero che se un linguaggio L non è libero dal contesto allora neanche il suo complemento L^c è libero dal contesto? Perché?

SOLUZIONE

1. Vero. Supponiamo per assurdo che L^c sia regolare. Allora grazie alla chiusura dei linguaggi regolari rispetto alla complementazione, anche il complemento di L^c dev'essere regolare. Ma il complemento di L^c è L, che per ipotesi non è regolare. Assurdo.
2. Falso. Un esempio di linguaggio non libero dal contesto è $L = \{a^n b^n c^n\}$, non riconoscibile da nessun automa a pila. Il suo complemento è invece libero dal contesto. Si tratta infatti del linguaggio le cui stringhe o sono del tipo $a^n b^m c^l$ con $n \neq m$ o $n \neq l$ oppure non sono del tipo $a^* b^* c^*$. Chiamiamo $L_1 = \{a^n b^m c^l \mid n \neq m \text{ o } n \neq l\}$ e $L_2 = a^* b^* c^*$ e sia L_2^c il complemento di L_2. In sostanza, L^c è dato dall'unione di L_1 e L_2^c, che sono entrambi liberi dal contesto (e quindi anche L^c è libero dal contesto, in quanto la famiglia dei linguaggi liberi dal contesto è chiusa rispetto all'unione). Infatti, per riconoscere L_1 basta un automa a pila

non deterministico, mentre notiamo che L_2 è un linguaggio regolare, e quindi anche il suo complemento L_2^c è regolare (e quindi anche libero dal contesto).

Esercizio 1.35. Si ricordi che la stringa inversa di una generica stringa w, qui denotata w^R, è ottenuta da w scrivendo i suoi caratteri in ordine inverso, ad esempio $(abaabb)^R = bbaaba$, $(aaabbb)^R = bbbaaa$. L'operazione di inversione viene estesa dalle stringhe ai linguaggi nel modo naturale:

$$L^R = \{w^R \mid w \in L\}.$$

Si dica, con brevi spiegazioni, quali delle seguenti famiglie di linguaggi sono chiuse rispetto all'operazione di inversione:

1. linguaggi non contestuali;
2. linguaggi regolari;
3. linguaggi generati da grammatiche non ristrette;
4. linguaggi non contestuali deterministici.

SOLUZIONE

1. Data una grammatica G non contestuale che genera il linguaggio L, la grammatica G^R ottenuta trasformando ogni produzione $A \to \alpha$ di G, nella produzione $A \to \alpha^R$, genera evidentemente L^R.

2. In particolare, se G è anche regolare, ogni produzione del tipo $A \to Ba$ viene trasformata in $A \to aB$ ottenendo quindi una grammatica che genera L^R e soddisfa ancora la definizione di grammatica regolare (è noto infatti che le grammatiche lineari a sinistra o lineari a destra sono tra loro equivalenti e generano entrambe tutti e solo i linguaggi regolari).

3. Anche i linguaggi generati da grammatiche non ristrette sono chiusi rispetto all'inversione; infatti queste grammatiche sono equivalenti alle macchine di Turing (MT). Data quindi una MT che riconosca L basta costruirne una che inverta la stringa di ingresso in un nastro ausiliario e successivamente simuli il comportamento della MT originaria usando il nastro ausiliario al posto del nastro di ingresso.

4. I linguaggi non contestuali deterministici non sono invece chiusi rispetto all'inversione. Ciò si può dedurre ricordando che $L = \{a^n b^n\} \cup \{a^n b^{2n}\}$ è non-deterministico e osservando invece che $L_1 = \{1a^n b^n\} \cup \{a^n b^{2n}\}$ è deterministico (la lettura del primo simbolo permette di stabilire subito se si dovranno contare 2 b per ogni a oppure una solo b). L_1^R però non è deterministico perché a inizio stringa non è possibile stabilire quale sarà il rapporto corretto tra il numero di b e quello di a.

Esercizio 1.36. Si consideri il linguaggio L fatto di tutte e sole le stringhe sull'alfabeto $\{a, b, c\}$ della forma $w.u.w^R$, dove $w \in \{a, b\}^+$ e $u \in \{a, b, c\}^*$.

1. Dire qual è il formalismo (o quali sono i formalismi, se più di uno, laddove per "formalismo" si intende automa, grammatica o logica MFO), che è in grado di riconoscere/generare/definire L ed è a potere espressivo minimo.
2. Usando il formalismo identificato al punto 1, definire l'automa/grammatica/formula logica che riconosce/genera/definisce L.
3. Si consideri il linguaggio L' fatto di tutte e sole le stringhe della forma wu^Ruw^R, con ancora $w \in \{a, b\}^+$ e $u \in \{a, b, c\}^*$.
   Qual è il formalismo (o quali sono i formalismi, se più di uno) a potere espressivo minimo tra automi, grammatiche e logica MFO che è in grado di riconoscere/generare/definire L'? Scrivere un automa/grammatica/formula logica nel suddetto formalismo (o in uno dei suddetti formalismi) che riconosce/genera/definisce L'.

SOLUZIONE

1. Il formalismo richiesto è MFO, in quanto L è semplicemente fatto di tutte e sole le stringhe in cui il primo simbolo è a o b, ed è uguale all'ultimo simbolo.
2. Una possibile formula MFO che definisce L è la seguente:

$$\exists x \exists y (x = 0 \wedge last(y) \wedge x < y \wedge ((a(x) \wedge a(y)) \vee (b(x) \wedge b(y))))$$

3. Il linguaggio L' è semplicemente fatto di stringhe della forma vv^R in cui il primo simbolo in v deve essere a o b. Di conseguenza, i formalismi a potere espressivo minimo che sono in grado di riconoscere/generare L' sono gli automi a pila nondeterministici e le grammatiche non contestuali.

   Una grammatica noncontestuale che genera L' è la seguente:

$$S \to aAa \mid bAb; \quad A \to aAa \mid bAb \mid cAc \mid \varepsilon$$

Esercizio 1.37. Si consideri il linguaggio $L = \{a^m b^n \mid m \neq n,\ m, n > 0\}$. Esempi di stringhe di L sono: $aabbb$ e $aaabb$, mentre ab, $aabb$, aba, sono stringhe non appartenenti ad L.

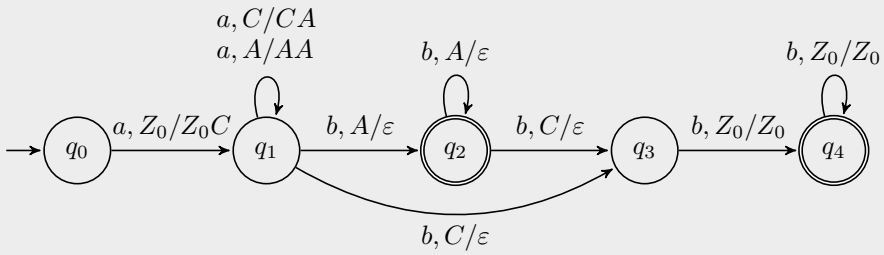
- Si fornisca una grammatica, preferibilmente a potenza generativa minima tra le classi conosciute, che generi L.
- Si fornisca un automa, preferibilmente a potenza riconoscitiva minima tra le classi conosciute, che riconosca L.

SOLUZIONE

Si può riconoscere L con una grammatica non-contestuale:

$$
\begin{aligned}
S &\to aCb \\
C &\to aCb \mid A \mid B \\
A &\to aA \mid a \\
B &\to bB \mid b
\end{aligned}
$$

Si può riconoscere L con un automa a pila deterministico:



2
Computabilità

2.1 Ricorsività di insiemi

Esercizio 2.1. Si denoti con M_i la i-esima macchina di Turing e si denoti con $L(M_i)$ il linguaggio da essa riconosciuto. Si considerino i seguenti insiemi:

1. $S_1 = \{i \mid L(M_i)$ non contiene nessuna stringa di lunghezza pari$\}$
2. $S_2 = \{i \mid L(M_i)$ contiene almeno una stringa di lunghezza pari$\}$

Per entrambi gli insiemi, si indichi, motivando opportunamente la risposta, se sono ricorsivi o ricorsivamente enumerabili.

SOLUZIONE

Nessuno dei due è ricorsivo per il teorema di Rice. Infatti la proprietà di contenere almeno una stringa di lunghezza pari è una proprietà del linguaggio riconosciuto (funzione calcolata) ed esiste almeno una macchina di Turing il cui linguaggio riconosciuto abbia tale proprietà (ad esempio il linguaggio A^*, con A alfabeto della macchina) e una senza (ad esempio il linguaggio $\emptyset$).

S_2 è ricorsivamente enumerabile, come si può vedere mediante enumerazione diagonale di tutte le stringhe di lunghezza crescente e pari e del numero di passi di simulazione da compiere: se esiste una stringa di lunghezza pari accettata, prima o poi ce ne si accorge.

S_1 non è ricorsivamente enumerabile perché lo è il suo complemento S_2, e quindi se lo fosse anche S_1, S_1 e S_2 dovrebbero essere entrambi ricorsivi, in contraddizione con quanto mostrato in precedenza.

Esercizio 2.2. Per ciascuno dei seguenti insiemi si stabilisca, fornendo un'opportuna motivazione, se è ricorsivo.

- $S_1 = \{i \mid i$ è un numero primo$\}$
- $S_2 = \{i \mid i$ è un numero primo minore di 100$\}$
- $S_3 = \{i \mid i$ è l'indice di una macchina di Turing che stabilisce se il suo ingresso è un numero primo$\}$

- $S_4 = \{i \mid i$ è un indice, minore di 100, di una macchina di Turing che stabilisce se il suo ingresso è un numero primo$\}$
- $S_5 = \{i \mid i$ è l'indice di una macchina di Turing con un numero dispari di stati$\}$
- $S_6 = \{i \mid i$ è un indice, minore di 100, di una macchina di Turing con un numero dispari di stati$\}$

SOLUZIONE

S_2, S_4 e S_6 sono ricorsivi in quanto finiti.

S_1 è ricorsivo perché la sua funzione caratteristica è computabile (deve stabilire se un numero è primo, il che è decidibile).

S_3 non è ricorsivo per il teorema di Rice (all'insieme $\mathcal{F} = \{f\}$, dove f è la funzione che stabilisce se il suo argomento è primo o meno, si applicano chiaramente le condizioni del teorema, dal momento che $\mathcal{F}$ non è né vuoto né l'insieme di tutte le funzioni computabili).

S_5 è ricorsivo. Basta infatti ispezionare il diagramma degli stati ricavato dall'enumerazione di Gödel per stabilire se il numero di stati è pari o dispari.

Si noti che avere un numero dispari di stati non è una proprietà della funzione calcolata dalla macchina, quindi non si può applicare il teorema di Rice. Si consideri, infatti, una macchina M con un numero dispari di stati. La macchina M', con esattamente gli stessi stati e le stesse transizioni di M ma con uno stato aggiuntivo q, non connesso agli altri stati, ha un numero pari di stati e calcola la stessa funzione di M. Poiché M (con la proprietà) e M' (senza la proprietà) calcolano la stessa funzione, non si tratta di una proprietà della funzione calcolata.

Esercizio 2.3. Si consideri l'insieme S definito nel modo seguente:

$S = \{i \mid i$ è l'indice di una macchina di Turing che accetta almeno una stringa$\}$

1. S è ricorsivo?
2. S è ricorsivamente enumerabile?
3. Il complemento di S è ricorsivamente enumerabile?

Si considerino ora gli insiemi seguenti:

- $S_1 = \{\, i \mid i$ è un numero naturale e non è l'indice di una macchina di Turing che accetta almeno una stringa $\}$
- $S_2 = \{\, i \mid i$ è l'indice di una macchina di Turing che non accetta alcuna stringa $\}$
- $S_3 = \{\, i \mid i$ è l'indice di una macchina di Turing in cui almeno una stringa non è accettata $\}$

4. Qualcuno degli insiemi S_1, S_2 e S_3 rappresenta il complemento di S?

SOLUZIONE

1. Ricadiamo chiaramente nelle ipotesi di applicazione del teorema di Rice: l'insieme di funzioni considerate non è né l'insieme vuoto (c'è almeno una macchina di Turing che accetta almeno una stringa), né l'insieme di tutte le funzioni computabili (non tutte le macchine di Turing accettano almeno una stringa), pertanto l'insieme S non è ricorsivo. Osserviamo inoltre che il problema di decisione associato a S è il complemento del problema della *emptiness* per le macchine di Turing (notoriamente indecidibile), ossia se una generica macchina di Turing riconosca il linguaggio vuoto.

2. Per stabilire che S è ricorsivamente enumerabile si può, ad esempio, mostrare che S è l'insieme di definizione di una funzione f (parziale e) computabile. Tale funzione f è implementata da una macchina di Turing che procede come segue. Per un dato indice i, simula il comportamento della i-esima macchina di Turing M_i; partendo da $n = 0$ e $m = 1$, verifica, per ogni stringa x di lunghezza n, se in m passi M_i accetta x, e procede enumerando (n, m) con la classica tecnica diagonale $(0,0), (0,1), (1,0), (0,2), (1,1), (2,0), \ldots$ fino a che non trova una stringa accettata da M_i, se esiste. Si noti che, fissato n, è sempre possibile enumerare tutte le stringhe di lunghezza n in tempo finito. Pertanto, se esiste una stringa di lunghezza n' accettata da M_i in un numero di passi m', questa viene prima o poi enumerata con il procedimento sopra illustrato; se non esiste, tale procedimento non termina.

3. Il complemento di S non può essere ricorsivamente enumerabile. Se lo fosse, sia S sia il suo complemento sarebbero ricorsivamente enumerabili (vedi punto 2), pertanto si potrebbe concludere che S è ricorsivo, ma al punto 1 abbiamo mostrato che non lo è.

4. - L'insieme S_1 è esplicitamente definito come il complemento $\mathbb{N} \setminus S$ di S, cioè l'insieme degli indici che non stanno in S.
   - L'insieme S_2 è definito come l'insieme degli indici che godono della proprietà opposta (l'opposto di accettare almeno una stringa è non accettare alcuna stringa) di quella goduta dagli indici che stanno in S, e quindi è ancora una volta il complemento di S.
   - L'insieme S_3 invece non è il complemento di S, infatti una stessa macchina di Turing può accettare una stringa (e quindi avere l'indice in S) e non accettarne un'altra (e quindi avere l'indice anche in S_3).

Esercizio 2.4. Si consideri il seguente insieme.

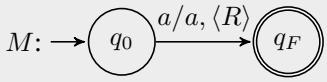
$S = \{i \mid$ La macchina di Turing con indice i termina la sua esecuzione in meno di 100 passi per qualche ingresso $\}$

S è ricorsivamente enumerabile? S è ricorsivo? Giustificare brevemente le risposte.

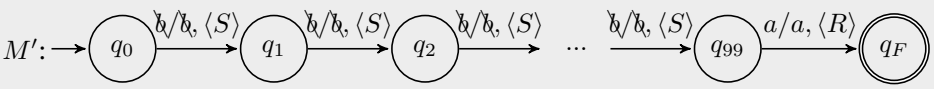
SOLUZIONE

S è ricorsivo (e quindi anche ricorsivamente enumerabile). Infatti basta verificare se la macchina si arresta entro 99 passi per qualche stringa di lunghezza < 100. Tali stringhe sono in numero finito, quindi la verifica avviene in un numero finito di passi. Non occorre considerare stringhe più lunghe perché, se anche ci fosse una stringa più lunga di 100 caratteri tale per cui la macchina si arresti in meno di 100 mosse, e quindi avendo letto un prefisso di lunghezza minore di 100, necessariamente vi sarebbe anche una stringa con lo stesso prefisso ma lunga meno di 100 che farebbe arrestare la macchina nello stesso numero di mosse.

Si noti che, nonostante S sia un insieme di indici di macchine di Turing, in questo caso non è applicabile il teorema di Rice, in quanto la proprietà delle macchine di Turing considerate per l'insieme S (terminare in meno di 100 passi per qualche ingresso) non è una proprietà della funzione calcolata, bensì una proprietà "strutturale" della macchina. Infatti esistono certamente due macchine diverse, una con la proprietà e una senza, che calcolano la stessa funzione. Ad esempio, una semplice macchina M (mostrata di seguito, a nastro singolo) che accetta tutte e sole le stringhe che iniziano con la a può farlo in una singola mossa (e quindi in meno di 100 passi), con una transizione dallo stato iniziale a uno stato finale a fronte della lettura del simbolo a.



Un'altra macchina M', mostrata sotto, può accettare però esattamente le stesse stringhe compiendo prima 99 mosse "inutili" (si pensi ad esempio a un diagramma degli stati con 101 stati e una catena lineare di 99 transizioni dallo stato iniziale al penultimo stato della catena in cui i nastri non vengono alterati e le testine non vengono spostate, con infine un'ultima transizione dal penultimo all'ultimo stato a fronte della lettura di una a). Poiché M accetta in meno di 100 passi mentre M' no e $L(M) = L(M')$ (cioè calcolano la stessa funzione), possiamo escludere che si tratti di una proprietà della funzione calcolata e quindi il teorema di Rice non è applicabile.



Esercizio 2.5. Con riferimento alla consueta notazione secondo cui f_i indica la funzione associata alla macchina di Turing (MT) con indice i, si considerino i seguenti insiemi:

- $S_1 = \{i \mid f_i$ è una funzione totale $\}$
- $S_2 = \{i \mid f_i$ è una funzione computabile $\}$
- $S_3 = \{i \mid i$ è l'indice di una MT che si arresta per qualunque ingresso $\}$
- $S_4 = \{i \mid f_i$ può essere calcolata da un automa a stati finiti $\}$
- $S_5 = \{i \mid i$ è l'indice di una MT in cui lo stato iniziale è anche finale $\}$

Per ciascuno degli insiemi S_1, S_2, S_3, S_4, S_5, dire se è ricorsivo.

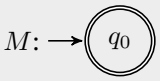
SOLUZIONE

Notiamo che gli insiemi S_1 e S_3 sono lo stesso insieme. Il problema di determinare se una funzione calcolata da una MT sia totale è notoriamente indecidibile (come si può determinare anche applicando direttamente il teorema di Rice), pertanto S_1 e S_3 non sono ricorsivi.

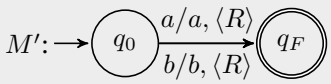
L'insieme S_2 è banalmente l'insieme di tutti i numeri naturali, poiché ogni funzione calcolata da una MT è ovviamente computabile (per definizione). Pertanto l'insieme S_2 è ricorsivo (come si nota applicando ad esempio il teorema di Rice oppure osservando che la sua funzione caratteristica è la funzione costante $c_{S_2}(x) = 1$, che è evidentemente computabile).

L'insieme S_4 non è ricorsivo (per il teorema di Rice) poiché l'insieme di funzioni calcolabili mediante un automa a stati finiti non è né l'insieme vuoto, né l'insieme di tutte le funzioni computabili.

Nel caso dell'insieme S_5, la proprietà considerata riguarda la struttura delle MT e non le funzioni da esse calcolate, pertanto il teorema di Rice non è direttamente applicabile. Per rendersene conto, basta considerare che esistono due diverse MT, una con la proprietà e una senza, che calcolano la stessa funzione. Ad esempio, la macchina M (mostrata sotto) con un solo stato (iniziale e finale) e alfabeto A riconosce il linguaggio A^*.



La macchina M' (mostrata di seguito, a nastro singolo, per l'alfabeto $A = \{a, b\}$) con due stati (q_0, iniziale, e q_F, finale), con una transizione da q_0 a q_F che avviene a fronte di qualunque simbolo trovato sul nastro, riconosce lo stesso linguaggio A^*.



Poiché M (con la proprietà) e M' (senza la proprietà) riconoscono lo stesso linguaggio (calcolano la stessa funzione), non si tratta di una proprietà della funzione calcolata.

L'insieme S_5 è comunque ricorsivo e una semplice procedura di decisione per S_5 è la seguente:

- mediante l'enumerazione di Gödel, generare la struttura della i-esima MT (cioè una stringa che rappresenta, secondo opportune convenzioni, il diagramma di stato della MT);
- data la stringa del punto precedente, mediante banale ispezione è immediato determinare se lo stato iniziale sia anche finale (le convenzioni di rappresentazione della stringa includono ovviamente anche l'indicazione di quali stati siano finali).

Esercizio 2.6. Sia $d : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ la biiezione tra $\mathbb{N}$ e $\mathbb{N} \times \mathbb{N}$ definita come segue:

$$d(x, y) = \frac{(x + y)(x + y + 1)}{2} + x$$

Si considerino i seguenti insiemi:

1. $S_1 = \{d(x, y) \mid f_y(x) \neq \perp\}$
2. $S_2 = \{y \mid f_y(0) \neq \perp\}$
3. $S_3(k) = \{x \mid f_k(x) \neq \perp\}$, dove k è un parametro.

Per ciascuno dei predetti insiemi, si dica se esso è ricorsivo, motivando brevemente la risposta.

SOLUZIONE

1. S_1 non è ricorsivo. Infatti la funzione d stabilisce banalmente una biiezione tra $\mathbb{N}$ e $\mathbb{N} \times \mathbb{N}$, e se si potesse stabilire l'insieme delle coppie di numeri (x, y) tali per cui $f_y(x) \neq \perp$ allora il problema dell'arresto risulterebbe decidibile, il che è assurdo.
2. S_2 non è ricorsivo. Sia $\mathcal{F}$ l'insieme di funzioni calcolabili definite nel punto 0; tale insieme non è né l'insieme di tutte le funzioni computabili (esiste, ad esempio, la funzione computabile indefinita ovunque), né l'insieme vuoto (esiste, ad esempio, la funzione calcolabile $f(x) = 1$, definita ovunque). L'insieme S_2 di indici delle macchine di Turing che calcolino funzioni in $\mathcal{F}$ non è pertanto ricorsivo per il teorema di Rice.
3. La ricorsività di $S_3(k)$ dipende da k. Per alcuni valori di k si sa esattamente per quali valori di ingresso x la funzione $f_k(x)$ è definita. Ad esempio, se $f_k(x)$ fosse la funzione x^2, si saprebbe che è definita sempre e quindi il problema sarebbe decidibile (e deciso). Per altri valori

di k, invece, il problema è indecidibile. Se, ad esempio, $f_k(x)$ fosse la funzione $f_x(0)$ (che è certamente calcolabile e quindi necessariamente esiste un indice k tale che $f_k(x) = f_x(0)$), non se ne potrebbe stabilire l'insieme di definizione, come ad esempio osservato al punto 2.

Esercizio 2.7. Si risponda ai seguenti quesiti, giustificando brevemente ma con chiarezza la risposta:

1. Si consideri l'insieme $S = \{y \mid \forall x(f_y(x) \neq \perp \rightarrow f_y(x) = x^2)\}$. S è ricorsivo? S è ricorsivamente enumerabile?
2. Si consideri la funzione seguente:
$$g(x,y) = \begin{cases} 1, \text{ se } \forall z(f_x(z) \neq \perp \wedge f_y(z) \neq \perp \rightarrow f_x(z) = f_y(z)), \\ 0, \text{ altrimenti.} \end{cases}$$
g è computabile?
3. Si consideri ora la funzione g', variazione della precedente g:
$$g'(x,y,z) = \begin{cases} 1, \text{ se } f_x(z) \neq \perp \wedge f_y(z) \neq \perp \wedge f_x(z) = f_y(z), \\ \perp, \text{ altrimenti.} \end{cases}$$
g' è computabile?

SOLUZIONE

1. S non è ricorsivo: ciò è una diretta conseguenza del teorema di Rice. Infatti, la proprietà di restituire il quadrato di x quando l'esecuzione termina sull'ingresso x è evidentemente una proprietà non banale della funzione calcolata, ed esiste almeno una funzione computabile con questa proprietà (ad esempio $f(x) = x^2$), ma non tutte le funzioni computabili ce l'hanno (ad esempio $f'(x) = x$).

   D'altro canto il complemento di S,
$$\overline{S} = \{y \mid \exists x(f_y(x) \neq \perp \wedge f_y(x) \neq x^2)\}$$
   è ricorsivamente enumerabile: i suoi elementi possono essere enumerati mediante una classica tecnica "diagonale" (su 3 dimensioni) che consideri nell'ordine tutte le funzioni computabili, per tutti i valori dei dati di ingresso e per numero crescente di passi di computazione individuando quindi i valori di y per i quali la condizione $f_y(x) \neq x^2$ sussiste. Di conseguenza S non è ricorsivamente enumerabile altrimenti S e $\overline{S}$ sarebbero entrambi ricorsivi.
2. Sia K il numero di Gödel di una macchina di Turing che calcola la funzione $f(x) = x^2$: risulta allora $g(x,K) = p(x)$, dove p è la funzione

caratteristica dell'insieme S di cui al punto 1; quindi $g(x, K)$ non è computabile, ed a maggior ragione non lo è $g(x, y)$.

3. g' è computabile: una macchina di Turing può calcolare g' simulando il calcolo di $f_x(z)$ e $f_y(z)$ e, se entrambe le computazioni terminano, verifica se $f_x(z) = f_y(z)$, altrimenti continua con la sua simulazione indefinitamente, il che è compatibile con la definizione di g'.

Esercizio 2.8. Sia A un insieme semidecidibile e B un insieme decidibile. È facile verificare che l'insieme $C = A \setminus B$ è un insieme semidecidibile: si fornisca all'uopo una funzione calcolabile (eventualmente anche parziale) che abbia come immagine C.

SOLUZIONE

Ricordiamo che il fatto che A sia semidecidibile significa che esso ammette una funzione totale e computabile g_A "generatrice" degli elementi di A, ossia $A = \{y \mid y = g_A(x), x \in \mathbb{N}\}$. Inoltre, poiché B è decidibile, la sua funzione caratteristica $c_B(x) =$ if $x \in B$ then 1 else 0 è computabile (e totale). Dal momento che gli elementi di C sono presenti in A ma non in B, una funzione g_C che "generi" tutti gli elementi di C dovrà generare tutti gli elementi di A (come g_A) tranne quelli presenti in B (potendo verificare l'appartenenza mediante c_B). Detto x un numero naturale, il corrispondente elemento di A generato da g_A è $g_A(x)$ e se ne verifica l'appartenenza a B verificando il valore di $c_B(g_A(x))$. La funzione richiesta può allora essere definita come segue: $g_C(x) =$ if $c_B(g_A(x)) = 0$ then $g_A(x)$ else $\perp$.

Esercizio 2.9. Si dica, motivando adeguatamente la risposta, se il seguente insieme T è ricorsivo:

$$T = \{i \mid \text{la macchina } M_i \text{ possiede } i \text{ stati senza transizioni uscenti}\}.$$

SOLUZIONE

T è ricorsivo: basta ricostruire il diagramma di stato di M_i tramite l'enumerazione di Gödel $\mathcal{E}$, e contare il numero di stati con la caratteristica richiesta.

Esercizio 2.10.

1. Siano dati due automi a pila deterministici con linguaggi L_1 e L_2: è ricorsivo il linguaggio $L_1 \cap L_2$?

2. Siano dati due automi a pila deterministici con linguaggi L_1 e L_2 e una macchina di Turing M: è decidibile se M calcola $L_1 \cap L_2$?

SOLUZIONE

1. Sì. Un insieme è ricorsivo se la sua funzione caratteristica è computabile. Occorre quindi che sia computabile una funzione che, data una generica stringa x, restituisca 1 o 0 a seconda che x appartenga o meno al linguaggio $L_1 \cap L_2$. Questo test di appartenenza si può effettuare algoritmicamente, verificando singolarmente l'appartenenza di x all'uno e all'altro automa. Ciascuno di questi test richiede infatti tempo finito, assumendo che ciascun automa a pila sia aciclico; se non fosse aciclico, lo si potrebbe comunque trasformare in un equivalente automa aciclico.

   Osserviamo inoltre che la non chiusura del formalismo degli automi a pila deterministici rispetto all'intersezione non ha alcuna ripercussione sulla ricorsività del linguaggio $L_1 \cap L_2$.

2. No, per il teorema di Rice: si tratta del consueto problema di determinare la *correttezza* di una generica macchina M. In particolare, la proprietà di riconoscere $L_1 \cap L_2$ è una proprietà non banale della funzione calcolata (linguaggio riconosciuto) dalla macchina M (esiste certamente una macchina che riconosce $L_1 \cap L_2$ per la ricorsività mostrata al punto precedente, così come ne esiste certamente una che riconosce un linguaggio diverso).

   Ovviamente la domanda non è chiusa (la macchina M è generica, anche se il testo del problema non riporta esplicitamente l'aggettivo "generico").

Esercizio 2.11. Sia L_1 un linguaggio ricorsivamente enumerabile, L_2 un linguaggio regolare e $L = L_1 \cap L_2$ la loro intersezione. Per ciascuna delle seguenti classi di linguaggi, si argomenti se L può farne parte, deve necessariamente farne parte o non può farne parte:

1. linguaggi regolari;
2. linguaggi ricorsivi;
3. linguaggi ricorsivamente enumerabili.

Motivare opportunamente e fornire esempi laddove appropriato.

SOLUZIONE

1. Può farne parte: se L_1 è regolare allora anche L è regolare (la famiglia dei linguaggi regolari è chiusa rispetto all'intersezione); ad esempio, se $L_1 = \emptyset$ abbiamo $L = \emptyset$. Può non farne parte: se L_1 non è regolare e $L_2 = A^*$, dove A è l'alfabeto, allora L non è regolare.

2. In modo simile al punto precedente, se L_1 è ricorsivo allora anche L è ricorsivo (la famiglia dei linguaggi ricorsivi è chiusa rispetto all'inter-

sezione), altrimenti potrebbe non esserlo. La ragione è che L_2, essendo regolare, è anche ricorsivo.

3. L è certamente ricorsivamente enumerabile in quanto intersezione di due linguaggi ricorsivamente enumerabili (la famiglia dei linguaggi ricorsivamente enumerabili è chiusa rispetto all'intersezione), dato che L_2, essendo regolare, è anche ricorsivamente enumerabile.

Esercizio 2.12. Per entrambi i punti seguenti si assuma che L sia un linguaggio infinito costruito su un alfabeto A.

1. Sia L un linguaggio ricorsivo. Esiste necessariamente una enumerazione algoritmica $\mathcal{E}$ che enumera tutte le stringhe di L in ordine lessicografico? Motivare brevemente la risposta.

2. Sia $\mathcal{E}$ una enumerazione algoritmica che enumera tutte le stringhe di un linguaggio L in ordine lessicografico. L è necessariamente ricorsivo? Motivare brevemente la risposta.

SOLUZIONE

1. L è ricorsivo, quindi la sua funzione caratteristica c_L è computabile. Si noti che il concetto di funzione caratteristica è applicabile non solo ad insiemi di numeri naturali ma anche a insiemi di stringhe (che possono peraltro essere agevolmente codificate come numeri naturali). In altre parole, la calcolabilità di c_L implica che esiste una macchina di Turing M che termina sempre e scrive 1 in uscita se la stringa in ingresso appartiene a L, 0 altrimenti. L'enumerazione algoritmica $\mathcal{E}$ richiesta si ottiene come segue: basta enumerare tutte le stringhe di A^* in ordine lessicografico (cosa sempre possibile) e, per ciascuna di esse, testarne, in tempo finito, l'appartenenza ad L tramite M (che è disponibile e termina sempre per ipotesi, essendo L ricorsivo), stampandola in uscita solo se appartiene a L.

2. L è enumerabile algoritmicamente mediante $\mathcal{E}$ in ordine lessicografico. Si può quindi costruire una macchina M che calcola la funzione caratteristica c_L di L come segue (che quindi risulta calcolabile): quando riceve una stringa x in ingresso, M esegue l'enumerazione $\mathcal{E}$ e legge la lista di tutte le stringhe di L in ordine lessicografico fino a che non compare la prima stringa che lessicograficamente stia dopo x (tale stringa deve necessariamente esistere, perché L è infinito). Se x è apparso nell'enumerazione, allora M stampa 1 in uscita, altrimenti stampa 0. Poiché abbiamo esibito una macchina di Turing (M) che calcola c_L, allora c_L è calcolabile e quindi L è ricorsivo.

Esercizio 2.13. Siano $L_1, L_2, \ldots, L_k$, $k > 1$ linguaggi definiti su un alfabeto Σ, tali che valgano le seguenti proprietà:

- $\forall i, j, i \neq j : L_i \cap L_j = \emptyset$,
- $L_1 \cup L_2 \cup \ldots \cup L_k = \Sigma^*$,
- $\forall i, L_i$ è semidecidibile.

Si dica, giustificando brevemente la risposta, se le seguenti affermazioni sono vere o false:

1. Tutti i linguaggi $L_1, L_2, \ldots, L_k$, sono ricorsivi.
2. È possibile che alcuni di essi siano regolari.
3. Tutti i linguaggi $L_1, L_2, \ldots, L_k$ sono necessariamente regolari.

SOLUZIONE

1. Poiché i linguaggi sono tra loro disgiunti e la loro unione è l'intero Σ^*, enumerando ordinatamente (grazie alla semidecidibilità, cioè ricorsiva enumerabilità di ciascun linguaggio) tutte stringhe di $L_1, L_2, \ldots, L_k$ in modalità "round robin" (ossia, una stringa di L_1, poi una di L_2, ..., una di L_k e daccapo) prima o poi una qualsiasi stringa x deve comparire in una e una sola delle sequenze L_i, e quindi si può decidere a quale linguaggio appartiene. Potendo quindi concludere per ciascuno dei linguaggi $L_1, L_2, \ldots, L_k$ se una generica stringa x appartiene (grazie alla semidecidibilità) o non appartiene (grazie alla verifica di appartenenza ad un altro linguaggio tra quelli dati) ad esso, tutti questi linguaggi sono ricorsivi.
2. È certamente possibile che qualcuno degli L_i (o anche tutti) sia regolare. Ad esempio, i linguaggi $L_1 = \Sigma^*$ e $L_2 = \emptyset$ soddisfano le condizioni e sono regolari.
3. Non è necessario. Ad esempio, i linguaggi $L_1 = \{a^n b^n \mid n \geq 0\}$ e $L_2 = \Sigma^* \setminus L_1$ soddisfano le condizioni ma non sono regolari.

2.2 Computabilità di funzioni

Esercizio 2.14. Si denoti, come di consueto, con f_y la funzione calcolata dalla macchina di Turing con indice y. Per ciascuna delle seguenti funzioni si dica se è computabile, motivando opportunamente la risposta:

1. $g_1(y) = \begin{cases} 1 \text{ se } f_{10}(10) > 10 \\ 0 \text{ altrimenti} \end{cases}$

2. $g_2(y) = \begin{cases} 1 \text{ se } f_{10}(10) > 10 \\ \perp \text{ altrimenti} \end{cases}$

3. $g_3(y) = \begin{cases} 1 \text{ se } f_y(10) > 10 \\ 0 \text{ altrimenti} \end{cases}$

4. $g_4(y) = \begin{cases} 1 \text{ se } f_y(10) > 10 \\ \perp \text{ altrimenti} \end{cases}$

5. $g_5(y, x) = \begin{cases} 1 \text{ se } f_y(10) > x \\ 0 \text{ altrimenti} \end{cases}$

6. $g_6(y, x) = \begin{cases} 1 \text{ se } f_y(10) > x \\ \perp \text{ altrimenti} \end{cases}$

SOLUZIONE

1. Sì: la condizione $f_{10}(10) > 10$ è una domanda chiusa. Non avrebbe senso, invece, dire che è computabile perché basta vedere il risultato di $f_{10}(10)$ e, a seconda di quello, restituire 1 o 0: il calcolo di $f_{10}(10)$ potrebbe infatti non terminare.

2. Sì: come sopra.

3. No per il teorema Rice: l'insieme di funzioni $\mathcal{F} = \{f_y \mid f_y(10) > 10\}$ non è né l'insieme vuoto, né l'insieme di tutte le funzioni computabili. Ad esempio, la funzione costante $f(x) = 11$ appartiene ad $\mathcal{F}$, mentre la funzione costante $g(x) = 9$ no. Quindi, per il teorema di Rice, l'insieme degli indici delle macchine di Turing che calcolano funzioni in $\mathcal{F}$ non è ricorsivo; poiché g_3 è la funzione caratteristica di tale insieme, g_3 non è calcolabile.

   Osserviamo che dire che, se la macchina di Turing non termina, non ce ne si accorge e quindi la funzione g_3 non è computabile è solo un'intuizione (non sufficiente), ma non una prova della non computabilità!

4. Sì: basta mettere in esecuzione la macchina y-esima con l'ingresso 10 (recuperandone il suo diagramma degli stati con l'enumerazione di Gödel). Se termina, ce ne si accorge in tempo finito e si può confrontare l'esito con 10 (restituendo 1 se l'esito è maggiore ed entrando in un ciclo infinito negli altri casi); se non termina, questo è compatibile con la definizione della funzione g_4.

   Qui la macchina da mettere in esecuzione è una sola e con uno specifico ingresso, quindi non c'è nessun bisogno di enumerazione diagonale.

5. No: il problema di stabilire se $f_y(10) > x$ è una generalizzazione di quello del caso della funzione g_3, quindi se sapessimo risolverlo, sapremmo valutare anche il test della funzione g_3, che però non è calcolabile.

   Qui il teorema di Rice non è utilizzabile direttamente: non abbiamo un insieme di funzioni, come richiede il teorema, bensì un insieme di insiemi di funzioni (uno per ogni valore della x). Quindi basta mostrare che questa è una generalizzazione del punto 3, già non computabile. Ovviamente il concetto di riduzione va utilizzato nella direzione giusta, ossia dal problema non computabile (3) a quello studiato (5).

6. Sì: in modo analogo al caso della funzione g_4.

   Ovviamente la riduzione da 4 a 6 non è utile, perché così staremmo solo mostrando che 6 è una generalizzazione di un problema computabile.

Esercizio 2.15. Si consideri una macchina di Turing universale M fissata, a nastro singolo. Sia A l'alfabeto del nastro di M.

1. È calcolabile la funzione $f : \begin{cases} 1 & \text{se } M(s) \neq \bot \\ 0 & \text{se } M(s) = \bot \end{cases}$ che riceve in ingresso una qualunque stringa $s \in \bigcup_{i=0}^{100} A^i$?
2. È calcolabile la funzione $g : \begin{cases} 1 & \text{se } M(s) \neq \bot \\ \bot & \text{se } M(s) = \bot \end{cases}$ che riceve in ingresso una qualunque stringa $s \in \bigcup_{i=2k,k\in\mathbb{N}} A^i$?

SOLUZIONE

1. Sì. Il dominio della funzione da calcolare è finito, quindi essa è rappresentabile da una tabella di dimensioni finite, contenente la corrispondenza ingresso-uscita tra la stringa s ed il comportamento della macchina M quando s viene data in ingresso. In altre parole, consideriamo quei programmi composti solo da istruzioni `if`, una per ognuna delle n stringhe in $\bigcup_{i=0}^{100} A^i$ (dove n è un numero finito e quindi anche il programma ha lunghezza finita); in ciascuna `if` si confronta la stringa in ingresso a una delle stringhe in $\bigcup_{i=0}^{100} A^i$ e si restituisce 1 o 0 in caso di uguaglianza. I programmi fatti in questo modo sono 2^n, perché, per ognuna delle n stringhe in $\bigcup_{i=0}^{100} A^i$, ciascun programma può compiere due scelte diverse (restituire 1 o 0). Ciascuno di questi programmi è banalmente computabile e, anche se non sappiamo quale, uno di questi codifica in maniera corretta la funzione f, che è quindi calcolabile. A scopo illustrativo, qui di seguito sono riportate le rappresentazioni sintetiche delle n possibili implementazioni di $f(x)$, tutte

computabili, una delle quali è quella corretta (per semplicità, con alfabeto $A = \{a, b\}$). Il valore di n non è particolarmente importante, una volta che si è osservato che è finito; in questo caso indica il numero di stringhe di lunghezza non superiore a 100, e quindi vale $n = \sum_{i=0}^{100} |A|^i$.

Programma #1	Programma #2	...	Programma #n
`if` $x = \varepsilon$ `return` 0	`if` $x = \varepsilon$ `return` 1	...	`if` $x = \varepsilon$ `return` 1
`if` $x = a$ `return` 0	`if` $x = a$ `return` 0	...	`if` $x = a$ `return` 1
`if` $x = b$ `return` 0	`if` $x = b$ `return` 0	...	`if` $x = b$ `return` 1
`if` $x = aa$ `return` 0	`if` $x = aa$ `return` 0	...	`if` $x = aa$ `return` 1
`if` $x = ab$ `return` 0	`if` $x = ab$ `return` 0	...	`if` $x = ab$ `return` 1
$\vdots$	$\vdots$	$\ddots$	$\vdots$
`if` $x = b^{100}$ `return` 0	`if` $x = b^{100}$ `return` 0	...	`if` $x = b^{100}$ `return` 1

2. Sì. Il dominio della funzione in questo caso è infinito, ma g può essere calcolata emulando $M(s)$ sino alla sua eventuale terminazione. Se $M(s)$ termina, allora g restituisce 1; se non termina, allora nemmeno l'emulazione termina, risultando comunque conforme al comportamento richiesto.

Esercizio 2.16. Si consideri la seguente funzione e si dica, motivando adeguatamente la risposta, se essa sia calcolabile:

$$w(x, y) = \begin{cases} f_y(x) & \text{se } f_y(y) = f_x(x) \neq \perp \\ \perp & \text{altrimenti} \end{cases}$$

SOLUZIONE

La funzione è calcolabile: una macchina di Turing che la calcola non deve fare altro che calcolare $f_y(y)$ e $f_x(x)$: se entrambi terminano, controlla se sono uguali e, se lo sono, ne restituisce il valore, altrimenti entra in un ciclo infinito e non termina. Se uno dei due non termina, neanche la macchina di Turing terminerà, conformemente con il comportamento richiesto.

Esercizio 2.17. Si consideri la funzione $f(x, y) = f_z(x)$, dove $z = f_y(y)$. La funzione f è computabile? Motivare brevemente la risposta.

SOLUZIONE

$f(x, y)$ può essere calcolata mediante il seguente algoritmo:

1. Calcolo $f_y(y)$.
   (a) Se il calcolo non termina (proprietà indecidibile), allora $f(x, y)$ non è definita per alcuna x.

(b) Se il calcolo di $f_y(y)$ termina, ottengo il risultato z e inizio il calcolo di $f_z(x)$.
   i. Se quest'ultimo calcolo non termina, allora $f(x, y)$ non è definita per la coppia $\langle y, x\rangle$.
   ii. Se termina, ottengo invece il risultato desiderato.

Tutti i passaggi sono calcolabili (corrispondono a un "calcolo" che si può effettuare con una macchina di Turing), anche se non terminano necessariamente. Pertanto $f(x, y)$ è calcolabile, anche se non totale.

Esercizio 2.18. Si definisce il numero reale t nel modo seguente:

- La parte intera di t è 0
- L'espansione decimale di t è tale che la i-esima cifra dopo la virgola vale:
  - 0 se la funzione f_i non è computabile;
  - altrimenti 1, se f_i non è totale;
  - altrimenti 2, se f_i non è la funzione che calcola il quadrato del suo ingresso;
  - altrimenti 3.

Si indichi con la notazione $t[n]$ la sequenza delle prime n cifre nell'espansione decimale di t. Si considerino quindi le funzioni g e h definite come segue:

- $g(x) = t[100]$ se $x = 0$, altrimenti $\perp$;
- $h(x) = t[x]$.

Sono calcolabili le funzioni g e h? Motivare opportunamente la risposta.

SOLUZIONE

Notiamo intanto che ogni funzione f_i è computabile (per definizione!), quindi non ci sono 0 nell'espansione decimale di t.

La funzione g è certamente calcolabile, in quanto la condizione $x = 0$ è decidibile e $t[100]$ è una costante (numero di 100 cifre comprese tra 1 e 3). Per ognuna delle 3^{100} possibili sequenze di 100 cifre comprese tra 1 e 3, è senz'altro possibile costruire una macchina di Turing che restituisca tale sequenza quando $x = 0$ (e che non si arresti altrimenti). Quindi, anche se non sappiamo necessariamente quale sequenza corrisponda a $t[100]$, esiste comunque una macchina di Turing che effettua il calcolo desiderato.

La funzione h non è invece calcolabile. Se lo fosse, sarebbe possibile decidere il problema della totalità di una funzione, che è notoriamente indecidibile. Per decidere se una generica funzione f_i sia totale, basterebbe infatti calcolare $h(i)$ e osservare se l'ultima cifra del risultato sia 1 (nel qual caso la funzione non è totale) o maggiore di 1 (nel qual caso la funzione è totale).

Esercizio 2.19. Si consideri la funzione $g(y) =$ if $f_y(y) = y$ then $y+1$ else $f_y(y)$. Si dica se g è totale e se è calcolabile, motivando adeguatamente la risposta.

SOLUZIONE

La funzione g non è totale. Si consideri ad esempio il caso in cui y è l'indice di una macchina di Turing che calcola la funzione ovunque indefinita (ossia $\forall x(f_y(x) = \perp)$). Non varrà quindi la condizione $f_y(y) = y$, poiché $f_y(x) = \perp$, e ci troviamo allora nel ramo else della funzione, il cui valore è $f_y(y)$ (cioè $\perp$), e quindi g è indefinita per questo specifico valore di y e pertanto non è totale. Inoltre, g è calcolabile, poiché le varie componenti della definizione della g sono calcolabili (cioè eseguibili da una macchina di Turing): la f_y è calcolabile per definizione e l'uguaglianza e l'incremento di 1 sono pure banalmente calcolabili. Operativamente, il calcolo può avvenire simulando la y-esima macchina di Turing con y in ingresso. Se essa termina, si controlla il risultato t e, se esso è uguale a y, si somma 1; altrimenti va bene t, anche se non dovesse essere definito.

Esercizio 2.20. Sia m una funzione binaria che, dato un indice di macchina di Turing (MT) y e un ingresso x, restituisce il numero di mosse che impiega tale MT a terminare l'esecuzione su tale ingresso; m è indefinita qualora la macchina non termini.

1. La funzione m è calcolabile?
2. È decidibile il problema di stabilire se, data una generica MT con indice y, esista un ingresso x tale per cui
   (a) $m(y, x) = 0$?
   (b) $m(y, x) \leq y$?
   (c) $m(y, x) \neq \perp$?

Spiegare brevemente ma esaurientemente e con chiarezza le risposte.

SOLUZIONE

1. Chiaramente m è calcolabile: è possibile infatti implementare un programma che emula l'esecuzione della y-esima MT contando il numero di passi eseguiti. A fine esecuzione viene restituito tale numero. Se invece l'esecuzione della macchina emulata non termina, allora neanche il programma termina, compatibilmente con la definizione di m.
2. (a) Decidibile, in quanto l'unico modo per terminare in 0 mosse è che lo stato iniziale della MT sia anche finale. Questo si può vedere semplicemente per ispezione della funzione di transizione della MT, che è ricostruibile mediante l'enumerazione di Gödel. Osserviamo che non è invece applicabile il teorema di Rice in questo ca-

so. Infatti, il linguaggio riconosciuto da una macchina con questa proprietà è il linguaggio A^*, con A alfabeto della macchina. Tuttavia è semplice costruire una macchina senza questa proprietà (cioè con uno stato iniziale che non è anche finale) che riconosca lo stesso linguaggio (basta includere una transizione per ogni simbolo di A dallo stato iniziale a uno stato finale). Quindi, poiché esistono due macchine, una con la proprietà e una senza, che calcolano la stessa funzione (riconoscono lo stesso linguaggio), possiamo concludere che non si tratti di una proprietà della funzione calcolata dalla MT e pertanto il teorema di Rice non è applicabile.

(b) Decidibile. Benché generico, y è dato e sappiamo che in al più y passi la MT può leggere al più y caratteri dell'ingresso. Il numero di stringhe diverse di al più y caratteri è finito; per ciascuna di esse basta verificare se la MT si arresta entro y passi.

(c) Indecidibile. Il quesito corrisponde a chiedere se una generica MT calcoli una funzione definita in almeno un punto. L'insieme di funzioni calcolabili con questa proprietà chiaramente non è vuoto (ad esempio la funzione $f(x) = 1$ è calcolabile e definita ovunque) e non è l'insieme di tutte le funzioni computabili (la funzione $g(x) = \perp$ è indefinita ovunque e calcolabile da una MT senza stati finali). Sussistono allora le condizioni per applicare il teorema di Rice, da cui consegue che l'insieme di indici di MT con la proprietà desiderata non è ricorsivo e pertanto il problema è indecidibile.

Esercizio 2.21. Sia m il numero di Gödel di una macchina di Turing che calcola la funzione $f(x)$. Si dica, motivando la risposta, se le seguenti funzioni sono calcolabili:

1. $g_1(x) = f(m)$;
2. $g_2(x) = f(x)$;
3. $g_3(x) = f(x) - 9$ se $f(x) > 10$, $\perp$ altrimenti;
4. $g_4(x) = f_x(x)$ se $x = m$, 1 altrimenti.

SOLUZIONE

Sono tutte funzioni computabili, infatti:

1. g_1 è costante, in quanto $f(m)$ è uno specifico valore (eventualmente $\perp$);
2. g_2 è computabile per definizione, in quanto la funzione f è calcolata dalla macchina di Turing con indice m, come da specifica;
3. per calcolare g_3 si inizia a calcolare $f(x)$ usando la macchina con indice m con ingresso x. In caso di terminazione, si verifica che il risultato sia

maggiore di 10 e se ne sottrae 9; se il risultato non è maggiore di 10, basta entrare in un loop infinito affinché $g_3(x) = \perp$; se invece il calcolo non termina, il valore della g_3 è $\perp$, compatibilmente con la definizione;

4. si confronta x con m: se sono uguali, si restituisce $f_x(x)$, ossia $f_m(m)$, cioè $f(m)$, che, come abbiamo visto, è calcolabile (e costante); se non lo sono, si restituisce 1 (valore costante, quindi certamente calcolabile).

Esercizio 2.22. Si consideri il seguente programma P codificato in uno pseudo-linguaggio ispirato al C.

```
P: {
    int x, y, z;
    read (x, y, z);
    while (x != y) {
        x = x - y;
        z = z + y;
    }
    write z;
}
```

Sia $f(x, y, z)$ la funzione calcolata da P. Si dica, giustificando brevemente la risposta, se la seguente funzione:

$$g(x, y, z) = \begin{cases} 1 \text{ se } f(x, y, z) \neq \perp \\ 0 \text{ altrimenti} \end{cases}$$

è calcolabile o no.

SOLUZIONE

L'esecuzione di P termina se e solo se i dati forniti in ingresso sono tali che x e y siano dello stesso segno, x sia $\geq y$ se entrambi positivi o viceversa se entrambi negativi, e sia, in modulo, un suo multiplo; più precisamente:

$$g(x, y, z) = \begin{cases} 1 \text{ se } x = y \vee ((x > y > 0 \vee x < y < 0) \wedge \exists k(k > 1 \wedge |x| = k \cdot |y|)) \\ 0 \text{ altrimenti} \end{cases}$$

ed è evidentemente calcolabile, dal momento che tutte le condizioni della formula qui sopra possono essere verificate algoritmicamente.

Esercizio 2.23. Una macchina di Turing si dice n-generatrice se, quando riceve 0 in ingresso, termina entro $n + 1$ passi scrivendo il numero n in uscita ($n \geq 0$).

Fissata una qualsiasi enumerazione algoritmica $\mathcal{E}$ delle macchine di Turing, il *generatore minimo* di un numero n, indicato con $gm(n)$, è il più piccolo indice, secondo $\mathcal{E}$, di macchina di Turing n-generatrice. Si risponda alle seguenti domande fornendo brevi ma chiare spiegazioni per le risposte date:

1. La funzione gm è totale?
2. La funzione gm è computabile?

SOLUZIONE

1. La funzione è certamente totale, in quanto, per qualunque numero n, si può facilmente costruire una macchina di Turing n-generatrice. Basta, infatti, compiere un passo di stampa per ogni cifra di n (per un totale di un passo se $n = 0$ e di $\lfloor \log_2 n \rfloor + 1$ passi se $n > 0$, quindi sempre entro $n + 1$ passi). Tale macchina potrebbe non essere la n-generatrice con l'indice più piccolo, ma la sua esistenza implica che la funzione gm è definita nel punto n.

2. Sì. Infatti, per quanto osservato al punto 1, si può facilmente trovare un limite superiore al valore di $gm(n)$. A tale scopo, sia i l'indice della macchina esibita al punto 1. Basta a questo punto, mediante una macchina di Turing universale, testare, in successione, ciascuna delle macchine di Turing con indice da 0 a i per $n + 1$ passi a fronte dell'ingresso 0: il primo indice per il quale la macchina termina e produce n in uscita è il valore $gm(n)$ cercato.

Esercizio 2.24. Si consideri l'insieme di macchine di Turing (MT) a nastro singolo con n stati definite su un alfabeto prefissato. Come noto, c'è un numero finito di tali MT a n stati. Iniziando l'esecuzione con il nastro con tutti blank, alcune di queste MT terminano, mentre altre non terminano.

1. Dire se è decidibile il problema di stabilire se, data una generica MT dell'insieme considerato, questa termina partendo con il nastro con tutti blank.

Si definisce $S(n)$ il numero di passi eseguiti dalla MT che termina più tardi, tra quelle a n stati che terminano (iniziando l'esecuzione con il nastro con tutti blank). Tale numero, noto anche come l'n-esimo numero "alacre castoro", è definito in modo univoco e, intuitivamente, rappresenta la durata dell'esecuzione della MT più "indaffarata" tra quelle che terminano. La funzione $S(n)$ cresce molto velocemente al crescere di n.

2. Dimostrare che la funzione $S(n)$ non è computabile.

SOLUZIONE

1. Il problema di stabilire se, data una generica MT, questa termina partendo con il nastro con tutti blank è indecidibile. Questo si può vedere ad esempio usando il teorema di Rice. Infatti, l'insieme delle MT che terminano come descritto non è né l'insieme vuoto (ad esempio, la macchina con un solo stato, iniziale e finale, termina se il nastro ha tutti blank), né l'insieme di tutte le funzioni calcolabili (ad esempio, la macchina con un solo stato, iniziale e non finale, non termina mai).

2. Per dimostrare che la funzione $S(n)$ non è computabile procediamo per assurdo. Supponiamo di essere in grado di calcolare $S(n)$ per ogni n. Allora possiamo decidere se una qualunque MT con n stati termina iniziando la sua esecuzione con il nastro d'ingresso con tutti blank. Infatti basta mettere in esecuzione la macchina: se si ferma entro $S(n)$ passi, sappiamo che termina; se non si ferma entro $S(n)$ passi, allora sappiamo che non terminerà mai, poiché $S(n)$ è il numero massimo di passi eseguibili da una MT che termina. Ma stabilire se una MT termina iniziando la sua esecuzione con il nastro d'ingresso vuoto è indecidibile, come dimostrato in precedenza. Assurdo.

2.3 Decidibilità e semidecidibilità

Esercizio 2.25. Uno stato q di una macchina di Turing M viene detto *utile* se esiste una stringa w tale per cui q viene raggiunto durante l'esecuzione di M quando w si trova in ingresso, ovvero, detto q_0 lo stato iniziale di M, q è utile se

$$\exists w, x, y, \alpha_1, \ldots, \alpha_k, \beta_1, \ldots, \beta_k \ \langle q_0, \uparrow w, \uparrow Z_0, \ldots, \uparrow Z_0 \rangle \vdash_M^* \langle q, x \uparrow y, \alpha_1 \uparrow \beta_1, \ldots, \alpha_k \uparrow \beta_k \rangle,$$

con $x \cdot y = w$, dove $\uparrow$ indica la posizione delle testine nei nastri.

1. È decidibile stabilire, data una macchina di Turing M e un suo stato q, se q sia utile?
2. È semidecidibile il problema di cui sopra?

SOLUZIONE

1. Il problema è indecidibile. Se fosse decidibile potremmo infatti decidere anche il problema della *emptiness* (cioè il problema di stabilire se il linguaggio riconosciuto è vuoto) per una generica macchina M; la emptiness è notoriamente indecidibile (se ne può ricavare immediatamente l'indecidibilità mediante il teorema di Rice). Infatti basterebbe decidere se lo stato finale di M è utile (o almeno uno degli stati finali di M, nel caso ce ne sia più di uno) per stabilire se M accetta almeno una stringa o meno.
2. Il problema è semidecidibile, poiché, mediante una enumerazione diagonale di tutte le possibili coppie $\langle w, i \rangle$ (dove w è la stringa in ingresso e i è il numero di passi di esecuzione di M testati, ad esempio, mediante emulazione), ci si può accorgere se, prima o poi, lo stato q venga raggiunto.

Esercizio 2.26. Nel corso di *Prova finale di Algoritmi* si richiede di sviluppare in C una funzione f da $\mathbb{N}$ a $\mathbb{N}$. Un generatore di test, fornito agli studenti, enumera gli ingressi per f e le corrispondenti uscite attese.

1. È decidibile il problema di stabilire se la codifica di f fatta da un generico studente è corretta rispetto ad ogni possibile caso di test fornito dal generatore?

2. Ada e Pasquale si sono consultati prima di scrivere le proprie implementazioni. È decidibile il problema di stabilire se le loro codifiche di f sono equivalenti?

3. È semidecidibile il problema del punto 1?

SOLUZIONE

1. No per il teorema di Rice (la correttezza è una proprietà non banale della funzione calcolata: esiste almeno una funzione corretta, ed almeno una funzione non corretta).

2. Sì: è una domanda chiusa, in quanto i due programmi sono stati realizzati, quindi sono fissati.

3. No: come noto, la correttezza non è nemmeno semidecidibile. In particolare, testando uno dopo l'altro i vari casi di test (con tecnica diagonale mediante enumerazione, dove le dimensioni considerate sono i possibili ingressi e il numero di passi di emulazione) si può eventualmente trovare, se c'è, una discrepanza tra l'uscita predetta dal generatore e quella ottenuta con la codifica di f; è quindi semidecidibile la presenza di una discrepanza, ed essendo indecidibile il problema della correttezza (cioè assenza di discrepanze), allora la correttezza non può essere nemmeno semidecidibile.

Esercizio 2.27.

1. Dire se è decidibile il problema di stabilire se, data una macchina di Turing (MT) deterministica e una sequenza di suoi stati (con lo stato iniziale come primo stato della sequenza), esiste una stringa x tale che la MT attraversa esattamente, uno per uno, la sequenza di stati desiderata durante l'esecuzione della MT con x in ingresso.

2. Dire se è semidecidibile il problema del punto 1.

3. Dire se è decidibile il problema di stabilire se, data una MT deterministica e una sequenza di suoi stati, esiste una stringa x tale che la MT, durante l'esecuzione con x in ingresso, attraversa gli stati desiderati nell'ordine desiderato, ma potrebbe, tra uno stato e il successivo della sequenza, attraversarne anche altri.

SOLUZIONE

1. Decidibile. Sia n la lunghezza della sequenza di stati data. Negli $n-1$ passi necessari per attraversare gli n stati si possono leggere al più $n-1$ caratteri dal nastro d'ingresso. Basta allora testare l'esecuzione della macchina con tutte le stringhe di $n-1$ caratteri, che sono in

numero finito, per $n-1$ passi e verificare se si attraversano gli stati nella sequenza data.

2. Semidecidibile, poiché, come mostrato sopra, il problema è decidibile.

3. Indecidibile. È sufficiente notare che, prendendo come coppia di stati da cui partire ed in cui arrivare lo stato iniziale e uno stato finale, possiamo determinare se la funzione calcolata dalla MT è definita in almeno un punto, che è il complemento del problema della *emptiness*, problema notoriamente indecidibile per le macchine di Turing (come si può evincere anche applicando il teorema di Rice).

Esercizio 2.28. Si dica, motivando opportunamente la risposta, se i problemi seguenti sono decidibili:

1. Stabilire se, dato un generico automa a stati finiti, esso accetta tutte le stringhe in Σ^*, con $\Sigma = \{0, 1\}$;

2. Stabilire se, dato un generico automa a stati finiti, esso accetta tutte e sole le stringhe che codificano numeri primi espressi in notazione binaria, con $\Sigma = \{0, 1\}$.

Come cambierebbe la risposta ai quesiti precedenti se fosse data una macchina di Turing al posto di un automa a stati finiti?

SOLUZIONE

1. Decidibile poiché è decidibile l'equivalenza tra automi a stati finiti. Basta quindi considerare un automa a stati finiti A, come quello mostrato qui sotto, che accetti tutte le stringhe in Σ^* e stabilire se l'automa dato sia equivalente ad A.



2. Decidibile (e deciso negativamente) poiché la risposta è sempre no: nessun automa a stati finiti è in grado di riconoscere il linguaggio dato.

Le varianti con macchina di Turing sono tutte indecidibili per il teorema di Rice.

Nel primo caso, infatti, il problema in questione riguarda una proprietà non banale del linguaggio riconosciuto (funzione calcolata): esiste certamente una macchina di Turing che accetta tutte le stringhe in Σ^* (come l'automa A) ma non tutte le macchine di Turing lo fanno. Osserviamo anche che il fatto che il linguaggio riconosciuto contenga tutte le stringhe non rende la proprietà banale: le proprietà banali per il teorema di Rice sono quelle che

appartengono a tutte le macchine (o a nessuna macchina) – non quindi a tutte le stringhe.

Nel secondo caso valgono considerazioni analoghe: il test di primalità può essere codificato con un algoritmo (ne esistono vari) e quindi esiste certamente una macchina di Turing che accetta tutti e soli i numeri primi, ma ovviamente non tutte le macchine lo fanno.

Esercizio 2.29. È decidibile il problema della soddisfacibilità di una formula MFO? Motivare adeguatamente la risposta.

SOLUZIONE

La soddisfacibilità di una formula MFO è decidibile: infatti questa logica definisce una sottofamiglia dei linguaggi regolari ed esistono algoritmi che, data una formula MFO, costruiscono un automa a stati finiti ad essa equivalente, ossia che riconosce l'insieme delle stringhe (linguaggio) che soddisfano la formula. Quindi una formula F è soddisfacibile se e solo se un automa a stati finiti ad essa equivalente riconosce un linguaggio non vuoto. Questo problema è decidibile – è infatti il complemento del problema della *emptiness*, che per gli automi a stati finiti è decidibile, grazie al *pumping lemma*.

Esercizio 2.30. Dimostrare che la famiglia di insiemi semidecidibili è chiusa rispetto all'intersezione, cioè, dati due insiemi qualunque S_1 e S_2, entrambi semidecidibili, la loro intersezione $S_{12} = S_1 \cap S_2$ è pure semidecidibile. A questo scopo, si risponda alle seguenti domande:

1. Delineare una procedura $\texttt{semDec}_{S_{12}}$ che "semidecide" S_{12}, cioè tale che, per $x \in S_{12}$, $\texttt{semDec}_{S_{12}}(x)$ termini e restituisca "vero", mentre, per $x \notin S_{12}$, $\texttt{semDec}_{S_{12}}(x)$ non termini; si può assumere l'esistenza di simili procedure $\texttt{semDec}_{S_1}$ e $\texttt{semDec}_{S_2}$ che semidecidono S_1 e S_2.
2. Delineare una procedura che enumeri S_{12}.
3. Con l'assunzione che $S_{12} \neq \varnothing$, scrivere una definizione della funzione generatrice $g_{S_{12}}$ di S_{12} (con questo termine si intende la funzione totale e calcolabile la cui immagine sia S_{12}).

Suggerimento: sfruttare il fatto che le funzioni generatrici g_{S_1} di S_1 e g_{S_2} di S_2 esistono; assicurarsi che $g_{S_{12}}$ sia totale e computabile e che la sua immagine coincida con S_{12}.

SOLUZIONE

1. $\texttt{semDec}_{S_{12}}$ per ogni ingresso x esegue prima $\texttt{semDec}_{S_1}$ su x e, se termina, esegue $\texttt{semDec}_{S_2}$ su x; se entrambe terminano e restituiscono "vero", si arresta e restituisce "vero".

2. Si effettua un'esecuzione "diagonale" (*dovetailing*) di $\texttt{semDec}_{S_{12}}$ con valori crescenti dei propri argomenti e del numero di passi da eseguire per ogni argomento. Quando il numero di passi considerato è sufficiente per far arrivare $\texttt{semDec}_{S_{12}}$ a terminazione, si emette l'argomento.

3. Si consideri la biiezione $d(x,y) = \frac{(x+y)(x+y+1)}{2} + 1$ tra coppie di interi e interi. Sia $\langle x,y\rangle = d^{-1}(n)$ e K un qualunque elemento di S_{12} (che per ipotesi non è vuoto), allora $g_{S_{12}}(n) = \begin{cases} g_{S_1}(x) \text{ se } g_{S_1}(x) = g_{S_2}(y) \\ K \text{ altrimenti} \end{cases}$. Essenzialmente, procediamo enumerando tutte le coppie $\langle x,y\rangle$ di valori dei domini di g_{S_1} e g_{S_2}: l'immagine di $g_{S_{12}}$ è composta da quei valori z che sono nell'immagine di g_{S_1} e di g_{S_2}, cioè tali per cui esistono x la cui immagine tramite g_{S_1} è z e y la cui immagine tramite g_{S_2} è pure z. Qualora x o y non appartenessero all'insieme di definizione delle rispettive funzioni o avessero immagini diverse, si restituisce un valore K di S_{12}. In questo modo l'immagine di $g_{S_{12}}$ contiene tutti e soli gli elementi di S_{12}.

Esercizio 2.31. Un linguaggio L viene detto *senza prefissi* se non contiene alcuna stringa che sia un prefisso proprio di un'altra stringa di L. È decidibile il problema di stabilire se un generico automa a stati finiti riconosca un linguaggio senza prefissi? Cambierebbe la risposta se, al posto di un automa a stati finiti, fosse data una generica macchina di Turing?

SOLUZIONE

Per l'automa a stati finiti, il problema è decidibile. Basta infatti effettuare qualche controllo strutturale sull'automa, come indicato di seguito.

Per ogni stato finale q raggiungibile dallo stato iniziale (con un cammino che corrisponderà quindi a una qualche stringa in ingresso x, accettata), si verifica se c'è un altro stato finale q' (non necessariamente diverso da q) raggiungibile da q con un cammino non vuoto (chiaramente, se $q' = q$ significa che c'è un ciclo). Se esiste un tale cammino tra q e q', esso corrisponde ad un suffisso y aggiunto a x e la stringa xy è pure accettata; x (accettata) è quindi un prefisso della stringa xy (accettata) e allora il linguaggio riconosciuto non è senza prefissi. In tutti gli altri casi il linguaggio è senza prefissi. Poiché la raggiungibilità è un problema decidibile (mediante ad esempio una ricerca in profondità o tramite algoritmi specifici noti in letteratura), è decidibile anche il problema in questione.

Nel caso della macchina di Turing il problema è invece indecidibile per il teorema di Rice: la capacità di riconoscere un linguaggio senza prefissi è una proprietà della funzione calcolata dalla macchina. Osserviamo, infatti, che l'accettazione delle stringhe di uno specifico linguaggio è determinata dalla funzione calcolata dalla macchina, che infatti associa ad ogni possibile stringa in ingresso (argomento della funzione) un valore booleano – in particolare, 1 per le stringhe accettate e 0 per le altre.

Sussistono le condizioni per poter applicare il teorema di Rice: detta $\mathcal{P}$ la proprietà in questione (qui, quella di riconoscere un linguaggio senza prefissi), l'insieme $\mathcal{F}$ di funzioni con la proprietà $\mathcal{P}$ non è banale. Ci basta mostrare che esiste almeno un linguaggio senza prefissi (ad esempio il linguaggio $L_1 = \emptyset$) ed almeno un linguaggio non senza prefissi (ad esempio il linguaggio $L_2 = \{a, aa\}$); ad essi corrispondono, rispettivamente, due opportune funzioni (calcolabili) f_1 (che associa a ogni stringa lo 0) e f_2 (che associa ad a e aa l'1 e a tutte le altre stringhe lo 0). Pertanto $\mathcal{F}$ non è né l'insieme vuoto (infatti esiste la funzione calcolabile f_1, che ha la proprietà $\mathcal{P}$), né l'insieme di tutte le funzioni computabili (infatti esiste la funzione f_2, che non ha la proprietà $\mathcal{P}$).

Esercizio 2.32. Si consideri il problema di stabilire se, date due generiche macchine di Turing M_1 e M_2, esista una stringa accettata da entrambe.

1. Tale problema è decidibile?
2. È semidecidibile?

Si consideri il problema di stabilire se il linguaggio riconosciuto da una macchina di Turing è vuoto.

3. Tale problema è decidibile?
4. È semidecidibile?

Motivare opportunamente le proprie risposte.

SOLUZIONE

1. Il problema non è decidibile. Supponiamo per assurdo che esista un algoritmo A in grado di risolvere il problema, ossia che, dati due indici i e j, restituisca "vero" se le macchine di Turing con indici i e j accettano una stringa in comune e "falso" altrimenti. Allora l'algoritmo A sarebbe anche in grado di risolvere il problema quando uno dei due indici è fissato; sia j l'indice di una macchina di Turing, qui indicata con $\hat{M}$, che accetti almeno una stringa (certamente ne esiste una con questa proprietà). Per il teorema di Rice, l'insieme degli indici delle macchine di Turing che accettano una stringa in comune con $\hat{M}$ non è ricorsivo e pertanto non è decidibile stabilire se una generica macchina di Turing accetti una stringa in comune con $\hat{M}$. Quindi non può esistere un algoritmo A in grado di risolvere il problema di partenza.

   Per mostrare che sussistono le condizioni di applicazione del teorema di Rice, osserviamo che la proprietà $\mathcal{P}$ di accettare una stringa in comune con $\hat{M}$ è una proprietà della funzione calcolata dalla macchina di Turing (che infatti associa ad ogni stringa in ingresso un 1 o uno

0 a seconda che la stringa venga accettata o meno) e che l'insieme $\mathcal{F}$ di funzioni con la proprietà $\mathcal{P}$ è non banale. Infatti $\mathcal{F}$ non è né vuoto (certamente $f_j \in \mathcal{F}$) né l'insieme di tutte le funzioni calcolabili (la funzione associata a una macchina che non accetta nessuna stringa non è in $\mathcal{F}$). Si noti che l'ipotesi che $\hat{M}$ accetti almeno una stringa rende la proprietà $\mathcal{P}$ non banale: se $\hat{M}$ non accettasse alcuna stringa, $\mathcal{F}$ sarebbe vuoto, poiché nessuna macchina potrebbe avere stringhe in comune con essa.

2. Il problema è però semidecidibile: è possibile infatti stabilire un principio enumerativo (*dovetailing*) di tutte le coppie $\langle m, n\rangle$, dove m indica il numero di passi di esecuzione delle macchine di Turing e n è la lunghezza della stringa da passare in ingresso alle macchine di Turing; per ciascuna coppia $\langle m, n\rangle$ si provano tutte le (finite) stringhe di lunghezza n. In questo modo, è possibile testare tutte le stringhe in ingresso con un qualunque numero di passi su entrambe le macchine, e, se esiste una stringa accettata da entrambe, prima o poi lo si scoprirà.

3. Il problema non è decidibile: l'insieme $\{\emptyset\}$ (ossia l'insieme contenente solo il linguaggio vuoto) è un particolare insieme di linguaggi, non vuoto (!) e certo non contenente tutti i linguaggi riconosciuti da macchine di Turing; quindi si può applicare anche in questo caso il teorema di Rice. Peraltro, questo è il problema della *emptiness* (stabilire se il linguaggio riconosciuto sia vuoto), che è notoriamente indecidibile per le macchine di Turing.

4. Il problema non è neanche semidecidibile, poiché è invece semidecidibile il problema complemento, ossia il problema di stabilire se un linguaggio non è vuoto. Per mostrarlo, basta infatti enumerare con tecnica diagonale le coppie $\langle x, y\rangle$, con y mosse della macchina sulla generica stringa x: se esiste una stringa accettata, prima o poi essa viene individuata da questa enumerazione. Se fossero semidecidibili sia il problema diretto, sia il problema complemento, allora il problema dovrebbe essere anche decidibile, ma abbiamo mostrato al punto precedente che non lo è.

Esercizio 2.33. Un automa (macchina) si dice *pari* se accetta solo stringhe di lunghezza pari. Per ciascuno dei seguenti problemi, dire se sono decidibili o meno, motivando la risposta.

1. Stabilire se un generico automa a stati finiti è pari.
2. Stabilire se una generica macchina di Turing è pari.

SOLUZIONE

1. Decidibile. Si consideri infatti un automa a stati finiti D che accetta tutte e sole le stringhe di lunghezza dispari sull'alfabeto di riferimento (semplicemente realizzabile con soli due stati, di cui uno solo di accettazione). Dato un generico automa a stati finiti A, è sempre possibile costruire un altro automa a stati finiti B che riconosce il linguaggio $L(B) = L(D) \cap L(A)$, grazie alla proprietà di chiusura dei linguaggi regolari rispetto all'intersezione. Ora, l'automa A è pari se e solo se $L(B)$ è vuoto, e la vuotezza (o vacuità, nota anche come *emptiness*) di un linguaggio regolare è un problema notoriamente decidibile, grazie al *pumping lemma*.

2. Indecidibile per il teorema di Rice. La "parità" di una macchina di Turing è certamente una proprietà della funzione da essa calcolata, in quanto non esiste una funzione calcolata da una macchina di Turing pari che sia anche calcolata da una macchina di Turing non pari. Basta quindi constatare che l'insieme di funzioni calcolate da macchine di Turing pari non è né vuoto né l'insieme di tutte le funzioni computabili per concludere che l'insieme degli indici di macchine di Turing pari non è ricorsivo.

Esercizio 2.34. Il Dr. Correction ha approntato un programma, chiamato C_0 (scritto in un linguaggio Turing-completo), che riceve in ingresso un compito di informatica teorica (opportunamente codificato come sequenza di bit) e restituisce in uscita il voto corrispondente. Il Dr. Correction ha testato, con soddisfazione, C_0 sui compiti dei propri studenti.

Il Prof. Halting aveva però già implementato un programma, chiamato H_0 (sempre scritto in un linguaggio Turing-completo), con la stessa finalità e si chiede se i due programmi (C_0 e H_0) valutino tutti i compiti possibili esattamente allo stesso modo. Si supponga che i compiti possibili siano un'infinità numerabile.

1. È decidibile il quesito del Prof. Halting?

Il programma C_0 diventa molto popolare, ma il suo utilizzo è a pagamento. Per questo motivo, molti docenti si cimentano nell'implementazione di Programmi per la Correzione di Compiti di Informatica Teorica (PCCIT) che si comportino esattamente come C_0. Non essendo però certi dell'attendibilità della propria implementazione, si chiedono se ci sia un modo per verificarla.

2. È decidibile il problema di stabilire se un generico PCCIT valuti tutti i compiti esattamente allo stesso modo di C_0?

3. Dopo mesi di alacre lavoro, il Dr. Correction riesce a dimostrare che C_0 restituisce sempre un voto a fronte di qualunque compito di informatica teorica. Alla luce di questo fatto, dire se cambierebbero (e come) le risposte per i punti precedenti.

SOLUZIONE

1. La domanda è chiusa, in quanto riguarda l'equivalenza tra due specifici programmi (C_0 e H_0) e quindi la risposta alla domanda non dipende da alcun parametro. Il problema è pertanto decidibile.

2. Dal testo deduciamo che un generico PCCIT è assimilabile a un generico programma/funzione implementato con una macchina di Turing. Il quesito è allora quello dell'equivalenza tra una generica macchina di Turing con una macchina data, che è notoriamente indecidibile. Possiamo anche applicare il teorema di Rice. Detta $\mathcal{P}$ la proprietà di calcolare la stessa funzione f calcolata da C_0, l'insieme $\mathcal{F}$ di funzioni computabili con la proprietà $\mathcal{P}$ è non banale: $\mathcal{F}$ non è vuoto in quanto f è calcolabile (è calcolata da C_0); non è nemmeno l'insieme di tutte le funzioni computabili in quanto esistono certamente funzioni computabili diverse da f (ad esempio $g(x) = x^2$). Concludiamo allora che l'insieme di indici di macchine di Turing che calcolano la specifica funzione calcolata da C_0 non è ricorsivo, pertanto non è possibile stabilire se un generico PCCIT (identificato da un generico indice di macchina di Turing) si comporti allo stesso modo di C_0.

3. La condizione aggiuntiva descritta nel testo è che la funzione calcolata da C_0 è sempre definita (ossia C_0 termina sempre). Tuttavia, nulla cambia alle risposte ai quesiti precedenti. La prima domanda resta chiusa. Nella seconda domanda, l'insieme di indici di macchine di Turing che calcolano la specifica funzione calcolata da C_0 è comunque non ricorsivo per il teorema di Rice.

Esercizio 2.35.

1. Dire se è decidibile il problema di stabilire se, data una generica grammatica regolare G, esiste una sequenza di produzioni che portano ad una parola fatta solo di simboli terminali. Dire se il problema è semidecidibile.

2. Dire se è decidibile il problema di stabilire se, data una generica grammatica non ristretta G, esiste una sequenza di produzioni che portano ad una parola fatta solo di simboli terminali. Dire se il problema è semidecidibile.

SOLUZIONE

1. Siccome per ogni grammatica regolare G possiamo costruire un automa a stati finiti equivalente che riconosca lo stesso linguaggio generato da G (e viceversa), questo problema è lo stesso del problema di decidere se il linguaggio riconosciuto da un automa a stati finiti sia non vuoto (che è il complemento del problema della *emptiness*). Il problema è quindi decidibile (e quindi anche semidecidibile), perché, come è noto, lo è il

problema della emptiness per gli automi a stati finiti. Detto Q l'insieme degli stati dell'automa, un modo per decidere il problema consiste nel controllare l'accettazione di tutte le parole con lunghezza $\leq |Q|$, che sono in numero finito: se almeno una parola è accettata, il linguaggio non è vuoto, altrimenti è vuoto. Grazie al *pumping lemma*, sappiamo che non serve controllare le infinite stringhe di lunghezza superiore a $|Q|$.

2. È noto che, data una qualunque grammatica non ristretta G, è possibile costruire un'equivalente macchina di Turing che riconosce lo stesso linguaggio generato da G (e viceversa). Quindi, questo problema è equivalente al problema di decidere se il linguaggio riconosciuto da una generica macchina di Turing è non vuoto. Il problema è indecidibile, in quanto il suo complemento (*emptiness*) è notoriamente indecidibile per le macchine di Turing.

   Un modo per mostrarlo è per riduzione dal problema della terminazione del calcolo (*halting problem*). Se fossimo in grado di decidere la emptiness di una generica macchina di Turing, allora saremmo anche in grado di decidere la terminazione. Si considerino infatti una generica macchina di Turing M e un generico ingresso x. Per ogni coppia $\langle M, x\rangle$, costruiamo una macchina M' che, a fronte dell'ingresso x', funziona nel modo seguente: se $x' \neq x$, allora M' non termina; se $x' = x$, allora M' mette in esecuzione M su x e, se M termina, allora anche M' termina accettando x. In altre parole, il linguaggio riconosciuto da M' è non vuoto se e solo se M accetta x. Quindi, se fosse decidibile la emptiness per una generica macchina M', allora potremmo anche decidere il problema della terminazione del calcolo per una generica macchina M e un generico ingresso x. Poiché quest'ultimo problema è notoriamente indecidibile, allora anche il problema della emptiness deve essere indecidibile.

   Un altro modo per mostrarlo è mediante il teorema di Rice: riconoscere il linguaggio vuoto è una proprietà (non banale) della funzione calcolata da una macchina di Turing. Infatti l'insieme delle funzioni calcolabili con questa proprietà non è vuoto (c'è una macchina che riconosce il linguaggio vuoto) né l'insieme di tutte le funzioni calcolabili (c'è una macchina che riconosce un linguaggio non vuoto).

   Il problema è invece semidecidibile, in quanto basta usare un'enumerazione diagonale (*dovetailing*) delle stringhe x in ingresso e del numero n di passi da simulare: per ogni coppia $\langle x, n\rangle$ si simula l'esecuzione della macchina per n passi sulla stringa x e, se esiste una stringa in ingresso per cui la macchina termina in un certo numero di passi, ce ne si può quindi accorgere, concludendo che il linguaggio riconosciuto dalla macchina non è vuoto; se invece il linguaggio è vuoto, questo procedimento continua all'infinito.

Esercizio 2.36. Una macchina di Turing (MT) M_1 viene detta più veloce di una MT M_2 qualora, per ogni ingresso $x \in \mathbb{N}$, M_1 non impieghi mai più mosse di M_2 per completare la propria esecuzione su x (convenzionalmente, qualora una MT non termini o termini in uno stato non finale per un qualche ingresso, nessun'altra MT impiega più mosse di essa per quell'ingresso). Sia ora M_u una MT che calcola una funzione che è indefinita ovunque.

1. È decidibile il problema di stabilire se, data una generica MT, essa sia più veloce di M_u?
2. Il problema precedente è deciso?
3. È decidibile il problema di stabilire se, data una generica MT, M_u sia più veloce di essa?
4. Il problema precedente è deciso?
5. È decidibile il problema di stabilire se, date due generiche MT M_1 e M_2, M_1 sia più veloce di M_2?

SOLUZIONE

1. Decidibile, perché chiaramente ogni macchina è più veloce di M_u.
2. Anche deciso, perché la risposta è sempre nota (sì).
3. Indecidibile, perché equivale a decidere se la funzione calcolata da una generica MT sia ovunque indefinita, il che è indecidibile per il teorema di Rice. Infatti, l'insieme $\mathcal{F}$ di funzioni calcolabili con la proprietà di essere ovunque indefinite non è né vuoto (contiene la funzione $f(x) = \perp$, calcolata da M_u), né l'insieme di tutte le funzioni computabili (non contiene, ad esempio, la funzione $g(x) = 1$, banalmente computabile). Sussistono quindi le condizioni per applicare il teorema di Rice e ne deriva che l'insieme S di indici di MT che calcolano la funzione ovunque indefinita non è ricorsivo, cioè non è possibile determinare se un generico indice di MT appartenga a S e quindi il problema in questione è indecidibile.
4. Non deciso poiché indecidibile.
5. Indecidibile, perché un suo caso particolare è indecidibile. Se questo problema fosse decidibile, sapremmo infatti risolverne qualunque sua istanza, incluso il problema 3.

Esercizio 2.37.

1. Si considerino le famiglie $\mathcal{M}_1$ e $\mathcal{M}_2$ di macchine di Turing (MT) che calcolano funzioni totali definite sui numeri naturali e periodiche, rispettivamente di periodo T_1 e T_2. È decidibile il problema di stabilire se, date due MT appartenenti, rispettivamente, a $\mathcal{M}_1$ e $\mathcal{M}_2$ esse siano equivalenti?

2. Si consideri la famiglia $\mathcal{M}^2$ di MT deterministiche che hanno complessità spaziale quadratica, ossia $M \in \mathcal{M}^2 \rightarrow S_M(n) = n^2$, dove n è la lunghezza della stringa in ingresso[1]. È decidibile il problema di stabilire se, data una qualsiasi MT $M \in \mathcal{M}_2$ e una stringa x, $x \in L(M)$?

3. (parte opzionale) Cambia la risposta alla domanda 2 se le MT in questione sono nondeterministiche? NB: si assuma che la complessità spaziale di una MT nondeterministica sia $S_M(n) = n^2$ se *tutte* le sue possibili computazioni su un ingresso di lunghezza n non occupano più di n^2 celle di memoria.

SOLUZIONE

I 3 problemi sono tutti decidibili:

1. Le funzioni calcolate da MT di $\mathcal{M}_1$ e di $\mathcal{M}_2$ sono tutte periodiche di periodo $T_1 \cdot T_2$. Date due macchine $M_1 \in \mathcal{M}_1$ e $M_2 \in \mathcal{M}_2$, se ne può verificare l'equivalenza controllando che restituiscano lo stesso risultato per tutti i valori di x tali che $0 \leq x \leq T_1 \cdot T_2$: questa procedura termina sempre, poiché il numero di controlli è finito e ciascuna singola esecuzione termina, dal momento che le famiglie $\mathcal{M}_1$ e $\mathcal{M}_2$ calcolano funzioni totali.

2. È sufficiente simulare l'esecuzione della MT data sulla stringa in ingresso: siccome esiste solo un numero finito di configurazioni possibili che occupino al massimo n^2 celle di memoria, la simulazione potrà in un numero finito di mosse o *i*) fermarsi accettando la stringa, o *ii*) fermarsi rifiutando la stringa o *iii*) ritrovarsi in un'identica configurazione già incontrata in precedenza: in tal caso la macchina sarebbe costretta ad "andare in loop" e quindi si può concludere che la stringa va rigettata.

3. In questo caso sono possibili più computazioni per ogni singola stringa ma quelle che occupano non più di n^2 celle di memoria (e non vanno in loop) sono comunque finite: è quindi sufficiente enumerarle tutte e verificare se almeno una di esse accetta la stringa in ingresso.

Esercizio 2.38. Il dott. E. Gold ha costruito un programma che riceve in ingresso un array di al più 5 stringhe di 3 caratteri costruite sull'alfabeto $\{a, b\}$ e che restituisce in uscita la versione ordinata alfabeticamente dello stesso array. Ad esempio, dato l'array $[abb, baa, aba]$, il programma deve restituire $[aba, abb, baa]$.

1. È decidibile stabilire se il programma costruito dal dott. E. Gold è corretto rispetto alle specifiche appena fornite?

[1] Si ricorda che una MT M ha complessità spaziale $S_M(n) = n^2$ se, durante ogni esecuzione con una stringa di ingresso lunga n, non occupa mai più di n^2 celle di memoria (escludendo le celle del nastro di ingresso).

2. È decidibile stabilire se il generico array passato al programma sia già ordinato?

Incerto sul punto 1, il dott. E. Gold si accorge però che gli array che si possono passare in ingresso al programma sono in numero finito.

3. Quanti sono i diversi array di al più 5 stringhe di 3 caratteri costruite su $\{a, b\}$?

Forte di questa intuizione, ha quindi intenzione di implementare un altro programma che funga da "tester" per stabilire la correttezza del proprio (e di qualunque altro) programma che operi sullo stesso dominio di ingresso: dato in ingresso il codice sorgente di un programma, il tester lo esegue con tutti i possibili (finiti) ingressi e controlla che il risultato sia sempre quello atteso.

4. Si può stabilire, tramite il tester, se un generico programma risponda alle specifiche?

SOLUZIONE

1. Sì, è una domanda chiusa: si tratta di un singolo programma che può essere solo corretto o scorretto, quindi la risposta a questa domanda non dipende da alcun parametro.

2. Sì, basta verificare se la versione ordinata dell'array coincide con quella passata in ingresso ed esistono algoritmi noti che implementano l'ordinamento in modo corretto.

3. Le diverse stringhe di 3 caratteri su $\{a, b\}$ sono $2^3 = 8$. I diversi array di i stringhe da 3 caratteri su $\{a, b\}$ sono quindi 8^i. In totale, il numero di diversi array di al più 5 stringhe è quindi $\sum_{i=0}^{5} 8^i$ (cioè 37449, ma il numero specifico non è importante).

4. No, questo non è possibile: stabilire se la funzione calcolata da un generico programma abbia una qualche proprietà non banale è indecidibile per il teorema di Rice. La proprietà di ordinare correttamente l'array è infatti certamente non banale, in quanto esiste una funzione calcolabile che ce l'ha (quella di un algoritmo di ordinamento corretto!) ed una funzione calcolabile che non ce l'ha (quella di un qualunque algoritmo che non calcoli l'ordinamento). Pertanto non sarà possibile stabilire la questione nemmeno con l'ausilio del tester, che in effetti potrebbe incappare in un loop infinito quando testa il generico programma su un qualche ingresso.

Esercizio 2.39.

1. Dire se è decidibile il problema di stabilire se una generica macchina di Turing (MT) riconosce il linguaggio $L(F)$ definito dalla seguente formula

MFO F:

$$\begin{array}{c} \exists x(x = 1 \wedge a(x)) \\ \wedge \\ \forall x \forall y \left(\begin{array}{c} y = x + 1 \rightarrow (x = 0 \rightarrow b(x)) \\ \wedge \\ (b(x) \rightarrow c(y)) \end{array} \right) \end{array}$$

2. Dire se è computabile la seguente funzione:

$$g(x) = \begin{cases} 1 \text{ se la funzione } f_x \text{ calcolata dalla } x\text{-esima MT è tale che:} \\ \qquad f_x(y) = \begin{cases} 1 \text{ se la } y\text{-esima MT riconosce il linguaggio } L(F) \\ 0 \text{ altrimenti} \end{cases} \\ 0 \text{ altrimenti} \end{cases}$$

SOLUZIONE

1. Il linguaggio definito da F è il linguaggio vuoto (in posizione 1 ci dovrebbero essere sia a che c, il che è impossibile). Il problema dato coincide quindi con il decidere se la MT riconosca il linguaggio vuoto. Questo problema, noto come problema della *emptiness*, è notoriamente indecidibile per le macchina di Turing. L'indecidibilità si può anche mostrare con il teorema di Rice (almeno una macchina riconosce il linguaggio vuoto, ma non tutte lo fanno).

2. In questo caso la funzione è banalmente computabile, perché corrisponde alla funzione costante $g(x) = 0$. Questo deriva dal fatto che per nessun valore di x abbiamo $g(x) = 1$, dal momento che questo implicherebbe che la funzione f_x (per definizione calcolabile) decida il problema della emptiness, che abbiamo mostrato essere indecidibile al punto precedente, e quindi non può esistere nessuna funzione calcolabile che lo decida. In altre parole, f_x dovrebbe essere la funzione caratteristica dell'insieme di indici di MT che riconoscono il linguaggio vuoto, ma, come abbiamo visto, tale insieme non è ricorsivo per il teorema di Rice.

   Un altro modo di vedere le cose è che $g(x)$ è la funzione caratteristica di un insieme vuoto di indici di funzioni (vuoto perché non esiste nessuna funzione con la proprietà desiderata, ovvero quella di decidere la emptiness). Per il teorema di Rice, tale insieme è banalmente ricorsivo e quindi $g(x)$ è necessariamente calcolabile.

Esercizio 2.40. Si consideri un semplice linguaggio di programmazione LP_1 che tratta solo interi (NB: l'insieme matematico degli interi, non quello limitato del C) e array (non limitati a compile-time) di interi, ed è dotato delle normali istruzioni di assegnamento (*variabile* = *espressione*), dell'istruzione condizionale `if` (con `then` e `else`) e del ciclo `while`. Si consideri poi un linguaggio LP_2 che

differisce da LP_1 in quanto invece del ciclo `while` ha il costrutto `for` definito da questa sintassi:

`for` *variabile_intera* = *costante_intera_1* `to` *costante_intera_2* `do` {*corpo_del_ciclo*}

con *costante_intera_2* $\geq$ *costante_intera_1* e la cui semantica consiste dei seguenti passi:

- inizializzare la variabile intera, diciamo i, con il valore *costante_intera_1*;
- eseguire il corpo del ciclo;
- incrementare di un'unità i;
- uscire dal ciclo quando i è maggiore di *costante_intera_2*.

NB: il *corpo_del_ciclo* non può contenere assegnamenti ad i.

È decidibile il problema di stabilire se, dato un generico programma scritto in LP_1 ne esiste un altro ad esso equivalente in LP_2? Spiegare brevemente la risposta.

SOLUZIONE

È ben noto che i costrutti base di LP_1 sono sufficienti per dare al linguaggio la potenza della macchina di Turing (è un facile esercizio codificare una qualsiasi macchina di Turing in LP_1). Al contrario, LP_2 non ha tutta la potenza della macchina di Turing (tra l'altro i suoi programmi terminano sempre). Quindi i problemi (in questo caso funzioni definite su numeri interi) risolvibili da LP_2 sono un sottoinsieme proprio e non vuoto dei problemi ricorsivamente enumerabili. Se fosse possibile decidere il problema suddetto sarebbe perciò anche possibile decidere se una generica macchina di Turing risolve un problema della famiglia dei problemi risolvibili con LP_2, violando il teorema di Rice.

Esercizio 2.41. In matematica, si definiscono *numeri primi gemelli* due numeri primi che differiscano tra loro di 2. La *congettura dei numeri primi gemelli* afferma che esistono infinite coppie di numeri primi gemelli.

1. Ad oggi, la più grande coppia di numeri primi gemelli trovata è $3756801695685 \cdot 2^{666669} \pm 1$. Si tratta di numeri con ben 200700 cifre, ma non è ancora stato dimostrato l'enunciato della congettura, che potrebbe quindi anche essere falsa.

   Che cosa si può concludere in merito alla decidibilità dell'enunciato della congettura?

2. Sia f una funzione definita come segue:

$$f(x) = \begin{cases} 1 \text{ se esiste una coppia di numeri primi gemelli maggiori di } x; \\ 0 \text{ altrimenti.} \end{cases}$$

   La funzione f è calcolabile?

SOLUZIONE

1. Si tratta di una questione certamente decidibile poiché la formula che la specifica è chiusa. La verità o falsità della congettura non dipende infatti da alcun parametro.

2. Se la congettura dei numeri primi gemelli è vera, allora la funzione f vale costantemente 1, ed è pertanto calcolabile (ad esempio, mediante un algoritmo con una singola istruzione `return 1`). Se la congettura è falsa, allora esiste un numero k oltre il quale non è più vero che esistono due numeri primi gemelli (e prima del quale esistono). Pertanto la funzione f sarà fatta a "scalino", e sarà pari a 1 per ingressi minori di k, e pari a 0 per ingressi maggiori o uguali a k. Anche in questo caso la funzione è certamente calcolabile (un algoritmo che la implementi richiede una singola istruzione del tipo `if x<k then return 1 else return 0`). In definitiva, anche se non sappiamo se la congettura sia vera o falsa e anche se, nel caso sia falsa, non conosciamo l'esatto valore di k, per ambo i casi e per qualunque valore di k esiste un algoritmo che codifica correttamente il comportamento della funzione f, che pertanto è calcolabile.

Esercizio 2.42. Una macchina di Turing M_i (che calcola la funzione f_i) è detta *riproducibile* se esiste un'altra macchina di Turing M_j (che calcola la funzione f_j) tale per cui $f_i = f_j$.

Si consideri l'insieme delle macchine di Turing definite sull'alfabeto di due caratteri $\{0, 1\}$. Al suo interno siano:

- $\mathcal{F}$ l'insieme delle funzioni calcolate da macchine di Turing riproducibili.
- $\mathcal{G}$ l'insieme delle funzioni calcolate da macchine di Turing riproducibili e con meno di 10 stati.
- $\mathcal{H}$ l'insieme delle funzioni calcolate da macchine di Turing riproducibili e con più di 10 stati.

Dire se i seguenti problemi sono decidibili:

1. Stabilire se una generica macchina di Turing calcoli una funzione in $\mathcal{F}$.
2. Stabilire se una generica macchina di Turing calcoli una funzione in $\mathcal{G}$.
3. Stabilire se una generica macchina di Turing calcoli una funzione in $\mathcal{H}$.

SOLUZIONE

1. Decidibile e anche deciso. Infatti, ogni macchina di Turing è riproducibile: come è noto, data una funzione calcolabile, esistono infinite macchine di Turing che la calcolano. Quindi $\mathcal{F}$ coincide con l'insieme universo di tutte le funzioni calcolabili.

2. Indecidibile per il teorema di Rice. Infatti, l'insieme delle macchine di Turing con meno di 10 stati e con alfabeto di due caratteri è finito (proprietà usata ad esempio anche per enumerare le macchine di Turing), pertanto sarà finito (e non vuoto) anche l'insieme di funzioni calcolabili calcolate da dette macchine di Turing. Allora l'insieme $\mathcal{G}$ non è né l'insieme vuoto, né l'insieme di tutte le funzioni computabili, da cui l'indecidibilità per il teorema di Rice.

3. Decidibile per il teorema di Rice. Infatti, ogni funzione calcolabile è calcolabile (anche) mediante una macchina con più di 10 stati. Come scritto in precedenza, per ogni funzione computabile esistono infinite macchine di Turing che la calcolano, mentre le macchine di Turing con un numero di stati minore o uguale a 10 sono in numero finito, quindi ogni funzione calcolabile dev'essere necessariamente calcolata anche da macchine con più di 10 stati.

Esercizio 2.43. Dire, motivando la risposta, se i seguenti problemi sono decidibili:

1. Stabilire se, data una generica macchina di Turing, esista un'altra macchina ad essa equivalente (dove "equivalente" significa "che calcola la stessa funzione").

2. Stabilire se tutte le macchine di Turing che terminano sempre siano equivalenti a una qualche altra macchina di Turing che termina sempre entro 10 mosse.

SOLUZIONE

1. Il problema è decidibile (ed anche deciso positivamente) in quanto è noto che ogni macchina di Turing ammette infinite macchine di Turing ad essa equivalenti. Per trovare una macchina equivalente basta, ad esempio, aggiungere al diagramma degli stati altri stati non utilizzati dalla macchina.

2. In questo caso la domanda è chiusa. Ce ne si può rendere conto ad esempio osservando che un'ipotetica formula, come la seguente, che descriva il problema in questione è chiusa e non dipende quindi da alcun parametro:

$$\forall M(t(M) \rightarrow \exists M'(e(M, M'))),$$

dove $t(M)$ indica che la macchina M termina sempre e $e(M, M')$ indica che le macchine M e M' sono equivalenti.

Poiché la domanda è chiusa, il problema è certamente decidibile. Mediante semplici considerazioni legate alla modalità di esecuzione di

una macchina di Turing (non richieste dall'esercizio), si può concludere che il problema è anche deciso (negativamente). Infatti, per la sola lettura di un ingresso x occorrono almeno $|x|$ mosse. Pertanto una macchina che non esegue mai più di 10 mosse non può calcolare una funzione che dipenda da un ingresso arbitrariamente grande (come ad esempio $f(x) = x$), dal momento che non può neanche leggerlo. Allora non è vero che tutte le macchine di Turing hanno la proprietà descritta.

Esercizio 2.44. Dato un generico trasduttore a stati finiti T per il quale

- O è l'alfabeto di uscita di T,
- I è l'alfabeto di ingresso di T,
- $\tau : I^* \to O^*$ è la traduzione definita da T,

e una stringa $y \in O^*$, è decidibile il problema di stabilire se esista una stringa $x \in I^*$ tale che $y = \tau(x)$?

Giustificare brevemente la risposta.

SOLUZIONE

Premessa: la lunga storia della teoria degli automi dimostra che molto raramente un problema relativo agli automi a stati finiti è indecidibile; ciò grazie alle sue proprietà essenziali, riassunte nel *pumping lemma*.

Il problema in oggetto non costituisce un'eccezione. Infatti, anche se in generale non è detto che $|x| \leq |\tau(x)|$ a causa di eventuali mosse che non scrivono nulla in uscita, è possibile enumerare le stringhe in ingresso in modo tale da garantire che l'insieme traduzione di un numero finito di esse contenga tutte le stringhe di uscita z tali che $|z| \leq |y|$. Precisamente si potrebbe procedere come segue.

Si enumerino tutte le stringhe $x \in I^*$ per lunghezza crescente e se ne calcolino le rispettive $\tau(x)$ finché $|\tau(x)| > |y|$. Se durante tale enumerazione si individua una sottostringa w di x tale che $x = zws$, $\delta^*(q_0, z) = q$, $\delta^*(q, w) = q$ e $\eta^*(q, w) = \varepsilon$ (cioè w corrisponde a un ciclo in cui non viene scritto nulla in uscita), w può essere eliminata dalla stringa x senza evidentemente cambiare il risultato della traduzione. Rimangono quindi da prendere in esame solo stringhe "a traduzione crescente" ovvero quelle per cui l'aggiunta di k caratteri in ingresso (con k comunque inferiore al numero di stati $|Q|$ dell'automa) comporta l'aggiunta di almeno un carattere in uscita:

$$\exists k(k < |Q| \wedge \forall x \forall z(|z| \geq k \to |\tau(xz)| > |\tau(x)|))$$

Poiché, limitandoci alle stringhe a traduzione crescente, non perdiamo stringhe in uscita e produciamo almeno un carattere in uscita ogni $< |Q|$ caratteri letti in ingresso, abbiamo la garanzia di produrre tutte le possibili traduzioni di lunghezza $\leq |y|$ se consideriamo tutte le stringhe in ingresso di lunghezza

$< |Q| \cdot |y|$, che sono in numero finito. Non serve quindi considerare stringhe in ingresso di lunghezza $\geq |Q| \cdot |y|$, perché o producono traduzioni già prodotte da stringhe più corte o producono traduzioni più lunghe di $|y|$.

Enumeriamo quindi tutte le finite stringhe di I^* di lunghezza $< |Q| \cdot |y|$ e le loro traduzioni tramite T e verifichiamo se tra esse figura la stringa y, decidendo così il problema in oggetto.

Esercizio 2.45. Si consideri il gioco del "forza 4", in cui ci sono 2 giocatori, uno con pedine rosse ed uno con pedine gialle, in cui vince chi per primo mette in fila (orizzontalmente, verticalmente, o diagonalmente) 4 pedine del proprio colore in uno schema di dimensione 7 colonne per 6 righe. Una mossa del gioco consiste nello scegliere una colonna, e nel fare scendere una pedina lungo la colonna: se la colonna al momento della mossa contiene k pedine, in seguito alla mossa la nuova pedina occuperà la posizione $k + 1$ (assumendo che la prima posizione sia la #1); non si possono depositare pedine in una colonna piena e inzialmente tutte le colonne sono vuote.

1. È decidibile il problema di stabilire se, data una configurazione del gioco (cioè una distribuzione di pedine nello schema), esiste una sequenza di mosse che portano un giocatore a vincere?
2. È decidibile il problema del punto 1 se lo schema di gioco, invece che essere 6×7, è di dimensione arbitraria $h \times k$?
3. È decidibile il problema del punto 1 se lo schema di gioco, invece che essere 6×7, non ha limiti né nelle righe né nelle colonne?
4. Si consideri di avere a disposizione una funzione che, date 2 configurazioni del gioco ed un colore, dice quale delle 2 è la "migliore" per quel colore (una configurazione potrebbe essere "migliore" perché, per esempio, rende più probabile al giocatore del colore dato di vincere la partita):
   (a) È decidibile il problema di stabilire se, dato un generico programma che prende in ingresso una configurazione del forza 4 (schema 6×7), ed un colore che deve muovere, questo restituisce sempre la mossa migliore per quel colore?
   (b) È semidecidibile il problema del punto 4a?

SOLUZIONE

1. Decidibile. Lo schema è finito e si possono quindi analizzare tutte le (finite) sequenze di gioco disponibili a partire dalla configurazione data.
2. Decidibile. Anche in questo caso lo schema è finito, quindi valgono le considerazioni fatte al punto precedente.

3. Decidibile, anzi deciso (positivamente): essendoci un numero illimitato di posizioni, se un giocatore gioca in modo "stupido" (posizionando, ad esempio, ogni propria pedina lontano da tutte le altre) è sempre possibile che l'altro vinca (e quindi esiste certamente una sequenza di mosse che porta un giocatore a vincere).

4. (a) Indecidibile. Anche se la funzione ha un dominio finito, i possibili programmi sono infiniti e la proprietà richiesta è una proprietà della funzione calcolata. In particolare, proprio perché il dominio è finito, è banale costruire un programma/funzione che restituisca sempre la mossa migliore per un dato colore (e quindi questa funzione è calcolabile), ma ovviamente non tutte le funzioni calcolabili calcolano questa funzione: sussistono quindi le ipotesi del teorema di Rice.
   (b) Semidecidibile. Come indicato al punto precedente, si può costruire un programma, chiamiamolo P, che restituisca sempre la mossa migliore per tutti i possibili ingressi (coppie di configurazioni e colore). Se un programma calcola la funzione in modo corretto, ciò può essere stabilito eseguendolo su tutti i possibili ingressi (il dominio è finito) e confrontandone ogni volta il risultato con quello di P a fronte dello stesso ingresso.

2.4 Riduzioni

Esercizio 2.46.

1. Dire se è decidibile il problema di stabilire se due programmi, uno scritto in C e uno scritto in Java, calcolano la stessa funzione.

2. Dire se è decidibile il problema di stabilire se due programmi, entrambi scritti in C, sono identici, salvo per il fatto che i nomi delle variabili usate sono diversi tra un programma e l'altro.

3. Dire se è decidibile il problema di stabilire se, dato un programma C che fa uso di puntatori, esiste un'esecuzione in cui, ad un certo punto, due variabili puntano entrambe allo stesso indirizzo (fenomeno dell'*aliasing*).

SOLUZIONE

1. L'equivalenza di programmi C e Java è indecidibile. Lo si può vedere riducendo il problema dell'equivalenza tra due programmi C a quella tra un programma C e un programma Java: ogni programma C può essere trasformato in un equivalente programma Java, e se fosse decidibile l'equivalenza tra programma Java e programma C, allora potremmo anche decidere l'equivalenza tra programmi C, che è notoriamente indecidibile.

2. Il problema è decidibile, in quanto si tratta di leggere i 2 programmi parola per parola. Laddove si trovano dichiarazioni di variabili (che devono essere in corrispondenza una con l'altra), si memorizza una corrispondenza tra la variabile (chiamiamola v_1) del programma P_1 e quella (v_2) del programma P_2, e si controlla poi che ogni volta che nel programma P_1 compare la variabile v_1, nel programma P_2 compaia v_2. In questo modo, dopo aver letto per intero i programmi P_1 e P_2, se non si sono trovate discrepanze, si conclude che i programmi sono effettivamente identici a meno dei nomi delle variabili, altrimenti no.

3. Il problema non è decidibile, lo si può vedere per riduzione dal problema della terminazione (*halting problem*) dei programmi C. Più precisamente, si consideri un generico programma P, espresso in C. Nel caso in cui P usi puntatori, è possibile sempre ri-esprimere lo stesso calcolo in C senza usare puntatori (ad esempio, il noto teorema di Böhm e Jacopini afferma che è possibile calcolare qualunque funzione computabile facendo uso soltanto di sequenze di sottoprogrammi, istruzioni condizionali (`if`) e iterazioni (`while`)). Assumiamo dunque che P non faccia uso di puntatori.

   Dato quindi il programma P ed un generico suo ingresso a, possiamo scrivere un nuovo programma P' che prende come ingresso a e, in sequenza

   - dichiara due puntatori, x e y (che non compaiono in P, dato che in P non compare alcun puntatore);
   - assegna loro due valori diversi;
   - esegue P su a e, se la computazione termina, assegna x a y, creando quindi un alias tra x e y.

   Chiaramente P termina sull'ingresso a se, e solo se, in P' ad un certo punto x e y puntano allo stesso indirizzo. Quindi, se il problema desiderato fosse decidibile, lo sarebbe anche quello della terminazione di P su un generico ingresso a, il che è impossibile.

Esercizio 2.47.

1. Data una generica macchina di Turing non deterministica M e una generica stringa x, è decidibile stabilire se esistono almeno due cammini distinti di computazione effettuati da M che portano all'accettazione della stringa x?

2. Il problema è semidecidibile?

3. Come cambiano le risposte alle domande precedenti se M è un automa a stati finiti?

SOLUZIONE

1. Possiamo esibire una riduzione dal problema dell'arresto al problema in questione, che quindi è indecidibile. Infatti, sia M' una generica macchina non deterministica. Costruiamo una nuova macchina M'' uguale a M' ma con l'aggiunta di un cammino non deterministico che, dallo stato iniziale, accetta x in modo banale, ossia compiendo un passo verso un nuovo stato per ogni carattere di x. Se potessimo decidere se M'' ammette almeno due cammini che accettano x potremmo allora decidere se la (generica) macchina M' accetta la (generica) stringa x.

2. Il problema è semidecidibile perché, se esistono due cammini che accettano x, prima o poi li trovo provando tutti i possibili cammini, attraversando in ampiezza (*breadth-first*) l'albero delle possibili computazioni nondeterministiche.

3. Con un automa a stati finiti il problema diventa decidibile perché possiamo, ad esempio, enumerare tutti i (finiti) cammini di lunghezza $|x|$ sull'automa a partire dallo stato iniziale e vedere se almeno due di essi accettano x.

Esercizio 2.48. Dato un ingresso x ed una macchina di Turing (MT) deterministica M_y con indice y, si definisce *lunghezza finita* dell'esecuzione di M_y su x il numero totale di transizioni (realizzate da M_y leggendo l'input x) che portano la macchina M_y dalla configurazione iniziale ad una configurazione di arresto (se la computazione non termina, la lunghezza finita non è definita).

È decidibile stabilire se un generico ingresso x induca in una generica MT M_y un'esecuzione di lunghezza finita pari?

SOLUZIONE

Sia P il problema in oggetto. Mostriamo che P non è decidibile mediante una riduzione dal problema dell'arresto (*halting problem*) a P. L'idea è di costruire una macchina che accetti la stringa x in ingresso con un'esecuzione di lunghezza finita pari se e solo se la macchina M_y accetta x. Otteniamo questo risultato raddoppiando artificiosamente la lunghezza dell'esecuzione di M_y.

1. Se M_y non è completa, costruisco una macchina M'_y completa in cui ogni esecuzione o termina in accettazione o va in loop. Altrimenti, $M'_y = M_y$. Sappiamo dalla teoria che questa costruzione è sempre possibile; inoltre non viene intaccata la lunghezza dei cammini di accettazione (che resta quindi pari in M'_y se e solo se era pari in M_y).

2. A partire da M'_y (che assumiamo, per comodità, essere a nastro singolo), costruisco una macchina M''_y in cui ogni transizione t di M'_y tra

uno stato q_1 e uno stato q_2 è sdoppiata in due transizioni che passano attraverso uno stato intermedio q_t. Definisco quindi la funzione di transizione di stato di M''_y in questo modo:

per ogni transizione t del tipo $q_1 \xrightarrow{A/C,\langle N\rangle} q_2$ in M'_y (dove A è il carattere nella posizione corrente del nastro, C è il carattere che rimpiazza A e $N \in \{L, R, S\}$ è lo spostamento della testina), la macchina M''_y conterrà le due transizioni seguenti:

- $q_1 \xrightarrow{A/C,\langle S\rangle} q_t$,
- $q_t \xrightarrow{C/C,\langle N\rangle} q_2$.

In altre parole, ogni passo di M'_y corrisponde a due passi di M''_y. Lo stato iniziale e gli stati finali di M''_y sono gli stessi di M'_y (quindi nessuno degli stati q_t aggiunti in M''_y è finale o iniziale). Quindi, dato un generico ingresso x, M''_y o non termina oppure accetta con un'esecuzione di lunghezza pari e l'accettazione avviene se e solo se M'_y accetta x. Allora risolvere P su M''_y equivale a stabilire se M_y termina su x. Poiché quest'ultimo problema è notoriamente indecidibile, lo è anche P.

Parte II

Algoritmi e strutture dati

3
Ricorrenze, analisi e notazione asintotica

Esercizio 3.1.

1. Si consideri il seguente frammento di pseudocodice:

```
1  for i := 0 to n
2      P1(n)
```

dove P1(n) è definito nel seguente modo:

```
P1(n)
1  if n < 1
2      STAMPA(n)
3  for j := 1 to 3
4      P1(n/2)
5  for j := 1 to n
6      for k := 1 to n
7          STAMPA(j, k)
```

Dire quale è la complessità temporale del frammento di codice in funzione del parametro n.

2. Si consideri il seguente frammento di pseudocodice:

```
1  i := n
2  j := 1
3  while (i > 0)
4      P2(j)
5      i := i/3
6      j := j + 1
```

dove P2(n) è definito nel seguente modo:

```
P2(n)
1  if n < 1
2        STAMPA(n)
3  for l := 1 to 4
4        P2(n/2)
5  for k := 1 to n
6        for j := k + 1 to n
7              STAMPA(k, j)
```

Dire quale è la complessità temporale del frammento di codice in funzione del parametro n.

3. Sia dia la complessità del seguente algoritmo:

```
P3(n)
1  if n < 1
2        STAMPA(n)
3  for i := 1 to 9
4        P3(n/3)
5  P2(n)
```

dove P2(n) è definito come al punto 2.

NB: Tutte le variabili nei frammenti di codice precedenti sono da intendersi di tipo intero, quindi anche le operazioni su di esse sono da intendersi come operazioni sugli interi.

SOLUZIONE

1. P1 è un algoritmo ricorsivo la cui ricorrenza è $T(n) = 3T(\frac{n}{2}) + n^2$, la cui soluzione è, applicando il teorema dell'esperto, $T(n) = \Theta(n^2)$. Infatti, $f(n) = n^2$ è un polinomio, per cui è sufficiente confrontare il grado del polinomio, cioè 2, con $\log_b a = \log_2 3$; siccome vale $2 > \log_2 3$, siamo nel caso 3 del teorema dell'esperto, e la condizione aggiuntiva $af(\frac{n}{b} \leq cf(c)$ (per qualche $c < 1$ e n grande a sufficienza) è automaticamente verificata, per cui $T(n)$ è dell'ordine di grandezza di $f(n)$.

   Di conseguenza la complessità globale del frammento di pseudocodice è $\Theta(n \cdot n^2) = \Theta(n^3)$, in quanto P1 viene invocata n volte con parametro n.

2. In questo secondo caso la ricorrenza di P2 è $T(n) = 4T(\frac{n}{2}) + n^2$, che ha soluzione (sempre applicando il teorema dell'esperto) $T(n) = \Theta(n^2 \log(n))$ (come al punto precedente, $f(n)$ è un polinomio di grado 2, ma in questo caso abbiamo $\log_b a = \log_2 4 = 2$, per cui siamo nel caso 2 del teorema dell'esperto).

P2 viene invocata un numero $\log_3(n)$ di volte con valori crescenti del parametro, quindi il costo totale del frammento di pseudocodice è $\sum_{j=1}^{\log(n)} j^2 \log(j)$, che è $\Theta(\log^3(n)\log(\log(n)))$.
Per vedere che vale quest'ultima relazione, è sufficiente mostrare che, da un lato, vale $\sum_{j=1}^{\log(n)} j^2 \log(j) \leq \log^3(n)\log(\log(n))$ (quindi $\sum_{j=1}^{\log(n)} j^2 \log(j) = \mathcal{O}(\log^3(n)\log(\log(n)))$), e, dall'altro, che vale $\sum_{j=1}^{\log(n)} j^2 \log(j) \geq (\frac{\log(n)}{2})^3 \log(\frac{\log(n)}{2})$ (quindi abbiamo anche che $\sum_{j=1}^{\log(n)} j^2 \log(j) = \Omega(\log^3(n)\log(\log(n)))$). Si noti che $\sum_{j=1}^{\log(n)} j^2 \log(j) \leq \log(n)^3 \log(\log(n))$ è banalmente vera, in quanto vale $j \leq \log(n)$. Per mostrare invece che vale $\sum_{j=1}^{\log(n)} j^2 \log(j) \geq (\frac{\log(n)}{2})^3 \log(\frac{\log(n)}{2})$ si può notare che $\sum_{j=1}^{\log(n)} j^2 \log(j) \geq \sum_{j=\frac{\log(n)}{2}+1}^{\log(n)} j^2 \log(j) \geq (\frac{\log(n)}{2})^3 \log(\frac{\log(n)}{2})$ è vera, in quanto nell'ultima sommatoria vale sempre $j > \frac{\log(n)}{2}$.

3. In questo caso la ricorrenza di P3 è $T(n) = 9T(\frac{n}{3}) + n^2 \log_3(n)$ (si noti che, per semplificare i calcoli che seguono, usiamo $\log_3(n)$ invece che $\log(n)$, che è legittimo, in quanto il cambio di base non cambia l'ordine di grandezza del logaritmo). Tale ricorrenza non può essere risolta con il teorema dell'esperto, in quanto $n^2 \log_3(n)$ non è polinomialmente più grande di n^2. Tuttavia, tramite il metodo di sostituzione, è possibile mostrare che in questo caso vale $T(n) = \Theta(n^2 \log_3^2(n))$.
Per ottenere ciò, mostriamo innanzi tutto che vale $T(n) \leq cn^2 \log_3^2(n)$. Per l'ipotesi induttiva, abbiamo

$$\begin{aligned} T(n) &= 9T(\frac{n}{3}) + n^2 \log_3(n) \\ &\leq 9c(\frac{n}{3})^2 \log_3^2(\frac{n}{3}) + n^2 \log_3(n) \\ &= cn^2(\log_3^2(n) - 2\log_3(n) + \log_3^2(3)) + n^2 \log_3(n) \\ &= cn^2 \log_3^2(n) - 2cn^2 \log_3(n) + cn^2 + n^2 \log_3(n) \\ &\leq cn^2 \log_3^2(n) \end{aligned}$$

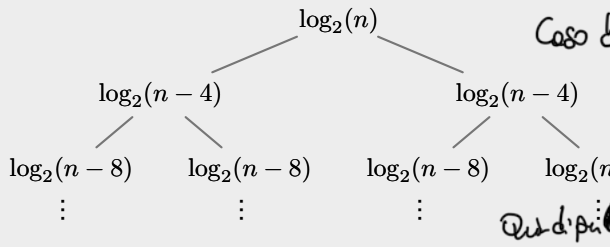
se è $-2cn^2 \log_3(n) + cn^2 + n^2 \log_3(n) \leq 0$, cioè se $c \geq \frac{\log_3(n)}{2\log_3(n)-1}$, che vale per $n \geq 3$ e $c \geq 1$. Inoltre, assumendo $T(1) = T(2) = 1$ come casi base, abbiamo che vale $T(3) = 9 + 9\log_3(3) = 9 \cdot 2$ e $T(9) = 81 \cdot 2 + 81 \cdot 2 \leq c81 \cdot 4$, che vale per $c \geq 1$.
Inoltre, si mostra, con calcoli analoghi a quelli sopra (con $\geq$ al posto di $\leq$), che vale $T(n) \geq cn^2 \log_3^2(n)$, se è $-2c\log_3(n) + c + \log_3(n) \geq 0$, che a sua volta è vero per $n \geq 1$ e $c \leq \frac{1}{2}$. In questo caso, abbiamo anche che vale $T(3) = 18 \geq \frac{1}{2} \cdot 9\log_3(3)$.

Esercizio 3.2. Trovare un limite asintotico superiore per la soluzione della seguente ricorrenza:

$$T(n) = 2T(n-4) + \Theta(\log(n))$$

SOLUZIONE

In questo caso la ricorrenza non ha una forma che permette di applicare il teorema dell'esperto (a destra, il termine in T è della forma $T(n-b)$ invece che $T(\frac{n}{b})$). Possiamo risolvere la ricorrenza formulando un'ipotesi tramite il metodo dell'albero di ricorsione, e poi verificarla con il metodo della sostituzione. Si noti che, per semplificare i conti, nel seguito usiamo $\log_2(n)$ invece che $\log(n)$, cosa che comunque è possibile in quanto vale $\log_2(n) = \Theta(\log(n))$. L'albero di ricorsione è il seguente:



Mediante il metodo dell'albero di ricorsione si ottiene l'ipotesi $\mathcal{O}(2^{\frac{n}{4}}\log_2(n))$; infatti, il costo di ogni livello è $\Theta(2^k \log_2(n-4k))$, e la profondità dell'albero è $k = \frac{n}{4}$. Poi si verifica che, con opportune costanti, vale $T(n) \leq c2^{\frac{n}{4}}\log_2(n)$.

In effetti, per arrivare a concludere che abbiamo $T(n) \leq c2^{\frac{n}{4}}\log_2(n)$, è utile mostrare un vincolo leggermente più stretto, e cioè che vale $T(n) \leq c2^{\frac{n}{4}}\log_2(n) - bn^2$. Infatti si ha:

$$\begin{aligned} T(n) &= 2T(n-4) + \log_2(n) \leq 2c2^{\frac{n-4}{4}}\log_2(n-4) - 2b(n-4)^2 + \log_2(n) \\ &= c2^{\frac{n}{4}}\log_2(n-4) - bn^2 - bn^2 + 16bn - 32 + \log_2(n) \\ &\leq (c2^{\frac{n}{4}}\log_2(n) - bn^2) - bn^2 + 16bn - 32 + \log_2(n) \leq c2^{\frac{n}{4}}\log_2(n) - bn^2 \end{aligned}$$

per b ed n tali che vale $-bn^2 + 16bn - 32 + \log_2(n) \leq 0$. Questo si ha, per esempio, per $n \geq 16$ e $b \geq 1$. Inoltre, prendendo come caso base $T(1) = T(2) = T(3) = T(4) = 1$, si ha che $T(8) = 2+3$, $T(12) = 10+\log_2(12) = 10+2+\log_2(3)$, $T(16) = 2(12+\log_2(3))+4 \leq c16\cdot 4 - 256b$, che è vero, per esempio, se $c \geq \frac{8b+1}{2}$ (perché in questo caso vale $c16 \cdot 4 - 256b \geq 32 \geq 2(12+\log_2(3))+4$).

Esercizio 3.3. Si consideri la seguente —inutile!— funzione, definita mediante pseudocodice, e che riceve in ingresso un numero intero n.

```
USELESS(n)
1  for i := 1 to n
2        AUX(i)
```

dove a sua volta la funzione AUX(m), che prende in ingresso un numero intero m, è definita nel modo seguente (ODD(M) restituisce `true` se m è un numero dispari, altrimenti restituisce `false`):

```
AUX(m)
1  if (m = 1)
2        return
3  else
4        if ODD(m)
5              AUX(m − 1)
6        else AUX(m/2)
```

Si valuti la complessità asintotica della funzione USELESS mediante la notazione $\mathcal{O}$, o, preferibilmente, mediante la notazione Θ.

SOLUZIONE

La funzione AUX ha una complessità che è soluzione della seguente equazione alle ricorrenze:

$$\begin{aligned} T(1) &= 1; \\ T(n) &= \text{if } \texttt{odd(n)} \text{ then } 1 + T(n-1) \\ &\qquad \text{else } 1 + T\left(\frac{n}{2}\right) \end{aligned}$$

Consideriamo due casi estremi:

a) $n = 2^k$ per qualche $k \in \mathbb{N}$; in tal caso, abbiamo

$$T(n) = 1 + T\left(\frac{n}{2}\right) = 1 + 1 + T\left(\frac{n}{4}\right) = 1 + 1 + 1 \ldots 1 + T(1) = 1 \cdot k$$

quindi $T(n)$ è $\mathcal{O}(\log(n))$.

b) n è dispari, $\frac{n-1}{2}$ è pure dispari e così via finché la ricorsione si chiude; in tal caso vale

$$\begin{aligned} T(n) &= 1 + T(n-1) = 1 + 1 + T\left(\frac{n-1}{2}\right) = 1 + 1 + 1 + T\left(\frac{n-3}{2}\right) \\ &= 1 + 1 + 1 + 1 + T\left(\frac{n-3}{4}\right) \ldots \end{aligned}$$

che è $\mathcal{O}(2\log(n)) = \mathcal{O}(\log(n))$. Quindi AUX ha una complessità $\mathcal{O}(\log(n))$ (e anche $\Theta(\log(n))$). USELESS chiama n volte AUX con parametro i che va da

1 a n; quindi per USELESS vale $T(n) \leq \sum_{i=1}^{n} c\log(i)$ che è $\mathcal{O}(n\log(n))$. Vale anche $T(n) \geq \sum_{i=\frac{n}{2}}^{n} c\log(i) \geq \frac{n}{2}\log(\frac{n}{2})$, che è $\Omega(n\log(n))$.

Quindi $T(n)$ è $\Theta(n\log(n))$.

Esercizio 3.4. Si risolva la seguente equazione alle ricorrenze:

$$T(n) = T(n-1) + n^2$$

SOLUZIONE

È facile sviluppare $T(n)$ in $n^2 + (n-1)^2 + (n-2)^2 + (n-3)^2 \ldots + c$ ossia $T(n) = \sum_{i=1}^{n} i^2$, che è $\Theta(n^3)$. Infatti, valgono le seguenti relazioni: $\sum_{i=1}^{n} i^2 \leq n^3$ e $\sum_{i=1}^{n} i^2 \geq (\frac{n}{2})^2(\frac{n}{2}) = \frac{n^3}{8}$.

Esercizio 3.5. Si calcoli la complessità del seguente frammento di codice:

```
PROC(n)
1  if (n < 5)
2        return 0
3  s := PROC(2 * n/5)
4  s := s + PROC(3 * n/5)
5  i := 1
6  while (i <= 3 * n)
7        s := s + 1
8        i := i + 2
9  return s
```

SOLUZIONE

Lo pseudocodice ricorsivo dell'esercizio ha una funzione di complessità definita dalla seguente ricorrenza:

$$T(n) = T\left(\frac{2}{5}n\right) + T\left(\frac{3}{5}n\right) + \Theta(n)$$

che si risolve mediante il metodo di sostituzione, dopo avere fatto l'ipotesi che la soluzione sia $\Theta(n\log(n))$ (tale ipotesi può essere ottenuta mediante l'albero di ricorsione, in modo analogo a quanto descritto in [2, capitolo 4] per la ricorrenza $T(n) = T(\frac{n}{3}) + T(\frac{2}{3}n) + \Theta(n)$).

Più precisamente, abbiamo quanto segue:

$$T(n) = T\left(\frac{2}{5}n\right) + T\left(\frac{3}{5}n\right) + dn \leq \frac{2}{5}cn\log\left(\frac{2}{5}n\right) + \frac{3}{5}cn\log\left(\frac{3}{5}n\right) + dn$$

$$
\begin{aligned}
&= \frac{2}{5}cn\left(\log(n) - \log\left(\frac{5}{2}\right)\right) + \frac{3}{5}cn\left(\log(n) - \log\left(\frac{5}{3}\right)\right) + dn \\
&= cn\log(n) - cn\left(\frac{2}{5}\log\left(\frac{5}{2}\right) + \frac{3}{5}\log\left(\frac{5}{3}\right)\right) + dn \\
&= cn\log(n) - cn\left(\log(5) - \frac{2}{5}\log(2) - \frac{3}{5}\log(3)\right) + dn \\
&\leq cn\log(n)
\end{aligned}
$$

se vale $-cn\left(\log(5) - \frac{2}{5}\log(2) - \frac{3}{5}\log(3)\right) + dn \leq 0$, che è vero se è $c \geq \frac{d}{\left(\log(5) - \frac{2}{5}\log(2) - \frac{3}{5}\log(3)\right)}$ (cioè, siccome abbiamo $\frac{1}{2} < \log(5) - \frac{2}{5}\log(2) - \frac{3}{5}\log(3) < 1$, $c \geq 2$ se è $d = 1$). Inoltre, ponendo come casi base $T(1) = T(2) = 1$, si ha che vale (usando $c = 2$ e $d = 1$) $T(3) = 2 + 3 \leq 2 \cdot 3\log(3)$.

In maniera analoga (con conti analoghi a quelli sopra, scambiando $\leq$ con $\geq$) si mostra che vale $T(n) \geq cn\log(n)$ se è $c \leq \frac{d}{\left(\log(5) - \frac{2}{5}\log(2) - \frac{3}{5}\log(3)\right)}$ (che è vero se vale $c \leq 1$ e $d = 1$).

Esercizio 3.6. Risolvere la seguente ricorrenza:

$$T(n) = 9T\left(\frac{n}{10}\right) + n\log(n)$$

SOLUZIONE

La ricorrenza si risolve applicando il teorema dell'esperto. Siamo nel caso 3 del teorema, per cui occorre verificare la condizione aggiuntiva, $af(\frac{n}{b}) \leq cf(n)$, con $f(n) = n\log(n)$. La condizione è verificata, in quanto diventa $\frac{9n}{10}\log(\frac{n}{10}) \leq cn\log(n)$, cioè $\frac{9n}{10}\log(n) - \frac{9n}{10}\log(10) \leq cn\log(n)$, che è banalmente verificata prendendo, per esempio, $c = \frac{99}{100}$. Di conseguenza, la soluzione è $T(n) = \Theta(n\log(n))$.

Esercizio 3.7. Si risolva la seguente ricorrenza:

$$T(n) = 8T(\frac{n}{6}) + n^{\frac{3}{2}}\log^2(n)$$

SOLUZIONE

La ricorrenza si risolve applicando semplicemente il teorema dell'esperto. Si ha che $\log_b a = \log_6 8$, quindi $n^{\log_6 8}$, che vale circa $n^{1.16}$, è polinomialmente più piccolo di $n^{\frac{3}{2}}\log^2(n)$. Siamo quindi nel caso 3 del teorema, e si ha che la condizione aggiuntiva $af(\frac{n}{b}) \leq cf(n)$ (con $f(n) = n^{\frac{3}{2}}\log^2(n)$) è verificata già per $c = 1$. Quindi, si ha anche che $T(n) = \Theta(n^{\frac{3}{2}}\log^2(n))$.

Esercizio 3.8. Si calcoli la complessità del seguente frammento di codice:

```
FUN(A)
1  n := A.length
2  res := 0
3  if (n > 5)
4        res := res + FUN(A[1..n/3]) + FUN(A[n/3 + 1..2 * n/3])
                    + FUN(A[2 * n/3 + 1..n])
5        res := res + FUN(A[n/6 + 1..n/2]) + FUN(A[n/2 + 1..5 * n/6])
6  for i := 1 to n
7        res := res + A[i]
8  return res
```

SOLUZIONE

La ricorrenza corrispondente all'algoritmo è $T(n) = 5T(\frac{n}{3}) + n$, che si risolve con il teorema dell'esperto (caso 1), e la cui soluzione è $\Theta(n^{\log_3 5})$. Infatti, in questo caso si ha che $f(n) = n$ è un polinomio di grado 1 per cui, per capire in quale caso del teorema dell'esperto ci troviamo, è sufficiente confrontare il grado di $f(n)$ con $\log_b a = \log_3 5 > 1$. Sono quindi soddisfatte le condizioni del caso 1 del teorema dell'esperto.

Esercizio 3.9. Si consideri la seguente ricorrenza.

$$T(n) = T(n-1) + T(n-2) + n^2$$

Indicare una funzione $g(n)$ per cui $T(n) = \mathcal{O}(g(n))$, spiegando come si è arrivati ad individuare $g(n)$.

SOLUZIONE

Applicando il Teorema 1 di [1], che copre il calcolo della complessità di ricorrenze lineari di ordine costante, cioè della forma $\sum_{i=1}^{h} a_i T(n-i) + cn^k$, dove abbiamo $h = 2$, $k = 2$ e $a = \sum_{i=1}^{h} a_i = 2$, si ha direttamente che vale $T(n) = O(n^2 2^n)$.

Esercizio 3.10. Si fornisca un limite asintotico superiore per l'espressione $T(n)$ definita dalla seguente equazione di ricorrenza:

$$T(n) = T\left(\frac{n}{2}\right) + n(2 + \sin(n)).$$

SOLUZIONE

Nonostante $f(n)$ (che in questo caso è $n(2 + \sin(n))$) sia polinomialmente più grande di $n^{log_b a} = n^{log_2 1} = n^0 = 1$, il teorema dell'esperto non si può applicare poiché è violata la condizione (di "regolarità") $af\left(\frac{n}{b}\right) \leq cf(n)$, che

dovrebbe applicarsi in questo caso (saremmo nel caso 3 del teorema):

$$\frac{n}{2}\left(2+\sin\left(\frac{n}{2}\right)\right) \leq cn(2+\sin(n))$$

per n grande e $c<1$. Infatti, le curve $\frac{n}{2}(2+sin\left(\frac{n}{2}\right))$ e $n(2+sin(n))$ oscillano e si intersecano continuamente, quindi a maggior ragione la relazione non vale moltiplicando $f(n)=n(2+\sin(n))$ per $c<1$.

Tuttavia, possiamo notare come l'espressione $f(n)=n(2+sin(n))$ sia maggiorata da $3n$, quindi vale $T(n)\leq T'(n)$, dove $T'(n)$ è definita dalla ricorrenza $T'(n)=T'(\frac{n}{2})+3n$. Possiamo risolvere la ricorrenza di $T'(n)$ con il teorema dell'esperto, ricadendo nel caso 3 e ottenendo $T'(n)=\Theta(n)$. Ipotizziamo allora che valga $T(n)=\mathcal{O}(n)$, ossia $T(n)\leq dn$, e procediamo per sostituzione:

$$T(n)=T\left(\frac{n}{2}\right)+n(2+\sin(n)) \leq d\frac{n}{2}+n(2+\sin(n)) \leq d\frac{n}{2}+3n$$

La relazione $d\frac{n}{2}+3n\leq dn$ è verificata per $d\geq 6$ (e $n>0$). Occorre poi verificare la condizione iniziale, che è soddisfatta, ad esempio, con $T(1)=1$.

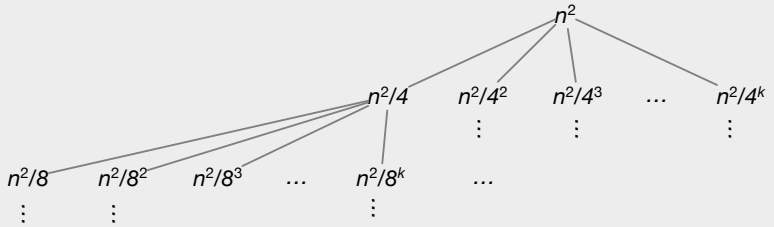
Esercizio 3.11. Trovare un limite superiore per la seguente ricorrenza:

$$T(n)=T\left(\lfloor\frac{n}{2}\rfloor\right)+T\left(\lfloor\frac{n}{4}\rfloor\right)+T\left(\lfloor\frac{n}{8}\rfloor\right)+\ldots+T\left(\lfloor\frac{n}{2^k}\rfloor\right)+n^2$$

dove k è una costante intera maggiore di 1 e n è una potenza di 2.

SOLUZIONE

Possiamo risolvere la ricorrenza usando il metodo della sostituzione. Per formulare l'ipotesi da verificare, possiamo disegnare il seguente albero di ricorsione.



Si noti che vale $\sum_{i=1}^{k}\frac{1}{4^k}<\sum_{i=1}^{\infty}\frac{1}{4^k}=\frac{1}{1-\frac{1}{4}}-1=\frac{1}{3}$, in quanto è noto (si veda [2, Appendice A]) che abbiamo $\sum_{i=0}^{\infty}x^k=\frac{1}{1-x}$ se vale $|x|<1$, e abbiamo anche che $\sum_{i=1}^{\infty}x^k=\frac{1}{1-x}-1=\frac{x}{1-x}$. Analogamente, abbiamo $\sum_{i=1}^{k}\frac{1}{8^k}<\frac{1}{7}$, $\sum_{i=1}^{k}\frac{1}{16^k}<\frac{1}{15}$, e così via.

Si noti che $\frac{1}{3} + \frac{1}{7} + \frac{1}{15} \ldots$ è maggiorata da $\sum_{i=1}^{\infty} \frac{1}{2^i} = 1$. Possiamo quindi intuire che la somma di tutti i termini dell'albero di ricorsione è maggiorata da cn^2, di conseguenza facciamo l'ipotesi che valga $T(n) \leq cn^2$, per una qualche costante positiva c, e procediamo a verificare l'ipotesi. Otteniamo:

$$\begin{aligned}
T(n) &= n^2 + T\left(\lfloor\frac{n}{2}\rfloor\right) + T\left(\lfloor\frac{n}{4}\rfloor\right) + T\left(\lfloor\frac{n}{8}\rfloor\right) + \ldots + T\left(\lfloor\frac{n}{2^k}\rfloor\right) \\
&= n^2 + \sum_{i=1}^{k} T\left(\lfloor\frac{n}{2^i}\rfloor\right) \\
&\leq n^2 + \sum_{i=1}^{k} c\frac{n^2}{4^i} \\
&= n^2\left(1 + \sum_{i=1}^{k} \frac{c}{4^i}\right) \\
&= n^2\left(1 + c\sum_{i=1}^{k} \frac{1}{4^i}\right) \\
&< n^2\left(1 + \frac{c}{3}\right)
\end{aligned}$$

L'espressione ottenuta soddisfa l'ipotesi se vale $n^2(1 + \frac{c}{3}) \leq cn^2$, ossia se vale (considerando $n \geq 1$) $c \geq \frac{3}{2}$.

Per verificare il caso base, poniamo $T(n) = 1$ per tutti gli $n < 2^k$. Notiamo che vale $T(1) = 1 \leq c \cdot 1^2$ per $c \geq \frac{3}{2}$, e, analogamente, $T(2) = 1 \leq c \cdot 2^2$, …, $T(2^{k-1}) = 1 \leq c \cdot 2^{2k-2}$.

In definitiva, vale $T(n) = O(n^2)$.

Esercizio 3.12. Si consideri il seguente algoritmo, descritto in pseudocodice, che prende in ingresso un array A di numeri interi:

```
FUN(A)
1  if A.length < 5
2        return FUN1(A)
3  a := A.length − 1
4  FUN(A[0 .. a − 1])
5  FUN(A[1 .. a])
6  FUN2(A)
```

Si supponga che FUN1 abbia complessità $\Theta(n)$ e FUN2 abbia complessità $\Theta(n^2)$, dove per entrambe le funzioni n è la lunghezza dell'array passato come argomento.

1. Scrivere la ricorrenza che definisce la complessità temporale $T_{\text{FUN}}(n)$ di FUN.

2. Dare un limite superiore asintotico per la funzione $T_{\text{FUN}}(n)$, giustificando opportunamente la risposta.

SOLUZIONE

Si ottiene una ricorrenza $T(n) = \Theta(1)$, per $n < 5$, altrimenti $T(n) = 2T(n-1) + \Theta(n^2)$. Da [1, Teorema 1] abbiamo la seguente maggiorazione: $T(n) = \mathcal{O}(n^2 2^n)$.

In aggiunta, è possibile dimostrare un limite asintotico più stretto, ossia $T(n) = O(2^n)$. Si può facilmente verificare che dimostrare $T(n) \leq c2^n$ risulta complicato, a causa del termine $\Theta(n^2)$. Proviamo quindi a dimostrare che vale $T(n) \leq c2^n - bn^2$, che ovviamente implica che $T(n) \leq c2^n$ vale.

La nostra ipotesi induttiva è dunque $T(n-1) \leq c2^{n-1} - b(n-1)^2$. Sostituendola nella ricorrenza, otteniamo:

$$\begin{aligned} T(n) &= 2T(n-1) + dn^2 \\ &\leq 2(c2^{n-1} - b(n-1)^2) + dn^2 \\ &= c2^n - 2bn^2 - 2b + 4bn + dn^2 \\ &\leq c2^n - 2bn^2 + 4bn + dn^2 \\ &\leq c2^n - 2bn^2 + \frac{b}{2}n^2 + dn^2 \\ &= c2^n - \frac{3}{2}bn^2 + dn^2 \leq c2^n - bn^2 \end{aligned}$$

L'ultima maggiorazione è valida se vale $-\frac{3}{2}b + d \leq -b$, cioè se vale $d \leq \frac{b}{2}$, quindi se $b \geq 2d$. Consideriamo la penultima maggiorazione, in cui si sfrutta la condizione $4bn \leq \frac{b}{2}n^2$. Innanzi tutto, notiamo che si potevano sfruttare anche altre condizioni, per esempio $4bn \leq \frac{b}{5}n^2$, in quel punto; la scelta di usare l'espressione $\frac{b}{2}n^2$ deriva dalla volontà di fare comparire un termine della forma pbn^2 che sia più piccolo di $2bn^2$ (in modo che $-2bn^2 + pbn^2$ sia negativo), ma che sia anche tale che $-2bn^2 + pbn^2$ sia più piccolo di $-bn^2$, per poter imporre opportuni vincoli su b e d in modo da poter soddisfare la condizione finale di essere $\leq c2^n - bn^2$. La condizione $4bn \leq \frac{b}{2}n^2$ (cioè $4 \leq \frac{n}{2}$) è valida per un valore di n sufficiente grande, ed in particolar modo per $n \geq 8$.

Di conseguenza, consideriamo come caso base $n = 8$. Notiamo che per la validità della dimostrazione non è necessario imporre vincoli sulla costante c, quindi possiamo sceglierla grande a sufficienza per verificare la tesi per $n = 8$, ovvero che $T(8) \leq c2^8 - b8^2 = 256c - 64b$.

Si poteva anche direttamente notare, senza effettuare ulteriori maggiorazioni, che vale

$$c2^n - 2bn^2 + 4bn + dn^2 \leq c2^n - bn^2$$

se, e solo se, vale

$$-bn + 4b + dn \leq 0$$

e quest'ultima disuguaglianza è verificata per $b > d$ e $n \geq \frac{4b}{b-d}$.

Esercizio 3.13.

1. Indicare la complessità asintotica della seguente funzione $\mathrm{F}(n)$ (si rammenta che in questo caso si fa riferimento al criterio di costo costante).

```
F(n)
1  sum := 0
2  for i := 1 to n
3        for j := 1 to i^2
4              if j mod i = 0
5                    for k := 1 to j
6                          sum := sum + 1
7  return sum
```

2. È possibile riscrivere la funzione $\mathrm{F}(n)$ in modo che, a parità di risultato calcolato, la sua complessità asintotica sia inferiore?

NB: L'operazione $a \mod b$ restituisce il resto della divisione tra gli interi a e b.

SOLUZIONE

1. La complessità è $\Theta(n^4)$. Infatti, il ciclo più esterno esegue $\Theta(n)$ iterazioni. Il secondo ciclo ne esegue $\Theta(i^2)$ (che sono quindi dell'ordine di grandezza di $\Theta(n^2)$), per ogni iterazione i del primo ciclo. Il terzo ne esegue $\Theta(j)$, ossia $\Theta(i^2)$, ma viene eseguito solo ogni i iterazioni del secondo ciclo, quindi complessivamente esegue $\Theta(i)$ (ossia $\Theta(n)$) iterazioni per ogni iterazione del secondo ciclo. L'iterazione interna ha una complessità di $\Theta(1)$. Complessivamente abbiamo $\Theta(n) \cdot \Theta(n^2) \cdot \Theta(n) = \Theta(n^4)$.

   Un calcolo più preciso (che aiuta anche a svolgere i passi successivi) è il seguente. Come detto sopra, il ciclo più interno viene eseguito j volte, ma solo quando j è un multiplo di i. Quindi, esso viene eseguito quando $j = i, 2i, 3i, \ldots, i \cdot i$. Possiamo quindi scrivere la seguente sommatoria, per calcolare quante volte viene eseguita l'istruzione di incremento di sum:

$$\sum_{i=1}^{n} \sum_{t=1}^{i} \sum_{k=1}^{t \cdot i} 1.$$

   Ora, la sommatoria più interna chiaramente vale $t \cdot i$ e si ha che

$$\sum_{t=1}^{i} t \cdot i = i \frac{i(i+1)}{2} = \frac{i^2(i+1)}{2} \tag{3.1}$$

sfruttando la nota formula di Gauss, secondo cui vale la seguente uguaglianza:

$$\sum_{t=1}^{i} t = \frac{i(i+1)}{2}.$$

Infine, la sommatoria completa ha la forma

$$\sum_{i=1}^{n} \frac{i^2(i+1)}{2}$$

che è dell'ordine di grandezza di $\Theta(n^4)$. Questo si può vedere anche ricordando che sono noti i seguenti risultati (si veda anche [2, Appendice A]):

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \tag{3.2}$$

$$\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4} \tag{3.3}$$

2. La parte ciclica della funzione si può riscrivere come segue, abbassando la complessità a $\Theta(n^2)$.

```
for i := 1 to n
    j := i
    while j ≤ i * i
        sum := sum + j
        j := j + 1
```

Una riscrittura equivalente, ottenuta raccogliendo il fattore i dal secondo ciclo, è la seguente:

```
for i := 1 to n
    for k := 1 to i
        sum := sum + k * i
```

che ha complessità $\sum_{k=1}^{i} k \cdot i = i\frac{i(i+1)}{2}$ (come già calcolato in (3.1)). Questo consente di sostituire il ciclo interno con una formula esplicita. La riscrittura seguente porta a una complessità di $\Theta(n)$.

```
for i := 1 to n
    sum := sum + i * (i * (i + 1)/2)
```

Si può ricavare una semplificazione estrema mediante le formule per la somma di quadrati e per la somma di cubi riportate in (3.2) e (3.3), rispettivamente. Abbiamo quindi che vale la formula seguente

$$\sum_{i=1}^{n} \frac{i(i+1)}{2} i = \frac{1}{2} \sum_{i=1}^{n} (i^3 + i^2) = \frac{n(n+1)(4n+2+3n^2+3n)}{2 \cdot 12}$$
$$= \frac{n(n+1)(3n^2+7n+2)}{24}$$

ottenendo così un'unica formula (polinomio di quarto grado in n) che dà direttamente il risultato.

return $n * (n+1) * (3 * n * n + 7 * n + 2)/24$

La complessità risultante è chiaramente $\Theta(1)$.

Esercizio 3.14. Sia $T[1\,..\,n][1\,..\,n]$ una matrice quadrata di dimensione $n \times n$. Si consideri la seguente procedura $\mathrm{F}(T, n)$.

```
F(T, n)
 1  v := ⌊n/2⌋
 2  if v ≤ 1
 3        return T
 4  w := ⌈n/2⌉
 5  if w = v
 6        w := w + 1
 7  A := T[1..v][1..v]
 8  A1 := F(A, v)
 9  B := T[w..n][w..n]
10  B1 := F(B, n − w + 1)
11  for i := 1 to n
12        for j := 1 to n
13              if i ≤ v and j ≤ v
14                    T[i][j] := A1[i][j]
15              elseif i ≥ w and j ≥ w
16                    T[i][j] := B1[i − w + 1][j − w + 1]
17              else T[i][j] := T[i][j] * T[i][j]
18  return T
```

1. Scrivere la ricorrenza associata alla funzione $\mathrm{F}(T, n)$ definita sopra.
2. Si valuti la complessità temporale della funzione $\mathrm{F}(T, n)$ risolvendo la ricorrenza definita al punto 1.

SOLUZIONE

Si denoti con $N = n^2$ il numero di elementi della matrice T. La complessità della procedura è espressa dalla seguente ricorrenza (per evitare ambiguità con il nome della matrice T, nella ricorrenza indichiamo con T_{F} il tempo necessario per eseguire la funzione F):

$$T_{\text{F}}(N) = 2T_{\text{F}}\left(\frac{N}{4}\right) + \Theta(N)$$

Per risolvere la ricorrenza è possibile applicare il terzo caso del teorema dell'esperto, ottenendo quindi $T_{\text{F}}(N) = \Theta(N) = \Theta(n^2)$.

Esercizio 3.15. Si considerino le seguenti funzioni:

```
FUNC1(n)
1  k := 0
2  i := 0
3  while i ≤ n
4        k := k + 1
5        i := i + k
6        (∗)
7  return k
```

```
FUNC2(m)
1  if m ≤ 1
2        return m
3  j := 1
4  while j ≤ m
5        j := j ∗ 3
6  return FUNC2(m/3) + FUNC2(m/3)
```

Si calcoli la complessità di FUNC1(n) quando al posto di $(*)$ si trovano le seguenti istruzioni:

1. FUNC2(10^5)
2. FUNC2(n)

SOLUZIONE

Il valore di i all'inizio della j-esima iterazione del ciclo in FUNC1 non è mai influenzato dall'esecuzione della funzione FUNC2. Esso è pari a $1 + 2 + 3 + 4 + 5 + \ldots + (j-1)$, e quindi è dell'ordine di j^2. Quindi, il ciclo viene eseguito un numero di volte che è $\Theta(\sqrt{n})$.

Calcoliamo innanzi tutto la complessità di FUNC2(m). Essa è data dalla seguente ricorsione: $T(m) = 2T\left(\frac{m}{3}\right) + \log_3(m)$. La ricorsione si risolve tramite il teorema dell'esperto, in quanto $\log_3(m)$ è polinomialmente più piccola di $m^{\log_3 2}$. Siamo quindi nel caso 1 del teorema dell'esperto, e la complessità di FUNC2(m) è $\Theta(m^{\log_3 2})$.

Nel caso 1 l'istruzione $(*)$ ha costo costante, indipendente da n. Quindi, in questo caso la complessità del codice è data dal numero di iterazioni del ciclo, ed è $\Theta(\sqrt{n})$.

Nel caso 2 a ogni iterazione del ciclo il costo dell'istruzione $(*)$ è $\Theta(n^{\log_3 2})$, quindi il costo totale è $\Theta\left(n^{\frac{1}{2}+\log_3 2}\right)$.

Esercizio 3.16. Si considerino le seguenti ricorrenze:

1. $T(n) = 16T\left(\frac{n}{2}\right) + n^5 \log(n)$
2. $T(n) = 4T\left(\frac{n}{64}\right) + \sqrt[4]{n}$
3. $T(n) = 2T\left(\frac{n}{2}\right) + (n\cos(n)\log(n))^2$

Per ogni ricorrenza, indicare un *limite asintotico superiore* per la funzione $T(n)$.

SOLUZIONE

Le prime due ricorrenze si risolvono con il teorema dell'esperto.

Più precisamente, nella prima ricorrenza si ha che $\log_b a = \log_2 16 = 4$, e n^4 è polinomialmente più piccolo di $n^5 \log(n)$, per cui siamo nel caso 3. In questo caso occorre anche verificare la condizione aggiuntiva, cioè che $af(\frac{n}{b}) \leq cf(n)$ vale per qualche $c < 1$ e n grande a sufficienza. Questo corrisponde a verificare che valga $16(\frac{n}{2})^5 \log(\frac{n}{2}) \leq cn^5 \log(n)$, cioè che valga $\frac{n^5}{2}(\log(n) - \log(2)) \leq cn^5 \log(n)$, che è banalmente verificata, ad esempio, per $c = \frac{3}{4}$ e $n \geq 1$. Quindi, vale $T(n) = \Theta(f(n)) = \Theta(n^5 \log(n))$ (quindi ovviamente anche $T(n) = \mathcal{O}(n^5 \log(n))$).

Nel caso della seconda ricorrenza, invece, $f(n)$ è della forma n^k, per cui basta confrontare k con $\log_b a$. Si ha che $\log_b a = \log_{64} 4 = \frac{1}{3} > \frac{1}{4} = k$, per cui siamo nel caso 1, e vale $T(n) = \Theta(n^{\log_b a}) = \Theta(\sqrt[3]{n})$ (e quindi $T(n) = \mathcal{O}(\sqrt[3]{n})$).

Nel caso della terza ricorrenza, vale $f(n) = (n\cos(n)\log(n))^2$, quindi $f(n)$ oscilla tra 0 e $(n\log(n))^2$, e quindi non è né una maggiorazione, né una minorazione di $n^{\log_b a} = n^{\log_2 2} = n$. Non si può quindi applicare il teorema dell'esperto. D'altro canto, è facile vedere che $T(n)$ è maggiorata da $(n\log(n))^2$. Infatti si ha che, per ogni n, vale $T(n) \leq T'(n) = 2T'(\frac{n}{2}) + (n\log(n))^2$; questo può facilmente essere visto per induzione, notando che valgono sia $T(\frac{n}{2}) \leq T'(\frac{n}{2})$, sia $f(n) \leq (n\log(n))^2$. Applicando il teorema dell'esperto (caso 3), si ottiene che $T'(n) = \mathcal{O}((n\log(n))^2)$; infatti anche in questo caso la condizione aggiuntiva $af(\frac{n}{b}) \leq cf(n)$, cioè $2(\frac{n}{2})^2 \log^2(\frac{n}{2}) \leq cn^2 \log^2(n)$ (o, equivalentemente, $\frac{n^2}{2}(\log^2(n) - 2\log(n) + \log^2(2)) \leq cn^2 \log^2(n)$), vale per $c = \frac{3}{4}$ e $n > 1$. Di conseguenza, vale anche $T(n) = \mathcal{O}((n\log(n))^2)$.

Esercizio 3.17. Si consideri la seguente funzione G, che riceve un array di interi A e restituisce un intero:

```
G(A)
 1  if A.length ≤ 1
 2       return 1
 3  k := G(A[1..n/2])
 4  j := G(A[n/2 + 1..n])
 5  for h := 1 to A.length
 6       i := 1
 7       while i ≤ A.length
 8            i := i * 2
 9            k := k + i * A[h]
10            j := j − i * A[h]
11  return (k + j)/2
```

Detta n la lunghezza dell'array A,

1. si scriva la ricorrenza associata al codice della funzione;
2. si fornisca un limite asintotico superiore (possibilmente stretto) per la complessità temporale di $\textsc{g}(A)$ in funzione di n.

SOLUZIONE

La ricorrenza associata al codice della funzione è:

$$T(n) = \begin{cases} \Theta(1) \text{ se } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n \log(n)) \text{ se } n > 1 \end{cases}$$

o anche

$$T(n) = \begin{cases} 1 \text{ se } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + n \log_2(n) \text{ se } n > 1 \end{cases}$$

La funzione $f(n) = n \log_2(n)$ cresce più velocemente di $\Theta(n^{\log_2 2}) = \Theta(n)$, ma non polinomialmente più velocemente. Pertanto non si può applicare il teorema dell'esperto.

Osserviamo che $T(n)$ cresce più velocemente di $T'(n) = 2T'\left(\frac{n}{2}\right) + n$ ma meno di $T''(n) = 2T''\left(\frac{n}{2}\right) + n^{1+\epsilon}$, per ogni $\epsilon > 0$ (le relazioni $T(n) \geq T'(n)$ e $T(n) \leq T''(n)$ si mostrano per banale induzione). Pertanto valgono sia $T(n) \in \Omega(n \log n)$ che $T(n) \in O(n^{1+\epsilon})$. Intuitivamente, ha quindi senso ipotizzare una complessità che sia più alta di $\Theta(n \log n)$, ma non polinomialmente più alta.

Formuliamo allora l'ipotesi che valga $T(n) \leq cn(\log_2 n)^2$ e procediamo per sostituzione.

$$\begin{aligned}
T\left(\frac{n}{2}\right) &\leq c\frac{n}{2}\left(\log_2\left(\frac{n}{2}\right)\right)^2 && \text{Ipotesi induttiva} \\
T(n) &\leq cn\left(\log_2\left(\frac{n}{2}\right)\right)^2 + n\log_2(n) && \text{Sostituzione} \\
cn\left(\log_2\left(\frac{n}{2}\right)\right)^2 + n\log_2(n) &\leq cn(\log_2 n)^2 && \text{Da verificare} \\
c\left(\log_2\left(\frac{n}{2}\right)\right)^2 + \log_2(n) &\leq c(\log_2 n)^2 && (n > 0) \\
c((\log_2 n)^2 - \left(\log_2\left(\frac{n}{2}\right)\right)^2) &\geq \log_2(n) \\
c((\log_2 n)^2 - (\log_2 n - 1)^2) &\geq \log_2(n) \\
c(2\log_2 n - 1) &\geq \log_2(n)
\end{aligned}$$

Quest'ultima relazione è sempre soddisfatta per $n > 1$ e $c \geq 1$.

Nel caso base, assumendo che valga $T(1) = 1$, notiamo che si ha che $c1(\log_2(1))^2 = 0 \not\geq T(1) = 1$. Tuttavia, è sufficiente mostrare che la disuguaglianza vale da un certo n in poi, per cui tramite la ricorrenza otteniamo $T(2) = 2T(1) + 2\log_2(2) = 4$ e $T(3) = 2T(1) + 3\log_2(3) = 2 + 3\log_2(3)$; inoltre $4 \leq c2(\log_2 2)^2$ e $2 + 3\log_2(3) \leq c3(\log_2 3)^2$ valgono entrambe per $c \geq 2$, e per $n > 3$ la ricorrenza non dipende più da $T(1)$. La relazione è soddisfatta anche nel caso base e quindi si ha che vale $T(n) = O(n(\log n)^2)$.

Si dimostra analogamente che $T(n) \geq dn(\log_2 n)^2$. Questa relazione è soddisfatta se

$$d(2\log_2 n - 1) \leq \log_2(n)$$

il che vale, ad esempio, per $d \leq 1/2$ e $n \geq 1$. Nel caso base vale $T(1) = 1 \geq d1(\log_2(1))^2 = 0$. Quindi vale $T(n) = \Omega(n(\log n)^2)$.

Complessivamente, abbiamo che vale $T(n) = \Theta(n(\log n)^2)$.

Un altro modo per risolvere la ricorrenza consiste nell'effettuare un cambio di variabile, ponendo $m = \log_2 n$:

$$\begin{aligned}
T(2^m) &= 2T(2^{m-1}) + 2^m m && \text{Pongo } m = \log_2 n, \text{ quindi } n = 2^m \\
T(2^m)/2^m &= 2T(2^{m-1})/2^m + m && \text{Divido per } 2^m \\
S(m) &= 2T(2^{m-1})/2^m + m && \text{Definisco } S(m) = T(2^m)/2^m \\
S(m) &= S(m-1) + m && \text{Quindi } S(m-1) = 2T(2^{m-1})/2^m \\
S(m) &= S(m-2) + m - 1 + m && \text{Sostituisco} \\
&\vdots \\
S(m) &= S(0) + 1 + 2 + \ldots + m && \text{Con } S(0) = T(2^0)/2^0 = 1 \\
S(m) &= \Theta(m^2) \\
T(2^m)/2^m &= \Theta(m^2) \\
T(n)/n &= \Theta((\log n)^2) \\
T(n) &= \Theta(n(\log n)^2)
\end{aligned}$$

Esercizio 3.18. Con la notazione $f(\mathcal{O}(n))$ indichiamo l'insieme delle funzioni $\{f(g(n)) \mid g(n) \in \mathcal{O}(n)\}$, cioè che sono date dalla composizione di una funzione f e di una funzione g, laddove la funzione g appartiene all'insieme $\mathcal{O}(n)$. Per esempio, la funzione $\frac{1}{4n^3+n}$ appartiene all'insieme $\frac{1}{\mathcal{O}(n^3)}$.

Si dica, giustificando opportunamente le risposte, se valgono le seguenti uguaglianze:

a) $\log(\mathcal{O}(n)) = \mathcal{O}(\log(n))$

b) $2^{\mathcal{O}(n)} = \mathcal{O}(2^n)$

c) $(\mathcal{O}(n))^k = \mathcal{O}(n^k)$

SOLUZIONE

a) Falsa. Si consideri ad esempio $f(n) = 2\log(n)$. Abbiamo chiaramente che vale $f(n) \in \mathcal{O}(\log(n))$. Tuttavia, vale anche $f(n) \notin \log(\mathcal{O}(n))$, cioè non è vero che $f(n)$ è una funzione della forma $\log(g(n))$, con $g(n) \in \mathcal{O}(n)$; infatti si ha che $f(n) = \log(n^2)$ e $n^2 \notin \mathcal{O}(n)$.

b) Falsa. Si prenda $f(n) = 2n$: certamente vale $f(n) \in \mathcal{O}(n)$, quindi $2^{f(n)} \in 2^{\mathcal{O}(n)}$, ma si ha anche che $2^{f(n)} = 2^{2n} \notin \mathcal{O}(2^n)$.

c) Vera. L'insieme $\mathcal{O}(n)$ è dato dalle funzioni g per cui esistono $c > 0, n_0 > 0$ tali che $g(n) \leq cn$ per $n > n_0$. Affinché valga $(g(n))^k \in \mathcal{O}(n^k)$ devono allora esistere $c' > 0, n'_0 > 0$ tali che $(g(n))^k \leq c'n^k$ per $n > n'_0$, che è senz'altro verificata per c' opportuno e $n'_0 = n_0$. Infatti, per $n > n_0$, vale $g(n) \leq cn$ e quindi $(g(n))^k \leq (cn)^k \leq c'n^k$ se $c' > c^k$.

L'altra direzione è analoga: sia h una funzione in $\mathcal{O}(n^k)$, cioè tale per cui esistono $c > 0, n_0 > 0$ tali che $h(n) \leq cn^k$ per $n > n_0$. Affinché valga $h(n) \in \mathcal{O}(n)^k$ si deve mostrare che esiste una funzione $g(n) \in \mathcal{O}(n)$ tale che vale $h(n) = g(n)^k$. Tale funzione è $\sqrt[k]{h(n)}$, basta mostrare che vale $\sqrt[k]{h(n)} \in \mathcal{O}(n)$. Ciò è vero, infatti vale $\sqrt[k]{h(n)} \leq (\sqrt[k]{c})n$ per $n > n_0$.

4
Complessità delle macchine astratte

Esercizio 4.1. È noto che esistono linguaggi riconoscibili in tempo lineare da automi a pila deterministici e da Macchine di Turing a k nastri, ma non da Macchine di Turing a nastro singolo. Esistono anche linguaggi regolari non riconoscibili in tempo lineare da Macchine di Turing a nastro singolo? Giustificare brevemente la risposta.

SOLUZIONE

No: una Macchina di Turing a nastro singolo, una volta posizionata all'inizio della stringa di ingresso, può comportarsi esattamente come un automa a stati finiti, usando il proprio organo di controllo. Siccome un automa a stati finiti esegue sempre uno spostamento a destra della testina di lettura per ogni mossa, la stessa cosa può fare la MT senza dover utilizzare altre porzioni del nastro.

Esercizio 4.2. Descrivere in modo sufficientemente preciso il funzionamento di una MT a nastro singolo che effettua la traduzione $\tau(w) = w^R$, con $w \in \{a, b\}^*$, ottimizzando la complessità spaziale.

Dare le complessità spaziale e temporale della MT ideata.

È possibile ottenere una complessità spaziale migliore mediante un modello di calcolo diverso dalla MT a nastro singolo? Motivare adeguatamente la risposta.

SOLUZIONE

La Macchina di Turing desiderata non deve fare altro che marcare il primo simbolo, per esempio facendolo diventare $\underline{a}$ o $\underline{b}$, per tenere traccia dell'inizio della stringa (in modo da sapere, nella passata successiva da destra a sinistra, quando si è giunti all'inizio della stringa), quindi andare in fondo alla stringa e cambiare l'ultimo simbolo in A se il primo era a, e in B se il primo era b, quindi ripetere l'operazione al contrario, andando a cambiare il primo simbolo in $\underline{A}$ (sempre per tenere traccia dell'inizio della stringa) se l'ultimo era a, e in $\underline{B}$ se l'ultimo era b, quindi proseguire così con il secondo ed il penultimo, ecc., fino ad esaurire tutta la stringa. A quel punto occorre

ripassare la stringa e cambiare le A (o $\underline{A}$) in a, e le B (o $\underline{B}$) in b. La complessità spaziale è esattamente n (con n lunghezza della stringa in ingresso) in quanto si usano solo le celle inizialmente occupate, e quella temporale è $\Theta(n^2)$.

È possibile ottenere una complessità spaziale migliore sfruttando una MT a 1 nastro, che usa il nastro per contare (in binario) quanti simboli ci sono nella stringa in ingresso (in modo da sapere poi quando si è arrivati all'inizio della stringa in ingresso nella successiva passata da destra a sinistra). Una volta arrivata in fondo al nastro di ingresso, la MT scorre all'indietro i simboli in ingresso, decrementando il contatore (fino a che questo non arriva a 0), e producendo in uscita una a se in ingresso legge una b, e una b se in ingresso legge una a (in modo da produrre la stringa inversa). In questo caso infatti la complessità spaziale è $\Theta(\log(n))$.

Si può inoltre vedere che incrementare (e decrementare) m volte un contatore binario ha complessità $\Theta(m)$ (se il contatore viene scritto a partire dalla cifra meno significativa, e se ad ogni modifica del contatore vengono visitate solo le cifre strettamente necessarie). Infatti, si consideri che ogni cifra in posizione h viene modificata un numero di volte pari a $\frac{m}{2^h}$; per esempio, la cifra in posizione 0, cioè quella meno significativa, è modificata sempre; quella in posizione 1 è modificata solo se la prima è 1, quindi la metà delle volte; quella in posizione 2 viene modificata solo se le prime 2 sono entrambe 1, quindi un quarto delle volte, e così via. Inoltre, quando il contatore viene incrementato da 0 a m, in $\frac{m}{2}$ casi (i valori pari compresi tra 0 e m) si modifica solo la cifra meno significativa, il che richiede 2 mosse, una per arrivare sulla cifra e modificarla, e una per tornare all'inizio del nastro; negli $\frac{m}{2}$ casi rimanenti (i numeri dispari), nella metà ulteriore dei casi (quindi in $\frac{m}{4}$ casi) la seconda cifra meno significativa è uno 0, quindi, modificata quella, il contatore ha il valore desiderato, e con 2 ulteriori mosse si torna all'inizio del nastro; negli $\frac{m}{4}$ casi rimanenti, nella metà di essi la terza cifra è uno 0, e così via. In definitiva, il numero di mosse da fare per incrementare il contatore m volte è dato dalla seguente espressione:

$$\sum_{h=0}^{\lfloor \log_2(m) \rfloor} \left(\frac{m}{2} \frac{2(h+1)}{2^h} \right) = m \sum_{h=0}^{\lfloor \log(m) \rfloor} \frac{1+h}{2^h} \leq m \sum_{h=0}^{\infty} \frac{1+h}{2^h} = \Theta(m)$$

ricordando che abbiamo $\sum_{h=0}^{\infty} \frac{1+h}{2^h} = 4$, in quanto vale $\sum_{h=0}^{\infty} \frac{1+h}{2^h} = \sum_{h=0}^{\infty} \left(\frac{1}{2^h} + \frac{h}{2^h} \right)$ e valgono anche $\sum_{h=0}^{\infty} x^h = \frac{1}{1-x}$ e $\sum_{h=0}^{\infty} h x^h = \frac{x}{(1-x)^2}$ se è $|x| < 1$ (si veda [2, Appendice A]). Analogamente per i decrementi.

Quindi, la complessità temporale della MT a 1 nastro è anch'essa migliore rispetto a quella della MT a nastro singolo, ed è $\Theta(n)$, in quanto si tratta di fare due passate sulla stringa, in cui ognuna ha costo lineare.

Esercizio 4.3. Si descriva una MT a k nastri che, dato in ingresso un valore n naturale maggiore di 0 codificato in unario, produce in uscita le prime n potenze

di 4, cioè $4^0, 4^1, 4^2, \ldots 4^{n-1}$, ognuna codificata in binario e seguita dal simbolo #. Dare le complessità temporale e spaziale della MT ideata.

SOLUZIONE

Per produrre i valori desiderati è sufficiente una MT a 1 nastro di memoria, che funziona nel seguente modo:

- Legge un simbolo dal nastro in ingresso o si ferma se non ci sono più simboli da leggere.
- Scrive 1 sul nastro di uscita.
- Copia tutti gli 0, se ce ne sono, (leggendoli da sinistra a destra) dal nastro di memoria al nastro di uscita, ed aggiunge in fondo al nastro di uscita un #.
- Mette due 0 sul nastro di memoria (la testina è ora in fondo al nastro di memoria). Si noti che aggiungere due 0 in fondo corrisponde a moltiplicare per 4.
- Torna all'inizio del nastro di memoria, leggendo gli 0 da destra a sinistra, e poi ricomincia da capo.

Per il calcolo della complessità spaziale conta solo il nastro di memoria, non il nastro di ingresso, né quello di uscita, quindi la complessità è $\Theta(n)$.

Per il calcolo della complessità temporale, il costo della i-esima iterazione del ciclo è $\Theta(2i)$, quindi il costo totale è $\sum_{i=\{1..n\}} \Theta(2i)$, che è $\Theta(n^2)$.

Esercizio 4.4. Si consideri il linguaggio L_q di figure definito sull'alfabeto $\{a, b\}$, composto da tutte e sole le figure quadrate fatte da righe di soli simboli a alternate a righe di soli simboli b, partendo con le a. Per esempio:

```
aaaa
bbbb
aaaa
bbbb
```

Si descrivano esaurientemente due Macchine di Turing che accettino L_q, una a k nastri e una a nastro singolo, sapendo che la figura è fornita sul nastro in ingresso riga per riga e senza separatori, e se ne studino le complessità spaziali e temporali. Per esempio, la figura precedente è fornita in ingresso come stringa `aaaabbbbaaaabbbb`.

SOLUZIONE

Macchina di Turing a k nastri

Per risolvere il problema bastano 2 nastri di memoria. Si legge la prima sequenza di a copiandola su 2 nastri di memoria; poi il primo nastro viene

utilizzato per contare il numero di elementi di ogni riga, mentre il secondo serve per contare il numero di righe. Chiaramente basta una passata sulla stringa in ingresso. Siano m gli elementi di una riga, la stringa è lunga $n = m^2$, mentre la memoria occupata risulta $2m$. Quindi: $S(n) = \Theta(\sqrt{n})$, $T(n) = \Theta(n)$.

Macchina a nastro singolo

L'idea è controllare la lunghezza di ogni riga, cambiando di volta in volta le a in A e le b in B riga per riga, da sinistra verso destra, a parte i primi elementi di ogni riga che verranno cambiati in X. In questo modo possiamo controllare che tutte le righe siano lunghe uguali, facendo un numero di mosse che è $\Theta(nm)$, in quanto si tratta di fare m passate, una per colonna, su un numero di elementi che è almeno $n - m$ (occorre scorrere per intero le prime $m - 1$ righe, mentre l'ultima riga può essere analizzata solo parzialmente, fino all'ultimo simbolo minuscolo ancora presente).

Per controllare che il numero di righe sia corretto, la macchina può controllare che il numero delle X sia uguale al numero di A della prima riga $+1$, ad esempio cambiando le X e le A una alla volta in altri simboli (per esempio $\underline{X}$ e $\underline{A}$). Si noti che anche in questo caso occorre fare un numero di mosse che è $\Theta(nm)$, in quanto occorre fare m passate, ed ogni passata si ferma alla prima X incontrata (per cambiarla in $\underline{X}$ e poi tornare indietro); la prima passata richiede quindi $\Theta(m)$ mosse, la seconda $\Theta(2m)$, ecc., fino all'ultima (la m-esima) che ne richiede $\Theta(m^2)$. In definitiva, il controllo della correttezza del numero di righe richiede un tempo che è $\Theta(m \cdot m^2) = \Theta(mn)$.

Abbiamo quindi $S(n) = \Theta(n)$ (si ricordi che il contenuto iniziale del nastro conta nel calcolo della complessità spaziale per Macchine di Turing a nastro singolo) e $T(n) = \Theta(n\sqrt{n})$.

Esercizio 4.5. Si consideri il problema di riconoscere le stringhe del linguaggio $L = a^{5^n}$, fatte cioè di sole a in numero uguale ad una potenza di 5.

Descrivere una ***MT a nastro singolo*** che riconosce il linguaggio L (non serve definire tutti i dettagli della MT, basta descriverla in modo sufficientemente preciso per calcolarne la complessità). Dire quali sono le complessità temporale e spaziale della macchina descritta.

SOLUZIONE

Per riconoscere il linguaggio desiderato è sufficiente una MT che, per ogni a letta in ingresso, incrementa un contatore rappresentato *in base* 5, il quale è memorizzato in fondo alla stringa in ingresso inizialmente nel nastro della Macchina di Turing a nastro singolo. Dopo avere contato tutte le a, è sufficiente controllare che il contatore sia fatto di un 1 seguito da soli 0. Leggere un simbolo costa lo scorrimento di tutto il nastro (per poter incrementare il contatore alla fine della stringa), e poi l'incremento del contatore alla fine, quindi $\Theta(s + \log(s))$, dove s è la lunghezza della stringa in

ingresso, cioè $s = 5^n$. Questo va fatto per ogni simbolo della stringa in ingresso, quindi questa fase costa $\Theta(s^2)$. Per capire se il numero di simboli in ingresso è della forma 5^n è poi sufficiente scorrere il contatore una volta, quindi il costo di questa seconda fase è $\Theta(\log(s))$, quindi il costo temporale totale è $T(s) = \Theta(s^2)$ nel caso pessimo. Il costo spaziale è ovviamente $S(s) = \Theta(s + \log(s)) = \Theta(s)$.

Una seconda, più efficiente maniera per risolvere il problema dato è quella di scorrere diverse volte la stringa, ogni volta riducendo di un quinto il numero di simboli. Una maniera per fare ciò potrebbe essere quella di marcare, alla prima passata, il primo e l'ultimo simbolo della stringa, e poi ad ogni successiva passata cancellare 4 simboli a ogni 5 letti, rimpiazzandoli con dei blank (si noti che non serve ogni volta ricompattare la stringa eliminando i blank intermedi, o la complessità peggiora). Se alla fine rimane un solo simbolo, la stringa di partenza aveva la lunghezza della forma desiderata. In questo caso il costo spaziale è sempre $S(s) = \Theta(s)$, ma quello temporale diventa $T(s) = \Theta(s \log(s))$ in quanto ogni passata costa s, ma il numero di passate è dell'ordine di $\log(s)$.

Esercizio 4.6. Descrivere una MT a nastro singolo che, data in ingresso una sequenza di n numeri naturali, ognuno rappresentato mediante esattamente 8 bit (per esempio, il numero 8 è rappresentato come `00001000`), separati uno dall'altro dal simbolo #, ordina la sequenza in ordine crescente (non serve definire tutti i dettagli della MT, basta descriverla in modo sufficientemente preciso per calcolarne la complessità).

Dire quali sono le complessità temporale e spaziale della macchina descritta in funzione del numero di valori nella sequenza.

Dire come cambiano (se cambiano) la complessità se queste vengono calcolate in funzione della lunghezza della stringa in ingresso.

SOLUZIONE

Una MT che risolve il problema desiderato basta che realizzi un algoritmo tipo INSERTION-SORT o BUBBLESORT. Siccome il numero di bit per codificare i valori nella sequenza è fissato, per implementare un BUBBLESORT, per esempio, basta leggere 8 bit, memorizzarli negli stati (senza fare ricorso al nastro), fare altrettanto con il valore precedente, e poi scambiare i valori nella sequenza se il secondo è più grande del primo (verifica che può essere fatta in modo immediato in base ai valori memorizzati). Si noti che le operazioni di memorizzazione dei numeri, di confronto, e di scambio hanno un costo costante, perché, come già rimarcato, il numero di bit è esattamente uguale a 8 (quindi memorizzare il valore corrisponde a leggerlo, e richiede 8 mosse, ecc.). Come in un BUBBLESORT classico, il secondo elemento va confrontato con gli elementi precedenti, poi altrettanto va fatto per il terzo, il quarto, ecc.

Quindi, il costo temporale della MT in funzione del numero n di valori nella sequenza è il classico $\mathcal{O}(n^2)$, mentre il costo spaziale è $\Theta(n)$, in quanto non viene usata memoria aggiuntiva rispetto ai valori già memorizzati.

I costi non cambiano se si usa, come parametro di dimensione dei dati, la lunghezza l della stringa invece che il numero di valori n, in quanto la relazione tra i due è banalmente lineare ($l = \Theta(n)$).

Si noti che usare algoritmi come COUNTING-SORT o MERGE-SORT non darebbe beneficio dal punto di vista delle complessità, in quanto questi richiederebbero memoria aggiuntiva, il che richiederebbe necessariamente di scorrere avanti e indietro il nastro ogni volta per modificarla, dando luogo quindi comunque ad una complessità almeno quadratica.

Esercizio 4.7. Si consideri il problema di convertire valori codificati in binario in valori codificati in ottale (cioè in base 8).

1. Definire una MT a k nastri che legge dall'ingresso un valore naturale codificato in binario (scritto partendo dal bit *meno* significativo) e scrive in uscita lo stesso valore, ma codificato in ottale (scritto sempre a partire dalla cifra *meno* significativa). Dire quali sono le complessità temporale e spaziale della MT a k nastri ideata.

2. Definire una MT a nastro singolo che esegue la trasformazione analoga a quella descritta al punto 1 (alla fine sul nastro deve rimanere il valore iniziale, ma codificato in ottale, partendo dalla cifra meno significativa). Dire quali sono le complessità temporale e spaziale della MT a nastro singolo ideata.

Si diano pure le definizioni delle MT a parole, ma in modo sufficientemente preciso per potere valutare la complessità delle MT ideate.

SOLUZIONE

1. Per ottenere la prima trasformazione è sufficiente leggere i simboli in ingresso a 3 a 3, e trasformare ogni terna nella corrispondente cifra ottale (per cui `000` diventa `0`, `001` diventa `1`, `010` diventa `2`, ecc.; attenzione che la lettura viene fatta dal bit meno significativo). La complessità spaziale è costante, perché nulla viene memorizzato nei nastri di memoria. La complessità temporale è chiaramente $\Theta(n)$, con n lunghezza della stringa in ingresso.

2. La seconda trasformazione invece richiede tempo $\Theta(n^2)$ (con n sempre la lunghezza della stringa in ingresso). Infatti, non solo ogni terna di cifre binarie in ingresso deve essere convertita in una cifra ottale (in questo caso viene fatta una vera e propria sostituzione, per cui si può pensare che l'ultima cifra di ogni terna venga trasformata nella cifra ottale, e le altre 2 vengano invece cancellate, riportandole a blank), ma,

alla fine della conversione delle terne, occorre eliminare gli spazi che si sono creati tra le cifre ottali, e questa operazione richiede un tempo $\Theta(n^2)$. Infatti spostare la seconda cifra della codifica ottale richiede 2 mosse, spostare la terza ne richiede 5, la quarta ne richiede 8, e la k-esima (con $k \geq 2$) ne richiede $2 + (k-2)3$; quindi, siccome il numero di cifre da spostare è $\Theta(\frac{n}{3})$, il totale del numero di mosse necessarie per fare gli spostamenti è $\Theta(\sum_{k=1}^{n} k) = \Theta(n^2)$. La complessità spaziale è invece n, in quanto al massimo sul nastro si trovano n simboli.

Esercizio 4.8. Si consideri il seguente linguaggio su alfabeto $\Sigma = \{a, b, c\}$:

$$L = \{w \cdot w \cdot c \cdot x \mid w \in \{a, b\}^+, |w| \leq 49, x \in \Sigma^+\}.$$

Si descriva una MT a nastro singolo che accetti L, valutandone la complessità spaziale e quella temporale.

SOLUZIONE

Il linguaggio è regolare, essendo w di lunghezza limitata. La MT potrà usare l'organo di controllo per assicurarsi che i primi 98 caratteri (al più) della stringa siano compatibili con il linguaggio, e dovrà poi controllare che vi sia una c seguita da un carattere qualunque per accettare. Quindi la macchina farà al più 100 passi, cioè $T(n) = \Theta(1)$; $S(n) = \Theta(n)$, visto che comunque la memoria sarà occupata dalla stringa in ingresso.

Si noti che la proprietà $S(n) \leq T(n)$ (che in questo caso è violata) vale per le MT a k nastri, in cui i nastri di memoria sono inizialmente tutti vuoti, a differenza del caso delle MT a nastro singolo).

Esercizio 4.9. Si consideri il linguaggio $L = \{wcwcw \mid w \in \{a, b\}^+\}$, costruito sull'alfabeto $\Sigma = \{a, b, c\}$. Si descriva una macchina RAM che riconosca L, minimizzando la complessità *spaziale* sia a criterio di costo costante che logaritmico.

Dire quali sono le complessità temporale e spaziale della macchina RAM ideata, sia a criterio di costo costante che a criterio di costo logaritmico.

SOLUZIONE

Basta vedere i fattori w come numeri binari, per comodità con le cifre meno significative a sinistra, dove ad esempio $a = 0$ e $b = 1$. Si utilizza una cella di memoria per codificare w (per esempio `M[1]`), moltiplicando per 2 tutte le volte che si legge una nuova a, e moltiplicando per 2 e sommando 1 ogni volta che si legge una nuova b. Alla lettura della prima c si codifica in maniera analoga il secondo fattore w in `M[2]`. Alla seconda c si controlla che `M[1]` sia uguale ad `M[2]`, poi si procede a codificare il fattore w finale in `M[1]`; l'ultimo controllo `M[1]` $=$ `M[2]` avviene a fine stringa.

Indichiamo con n la lunghezza della stringa in ingresso, e notiamo che una stringa di lunghezza n viene codificata con un numero dell'ordine di

grandezza di 2^n in quanto, a ogni simbolo letto, il contatore viene raddoppiato, e ciò viene fatto un numero di volte proporzionale a n. Di conseguenza, w, la cui lunghezza è $\Theta(\frac{n}{3})$, viene codificata con un numero dell'ordine di grandezza di $2^{\frac{n}{3}}$. Quindi, la complessità spaziale risulta pari a $\Theta(1)$ a criterio costante e $\Theta(n)$ a criterio logaritmico. Per quel che riguarda la complessità temporale, la macchina legge ad uno ad uno i simboli in ingresso, e aggiorna un contatore, il cui valore è dell'ordine di 2^n (per essere precisi, il contatore viene incrementato un numero di volte dell'ordine di $\frac{n}{3}$, e l'i-esimo raddoppio viene fatto su un valore dell'ordine di 2^i). Quindi, a criterio di costo costante, la complessità temporale è $\Theta(n)$. A criterio di costo logaritmico, invece, è $\Theta(\sum_{i=1}^{n} \log(2^i)) = \Theta(\sum_{i=1}^{n} i)$, cioè $\Theta(n^2)$.

Esercizio 4.10. Si definisca una macchina RAM che prende in ingresso un numero naturale n e calcola il valore

$$\sum_{i=0}^{3^n} \frac{i}{9}$$

Dire quali sono le complessità (spaziale e temporale) della macchina scritta, calcolate sia a criterio di costo costante che a criterio di costo logaritmico.

SOLUZIONE

La sommatoria può essere scritta in forma chiusa come $\frac{3^n(3^n+1)}{18}$, quindi basta calcolare 3^n con un opportuno ciclo, e poi calcolare il valore desiderato. Per calcolare 3^n la maniera più semplice (ma non la più efficiente) è eseguire un ciclo `for` come il seguente:

```
exp := 1
for i := 1 to n
    exp := exp * 3
```

che ha complessità temporale $\Theta(n)$ a costo costante, e $\Theta(n^2)$ a costo logaritmico, in quanto il suo costo è $\sum_{i=0}^{n} \log 3^i = \log 3 \sum_{i=0}^{n} i$ (si noti che, a criterio di costo logaritmico, il costo della moltiplicazione $v * c$, con c una costante, è $\log(v)$).

La complessità spaziale è $\Theta(1)$ a costo costante, e $\Theta(\log 3^n) = \Theta(n)$ a costo logaritmico.

Si può ottenere una complessità temporale migliore con il seguente algoritmo, che si basa sull'osservazione che, se indichiamo con b_i il valore della i-esima cifra della codifica binaria di n (con b_0 la cifra meno significativa, b_1 la seconda, ecc.), abbiamo $n = \sum_{i=0}^{\log_2(n)} b_i 2^i$ e $3^n = 3^{\sum_{i=0}^{\log_2(n)} b_i 2^i} = \prod_{i=0}^{\log_2(n)} 3^{b_i 2^i}$. Si noti, quindi, che occorre moltiplicare tra di loro tutti i fattori 3^{2^i} in cui si ha $b_i \neq 0$. Di conseguenza, nello pseudocodice seguente, la riga 5 determina

se il fattore b_i è diverso da 0 (cioè se è uguale a 1), la variabile exp tiene traccia del valore $\prod_{i=j}^{i} 3^{b_j 2^j}$, e $temp$ memorizza 3^{2^i}.

```
CALC(n)
1  m := n
2  exp := 1
3  temp := 3
4  while m > 0
5       if m mod 2 = 1
6            exp := exp * temp
7       temp := temp * temp
8       m := m/2
9  return exp * (exp + 1)/18
```

Il ciclo while in questo caso ha complessità $\Theta(\log(n))$ usando il criterio di costo costante, mentre usando il criterio di costo logaritmico è

$$\Theta\left(\sum_{i=0}^{\log_2(n)} \log \prod_{j=0}^{i} 3^{2^j}\right) = \Theta\left(\sum_{i=0}^{\log_2(n)} (2^{i+1} - 1)\right) = \Theta(n)$$

poiché vale $\log \prod_{j=0}^{i} 3^{2^j} = \sum_{j=0}^{i} \log 3^{2^j} = \log(3) \sum_{j=0}^{i} 2^j$ ed inoltre $\sum_{j=0}^{i} 2^j = 2^{i+1} - 1$, quindi anche $\sum_{i=0}^{\log_2(n)} 2^j = 2^{\log_2(n)+1} - 1 = 2n - 1$. La complessità spaziale naturalmente non cambia.

Esercizio 4.11. Si consideri il seguente linguaggio:

$$L = \{a^{n_1} b a^{n_2} b \dots a^{n_k} \mid 1 \leq k \leq 100, \forall 1 \leq i \leq k.(n_i \geq 1), \exists 1 \leq i < j \leq k.(n_i = n_j)\}$$

Si descrivano a grandi linee, ma in maniera sufficientemente precisa, una MT e una RAM che riconoscano L, e se ne valutino le complessità temporali – nel caso della RAM sia a criterio di costo costante che a criterio di costo logaritmico.

Sono ovviamente preferite soluzioni a minor complessità per entrambi i tipi di macchina.

SOLUZIONE

Soluzione mediante MT

Grazie al fatto che il parametro k è limitato e ≤ 100 è possibile progettare diverse MT che riconoscano L con complessità temporale $\mathcal{O}(n)$.

Un primo modo consiste nel costruire una MT con 100 nastri di memoria: ogni sequenza a^{n_i} viene memorizzata su un nastro diverso; successivamente tutte le testine vengono riposizionate sulla prima cella dei rispettivi nastri

e vengono spostate di una posizione a destra contemporaneamente: se due di esse giungono alla fine della propria sottostringa contemporaneamente la stringa originaria viene accettata.

Una MT con meno nastri invece memorizza la stringa in ingresso in due nastri di memoria e, per ogni frammento a^{n_i} di un nastro verifica se sull'altro nastro ne esiste un altro con lo stesso numero di a. Si noti che per evitare di considerare il caso $n_i = n_j$ quando abbiamo $i = j$ può essere opportuno marcare in qualche modo la porzione di nastro sotto esame, per esempio sostituendo la b iniziale con un altro carattere. La MT prende in considerazione il primo gruppo a^{n_1} dal primo nastro e scorre il secondo nastro per verificare se esiste un altro gruppo con lo stesso numero di a (per fare questo deve fare k volte andata e ritorno nello scorrimento di a^{n_1} impiegando un tempo $2kn_1 + n$; il processo viene ripetuto per tutti gli a^{n_i} del primo nastro, ossia k volte; si ha quindi una complessità quadratica in k, ma ancora lineare in $\mathcal{O}(n)$.

Una MT più sofisticata potrebbe memorizzare soltanto i numeri n_i codificandoli in binario e poi verificare l'esistenza di coppie di valori uguali alla stessa maniera. La prima fase richiede di incrementare ogni contatore n_i volte, che ha complessità $\Theta(n_i)$ (si veda la discussione nella soluzione dell'esercizio 4.2), per cui, in totale, la prima fase richiede un tempo $\sum_{i=1}^{k} \Theta(n_i)$, che è $\Theta(n)$; essendo il tempo necessario per realizzare la prima fase $\Theta(n)$, la seconda può poi essere completata in tempo $\mathcal{O}(k^2 \log(n))$, cioè in tempo $\mathcal{O}(\log(n))$ in quanto k è costante.

Soluzione mediante RAM

Una RAM può procedere con uno schema algoritmico uguale a quello utilizzato per la terza MT descritta sopra. A criterio di costo costante ciò richiede $\mathcal{O}(n)$ per la prima fase e $\mathcal{O}(k^2)$ per la seconda; essendo $k \leq 100$, il costo totale è $\mathcal{O}(n)$. A criterio di costo logaritmico, invece, la prima fase costa $\mathcal{O}(n \log(n))$, ma la seconda $\mathcal{O}(k^2 \log(n))$, ossia ancora $\mathcal{O}(\log(n))$, comunque dominata dalla fase precedente.

Esercizio 4.12.

1. Si descriva a grandi linee, ma in modo sufficientemente preciso per calcolarne la complessità, una Macchina di Turing (deterministica) a k nastri che riconosca il linguaggio $L = \{a^{2^n} \mid n \geq 0\}$. Dire quali sono, nel caso pessimo, le complessità temporale e spaziale della MT ideata.
2. Si descriva a grandi linee una macchina RAM che riconosce il linguaggio L del punto 1. Darne le complessità temporale e spaziale, calcolate sia a criterio di costo costante che a criterio di costo logaritmico.

SOLUZIONE

1. Per riconoscere il linguaggio desiderato, è sufficiente una MT con 2 nastri di memoria. La macchina scandisce la sequenza di a in ingres-

so, e per ogni a letta incrementa un contatore binario. Alla fine, la macchina controlla che il contatore sia della forma `10000...` (cioè un 1 seguito da un numero arbitrario di 0). La complessità spaziale della MT così ideata è $\Theta(\log(2^n))$, cioè $\Theta(n)$. La complessità temporale è $\Theta(2^n)$, perché, applicando il ragionamento visto nella soluzione dell'esercizio 4.2, modificare un contatore binario 2^n volte ha costo $\Theta(2^n)$, e la verifica successiva che il contatore sia della forma `10000...` ha costo $\Theta(\log_2(n))$.

Si noti che in questo caso n non è la lunghezza della stringa x in ingresso, ma vale la relazione $|x| = 2^n$, cioè $n = \log(|x|)$.

2. La macchina RAM si comporta in maniera simile alla MT: incrementa un contatore di 1 per ogni a letta. Alla fine, si continua a dividere il contatore per 2, controllando ogni volta il resto della divisione, fino a che o si ottiene un numero dispari (cioè con resto della divisione pari a 1) maggiore di 1 (in questo caso la stringa non è accettata), o si ottiene 1 (in questo caso la stringa è accettata).

   A criterio di costo costante la macchina RAM ha complessità spaziale $\Theta(1)$ e complessità temporale $\Theta(2^n)$. A criterio di costo logaritmico la complessità spaziale è $\Theta(\log(2^n))$, cioè $\Theta(n)$. La complessità temporale, invece, è $\Theta(n2^n)$. Infatti, in questo caso domina il costo della fase di conteggio ($\Theta(n2^n)$), che richiede 2^n incrementi del contatore; la fase di divisione, invece, ha costo $\mathcal{O}(\log(2^n)\log_2(2^n)) = \mathcal{O}(n^2)$, in quanto il contatore viene diviso al massimo $\log_2(2^n) = n$ volte (si noti che ogni moltiplicazione e divisione è tale che uno dei due termini è costante, per cui la sua complessità è $\log(v)$, con v il valore moltiplicato/diviso).

Esercizio 4.13. Si consideri il linguaggio su alfabeto $\{0, 1\}$ fatto dalle stringhe $x \cdot y$, dove il prefisso x, con $|x| = 5$, codifica in binario la posizione in y di un carattere che deve essere 1.

Si descrivano esaurientemente una Macchina di Turing a nastro singolo e una macchina RAM che accettano il linguaggio, valutandone sia la complessità spaziale che quella temporale con tutti i criteri di costo disponibili.

SOLUZIONE

La stringa x ha lunghezza fissata, dunque la posizione significativa più a destra nella stringa di ingresso sarà `11111` in binario, cioè 31 in decimale. Il numero di stringhe corrette, considerando i soli prefissi di lunghezza $5+31$, è finito, quindi basta un automa a stati finiti per accettare il linguaggio, che si fermerà al più dopo 36 mosse. Il funzionamento sarà praticamente analogo sia per la MT che per la RAM.

Di conseguenza, per la MT a nastro singolo si ha: $T(n) = \Theta(1)$ (il numero di mosse è finito) e $S(n) = \Theta(n)$ (la stringa in ingresso sarà comunque inte-

ramente sul nastro). Si noti che, in questo caso, vale $T(n) < S(n)$ (si veda anche la discussione nella soluzione dell'esercizio 4.8).

Per la RAM, invece, sia usando il costo costante, che quello logaritmico si ha: $T(n) = \Theta(1)$, $S(n) = \Theta(1)$.

Esercizio 4.14. Si consideri il seguente problema: data una stringa x costruita su un alfabeto A, restituire il primo simbolo di x che *non* compare più di una volta in x.

1. Si descriva una MT a k nastri che risolve il problema dato, e se ne diano le complessità temporale e spaziale.
2. Si descriva una macchina RAM che risolve il problema dato, e se ne diano le complessità temporale e spaziale, sia a costo costante che a costo logaritmico.

SOLUZIONE

1. Si osservi che i possibili stati della computazione attraversati dalla MT sono in numero finito. In particolare essi sono determinati dal memorizzare le seguenti informazioni.
   - Per ogni carattere dell'alfabeto, se esso è stato visto nessuna, una o più di una volta. L'informazione è codificabile in modo semplice in $3^{|A|}$ distinti valori.
   - L'ordine con cui vengono visti i caratteri. L'informazione è codificabile in $|A|!$ distinti valori.

   Ricordando che l'alfabeto ha cardinalità finita, è sufficiente un automa a stati finiti con $3^{|A|} \cdot |A|!$ stati per eseguire il calcolo. L'automa utilizza la memoria a stati finiti per tenere traccia di tutte le informazioni necessarie a mano a mano che scorre la stringa. La complessità temporale dell'automa a stati finiti è $\Theta(n)$, con $n = |x|$, quella spaziale è costante.
   La MT desiderata emula il comportamento dell'automa a stati finiti, e ha le stesse complessità dell'automa a stati finiti (quindi $S(n) = \Theta(1)$, $T(n) = \Theta(n)$).
2. Come sopra. È sufficiente emulare un automa a stati finiti tramite la macchina RAM, che quindi avrà le stesse complessità ($S(n) = \Theta(1)$, $T(n) = \Theta(n)$), sia a costo costante che a costo logaritmico.

Esercizio 4.15.

1. Definire (in modo sufficientemente preciso, ma senza darne tutti i dettagli) una Macchina di Turing a k nastri che, dato in ingresso un numero naturale n codificato in binario (con cifra più significativa a sinistra), scrive in uscita il valore di $n! \mod 31$.

Dare le complessità temporale e spaziale della MT definita.

2. Definire (in modo sufficientemente preciso, ma senza darne tutti i dettagli) una macchina RAM che legge dall'input due numeri naturali n e p e scrive in uscita il valore di $n! \mod p$.

   Dare le complessità temporale e spaziale della RAM definita, usando sia il criterio di costo costante, che quello di costo logaritmico.

SOLUZIONE

1. Per eseguire il calcolo in modo efficiente è sufficiente notare che, se vale $n \geq 31$, allora $n! \mod 31$ è sempre 0. I casi in cui vale $n < 31$, invece, sono finiti. Quindi, è sufficiente controllare se vale $n \geq 31$ (e per fare questo è sufficiente vedere se la lunghezza della codifica di n è maggiore di 5 bit, o se sono esattamente 5 valori 1); se ciò è vero, si produce in uscita 0, se no, occorre leggere i (massimo 5) bit di n e, a seconda del caso in cui siamo, produrre direttamente l'uscita richiesta. Si noti che, se vale $n < 31$, allora ci sono solo 31 possibili casi, quindi per questa parte basta un meccanismo simile ad un automa a stati finiti.

   Le complessità temporale e spaziale della MT sono entrambe costanti.

2. In maniera analoga a prima, basta controllare se vale $n \geq p$ e, se non vale, calcolare $n! \mod p$. Si noti che, per calcolare $n! \mod p$, non serve calcolare prima $n!$ e poi applicare il modulo; invece, sfruttando il fatto che vale, dalle proprietà dell'aritmetica modulare, $a \cdot b \mod p = ((a \mod p) \cdot (b \mod p)) \mod p$, si può applicare il modulo ad ogni passo di moltiplicazione, garantendo che il risultato parziale non sia mai maggiore o uguale a p. Di conseguenza, i valori manipolati in ognuna delle n moltiplicazioni non sono mai più grandi di p, e la complessità di ogni singola moltiplicazione, calcolata con criterio di costo logaritmico, è al massimo $\log^2(p)$.

   Quindi, la complessità temporale totale della RAM, calcolata a criterio di costo logaritmico, è $\mathcal{O}(p \log^2(p))$ (si ricordi che vale $n < p$). Calcolata a criterio di costo costante, invece, la complessità è $\mathcal{O}(p)$. La complessità spaziale invece è $\mathcal{O}(\log(p))$ a costo logaritmico, e costante a costo costante.

Esercizio 4.16. Si consideri la funzione $f(n) = 2^{2^n}$.

1. Definire una Macchina di Turing a k nastri che legge in ingresso il valore n codificato in binario e calcola la funzione $f(n)$ (il cui risultato deve essere prodotto in uscita, sempre codificato in binario), e darne le complessità temporale e spaziale.

2. Definire una macchina RAM che calcola la funzione $f(n)$, e darne le complessità temporale e spaziale, calcolate sia a criterio di costo costante che a criterio di costo logaritmico.

SOLUZIONE

1. Una possibile MT (a 2 nastri) che risolve il problema funziona nel modo seguente.

   Innanzi tutto, legge n dal nastro di ingresso, e lo memorizza nel primo nastro di memoria. Quindi, inizia un ciclo in cui inizialmente memorizza un 1 nel secondo nastro di memoria, e poi via via decrementa n di 1 e contemporaneamente aggiunge uno 0 al secondo nastro di memoria, fino a che il contenuto del primo nastro diventa 0. In questo modo la MT calcola 2^n e lo memorizza nel secondo nastro di memoria. A questo punto, la MT comincia a scrivere sul nastro di uscita, scrivendo inizialmente un 1, quindi, in maniera simile a prima, decrementa via via il valore memorizzato nel secondo nastro, e contemporaneamente aggiunge uno 0 sul nastro di uscita. In questo modo, quando il contenuto del secondo nastro diventa 0, in uscita si trova 2^{2^n} codificato in binario.

   La MT ha complessità spaziale pari a $\Theta(\log(2^n))$, cioè $\Theta(n)$ (la dimensione dell'uscita non viene contata nella complessità spaziale). La complessità temporale è dominata dal secondo ciclo (quello che calcola il risultato finale). Il ciclo viene eseguito 2^n volte, e a ogni iterazione viene modificato un contatore di dimensione $\Theta(\log(2^n))$. Quindi, la complessità temporale è $\mathcal{O}(n2^n)$. In effetti, si noti che, evitando di scorrere a ogni decremento tutto il contatore binario, ma fermandosi alla prima cifra uguale ad 1 (assumendo che il contatore sia memorizzato con la cifra meno significativa più a sinistra), la complessità di ogni decremento è meno di $\Theta(\log(2^n))$ e si può vedere che, in totale, la complessità è $\Theta(2^n)$ (si veda anche la soluzione dell'esercizio 4.2).

2. Lo pseudocodice corrispondente ad una macchina RAM che calcola la funzione desiderata è il seguente:

   ```
   FUNC(n)
   1  res := 2
   2  while n > 0
   3        res := res * res
   4        n := n - 1
   ```

   Il ciclo viene eseguito n volte, e il numero di celle occupate è costante. Quindi, a criterio di costo costante, la complessità spaziale è $\Theta(1)$ e quella temporale è $\Theta(n)$.

A criterio di costo logaritmico, invece, possiamo subito notare che il costo spaziale è $\Theta(\log(2^{2^n}))$, cioè $\Theta(2^n)$. Inoltre, possiamo notare che il valore di `res` alla i-esima iterazione del ciclo è pari a 2^{2^i}, quindi il costo temporale della i-esima iterazione (in cui 2^{2^i} viene moltiplicato per se stesso, assumendo che per eseguire la moltiplicazione si usi l'algoritmo "scolastico", che ha complessità dell'ordine del prodotto dei logaritmi dei valori moltiplicati) è $\Theta(\log^2(2^{2^i}))$, cioè $\Theta(2^{2i})$, quindi il costo temporale totale è dell'ordine di grandezza di $\sum_{i=1}^{n} \log^2(2^{2^i}) = \sum_{i=1}^{n} 2^{2i} = \sum_{i=1}^{n} 4^i = \frac{4^{n+1}-1}{3} - 1$ (si ricordi che vale $\sum_{k=0}^{n} x^k = \frac{x^{n+1}-1}{x-1}$, si veda anche [2, Appendice A]), che è chiaramente $\Theta(2^{2n})$.

Si noti che, sia per la MT che per la RAM, il calcolo del costo è stato fatto in funzione del valore n in ingresso. Se il calcolo venisse fatto in funzione della dimensione r della stringa che codifica n, occorre ricordare che vale la relazione $n = \Theta(2^r)$.

Esercizio 4.17. Si consideri il linguaggio

$$L = \left\{ a^{\frac{n(n+1)}{2}} \mid n > 0 \right\}.$$

1. Definire una MT a k nastri che riconosce il linguaggio L, e darne le complessità temporale e spaziale in funzione della lunghezza della stringa in ingresso.
2. Definire una macchina RAM che riconosce il linguaggio L, e darne le complessità temporale e spaziale, calcolate sia a criterio di costo costante che a criterio di costo logaritmico, calcolate in funzione del numero di simboli in ingresso.

SOLUZIONE

1. Come è noto (si veda anche [2, Appendice A]), vale l'espressione $\frac{n(n+1)}{2} = \sum_{i=1}^{n} i$ cioè $1 + 2 + 3 + \ldots + n$. Quindi, per riconoscere la stringa mediante una MT a k nastri basta memorizzare su un nastro un contatore che si inizializza a 1, e che via via viene incrementato. Ogni volta che il contatore viene incrementato, viene copiato su un secondo nastro, e poi la copia viene usata (decrementandola via via) per contare un corrispondente numero di a dall'input. Il linguaggio viene accettato se, ad un certo punto, la stringa in ingresso finisce proprio mentre la copia del contatore arriva a 0. La MT così ideata ha complessità spaziale $\Theta(\log(\sqrt{|s|})) = \Theta(\log(|s|))$, con $|s|$ lunghezza della stringa in ingresso. Infatti, il massimo valore memorizzato (in binario) nei nastri di memoria è n, il che richiede $\log_2(n)$ celle di memoria, ed inoltre vale $|s| = \Theta(n^2)$, cioè $n = \Theta(\sqrt{|s|})$. La complessità temporale è invece

$\Theta(|s|)$, perché, come visto anche nella soluzione dell'esercizio 4.2, incrementare (e decrementare) un contatore binario h volte, se il contatore è memorizzato a partire dalla cifra meno significativa e si fa attenzione a fermarsi alla prima cifra da modificare, costa $\Theta(h)$. Per la precisione, si noti che la MT esegue n incrementi (costo totale $\Theta(n)$), $1+2+3+\ldots+n$ decrementi (costo totale $\Theta(n^2)$), e n copie, laddove il costo totale delle copie è $\sum_{i=1}^{n}(\log_2(i)+1) = \Theta(n\log(n))$. In tutto, quindi, la complessità temporale della MT è $\Theta(n+n^2+n\log(n)) = \Theta(n^2) = \Theta(|s|)$.

2. La macchina RAM si comporta in maniera analoga (mantiene un contatore, che viene usato per contare via via $1, 2, 3, 4, \ldots$). Tale macchina RAM ha, a criterio di costo costante, costo spaziale $\Theta(1)$ (si usano solo 2 celle di memoria) e costo temporale $\Theta(|s|)$. A criterio di costo logaritmico, invece, il costo spaziale è $\Theta(\log(\sqrt{|s|})) = \Theta(\log(|s|))$ e quello temporale $\Theta(|s|\log(|s|))$.

Esercizio 4.18. Si consideri il linguaggio $L = \{uu \mid u \in \{a,b\}^+\}$

1. Si definisca in maniera sufficientemente precisa e dettagliata una MT *deterministica* M che riconosca L, preferibilmente con complessità temporale nella classe $\Theta(n)$, dove al solito $n = |x|$ è la lunghezza della stringa di ingresso. Si calcoli anche la funzione di complessità temporale $T_M(n)$ di M in modo preciso, non solo con riferimento alla sua classe Θ.

2. Si stabilisca se una MT *nondeterministica* N possa riconoscere lo stesso linguaggio L con una complessità temporale $T_N(n)$ tale che $T_N(n) < T_M(n)$ per ogni n. Nel caso positivo si descriva, anche a grandi linee ma in maniera sufficientemente precisa da poterne ricavare con esattezza la funzione di complessità temporale, una tale macchina N. Altrimenti si spieghi il motivo per cui una tale macchina N non può esistere.

3. Si delinei poi una macchina RAM che riconosca lo stesso linguaggio e se ne stimi la complessità temporale $T_R(n)$ facendo riferimento sia al criterio di costo costante che a quello logaritmico.

SOLUZIONE

1. Una MT "naturale" deterministica M che riconosca L può operare nel modo seguente.

   (a) Scandisce l'intera stringa x: n mosse (più un numero limitato di eventuali mosse extra per inizializzazione e riconoscimento di fine stringa). Nel fare ciò memorizza contemporaneamente il valore di $\frac{n}{2}$ in unario in un nastro di memoria e ricopia il valore di x in un altro nastro.

(b) Mediante $\frac{n}{2}$ mosse riporta la testina del nastro di ingresso a metà stringa.

(c) Mediante ulteriori $\frac{n}{2}$ mosse da destra a sinistra verifica che la prima metà della stringa nel nastro di ingresso coincida con la seconda metà della stringa nel nastro di memoria. In caso di confronto positivo la stringa è accettata.

Il totale delle mosse impiegate da M è dunque $2n + c$ (c essendo una costante piccola indipendente da n).

2. Una MT nondeterministica N può invece "indovinare" la metà della stringa di ingresso e operare quindi nel modo seguente.

   (a) Scandisce e copia la stringa di ingresso in un nastro di memoria, finché nondeterministicamente "indovina" di essere giunta alla sua metà: ciò comporta quindi $\frac{n}{2}$ mosse.

   (b) Mediante altre $\frac{n}{2}$ mosse riporta la testina del nastro di memoria a inizio stringa.

   (c) Mediante ulteriori $\frac{n}{2}$ mosse verifica che la parte restante della stringa di ingresso coincida con la stringa contenuta nel nastro di memoria. Il numero di mosse complessivo è perciò $\frac{3}{2}n + c'$.

3. La RAM, essendo deterministica, deve operare secondo le stesse linee di M (tenendo però presente che la definizione normale della RAM non permette alla macchina di riposizionare la testina di lettura); precisamente:

   (a) Legge e memorizza la stringa di ingresso, calcolandone la lunghezza mediante un contatore intero.

   (b) Dimezza il valore del contatore.

   (c) Per ogni valore di i, con $1 \leq i \leq \frac{n}{2}$, verifica che `M[i]` sia uguale a `M[i+n/2]`.

   Il costo complessivo, a criterio di costo costante è dunque $k_1 n + 1 + k_2 \frac{n}{2} + k_3$, laddove k_1 e k_2 sono costanti che indicano il numero di operazioni elementari (`READ`, `LOAD`, `STORE`, ecc.) all'interno dei due cicli (k_1 per il ciclo di lettura, e k_2 per il ciclo di controllo dell'uguaglianza dei simboli, e k_3 è il numero di operazioni necessarie per produrre in uscita il risultato.

   A criterio di costo logaritmico l'espressione precedente diventa $k_1 n \log(n) + \log(n) + k_2 \frac{n}{2} \log(n) + k_3$, dove per semplicità si è considerato, per ogni singola operazione che non opera solo su costanti, il suo costo massimo $\log(n)$.

Esercizio 4.19. Si consideri il linguaggio L costituito dalle stringhe della forma $x\#y$ con $x, y \in \{a, b\}^*$ e tali che:

(i) $|x| \neq |y|$,

(ii) $|x|$ pari e $|y|$ dispari o viceversa.

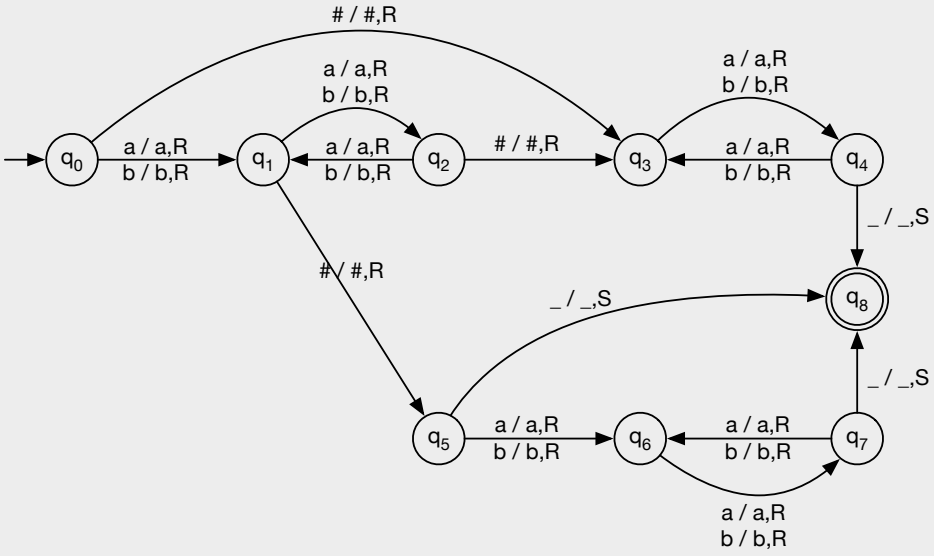
Si definisca (senza necessariamente codificarne tutti i dettagli) una Macchina di Turing a nastro singolo che riconosca il linguaggio L minimizzando la complessità temporale della macchina; se ne valutino le complessità spaziale e temporale asintotiche.

Si consideri il linguaggio L' ottenuto da L eliminando il vincolo (ii). Come cambierebbe la Macchina di Turing a nastro singolo per riconoscere L'? Come cambierebbero le complessità spaziale e temporale?

Cosa cambierebbe se si usasse una Macchina di Turing a k nastri invece di una Macchina di Turing a nastro singolo?

SOLUZIONE

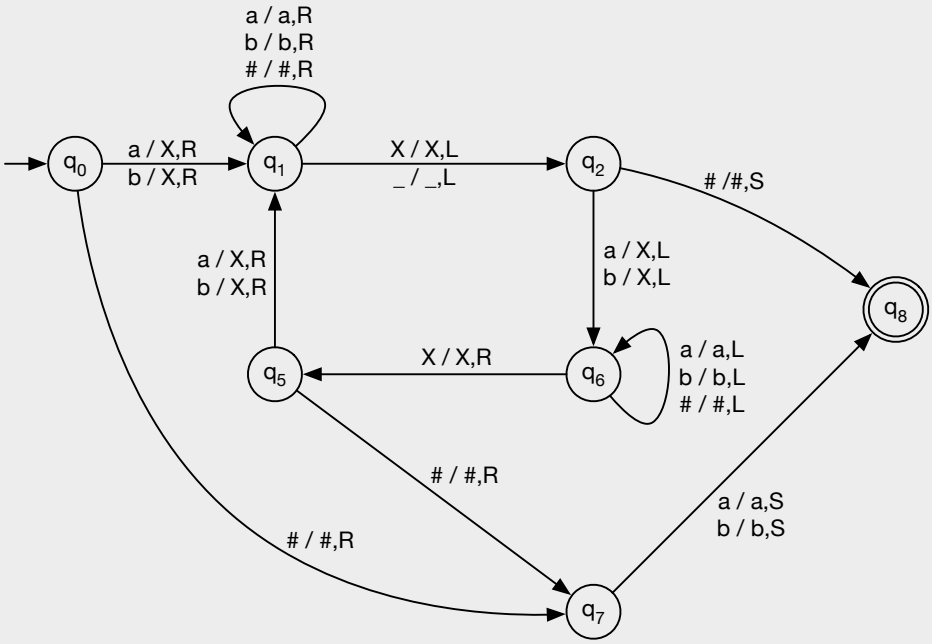
Una Macchina di Turing a nastro singolo che riconosce il linguaggio descritto opera con complessità temporale e spaziale $\Theta(n)$. Il linguaggio descritto è, infatti, un linguaggio regolare e il controllo sulla condizione (ii) (che implica la condizione (i)) può essere fatto usando solo gli stati come nella macchina seguente:



Si noti che la macchina arriva nello stato q_3 se il numero di elementi di x è pari (nel qual caso, tramite gli stati q_3 e q_4 si controlla che la lunghezza di y sia dispari); essa invece arriva in q_5 se la lunghezza di x è dispari (in questo caso si controlla tramite gli stati q_6 e q_7 che la lunghezza di y sia dispari).

La complessità spaziale è data dal fatto che l'unico nastro contiene la stringa di ingresso.

Se si rimuovesse il vincolo (ii) il linguaggio non sarebbe più regolare e una macchina che minimizza la complessità temporale potrebbe operare nel seguente modo:



Si noti che la macchina, quando entra nello stato q_1, legge un simbolo della parte x della stringa, e lo sostituisce con X. A quel punto, tramite l'autoanello su q_1 (e la transizione in q_2) trova il primo simbolo di y che non è X, e scrive al suo posto X (transizione in q_6). Quindi, tramite l'autoanello su q_6 (e la transizione in q_5), trova il prossimo simbolo di x che non è X, lo sostituisce con X (transizione in q_1), e ripete il tutto da q_1. La stringa viene accettata se, nel momento in cui una X è adiacente al separatore $\#$, dall'altra parte del separatore c'è un simbolo diverso da X. Si noti che, se viene presa la transizione da q_2 a q_8, allora la lunghezza di x è maggiore di quella di y, in quanto si può vedere che la transizione da q_1 a q_2 può essere presa solo se in x c'è ancora qualche simbolo diverso da X (altrimenti non si sarebbe ritornati in q_1, ma si sarebbe andati in q_7); se invece viene presa la transizione da q_7 a q_8, allora la lunghezza di x è minore.

La complessità temporale diventerebbe quindi $\Theta(n^2)$, mentre quella spaziale rimarrebbe pari a $\Theta(n)$.

Se invece che una Macchina di Turing a nastro singolo si usasse una Macchina di Turing a k nastri le complessità cambierebbero nel seguente modo.

Nel primo caso la macchina a k nastri opererebbe esattamente come la macchina a nastro singolo, mantenendo quindi una complessità temporale $\Theta(n)$, mentre la complessità spaziale diventerebbe $\Theta(1)$, poiché i nastri di memoria e ingresso sono distinti e quindi la memoria, in questo caso, non viene usata.

Rimossa la condizione (ii), una macchina con un nastro di memoria può riconoscere il linguaggio scorrendo la prima parte della stringa e copiandola nel nastro di memoria. Una volta raggiunto il simbolo # legge la seconda parte della stringa, avvolgendo indietro il nastro in cui è stata copiata la prima parte della stringa. Se il nastro raggiunge Z_0 prima che la stringa sia finita o la stringa finisce prima di arrivare a Z_0 sul nastro, la parola viene accettata.

La complessità temporale diventa quindi $\Theta(n)$, così come quella spaziale.

Esercizio 4.20. Si considerino le Macchine di Turing a nastro bidimensionale, in cui il nastro è costituito dal primo quadrante di un piano cartesiano a coordinate intere.

Si formalizzi la nozione di configurazione per tali Macchine di Turing; a partire da essa, si formalizzi anche le seguente definizione informale di complessità spaziale:

> La complessità spaziale è data dal *numero di celle del nastro visitate dalla testina* della macchina durante il suo funzionamento; al solito, essa è espressa come funzione della lunghezza della stringa (lineare) di ingresso.

SOLUZIONE

Una possibile definizione formale della complessità spaziale richiesta può essere fornita nel modo seguente.

In primo luogo occorre formalizzare il concetto di configurazione:

$$c = \langle q, x, memoria, \langle i, j \rangle, h \rangle$$

dove q rappresenta lo stato dell'organo di controllo, x la stringa in ingresso, $memoria : \mathbb{N} \times \mathbb{N} \to A$ indica il simbolo contenuto in ogni cella del nastro (con l'usuale vincolo che solo un numero finito di celle sia diverso da blank), $\langle i, j \rangle$ indica la posizione corrente della testina del nastro di memoria e h quella del nastro di ingresso. La transizione tra configurazioni è poi definita nel modo tradizionale. Inoltre, la configurazione iniziale c_0 è definita come segue:

$$c_0 = \langle q_0, x, memoria_0, \langle 0, 0 \rangle, 0 \rangle$$

con $memoria_0$ la funzione tale che $memoria_0(0,0) = Z_0$ e $\forall \langle i, j \rangle (\langle i, j \rangle \neq \langle 0, 0 \rangle \to memoria_0(i, j) = b)$.

La complessità spaziale di M è allora espressa dalla funzione:

$$S_M(n) = \max_{|x|=n} |\{\langle i, j\rangle | \exists c = \langle q, x, memoria, \langle i, j\rangle, h\rangle \wedge c_0 \vdash^* c\}|$$

Ossia la massima cardinalità, rispetto a tutte le stringhe di lunghezza n, dell'insieme delle coordinate delle posizioni della testina di memoria raggiunte durante il funzionamento della macchina a partire dalla configurazione iniziale.

Esercizio 4.21. Si consideri il linguaggio L' fatto di tutte e sole le stringhe sull'alfabeto $\{a, b\}$ della forma $a^{n_1}b^{m_1}a^{n_2}b^{m_2}a^{n_3}b^{m_3}\ldots$ e tali che esistono m_i, m_j (con $i \neq j$) tali che $m_i = m_j$.

1. Descrivere una Macchina di Turing (deterministica) a k nastri che riconosce il linguaggio L', e darne le complessità temporale e spaziale.
2. Descrivere una Macchina di Turing *nondeterministica* a k nastri che riconosce il linguaggio L', e darne le complessità temporale e spaziale.

SOLUZIONE

1. Una MT (deterministica) a 2 nastri di memoria che risolve il problema desiderato può memorizzare il contenuto della stringa nel primo nastro, quindi per ogni sequenza di m_i b, che per esempio può essere memorizzata nel secondo nastro, scorre il primo nastro e confronta la lunghezza m_j di ogni sequenza di b successiva con m_i; se ne trova una uguale, si ferma accettando, altrimenti passa ad analizzare la successiva sequenza di b, e così via.

   Il costo spaziale della MT descritta è ovviamente $\Theta(n)$, mentre il costo temporale è, nel caso pessimo, $\Theta(n \cdot$ numero di sequenze distinte di $b)$. Ora, al massimo ci sono sequenze di b distinte di lunghezza $m_1 = 1$, $m_2 = 2$, $m_3 = 3$, ecc. la cui somma è $\leq n$. Siccome vale $\sum_{i=1}^{j} i = \Theta(j^2)$, il numero massimo di sequenze distinte è $\leq \sqrt{n}$, quindi la complessità totale è, nel caso pessimo, $\Theta(n\sqrt{n})$.

2. Una Macchina di Turing nondeterministica (a 2 nastri di memoria) deve semplicemente "indovinare" le 2 sequenze di lunghezza uguale, memorizzarle nei nastri 1 e 2, e poi confrontarle per verificare che la loro lunghezza sia uguale. La complessità spaziale di tale MT è ancora $\Theta(n)$ nel caso pessimo (la lunghezza delle stringhe di b potrebbe per esempio essere dell'ordine di grandezza di $\frac{n}{2}$), mentre in questo caso anche la complessità temporale (che è la complessità dell'esecuzione di accettazione più corta) è $\Theta(n)$, perché si tratta di scorrere una volta il nastro di ingresso, e poi una volta le 2 stringhe di b "indovinate".

Esercizio 4.22. Si dica, spiegandone brevemente il motivo, se le seguenti affermazioni sono vere o false:

1. Dati 2 formalismi riconoscitivi F_1 e F_2, se F_1 è strettamente più potente di F_2, allora, preso un qualunque linguaggio L riconoscibile sia mediante F_1 che mediante F_2, L è sicuramente riconoscibile da F_1 con complessità temporale asintoticamente non superiore rispetto a F_2.
2. I linguaggi definibili mediante formule MFO sono riconoscibili da MT a nastro singolo con complessità temporale inferiore rispetto a quella delle macchine RAM.

Il criterio di costo adottato per ogni formalismo deve essere realistico anche in caso di dati "estremi" (numeri o strutture di grandezza illimitata).

SOLUZIONE

1. *Falsa*: è noto che il linguaggio wcw^R non è riconoscibile da una MT a nastro singolo con complessità temporale inferiore a $\Theta(n^2)$, mentre è riconoscibile in tempo $\Theta(n)$ mediante un automa a pila deterministico, che naturalmente è un formalismo meno potente rispetto alle MT.
2. *Falsa*: i linguaggi definibili mediante formule MFO sono una sottoclasse dei linguaggi regolari, quindi si possono riconoscere usando memoria finita. Data una MT a nastro singolo che riconosce un linguaggio, ad esempio comportandosi mossa per mossa come un automa a stati finiti, si può scrivere una RAM che esegue un'istruzione per ogni mossa della MT, sempre usando un numero finito di celle; ogni mossa costa un tempo $\mathcal{O}(1)$ anche a criterio di costo logaritmico perché i dati in ingresso sono caratteri di un alfabeto finito.

Esercizio 4.23. Si consideri una funzione $f(n)$ tale che

$$\lim_{n\to\infty} \frac{f(n)}{n^c} = 0$$

dove c è una costante positiva arbitraria.

Si può concludere che $f(n) = \mathcal{O}(n^{c-\varepsilon})$, per una qualche costante positiva $\varepsilon < c$? Motivare opportunamente la risposta.

SOLUZIONE

No, non si può concludere che l'affermazione è vera. La funzione

$$f(n) = \frac{n^c}{\log(n)}$$

soddisfa chiaramente il limite imposto. Tuttavia, non esistono 3 costanti

positive M, n_0 e ε (con $M, \varepsilon \in \mathbb{R}$, $n_o \in \mathbb{N}$) tali che per, $n > n_0$, risulti sempre $f(n) \leq Mn^{c-\varepsilon}$, ossia $\frac{n^c}{\log(n)} \leq Mn^{c-\varepsilon}$. Infatti, questo avviene se, e solo se, vale $\frac{n^\varepsilon}{\log(n)} \leq M$, ma, come noto, n^ε cresce più velocemente di $\log(n)$ per qualunque $\varepsilon > 0$, pertanto il loro rapporto non può essere limitato da una costante M.

Esercizio 4.24. Si considerino i formalismi delle Macchine di Turing deterministiche a 1 nastro di memoria (MT-1) e degli Automi a 2 Pile deterministici (A2P). Un A2P è un automa a pila deterministico che ha a disposizione una pila aggiuntiva. Dire, motivando opportunamente la risposta, quale è la relazione sussistente tra la complessità delle MT-1 e degli A2P.

SOLUZIONE

Si indichi con $T_{M1}(n)$ la complessità di una generica MT-1, e con $T_{A2P}(n)$ la complessità di un generico A2P. La complessità di un A2P che simula una MT-1 ha, nel caso pessimo, complessità $\mathcal{O}(T^2_{M1}(n))$. Infatti, un A2P può simulare la singola mossa di una MT-1 in questo modo. Esso usa le 2 pile per rappresentare il contenuto sia del nastro di ingresso della MT-1 che del nastro di memoria, per esempio memorizzando prima il contenuto del nastro di ingresso, seguito da un separatore (per esempio #), seguito dal contenuto del nastro di memoria. Lo A2P prima legge interamente il contenuto del nastro di ingresso, e lo memorizza nella pila di sinistra, quindi introduce il separatore ed il simbolo di inizio nastro. Per simulare la mossa della MT-1, lo A2P usa le 2 pile per simulare lo scorrimento dei 2 nastri (leggendo un simbolo dalla pila di sinistra e mettendolo nella pila di destra per scorrere da destra a sinistra, e leggendo un simbolo dalla pila di destra e mettendolo nella pila di sinistra per scorrere da sinistra a destra). Quindi, lo A2P simula prima lo scorrimento del nastro di ingresso, fino a giungere al simbolo corrente (che è opportunamente marcato), lo legge, e procede con la simulazione dello scorrimento del nastro di memoria, fino al carattere corrente. Quindi, modifica opportunamente la memoria per simulare il cambiamento dei simboli letti, e lo spostamento delle testine. La simulazione della singola mossa della MT-1 richiede alla peggio di scorrere interamente il nastro di ingresso ed il nastro di memoria, ed ha complessità $\mathcal{O}(T_{M1}(n))$, in quanto il contenuto del nastro di memoria è maggiorato dalla complessità temporale della MT-1. Poiché il numero di mosse da simulare è pari a $T_{M1}(n)$, la complessità globale è $\mathcal{O}(T^2_{M1}(n))$.

Viceversa, per simulare un A2P con una MT-1 è sufficiente usare il nastro di memoria per simulare il contenuto delle due pile, per esempio mettendo prima il contenuto della prima pila, seguito da un simbolo separatore, seguito da contenuto della seconda pila. La simulazione della singola mossa dell'A2P richiede di scorrere il nastro di memoria fino a giungere alla cima della pila di sinistra, proseguire fino a giungere alla cima della pila di destra, e modificare opportunamente il nastro per simulare il cambiamento dei

simboli in cima alla pila. A questo proposito, se le pile aumentano o diminuiscono di lunghezza, occorre scorrere il nastro di memoria, opportunamente spostando i simboli (per esempio, in caso di aggiunta di 2 simboli in cima alla pila di sinistra, il contenuto della pila di destra va tutto spostato di 2 posizioni a destra; viceversa, se il simbolo in cima alla pila di sinistra va cancellato, occorre spostare tutto il contenuto della pila di destra di una posizione a sinistra). Questo richiede, nel caso pessimo, complessità $\mathcal{O}(T_{A2P}(n))$, quindi, ancora una volta, la complessità totale di una MT-1 che simula un A2P è $\mathcal{O}(T^2_{A2P}(n))$.

5
Sintesi di algoritmi

5.1 Vettori, liste e matrici

Esercizio 5.1. Si consideri un array A di lunghezza n, contenente numeri relativi.

1. Definire un algoritmo che restituisce il sottoarray $A[i..j]$ di A di lunghezza massima tale che, per ogni $i \leq k \leq j$, si ha $A[i] \leq A[k] \leq A[j]$. Per esempio, nel caso dell'array $[5, 18, 2, 12, 4, 7, 13, -2, -5, 12]$ il sottoarray cercato è $[2, 12, 4, 7, 13]$.
2. Dire quale è la complessità temporale dell'algoritmo ideato.

SOLUZIONE

Una soluzione estremamente semplice al problema individua il sottoarray controllando, per ogni coppia di valori i, j, con $i \leq j$, se il sottoarray $A[i, j]$ corrispondente rispetta la proprietà desiderata tramite una scansione lineare dello stesso. Questo approccio tuttavia ha complessità $\mathcal{O}(n^3)$ in quanto compie n^2 scansioni di porzioni del vettore di lunghezza $\mathcal{O}(n)$.

È possibile tuttavia costruire una soluzione a tempo quadratico per il problema. L'approccio adottato è quello di analizzare, per ogni possibile valore di j, la porzione di vettore $A[1, j-1]$ alla ricerca del possibile valore di i corrispondente (si noti che, nel caso di sottoarray più breve che termina in j, $i = j$). Fissato il valore di j, individuare la posizione i_ok del minimo del sottoarray $A[1, j-1]$ consente anche di trovare la porzione più lunga dello stesso che rispetta la proprietà desiderata. Ripetendo il processo per tutti i valori di j, e selezionando il sottoarray di lunghezza massima, è possibile ottenere la soluzione. Si noti altresì che, nel caso più breve, uno qualunque degli elementi di A è considerabile come un sottoarray $A[i..i]$ che rispetta la proprietà: in questo caso il seguente algoritmo restituisce il singolo valore $A[i..i]$ come soluzione.

Nel caso siano presenti più sottoarray con la proprietà desiderata e lunghezza massima tra quelli presenti, è sufficiente che l'algoritmo ne restituisca uno qualunque tra essi.

Di seguito lo pseudocodice dell'algoritmo:

```
SOTTOARRAY(A)
 1  max_l := 1
 2  best_i := 0
 3  best_j := 0
 4  for j := A.length downto 1
 5       l := 0
 6       i_ok := j
 7       min := A[j]
 8       for i := j downto 1
 9            if A[i] > A[j]
10                 break
11            if A[i] < min
12                 i_ok := i
13                 min := A[i]
14       l := j − i_ok + 1
15       if max_l < l
16            max_l := l
17            best_i := i_ok
18            best_j := j
19  return A[best_i..best_j]
```

La complessità dell'algoritmo è $\Theta(n^2)$, data dai due cicli annidati.

Esercizio 5.2. Scrivere un algoritmo che, dato un array A di lunghezza n contenente numeri interi con segno, determina se esiste un sottoarray (ovvero una sequenza di elementi consecutivi dell'array) di A che sia a sua volta scomponibile in 2 sottoarray tali per cui la somma degli elementi nel primo sottoarray sia uguale alla somma degli elementi del secondo. L'algoritmo deve restituire il sottoarray più lungo tra quelli che soddisfano la condizione di cui sopra, se esiste. Indicare la complessità temporale asintotica dell'algoritmo scritto.

SOLUZIONE

Si noti che i possibili sottoarray di A sono n^2: ogni sottoarray è infatti definito univocamente dagli indici della cella iniziale i e della cella finale j, con $1 \leq i \leq j \leq n$. Un modo naturale per risolvere il problema consiste nell'elencare tutti gli n^2 sottoarray di A, e controllare se questi hanno la proprietà desiderata. Nel seguito, osserviamo come tale verifica si può fare in tempo $\mathcal{O}(n)$ per ogni sottoarray. Di conseguenza, la complessità temporale complessiva è $\Theta(n^3)$.

Dato un sottoarray di A, $A[i..j]$, corrispondente agli elementi di A compresi tra $A[i]$ e $A[j]$, estremi inclusi, di almeno due elementi ($i < j$), consideriamo un terzo indice k per suddividerlo in due sottoarray: $A_1 = A[i..k]$ e $A_2 = A[(k+1)..j]$.

Uno schema algoritmico per trovare se esiste una suddivisione di $A[i..j]$ in due sottoarray, A_1, A_2 con la proprietà desiderata è il seguente:

- si calcola la somma s degli elementi di $A[i..j]$;
- si inizializza k al valore di i, di conseguenza si ha che la somma degli elementi di A_1 è $A[i]$, quella degli elementi di A_2 è $s - A[i]$. Si noti che, date le somme degli elementi di A_1 e A_2 per un dato valore di k è possibile calcolare quelle relative alla suddivisione data da $k + 1$ sommando ad A_1, e sottraendo ad A_2, il valore $A[k + 1]$.
- Si procede quindi assegnando a k tutti i valori tra i e j, al calcolare le somme degli elementi di A_1 e A_2, e controllare se la condizione di uguaglianza è rispettata. Se lo è, il sottoarray è l'attuale $A[i..j]$. Il costo dell'operazione è $\mathcal{O}(j - i) = \mathcal{O}(n)$.

Lo schema sopra citato va iterato per tutte le possibili coppie di valori di i e j, con $1 \leq i < j \leq n$, ottenendo una complessità temporale di $\Theta(n^3)$.

Esercizio 5.3. Sia dato un array A di n interi che assumono valori nell'intervallo $[0, n + 1]$. L'array non contiene duplicati; i valori in esso contenuti saranno quindi tutti quelli compresi nell'intervallo considerato, tranne due. A non è ordinato.

Si specifichi un algoritmo che individui e produca in uscita i due valori mancanti in A sotto forma di un vettore R di due elementi, e se ne discuta la complessità.

SOLUZIONE

È possibile ispirarsi all'algoritmo del counting sort e utilizzare un array ausiliario B di $n + 2$ valori binari (0 e 1), tale che $B[i]$ sia posto a 1 se i è presente nell'array; altrimenti $B[i]$ è posto a 0. L'algoritmo risultante (riportato di seguito) ha complessità $\Theta(n)$.

```
MANCANTI(A)
 1  n := A.length
 2  ALLOCA(B, n + 2) // Vettore di n + 2 elementi
 3  ALLOCA(R, 2) // Vettore di 2 elementi
 4  for i := 1 to A.length
 5        B[A[i] + 1] := 1
 6  i := 1
 7  o := 1 // indice vettore dei risultati
 8  while (o ≤ 2) and (i ≤ n + 2)
 9        if B[i] = 0
10              R[o] := i − 1
11              o := o + 1
12        i := i + 1
13  return R
```

Esercizio 5.4. Sia dato un vettore di interi, A, ordinato in ordine non decrescente. Sia dato inoltre un valore intero $v > 0$. Si definisca un algoritmo che verifichi se A contiene due elementi a_i e a_j tali che $a_i - a_j = v$. Si progetti l'algoritmo, ottimizzandone la complessità temporale.

SOLUZIONE

Una soluzione banale, ma poco efficiente, controlla tutte le coppie $\langle i, j \rangle$, $A[i], A[j]$, con $1 \leq i < j \leq A.length$, per verificare se vale $A[j] - A[i] = v$. La complessità di tale soluzione è $\mathcal{O}(n^2)$. È possibile poi definire una soluzione con complessità $\mathcal{O}(n \log(n))$. Per ognuno degli n elementi dell'array, $A[i]$, si applica una ricerca binaria agli elementi dell'array per cercare un elemento pari a $v + A[i]$.

CERCACOPPIA-RICERCABINARIA(A, v)

```
1  for i := 1 to A.length
2        if RICERCABINARIA(A[i..A.length], v + A[i])
3              return true
4  return false
```

È infine possibile ottenere una soluzione più efficiente, di complessità $\mathcal{O}(n)$, scandendo gli elementi in modo lineare tramite un opportuno incremento di due indici, i e j:

CERCACOPPIA-LINEARE(A, v)

```
 1  i := 1
 2  j := 2
 3  while (i ≤ j) and (j < A.length)
 4        if (A[j] − A[i]) < v
 5              j := j + 1
 6        else
 7              if (A[j] − A[i]) > v
 8                    i := i + 1
 9              else // (A[j] − A[i]) = v
10                    return true
11  return false
```

Questa soluzione sfrutta il fatto che il vettore A sia ordinato in ordine crescente; di conseguenza spostare verso destra l'indice del secondo elemento della coppia incrementa la differenza tra i due valori della coppia, mentre spostare l'indice del primo elemento la decrementa.

Per analizzare la complessità della soluzione si noti che, ad ogni iterazione, si incrementa j oppure i – oppure si esce dalla funzione. A ogni iterazione vale $j \geq i$; si eseguono quindi $\mathcal{O}(n)$ iterazioni.

Esercizio 5.5. Un ipotetico nucleo di valutazione del sistema universitario di un'ipotetica nazione ha deciso di valutare i docenti universitari utilizzando la seguente procedura:

- A ciascun docente i è associato un indice di produttività IP_i (un valore intero non negativo).
- I docenti sono ordinati in modo che i loro IP siano in ordine non decrescente.
- Il nucleo di valutazione definisce una soglia, cioè un numero non negativo k, per cui avanzeranno in carriera i soli docenti con un IP strettamente maggiore di k.

Si scriva un algoritmo che, dato il vettore ordinato $IP[1..n]$ e la soglia k, calcola il numero totale di docenti che possono essere promossi. Se ne discuta la complessità.

SOLUZIONE

Una soluzione semplice, di complessità $\mathcal{O}(n)$, scandisce il vettore IP e si ferma al primo elemento con valore maggiore di k (il vettore è ordinato). Tale algoritmo ha complessità $\Theta(n)$ solo nel caso in cui tutti i docenti abbiano IP minore di k (caso pessimo).

Una soluzione più efficiente sfrutta la ricerca binaria per contare i valori di IP strettamente maggiori di k. Si usa quindi una funzione ricorsiva ispirata alla ricerca binaria, che chiamiamo SOPRASOGLIA(IP, k, i, j), che a ogni passo ricorsivo restituisce il numero di elementi maggiori di k nella partizione considerata:

```
SOPRASOGLIA(IP, k, i, j)
 1  if (i > j)
 2        return 0 // partizione vuota
 3  m := (i + j)/2
 4  if (IP[m] ≤ k)
 5        // esamino la partizione a destra di IP[m]
 6        // (gli elementi precedenti sono tutti ≤ k)
 7        return SOPRASOGLIA(IP, k, m + 1, j)
 8  else
 9        // elementi di IP[m, j] tutti sopra soglia; verifico la partizione sx
10        return (j − m + 1) + SOPRASOGLIA(IP, k, i, m − 1)
```

Così come per la ricerca binaria, la complessità di questo algoritmo si può esprimere con la ricorrenza $T(n) = T(\frac{n}{2}) + \mathcal{O}(1)$, che ha soluzione $T(n) = \mathcal{O}(\log(n))$.

Esercizio 5.6. Siano dati k array di $n > 0$ elementi ciascuno, già ordinati. Si descriva, preferibilmente mediante pseudocodice, un algoritmo per ottenere un

array ordinato contenente tutti e soli gli elementi degli array dati, valutandone la complessità in funzione di k e n.

SOLUZIONE

L'algoritmo più semplice è dato dall'effettuare la fusione in ordine dei k array. A questo scopo, si mantengono k indici che rappresentano il primo elemento ancora da fondere per ogni array. Si ripete quindi la ricerca del minimo tra gli elementi puntati dai k indici, per poi inserirlo nell'array del risultato, ed incrementare l'indice nel vettore originario. Si dovranno fare k confronti per ciascuno dei kn elementi, ottenendo una complessità temporale $\Theta(k^2 n)$.

In alternativa, è possibile ottenere un algoritmo più sofisticato, che ha però una complessità temporale minore, ovvero $\Theta(nk \log(k))$. L'idea dell'algoritmo è simile al *selection sort*, ossia trovare il minimo tra gli elementi che non sono ancora stati copiati nell'array ordinato e copiare questo minimo in coda all'array ordinato.

Possiamo ottenere una versione più efficiente del classico selection sort sfruttando il fatto che il nostro array è diviso in k array ordinati. Grazie a questo fatto, il minimo elemento da inserire in coda all'array ordinato da nk elementi può sempre essere ottenuto confrontando al più k elementi, che sono i minimi degli elementi che devono ancora essere copiati nell'array in uscita per ognuno dei k array. Ad esempio, nella prima iterazione, dobbiamo trovare il minimo tra tutti gli nk elementi: questo sarà necessariamente il minimo tra i primi elementi dei k array. Nella seconda iterazione, otteniamo l'elemento da copiare considerando gli stessi elementi dell'iterazione precedente, eccetto per l'elemento precedentemente copiato nell'array ordinato, che viene sostituito dal secondo elemento dell'array contenente l'elemento precedentemente copiato. Per ognuna delle nk iterazioni, dobbiamo calcolare il minimo di k elementi, ottenendo una complessità totale di $\Theta((nk)k) = \Theta(nk^2)$.

Tuttavia, possiamo migliorare la complessità asintotica utilizzando un BST bilanciato, ad esempio un albero rosso-nero. Costruiamo questo albero con i primi elementi di ognuno dei k array. Un nodo dell'albero contiene come chiave l'elemento e un dato che identifica l'array di appartenenza dell'elemento tra i k array in ingresso. Ogni iterazione cancella il minimo di questo albero, rimpiazzandolo con l'elemento successivo dell'array a cui apparteneva l'elemento cancellato dall'albero, informazione che può essere ricavata dal dato associato al nodo eliminato dall'albero. L'elemento cancellato dall'albero è inserito in coda all'array ordinato che si sta costruendo. La costruzione dell'albero bilanciato ha un costo $\Theta(k \log(k))$. Una singola iterazione dell'algoritmo ha costo $\Theta(\log(k))$, siccome sia la cancellazione del minimo elemento che l'inserimento dell'elemento successivo hanno entrambe costo $\Theta(\log(k))$ in un albero bilanciato. Dato che dobbiamo effettuare nk iterazioni, otteniamo una complessità totale di $\Theta(nk \log(k))$.

```
MERGEARRAYS(M)
 1  // M è un vettore di array, dunque una matrice k × n
 2  k := M.rows
 3  n := M.cols
 4  ALLOCA(C, k) // vettore di k indici dei successivi elementi da elaborare
 5  T := NIL
 6  for i := 1 to k // per ogni array
 7        ALLOCA(e, 1)
 8        e.key := M[i][1] // prendo il primo elemento
 9        e.row := i // tengo traccia dell'array di provenienza in M
10        RB-INSERT(T, e) // e lo inserisco nell'albero
11        C[i] := 2 // il prossimo elemento da elaborare è il secondo
12  ALLOCA(V, k · n) // vettore restituito come risultato
13  d := 1 // indice elemento destinazione in V
14  while d ≤ k · n
15        e := MIN(T)
16        V[d] := e.key // aggiungo il minimo al risultato
17        d := d + 1
18        RB-DELETE(T, e) // Costo O(log(k))
19        // Aggiunge l'elemento successivo della riga di e.key in M
20        if C[e.row] ≤ n
21              e.key := M[e.row][C[e.row]]
22              RB-INSERT(T, e)
23              C[e.row] := C[e.row] + 1
24  return V
```

Un metodo alternativo, ma che ottiene la stessa complessità computazionale, $\Theta(nk\log(k))$, consiste nell'usare la strategia della fase di *merge* dell'algoritmo MERGE-SORT su array via via più grossi: prima gli array di dimensione n, ricevuti in ingresso, fondendoli a coppie, poi i risultati di queste fusioni, che hanno dimensione $2n$, vengono fusi a coppie, e così via. Considerato che la fase di *merge* ha costo lineare nella lunghezza dei vettori che fonde, abbiamo quindi che $\log_2(k)$ passate di *merge* avranno costi proporzionali a $2n\frac{k}{2}$, $4n\frac{k}{4}$, $8n\frac{k}{8}$, e così via, ovvero ognuna di esse ha costo $\Theta(nk)$, da cui il risultato.

Esercizio 5.7. Sia L una lista semplice (cioè con puntatori solo all'elemento successivo) contenente n interi positivi memorizzati in ordine strettamente crescente. Si consideri il problema di trovare due puntatori a due elementi di L, i, e j, se esistono, tali che la somma degli elementi di L compresi tra i e j, estremi inclusi, sia uguale a un dato valore X.

Si descriva un algoritmo, mediante opportuno pseudocodice, per risolvere il problema (ossia stabilire se tali i, j esistono e nel caso produrli in uscita) e se ne valuti la complessità asintotica. Nel caso sia presente più di una coppia di valori con la proprietà desiderata, è sufficiente che l'algoritmo ne restituisca una tra esse.

SOLUZIONE

Una soluzione banale utilizza due puntatori ad elementi della lista e, per ogni passo in avanti fatto da uno di questi due puntatori, scandisce la lista con il secondo e calcola la somma degli elementi compresi tra i due puntatori. Questo approccio ha complessità $\Theta(n^2)$, in quanto, per ogni passo del primo puntatore, si effettua un intero scorrimento della lista con il secondo.

È tuttavia possibile fare meglio: il metodo precedente infatti non sfrutta in alcun modo il fatto che gli elementi della lista siano ordinati in ordine crescente. L'ordinamento degli elementi può essere sfruttato osservando che la somma degli elementi della lista compresi tra quello in posizione i e quello in posizione j, SOMMA(i, j), sarà sempre:

- minore di SOMMA$(i + 1, j + 1)$
- maggiore di SOMMA$(i - 1, j - 1)$

e, chiaramente SOMMA$(i, j-1) <$ SOMMA$(i, j) <$ SOMMA$(i, j+1)$, dato il fatto che tutti gli elementi sono positivi. Utilizzando queste relazioni, è possibile pensare una strategia di ricerca tra le possibili somme che impiega unicamente tempo lineare: la strategia inizia con il considerare la somma del solo primo elemento della lista ed incrementa la porzione sommata, spostando la posizione della fine j se la somma è troppo bassa. Simmetricamente, fa decrescere la somma incrementando la posizione dell'inizio i se questa è troppo alta. Di seguito, lo pseudocodice della soluzione:

```
FINDSUM(L, X)
 1  if L = NIL
 2        return false
 3  i := L.head
 4  j := L.head
 5  sum := j.key
 6  while (i ≠ j.next) and (j ≠ NIL) and (sum ≠ X)
 7        if sum < X
 8              j := j.next
 9              sum := sum + j.key
10        else
11              sum := sum − i.key
12              i := i.next
13  if sum = X
14        return (i, j)
15  else
16        return false
```

La complessità asintotica dell'algoritmo è chiaramente $\mathcal{O}(n)$ poiché i e j scorrono la lista una sola volta.

Esercizio 5.8. Si descriva un algoritmo, mediante opportuno pseudocodice, che calcoli il determinante del prodotto tra due matrici quadrate A e B di dimensioni $n \times n$, sotto l'ipotesi che A e B siano entrambe triangolari superiori (ossia abbiano tutti 0 al di sotto della diagonale principale) o, simmetricamente, entrambe triangolari inferiori. Si valuti la complessità asintotica dell'algoritmo in funzione della dimensione n delle matrici.

SOLUZIONE

Si rammenti che il prodotto tra una matrice A, di dimensioni $n \times m$, ed una matrice B, di dimensioni $m \times p$, è una matrice $n \times p$ il cui generico elemento in posizione (i, j) è dato da $\sum_{k=1}^{m} A[i][k] \cdot B[k][j]$. Si constata allora facilmente che il prodotto tra due matrici triangolari (ad esempio, superiori) è a sua volta una matrice triangolare superiore i cui elementi sulla diagonale principale sono il prodotto $A[i][i] \cdot B[i][i]$. Notoriamente il determinante di una matrice triangolare è il prodotto degli elementi sulla diagonale principale; esso è quindi dato dalla formula $\prod_{i=1}^{n} A[i][i] \cdot B[i][i]$, che evidentemente può essere calcolata in tempo $\Theta(n)$ a criterio di costo costante. Di seguito, lo pseudocodice dell'algoritmo:

```
CALCOLADET(A, B)
1  d := 0
2  for i = 1 to A.cols
3        d := d + A[i][i] · B[i][i]
4  return false
```

Osservazione. Si osservi che, anche partendo dall'assunto che i valori contenuti nella matrice siano limitati superiormente ed inferiormente da una costante, la complessità a costo logaritmico dell'algoritmo descritto è superiore a $\Theta(n)$. Si ha infatti che, nel caso pessimo il valore del determinante necessita di un numero di cifre n volte superiore a quello del singolo autovalore per essere rappresentato. Abbiamo quindi che le moltiplicazioni alla linea 3 dell'algoritmo risultano avere un costo pari a $(\log(n))^2$ (assumendo un algoritmo di moltiplicazione scolastico) risultando così la parte dominante del corpo del ciclo. La complessità totale a costo logaritmico è quindi $\Theta(n(\log(n))^2)$.

Si giunge allo stesso risultato anche ricordando che il determinante del prodotto tra due matrici è il prodotto dei due determinanti, calcolando quindi i determinanti di A e B in tempo $\Theta(n)$, e moltiplicandoli in tempo $\Theta(1)$ ($\Theta(n(\log(n))^2)$ e $\Theta((\log(n))^2)$ a costo logaritmico, rispettivamente).

Esercizio 5.9. Si consideri una matrice quadrata M di numeri interi positivi, avente lato n. La matrice contiene numeri interi ordinati in ordine crescente lungo ogni riga e ordinati in ordine decrescente lungo ogni colonna.

Si descriva un algoritmo di ricerca di un valore intero x all'interno della matrice stessa, avente minime complessità temporale e spaziale.

SOLUZIONE

È possibile individuare il valore cercato, o determinarne l'assenza, in tempo $\mathcal{O}(n)$ e spazio $\mathcal{O}(1)$. La soluzione si basa sul fatto che che la matrice, ordinata in questo modo, offra la possibilità di effettuare una ricerca considerando che tutti gli elementi in una riga successiva siano minori del corrente, e tutti quelli in una colonna successiva maggiori di esso. Nel caso il valore cercato non venga trovato, l'algoritmo restituisce $(-1, -1)$.

TROVAX(M, x)

```
 1  n := M.cols
 2  i := 1
 3  j := 1
 4  while (i ≤ n) and (j ≤ n)
 5        if M[i][j] = x
 6              return (i, j)
 7        if M[i][j] < x
 8              j := j + 1
 9        else
10              i := i + 1
11  return (−1, −1)
```

Esercizio 5.10. Si definisca *permutazione bidimensionale* una funzione biunivoca dall'insieme S di tutte le coppie ordinate di numeri interi tra 0 e $n-1$, per un dato intero n, a se stesso.

Si consideri una matrice M quadrata, di lato n. Ogni elemento della matrice è una coppia di numeri interi, x e y, compresi tra 0 e $n-1$. Si interpreti la matrice come una funzione $f(x, y) : (x, y) \mapsto M[x][y]$ con x e y interi, $1 \leq x \leq n$, $1 \leq y \leq n$.

1. Si realizzi in pseudocodice un algoritmo a complessità temporale minima che verifica se una matrice M rappresenta una permutazione bidimensionale, dimostrandone la minimalità della complessità.

2. Viene detta *orbita* in una permutazione un (sotto-)insieme degli elementi permutati tale per cui, applicando iterativamente la permutazione ognuno di essi va ad occupare, via via ad ogni iterazione, il posto di tutti gli altri dell'orbita. Si descriva un algoritmo che determina la dimensione dell'orbita massima della funzione rappresentata da una matrice M, assumendo che essa sia una permutazione bidimensionale.

SOLUZIONE

1. Una data matrice M di coppie è una permutazione bidimensionale se e solo se rappresenta una mappa biiettiva tra coppie. Come conseguenza, tutte le possibili coppie (i, j) con $1 \leq i \leq n$ e $1 \leq j \leq n$ devono

comparire una ed una sola volta in M. Per verificare questa condizione è sufficiente scandire la matrice M e tener traccia di quali coppie sono già state viste con una matrice di valori interi interpretati come booleani, V. La matrice V è inizializzata a 0, per indicare che prima di scandire M nessuna coppia è stata vista. Per ogni coppia di valori (i, j) si analizza quindi la cella opportuna di V indicizzata dalla coppia contenuta in $M[i][j]$: se essa è già impostata ad 1, allora la coppia in $M[i][j]$ è già presente in un'altra cella di M, dunque M non è una permutazione bidimensionale. In caso contrario, la cella di V in esame è impostata ad 1, in quanto la coppia contenuta in $M[i][j]$ è stata vista.

```
VERIFICAPERM(M)
1  n := M.cols
2  ALLOCA(V, n × n) // Matrice di elementi, tutti nulli
3  for i = 1 to n
4       for j = 1 to n
5            if V[M[i][j].x][M[i][j].y] = 1
6                 return false
7            else
8                 V[M[i][j].x][M[i][j].y] := 1
9  return true
```

La soluzione ha complessità $\mathcal{O}(n^2)$, minima in quanto è necessario almeno leggere l'intera matrice in ingresso allo scopo di verificare la biiettività della funzione rappresentata.

2. L'approccio più efficiente per determinare l'orbita massima è quello di scandire ogni orbita, sfruttando il fatto che esse sono necessariamente disgiunte tra loro. A questo scopo è necessario tener traccia di quali coppie (i, j) si è già scandita l'orbita: questo può essere effettuato con lo stesso meccanismo di tracciamento dell'algoritmo precedente, ovvero una matrice booleana. L'algoritmo di calcolo dell'orbita massima quindi scandisce M, controlla se una data cella non sia già stata vista in un'orbita e, se questo è il caso, percorre l'orbita stessa contando il numero di passi (e marcando, ad ogni passo, come "vista" la coppia). Per determinare il valore massimo della dimensione dell'orbita o_{max} è sufficiente confrontare, a valle di ogni scansione di orbita, il valore ottenuto con la stima di o_{max} corrente. Di seguito, lo pseudocodice dell'algoritmo:

```
ORBITAMAX(M)
 1  n := M.cols
 2  ALLOCA(V, n × n) // Matrice di elementi, tutti nulli
 3  o_max := 0
 4  for i = 1 to n
 5       for j = 1 to n
 6            if V[M[i][j].x][M[i][j].y] = 0 // Inizia nuova orbita
 7                 o := 1
 8                 c_i := M[i][j].x
 9                 c_j := M[i][j].y
10                 V[c_i][c_j] := 1
11                 while (c_i ≠ i) ∨ (c_j ≠ j) // Percorro orbita
12                      V[M[c_i][c_j].x][M[c_i][c_j].y] := 1
13                      tmp := c_i
14                      c_i := M[c_i][c_j].x
15                      c_j := M[tmp][c_j].y
16                      o := o + 1
17                 if o > o_max
18                      o_max := o
19  return o_max
```

La soluzione ha complessità $\mathcal{O}(n^2)$, sfruttando il fatto che ogni elemento può appartenere ad una sola orbita a causa della biunivocità della permutazione. Questo implica che ogni cella di M sia visitata al più due volte: una durante la scansione della sua orbita, ed una seconda, durante la scansione su M. Come sopra, la complessità è minima in quanto è necessario almeno leggere l'intera matrice in ingresso allo scopo di verificare a quale orbita appartenga ciascuno degli elementi. È possibile ottenere un guadagno (costante) arrestando la ricerca dell'orbita massima quando gli elementi ancora da ispezionare sono in numero minore di quelli dell'orbita massima trovata fino a quel punto.

5.2 Ordinamento

Esercizio 5.11. Descrivere in modo rigoroso (o in pseudocodice) un algoritmo che riceve in ingresso una sequenza S di numeri interi e la riordina nel modo seguente: il primo numero è il più grande di S, il secondo è il più piccolo di S, il terzo è il secondo più grande di S, il quarto è il secondo più piccolo di S, e così via. Per esempio, data in ingresso la sequenza $S = [4, 8, 23, 10, 17, 0, 3]$, l'algoritmo la deve riordinare nella seguente maniera: $[23, 0, 17, 3, 10, 4, 8]$. Si fornisca la complessità asintotica dell'algoritmo scritto.

L'algoritmo descritto deve avere complessità temporale minima; si fornisca una dimostrazione della minimalità della complessità temporale dell'algoritmo proposto.

SOLUZIONE

Un modo semplice di realizzare questo algoritmo consiste nell'ordinare l'array in ordine decrescente, con un algoritmo a complessità asintotica ottima, ovvero $\mathcal{O}(n \log(n))$, ad esempio HEAP-SORT.

Ottenuto l'array ordinato se ne costruisce uno nuovo con gli elementi ordinati dall'inizio se l'indice è pari e dalla fine se è dispari (complessità $\mathcal{O}(n)$). La complessità totale è pertanto $\mathcal{O}(n \log(n))$.

```
BUTTERFLYSORT(S)
1  HEAP-SORT(S) // HEAP-SORT agisce sul posto, ordine decrescente
2  ALLOCA(R, S.length)
3  for i := 1 to S.length
4        if i mod 2 = 1
5              R[i] := S[⌊i/2⌋ + 1]
6        else
7              R[i] := S[n − ⌊i/2⌋]
8  return R
```

È possibile dimostrare che la soluzione fornita ha complessità ottima, osservando che se, per assurdo, fosse disponibile un algoritmo più veloce di $\mathcal{O}(n \log(n))$ che ordina un array dato secondo il criterio indicato, questo algoritmo consentirebbe anche di ordinare l'array stesso in un ordine crescente o decrescente.

Detto infatti $\mathcal{A}(\cdot)$ l'algoritmo che ordina secondo l'ordine descritto nella consegna e indicata con $T_{\mathcal{A}}(n)$ la sua complessità temporale in funzione della lunghezza di S, possiamo costruire una procedura che ordina S in ordine decrescente come segue:

- si invoca $\mathcal{A}(\cdot)$ su S, ottenendo la sequenza R;
- tramite una scansione lineare si stampano prima gli elementi pari di R dall'inizio alla fine, poi i dispari dalla fine all'inizio.

Il costo di quanto fatto è quello di una scansione lineare $\Theta(n)$ più $T_{\mathcal{A}}(n)$. Sapendo che il limite inferiore per la complessità dell'ordinamento per confronto è $\mathcal{O}(n \log(n))$, abbiamo che la complessità per la proposta di $\mathcal{A}(\cdot)$ deve necessariamente essere maggiore o uguale a $\mathcal{O}(n \log(n))$. L'algoritmo proposto ha quindi complessità temporale ottima. Si osservi che è possibile ridurre anche la complessità spaziale dell'algoritmo, ($\mathcal{O}(n)$ nella sua formulazione precedente), a $\mathcal{O}(1)$: è sufficiente effettuare la permutazione degli elementi di S che trasforma l'ordine crescente in quello desiderato sul posto. Questo scambio può essere effettuato tramite un approccio affine a quello del CYCLE-SORT, ovvero seguendo le sequenze di scambi della permutazione (fissata) che porta un vettore ordinato in modo crescente, nell'ordine alternante richiesto.

Esercizio 5.12. Si consideri un vettore di n interi. Si descriva in pseudocodice la procedura che ordina il vettore in ordine crescente in base al valore del resto della divisione di ognuno degli elementi per 4. Elementi con lo stesso valore di resto della divisione possono trovarsi in qualunque ordine tra loro. Il vettore ordinato avrà quindi alla sua estrema sinistra tutti gli elementi con resto nullo rispetto alla divisione per quattro, seguiti da tutti quelli aventi resto 1, a loro volta seguiti da tutti quelli aventi resto 2, ed infine da tutti gli elementi aventi resto 3.

La procedura di ordinamento deve avere complessità temporale $\mathcal{O}(n)$ e complessità spaziale $\mathcal{O}(1)$.

SOLUZIONE

Si osservi che 4 è, banalmente, un valore costante. Di conseguenza, è possibile progettare un algoritmo di ordinamento che, in 4 passate, mimi il comportamento del SELECTION-SORT, ordinando gli elementi di una delle 4 classi di resto ad ogni passata. Esso quindi scorre il vettore mantenendo un indice (d nel seguito) che memorizza la posizione in cui salvare il prossimo elemento avente resto corretto. Scorrendo il vettore, se l'elemento corrente ha resto corretto, lo scambia con quello in posizione d, incrementando poi il valore di d.

Si osservi a questo punto che l'ultima passata è inutile: infatti, durante le prime tre passate sono stati posizionati correttamente gli elementi con resti $0, 1, 2$; gli elementi con resto 3 sono quindi già necessariamente in posizione corretta.

```
ORDINAMOD4(V)
1  n := V.length
2  d := 1
3  for r := 0 to 2
4       for i := 1 to n
5            if (V[i] mod 4 = r)
6                 SWAP(V[i], V[d])
7                 d := d + 1
```

L'algoritmo ha complessità $\mathcal{O}(3n) = \mathcal{O}(n)$.

Esercizio 5.13. È noto che il QUICKSORT ha un comportamento nel caso pessimo $\mathcal{O}(n^2)$; ad esempio la versione presentata qui di seguito ha complessità $\mathcal{O}(n^2)$ quando l'array in ingresso è già totalmente ordinato.

1. Si modifichi il codice della versione suddetta, riportata qui sotto per comodità, in modo che la complessità di esecuzione, se A è già ordinato, risulti $O(n \log(n))$.
   La nuova versione deve essere esclusivamente una modifica dell'algoritmo originario, non includere altri (sotto)algoritmi più adatti al caso specifico.

2. Qual è la complessità asintotica della nuova versione nel caso pessimo? Descrivere come è fatto il caso pessimo, illustrandolo con un esempio.

```
QUICKSORT(A, p, r)
1  if p < r
2        q := PARTITION(A, p, r)
3        QUICKSORT(A, p, q − 1)
4        QUICKSORT(A, q + 1, r)

PARTITION(A, p, r)
1  x := A[r]
2  i := p − 1
3  for j := p to r − 1
4        if A[j] ≤ x
5              i := i + 1
6              SWAP(A[i], A[j])
7  SWAP(A[i + 1], A[r])
8  return i + 1
```

SOLUZIONE

1. Il comportamento dell'algoritmo è determinato dalla scelta del "pivot", che nel nostro caso è l'ultimo elemento dell'array. Perciò se l'array è già ordinato la partizione è totalmente sbilanciata. In questo caso perciò conviene scegliere come pivot l'elemento mediano, in modo che la partizione lasci inalterato l'array e così via ricorsivamente ottenendo una complessità $O(n \log(n))$. La modifica dell'algoritmo in tal senso può semplicemente consistere nell'usare $A[(r + p)/2]$ come pivot al posto di $A[r]$ (riga 1). Alla fine della partizione, l'array tornerà nella permutazione originaria già ordinata e così via ricorsivamente.

2. Ovviamente anche la versione così modificata ha un comportamento $\mathcal{O}(n^2)$ nel caso pessimo, che si verifica quando l'array è tale che il pivot è sempre il massimo dell'array che rimane da ordinare. Quindi, prendendo come esempio un array di lunghezza pari, l'elemento di mezzo è il massimo, il secondo elemento più grande è l'ultimo, ecc. Un esempio di tale array è `[2, 4, 6, 8, 10, 5, 3, 7, 1, 9]`, che dà luogo alla seguente sequenza di stati del vettore ad ogni chiamata a Quicksort:

```
[2, 4, 6, 8, 10, 5, 3, 7, 1, 9]
[2, 4, 6, 8, 9, 5, 3, 7, 1, 10]
[2, 4, 6, 8, 1, 5, 3, 7, 9, 10]
[2, 4, 6, 7, 1, 5, 3, 8, 9, 10]
[2, 4, 6, 3, 1, 5, 7, 8, 9, 10]
[2, 4, 5, 3, 1, 6, 7, 8, 9, 10]
[2, 4, 1, 3, 5, 6, 7, 8, 9, 10]
```

```
[2, 3, 1, 4, 5, 6, 7, 8, 9, 10]
[2, 1, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Esercizio 5.14. Si vuole realizzare un software per la ricerca di anagrammi: si fornisce una parola in ingresso e si identificano gli anagrammi di tale parola all'interno di un elenco di n parole. La lunghezza di ciascuna parola è di al più m caratteri. Si descriva un algoritmo che risolva il problema in oggetto e se ne valuti la complessità temporale fornendo un opportuno limite asintotico superiore (in funzione di n e m).

SOLUZIONE

Una soluzione particolarmente inefficiente della ricerca di anagrammi consiste nell'enumerare le $\mathcal{O}(m!)$ permutazioni dei caratteri della parola in ingresso e verificare, per ogni permutazione, se essa corrisponda a una parola tra le n in elenco. Questo porta a una complessità di $\mathcal{O}(m!nm)$. Alternativamente, un semplice modo per verificare se due parole siano l'una un anagramma dell'altra consiste nell'ordinare alfabeticamente i caratteri di entrambe le parole e verificare che i risultati siano identici. Questo ha una complessità di $\mathcal{O}(m\log(m)+m) = \mathcal{O}(m\log(m))$. Si può anche utilizzare un COUNTING-SORT, con complessità di $\mathcal{O}(m+k)$ per un alfabeto di k caratteri. Tale complessità è $\mathcal{O}(m)$ se k è costante, come è ragionevole supporre ad esempio nel caso dei caratteri dell'alfabeto inglese ($k = 26$) o del codice ASCII ($k = 128$) o Unicode ($k = 17 \times 65536$). L'algoritmo si può quindi implementare verificando, per ciascuna delle n parole in elenco, se sia un anagramma della parola in ingresso, con una complessità di $\mathcal{O}(nm\log(m))$ (oppure $\mathcal{O}(nm)$ se si utilizza il COUNTING-SORT con alfabeto fisso).

5.3 Tabelle Hash

Esercizio 5.15. Si consideri il caso di una tabella di hash di 8 elementi, le cui chiavi sono costituite da numeri interi, gestita con la disciplina dell'indirizzamento aperto (*open addressing*). Per semplicità, si rappresenti lo stato di una tabella vuota nel seguente modo

[_,_,_,_,_,_,_,_]

considerando il bucket (slot) più a sinistra come quello di indice 0 e quello più a destra come quello di indice 7. Si indichi con $h(k,i)$ la funzione di hash $h : \mathbb{N}\times\mathbb{N} \to \{0,\ldots,7\}$, dove k indica il valore della chiave e i l'indice del passo di ispezione (alla prima ispezione $i = 0$).

1. Si mostri lo stato della tabella, dopo ognuna delle seguenti operazioni

   - inserimento di 2

- inserimento di 10
- inserimento di 18
- cancellazione di 10
- inserimento di 26

considerando $h(k,i) = (k+i) \bmod 8$.

2. Si consideri l'uso della seguente funzione di hash $h(k,i) = (k+g(k)i) \bmod 8$, dove la funzione $g(k)$:
   a) codifica k in binario naturale,
   b) toglie tutti gli zeri consecutivi presenti nelle posizioni meno significative della codifica binaria di k,
   c) interpreta il risultato come la codifica in binario naturale di un intero.

   La funzione indicata rappresenta una buona scelta per la tabella di cui sopra, ovvero consente alla sequenza di ispezione di visitare tutti i bucket?

SOLUZIONE

1. Stato della tabella, in cui ⊠ indica la marca di un elemento cancellato (*tombstone*):
   - inserimento di 2: `[_,_,2,_,_,_,_,_]`
   - inserimento di 10: `[_,_,2,10,_,_,_,_]`
   - inserimento di 18: `[_,_,2,10,18,_,_,_]`
   - cancellazione di 10: `[_,_,2,⊠,18,_,_,_]`
   - inserimento di 26: `[_,_,2,26,18,_,_,_]`
2. La funzione $g(k)$ ha la proprietà di garantire un risultato dispari (a tutti gli effetti, rimuove il fattore $2^i, i \in \mathbb{N}$ dalla fattorizzazione di k), tranne nel caso in cui il valore dato in ingresso sia zero. Si ha di conseguenza che la strategia di hashing doppio indicata avrà una sequenza di ispezione che tocca tutte le celle della tabella di hash, in quanto il risultato di $g(k)$ è coprimo con la lunghezza della tabella, salvo appunto nel caso di $h(0,i)$ per qualunque valore di i. La funzione di hash non è quindi una buona funzione di hash, considerando il criterio del raggiungimento di tutti i bucket (slot) durante le ispezioni.

Esercizio 5.16. Si consideri una tabella hash di 1024 elementi, organizzata con indirizzamento aperto. La funzione di hash utilizzata è $h(k) = 4k \bmod 1024$, la sequenza di ispezione è $h'(k,i) = (h(k) + 2i) \bmod 1024 = (4k + 2i) \bmod 1024$.

1. Qual è il numero massimo di elementi inseribili nella tabella?
2. Qual è il numero medio di tentativi per accedere ad un elemento dopo l'inserimento di 256 elementi? (si assuma che la sequenza di ispezione sia tale da rispettare l'ipotesi di hashing uniforme rispetto ai bucket (slot) che possono essere raggiunti da essa).

SOLUZIONE

1. Nel caso di una funzione di hash ideale, l'intera tabella di hash viene riempita, quindi il numero massimo possibile di elementi inseribili è 1024, trattandosi di una tabella organizzata con indirizzamento aperto (anche chiamato in letteratura *closed hashing*). Tuttavia, si noti che la funzione di hash utilizzata ha come codominio solamente indirizzi della tabella multipli di 4, e la sequenza di ispezione aggiunge multipli di 2 ad essi, ottenendo quindi una sequenza di ispezione che tocca unicamente indirizzi pari. Abbiamo quindi che tutti i bucket della tabella con indirizzo dispari saranno sicuramente inutilizzati: il numero massimo di elementi inseribili in tabella è quindi 512.
2. Data la funzione di hash e sequenza di ispezione, la tabella è equivalente ad una più piccola, con solamente 512 bucket. Di conseguenza, il rapporto tra bucket disponibili e bucket occupati (noto anche come fattore di carico) α è $\frac{256}{512}$. Ricordando che il numero medio di tentativi per accedere ad un elemento è dato da $\frac{1}{\alpha}\log(\frac{1}{1-\alpha})$, di conseguenza il numero medio di tentativi per accedere ad un elemento nel nostro caso è $(\frac{256}{512})^{-1}\log(\frac{1}{1-\frac{256}{512}})$, cioè circa $1,38$.

Esercizio 5.17. Si consideri la gestione delle tabelle hash mediante indirizzamento aperto. Sia m la dimensione della tabella, ovvero il numero di bucket (slot) disponibili. È noto che in tal caso per eliminare un elemento della sequenza di ispezione associata a una chiave k non basta assegnargli il valore NIL, perché questo potrebbe interrompere sequenze di ispezione che passano per il valore cancellato per poi terminare su altri.

Un rimedio normalmente utilizzato in questi casi consiste nel marcare l'elemento come cancellato (utilizzando un simbolo opportuno, detto *tombstone*, indicato nel seguito da $\boxtimes$), in modo tale che la ricerca di altri elementi lungo la sequenza non si interrompa. In questo modo però, a lungo andare la tabella tende a riempire tutte le posizioni che non contengono elementi validi con tombstone, di conseguenza rendendo tutte le sequenze di ispezione di lunghezza massima.

Si progetti un algoritmo detto COMPATTAMENTO il quale, per una data tabella T, ricevendo come argomento una chiave k, elimini dalla sequenza associata alla chiave k tutte le tombstone, ricompattando così la sequenza e recuperando la memoria inutilizzata. Si assuma a questo scopo che la tabella abbia almeno uno slot libero (contenente NIL).

Si rammenta che la notazione $T[p]$, data una posizione p della tabella, restituisce NIL, $\boxtimes$, o k, ossia la chiave contenuta nella posizione, se questa non è né tombstone né vuota. Si assuma inoltre che la funzione che genera la sequenza di ispezione $h(k,i)$ sia del tipo $h(k,i) = (f(k) + a \cdot i) \bmod m$ con a intero e positivo e f funzione calcolabile in tempo costante. Si valuti la complessità asintotica dell'algoritmo. Se l'algoritmo elimina anche altre tombstone oltre a quelle richieste, il risultato è ugualmente accettabile (anzi, è addirittura preferibile), purché non interrompa alcuna sequenza di ispezione valida. L'algoritmo prodotto è applicabile anche al caso in cui la funzione h sia una funzione di tipo doppio hashing?

SOLUZIONE

Si osservi che, data una sequenza di ispezione ottenuta come $h(k, i)$, al variare di i, è possibile che altre sequenze di ispezione $h(k', j), k \neq k'$ al variare di j, condividano parte delle posizioni ispezionate. Tenendo conto del fatto che le sequenze di ispezione sono generate da $h(k, i) = (f(k) + a \cdot i) \bmod m$, possiamo provare che, se due sequenze di ispezione coincidono per un passo, allora esse coincideranno in tutti i passi successivi. Assumiamo quindi che $h(k, i) = h(k', j)$, con $k \neq k'$ (i e j non sono necessariamente diversi) e mostriamo che $h(k, i + 1) = h(k', j + 1)$. Espandendo nell'ipotesi la definizione della sequenza di ispezione, otteniamo $f(k) + ai = f(k') + aj$, da cui abbiamo $f(k') = f(k) + a(i - j)$. Espandendo in $h(k, i + 1) = h(k', j + 1)$ la definizione della sequenza di ispezione otteniamo invece $f(k) + a(i + 1) = f(k') + a(j + 1)$; sostituendo la relazione appena ricavata, otteniamo $f(k) + a(i+1) = f(k)+a(i-j)+a(j+1)$, da cui, $f(k)+ai+a = f(k)+ai-aj+aj+a$, che è vera per ogni scelta possibile di i e j.

Considerando unicamente la sequenza di valori contenuti nella sequenza di ispezione con origine in $h(k, 0) = f(k) + a \cdot 0 = f(k)$, osserviamo che questa può contenere più corse (sequenze ininterrotte di oggetti dello stesso tipo) di tombstone, di cui una di queste può essere posizionata alla fine della sequenza di ispezione. Inoltre, nella sequenza di ispezione della chiave k è possibile che i bucket della tabella contengano chiavi diverse da k la cui sequenza di ispezione ha la stessa origine di k ($h(k, 0) = h(k', 0)$) oppure un'origine diversa ($h(k, 0) = h(k', i)$, $i > 0$).

Il raziocinio di funzionamento dell'algoritmo di compattamento sarà dunque il seguente:

- scandisce completamente la sequenza di ispezione con origine in $h(k, 0)$;
- considera le tombstone come posizioni in cui poter scrivere gli elementi trovati lungo la sequenza di ispezione, che abbiano effettivamente una chiave che collide con k;
- salva tutti gli elementi che incontra nella sequenza di ispezione tali per cui la loro chiave non collide con k (ovvero sono elementi di una sequenza con origine in $h(k', 0)$ che ha una posizione in comune con quella con origine in $h(k, 0)$). Gli elementi in questione vengono reinseriti a valle del compattamento.

Allo scopo di spostare gli elementi non tombstone nelle posizioni contenenti le tombstone che li precedono, l'algoritmo scandisce la sequenza di ispezione utilizzando due indici: uno tiene traccia della posizione della prima tombstone (contando dall'inizio della sequenza) disponibile per sovrascrittura, l'altro individua il primo elemento non tombstone che la segue. Nel momento in cui l'indice che scorre gli elementi raggiunge la fine della sequenza di ispezione, l'indice che tiene traccia delle tombstone disponibili per sovrascrittura indicherà l'inizio di una corsa di sole tombstone fino alla fine della sequenza stessa (dato che tutti gli elementi validi sono spostati

prima della posizione che indica). Abbiamo quindi che l'ultima sequenza di tombstone che precede il NIL che delimita la fine della sequenza può essere rimossa con certezza.

Di seguito, lo pseudocodice dell'algoritmo:

```
COMPATTAMENTO(T, k)
 1  i := 0
 2  pc := h(k, i) // posizione corrente nella sequenza
 3  while (T[pc] ≠ NIL) and (T[pc] ≠ ⊠)
 4        i := i + 1
 5        pc := h(k, i)
 6  if (T[pc] = NIL)
 7        return // Non ci sono tombstone
 8  else
 9        // T[pc] contiene una tombstone
10        L := []
11        pt := pc // posizione tombstone da riscrivere
12        j := i // indice in seq. ispezione della posizione tombstone
13        // Ripeti fino a quando tutte le tombstone sono in coda
14        while (T[pc] ≠ NIL)
15              // Cerca un nuovo elemento valido
16              i := i + 1
17              pc := h(k, i)
18              // scorrendo sulle tombstone (o fino alla fine)
19              while (T[pc] = ⊠)
20                    i := i + 1
21                    pc := h(k, i)
22              // Se T[pc] ha sequenza che non inizia in h(k, 0), lo salvo in L
23              if (h(T[pc], 0) ≠ h(k, 0))
24                    PUSH(L, T[pc])
25                    T[pc] := ⊠
26                    // altrimenti rimpiazzo una tombstone
27              elseif (T[pc] ≠ NIL)
28                    T[pt] := T[pc]
29                    T[pc] := ⊠
30              // Se la tombstone in pt è stata rimpiazzata, devo spostare pt
31              while (T[pt] ≠ ⊠) and (T[pt] ≠ NIL)
32                    j := j + 1
33                    pt := h(k, j)
34        while (T[pt] = ⊠)
35              T[pt] := NIL
36              j := j + 1
37              pt := h(k, j)
38        e := POP(L)
39        while e ≠ NIL
40              INSERISCI(T, e)
41              e := POP(L)
```

Si noti che entrambi gli indici, i e j, utilizzati nel programma percorrono al più tutta la sequenza di ispezione che parte da $h(k, 0)$, includendo anche gli elementi che non sono stati inseriti a partire dalla chiave k ma che si trovano in posizione $h(k, i)$ per qualche i. Quindi la complessità della prima parte dell'algoritmo, fino al reinserimento in T degli elementi di L, è $\mathcal{O}(n)$, dove n è la lunghezza complessiva della sequenza. La complessità di quest'ultima parte è $\mathcal{O}(pm)$, dove p è la lunghezza di L e m la lunghezza dell'intera tabella, poiché ogni inserimento, nel caso pessimo, costa $\mathcal{O}(r)$, con $1 \leq r \leq m$. È però probabile che nel caso medio anche questa parte dell'algoritmo abbia una complessità più vicina a $\mathcal{O}(p)$ che a $\mathcal{O}(pm)$ perché i nuovi inserimenti avverranno probabilmente in posizioni rese disponibili dalla passata precedente. Quindi la complessità totale dell'algoritmo è $\mathcal{O}(n+pm)$. Come sempre nel caso delle tabelle hash nel caso pessimo $p < n < m$ e quindi si ha una complessità $\mathcal{O}(m^2)$, caso comunque solitamente lontano dal caso medio: normalmente si può ipotizzare $p < n \ll m$.

Osservazione. Un algoritmo leggermente meno efficiente potrebbe più semplicemente accantonare in L tutti gli elementi non tombstone e poi reinserirli. In questo caso il valore p della formula precedente sarebbe maggiore e la passata di reinserimento in tabella rischierebbe di avvicinarsi maggiormente a una complessità quadratica.

Riguardo alle strategie di doppio hashing. Nel caso del doppio hashing non vale più il fatto che $h(k, i) = h(k', j)$ implichi $h(k, i + 1) = h(k', j + 1)$, e, di conseguenza, il funzionamento dell'algoritmo non è più garantito.

5.4 Alberi

Esercizio 5.18. Si consideri un albero ternario, ovvero un albero i cui nodi contengono tre puntatori ai figli, $N.left$, $N.center$, $N.right$. Ogni nodo dell'albero ha un'ulteriore proprietà, detta colore, $N.color$, che può valere $verde$, $arancione$ o $marrone$. Vogliamo individuare i sottoalberi più grandi in esso contenuti, costituiti unicamente da nodi arancioni. Si noti che i sottoalberi formati da nodi arancioni individuati non sono necessariamente sottoalberi completi. Ad esempio, nel caso in cui un nodo abbia un unico figlio arancione, il quale a sua volta ha un unico figlio arancione, l'algoritmo deve andare a selezionare il sottoalbero formato dai tre nodi in questione. Si progetti un algoritmo, corredato delle eventuali strutture dati necessarie, che restituisce la lista delle radici dei detti sottoalberi, ordinate in ordine decrescente rispetto al numero di nodi contenuti nei rispettivi sottoalberi.

SOLUZIONE

Un procedimento che raggiunge il limite inferiore di complessità ($\Omega(n)$ dato che è necessario almeno esaminare tutti i nodi dell'albero una volta) è il seguente. Si inizializza una lista L che conterrà coppie di valori formate da

una radice di un sottoalbero ed il conteggio dei nodi del sottoalbero stesso. Con un attraversamento posticipato (visita in post-ordine):

- si calcola se i tre sottoalberi sono costituiti da soli nodi arancioni;
- se un sottoalbero è costituito da soli nodi arancioni, ma il nodo corrente non lo è, si aggiunge un riferimento alla radice del sottoalbero alla lista L, con il relativo conteggio nodi;
- infine, si restituisce un valore booleano al chiamante che indica se il nodo è arancione e tutti i sottoalberi in lui radicati sono composti da nodi arancioni ed il conteggio dei nodi del sottoalbero;
- terminato l'attraversamento posticipato (complessità $\Theta(n)$), si possono ordinare le radici contenute nella lista utilizzando un COUNTING-SORT, dato che il range di valori ammissibili per le dimensioni dei sottoalberi è $\{0, \ldots, n\}$;

Di seguito è riportato lo pseudocodice dell'algoritmo:

```
TrovaSottoalberi(T)
1  L := CreaListaVuota()
2  (L, ta, count) := VisitaERiconosci(T, L)
3  if (ta) and T.root ≠ NIL
4        Inserisci(L, (T.root, count))
5  return counting-sort(L)

VisitaERiconosci(T, L)
 1  if (T = NIL)
 2        return (L, true, 0)
 3  (L, ta_l, c_l) := VisitaERiconosci(T.left, L)
 4  (L, ta_c, c_c) := VisitaERiconosci(T.center, L)
 5  (L, ta_r, c_r) := VisitaERiconosci(T.right, L)
 6  if (T.color ≠ arancione)
 7        if (ta_l) and T.left ≠ NIL
 8              Inserisci(L, (T.left, c_l))
 9        if (ta_c) and T.center ≠ NIL
10              Inserisci(L, (T.center, c_c))
11        if (ta_r) and T.right ≠ NIL
12              Inserisci(L, (T.right, c_r))
13  ta := ta_l and ta_c and ta_r and (T.color = arancione)
14  c := c_l + c_c + c_r + 1
15  return (L, ta, c)
```

La complessità totale è dunque $\Theta(n)$.

Esercizio 5.19. Si considerino alberi binari, in cui ciascun nodo contiene come informazione un numero intero. Si progetti un algoritmo che, ricevendo in ingresso un albero T e un intero *tot*, restituisce 1 se esistono almeno due cammini distinti dalla radice alle foglie in cui la somma dei numeri contenuti nei nodi è uguale a *tot* e 0 altrimenti. Due cammini sono distinti se differiscono anche solo per un nodo. Si calcoli l'ordine di grandezza (Θ) della complessità temporale dell'algoritmo proposto in funzione del numero n di nodi dell'albero.

SOLUZIONE

È possibile compiere l'intera ricerca in una sola visita di tutti i nodi dell'albero, tramite la seguente osservazione: è possibile mantenere la somma dei valori di un percorso a mano a mano che lo si visita. Da questo si ottiene che è anche semplice conservare la differenza tra detta somma e il totale *tot* desiderato. Nella misura in cui un percorso giunge al termine (ovvero si è arrivati ad una foglia), è possibile stabilire se la somma dei valori in esso sia quella desiderata semplicemente controllando se la rimanente distanza dal totale corrisponde alla chiave del nodo.

Di seguito è presentato lo pseudocodice di una implementazione ricorsiva dell'algoritmo di ricerca che segue l'idea di cui sopra.

```
CERCA(T, tot)
1  if (T.root ≠ NIL and CERCAAUX(T.root, tot) ≥ 2)
2        return 1
3  else
4        return 0

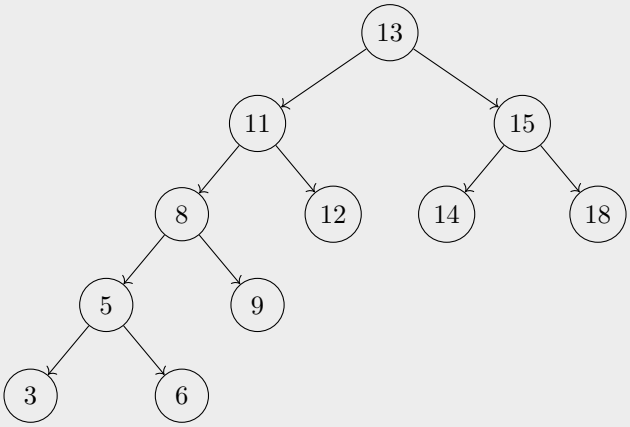
CERCAAUX(T, tot)
 1  if (T = NIL)
 2        return 0
 3  if (T.left = NIL) and (T.right = NIL)
 4        if tot = T.key
 5              return 1
 6        else
 7              return 0
 8  else
 9        l := CERCAAUX(T.left, tot − T.key)
10        r := CERCAAUX(T.right, tot − T.key)
11        return l + r
```

È facile constatare che la complessità temporale dell'algoritmo proposto è $\Theta(n)$: l'algoritmo visita infatti tutti i nodi dell'albero esattamente una sola volta.

Esercizio 5.20. Si consideri un array che sia il risultato dell'attraversamento posticipato sinistro di un albero binario di ricerca (Binary Search Tree, BST) non vuoto i cui nodi contengano numeri interi distinti, ordinati in ordine crescente (figlio di sinistra $<$ del padre $<$ del figlio di destra; linearizzato in: figlio di sinistra, figlio di destra, padre). Assumendo l'ipotesi che il BST di partenza avesse entrambi i figli per ogni nodo interno, si delinei, mediante opportuno pseudocodice, un algoritmo che, partendo dall'array linearizzato ricostruisca il BST originario. Indi se ne valuti la complessità asintotica.

SOLUZIONE

Un esempio di BST che soddisfi l'ipotesi dell'esercizio è fornito nella seguente figura:



La stampa risultante dall'attraversamento posticipato del BST in figura genera: $3, 6, 5, 9, 8, 12, 11, 14, 18, 15, 13$.

Si noti che (per un certo valore) non possono esistere più di due nodi contenenti quel valore, altrimenti si dovrebbe violare l'ipotesi che ogni nodo interno ha entrambi i figli.

Un primo modo semplice di riottenere il BST originale consiste nell'agire in maniera speculare alla stampa effettuata dalla visita: se la visita in post-ordine emette prima il sottoalbero sinistro, poi quello destro, dunque la radice, allora per riottenere lo stesso albero sarà sufficiente inserire prima la radice, dunque gli elementi del sottoalbero destro e poi gli elementi del sottoalbero sinistro. Questo corrisponde allo scorrere i valori stampati da destra a sinistra ed inserirli uno dopo l'altro. La complessità computazionale di questo metodo è di n volte il costo di un singolo inserimento in un BST. Nel caso ottimo, questo risulta in una complessità totale di $\mathcal{O}(n \log(n))$, ma nel caso in cui l'albero di partenza fosse quasi degenere la complessità diventa $\mathcal{O}(n^2)$. Infatti, se è vero che è impossibile avere un albero improprio, degenerato in una lista dati i vincoli sui figli dell'esercizio, è possibile comunque avere un'albero in cui tutti gli elementi della spina sinistra han-

no un solo figlio destro, foglia. Nel caso pessimo, un inserimento in questa struttura ha costo $\frac{n}{2}$.

È possibile avere una soluzione più efficiente di quella appena descritta, utilizzando una pila come struttura dati ausiliaria durante la costruzione dell'albero. L'osservazione da sfruttare a questo scopo è che gli elementi della stampa in post-ordine sono in ordine crescente, tranne quando si passa da figlio (necessariamente destro) a padre. Questo consente di individuare come ricostruire l'albero in una singola scansione. A questo scopo, si considerino gli elementi della stampa in post-ordine come contenuti in un vettore V.

```
RICOSTRUISCIALBERO(V)
 1  ALLOCA(N, 1) // Alloca un nuovo nodo dell'albero
 2  N.key := V[1]
 3  N.left := NIL
 4  N.right := NIL
 5  P := CREAPILAVUOTA()
 6  P := PUSH(P, N)
 7  i := 2
 8  while i ≤ V.length
 9       while V[i − 1] < V[i]
10            ALLOCA(M, 1) // Alloca un nuovo nodo dell'albero
11            M.key := V[i]
12            M.right := NIL
13            M.left := NIL
14            P := PUSH(P, M)
15            i := i + 1
16       ALLOCA(O, 1) // Alloca un nuovo nodo dell'albero
17       O.key := V[i]
18       O.right := POP(P)
19       O.left := POP(P)
20       P := PUSH(P, O)
21       i := i + 1
22  R := POP(P) // Preleva il nodo radice dell'albero ricostruito
23  T.root := R
24  return T
```

Esercizio 5.21. Si consideri un albero binario di interi, memorizzato mediante una struttura dati che prevede che ogni nodo punti ai propri figli ma non al padre e che può contenere elementi ripetuti.

1. Si vogliono cancellare tutte le foglie che, al momento della chiamata alla procedura, contengono un valore uguale a quello del padre. Si fornisca una breve traccia di un algoritmo per questo scopo e se ne valuti la complessità asintotica; essa è ottima?

2. Si voglia estendere l'algoritmo in modo tale che, a valle della procedura di eliminazione, si ottenga un albero in cui non ci siano foglie che contengono il valore del padre (cancellando eventuali nuove foglie contenenti il valore del padre). Fornire una traccia e la relativa complessità del nuovo algoritmo.

SOLUZIONE

1. È sufficiente una visita in ordine anticipato dell'albero: ogni volta che si effettua una chiamata ricorsiva il riferimento al nodo (padre) visitato viene passato alla chiamata come ulteriore parametro. Quando la visita raggiunge una foglia (evento verificabile controllando che entrambi i figli siano NIL), essa verifica se il padre ha lo stesso valore e in tal caso elimina la foglia. La complessità è lineare nel numero dei nodi dell'albero.

2. La soluzione più naturale consiste nell'iterare l'algoritmo precedente. Ogni iterazione ha quindi complessità lineare nel numero dei nodi; il numero di iterazioni al massimo è dato dalla profondità dell'albero. Quindi nel caso pessimo la complessità è $\mathcal{O}(n^2)$. Una soluzione alternativa invece, procede con una visita in ordine posticipato dove la visita ad un dato nodo consiste nel verificare se i propri figli sono foglie e, nel caso lo siano, se il loro valore coincide con il proprio. Se questo è il caso, le foglie vengono eliminate. Questo approccio consente di eliminare ricorsivamente anche i nodi che apparivano come interni a monte della chiamata alla procedura di eliminazione, al costo di una singola visita, ovvero $\mathcal{O}(n)$.

Esercizio 5.22. Si consideri un albero binario di ricerca T contenente $n > 0$ nodi. Si implementino in pseudocodice le seguenti procedure, e se ne analizzi la complessità temporale.

1. SIMMETRICO(T): La procedura controlla se l'albero ha struttura simmetrica rispetto all'asse verticale passante per la radice.

2. GEMELLI(T): La procedura controlla se i sottoalberi figli della radice hanno la stessa struttura, ovvero se i sottoalberi sono sovrapponibili con una traslazione orizzontale rigida.

3. STAMPA_STRATI(T): La procedura stampa il contenuto dell'albero per livelli, a partire dalla radice, stampando i nodi di ogni livello da sinistra a destra.

SOLUZIONE

1. La procedura che controlla se l'albero ha struttura simmetrica procede, dalla radice alle foglie, controllando se il nodo correntemente ispezio-

nato presenta asimmetrie (ovvero se una sola delle sue foglie è non-nulla). Se non sono presenti asimmetrie, e sono presenti sia il sottoalbero destro che sinistro, l'algoritmo procede confrontando il sottoalbero sinistro del figlio sinistro del nodo con quello destro del figlio destro (si ricordi la necessità di verificare la simmetria rispetto all'asse verticale). Effettuato questo controllo, l'algoritmo procede con il confrontare il sottoalbero destro del figlio sinistro con il sottoalbero sinistro del figlio destro.

```
SIMMETRICO(T)
1  return VISITASIM(T.root.left, T.root.right)

VISITASIM(L, R)
1  if (L = NIL) and (R = NIL)
2        return true
3  if (L = NIL) ∨ (R = NIL)
4        return false
5  return VISITASIM(L.left, R.right) and VISITASIM(L.right, R.left)
```

La complessità temporale dell'algoritmo è $\Theta(n)$, in quanto visita una ed una sola volta ogni nodo dell'albero.

2. Il controllo della proprietà di "gemellarità" dei figli della radice dell'albero può essere effettuato con una strategia analoga a quella precedente. Si osservi a questo scopo che, se è vero che i sottoalberi figli della radice sono gemelli, allora questo deve valere anche per i sottoalberi figli di un qualunque nodo posizionato nello stesso punto in entrambi. È quindi possibile riutilizzare lo schema algoritmico precedente, semplicemente andando a confrontare il sottoalbero sinistro del figlio sinistro con il sottoalbero sinistro del figlio destro di nodi nella stessa posizione; facendo seguito a questo confronto con il confronto dei rispettivi sottoalberi destri. Di seguito, lo pseudocodice della procedura:

```
GEMELLI(T)
1  return VISITAGEM(T.root.left, T.root.right)

VISITAGEM(L, R)
1  if (L = NIL) and (R = NIL)
2        return true
3  if (L = NIL) ∨ (R = NIL)
4        return false
5  return VISITAGEM(L.left, R.left) and VISITAGEM(L.right, R.right)
```

Come nel caso precedente, la complessità è $\Theta(n)$.

3. Un metodo a complessità temporale minima per stampare l'albero per strati utilizza una coda come struttura dati ausiliaria. L'idea dell'algoritmo è quella di "accodare per la stampa" i nodi, avendo cura di stampare il nodo che si trova all'inizio della coda prima di inserirne in coda i figli. È quindi sufficiente inserire nella coda la radice dell'albero per iniziare la procedura. Di seguito, lo pseudocodice dell'algoritmo:

```
STAMPASTRATI(T)
1  if (T.root ≠ NIL)
2        ENQUEUE(Q, T.root)
3  while ¬(IS_EMPTY(Q))
4        e := DEQUEUE(Q)
5        STAMPA(e)
6        if (e.left ≠ NIL)
7              ENQUEUE(Q, e.left)
8        if (e.right ≠ NIL)
9              ENQUEUE(Q, e.right)
```

L'algoritmo visita, e stampa, una sola volta ogni nodo, quindi la sua complessità temporale è $\Theta(n)$

Esercizio 5.23. Sia dato un albero binario di ricerca, contenente n chiavi intere, dove sono presenti chiavi duplicate.

1. Si realizzi la procedura TRUESUCCESSOR che, dato un riferimento ad un nodo x dell'albero, restituisce un riferimento al nodo contenente la più piccola chiave strettamente maggiore di $x.key$, o NIL se tale elemento non esiste.

2. Si realizzi la procedura BSTTOQUEUE che, dato un BST con duplicati, lo trasforma in una coda di riferimenti ai nodi, senza duplicati tra le chiavi, ordinata, dove il primo elemento in coda è l'elemento con chiave minima.

Si fornisca lo pseudocodice di tutte le procedure e se ne valuti la complessità temporale.

SOLUZIONE

1. La procedura TRUESUCCESSOR(x) si limita ad invocare la procedura TREE-SUCCESSOR(x) fino a quando il nodo successore non ha effettivamente chiave diversa da quello di partenza, dove TREE-SUCCESSOR è la procedura descritta in [2], capitolo 12, e riportata nel seguito, per individuare il successore in un BST.

```
TRUESUCCESSOR(x)
1  y := TREE-SUCCESSOR(x)
2  while y ≠ NIL and y.key = x.key
3        y := TREE-SUCCESSOR(y)
4  return y
```

La complessità risulta $T(n) \in \Theta(n)$: nel caso pessimo l'albero è costituito da un'unica chiave, ripetuta in ogni nodo. Allo scopo di ricavare la complessità anzidetta è utile ricordare che un attraversamento simmetrico di un albero effettua le stesse operazioni di una chiamata a TREE-MINIMUM, seguita da $n-1$ chiamate a TREE-SUCCESSOR, di cui si riporta il codice di seguito, insieme alla procedura TREE-MINIMUM, ad essa accessoria.

```
TREE-SUCCESSOR(x)
1  if x.right ≠ NIL
2        return TREE-MINIMUM(x.right)
3  y := x.p
4  while y ≠ NIL and x = y.right
5        x := y
6        y := y.p
7  return y
```

```
TREE-MINIMUM(x)
1  while x.right ≠ NIL
2        x := x.right
3  return x
```

Analizzando il codice di TREE-SUCCESSOR è possibile dimostrare che ogni arco dell'albero viene attraversato esattamente due volte, di conseguenza la procedura che combina TREE-MINIMUM e TREE-SUCCESSOR come sopra per visitare l'albero ha costo $\Theta(n)$. Dapprima, si osservi che ogni arco è attraversato in fase discendente solo dal codice di una chiamata a TREE-MINIMUM (o la prima tra tutte, o una effettuata da TREE-SUCCESSOR).

Dato un arco tra due nodi (x, y) si hanno due casi: y è figlio destro di x oppure è figlio sinistro.

Considerando il caso in cui sia figlio sinistro abbiamo che, prima di stampare y, tutto il suo sottoalbero sinistro deve essere stato stampato: di conseguenza l'arco (x, y) deve essere stato attraversato (andando da x a y) per accedervi. A valle della stampa del sottoalbero sinistro di x, è necessario stampare x stesso: il percorso che TREE-SUCCESSOR effettua, risalendo, a partire dal massimo del sottoalbero sinistro, include

necessariamente l'arco (x, y), che viene percorso da y a x. Effettuato ciò, non ci saranno più accessi al sottoalbero sinistro di x.

Considerando il caso in cui y sia figlio destro di x, abbiamo che, dopo la stampa di x, viene invocata TREE-SUCCESSOR, la quale, per raggiungere il minimo del sottoalbero radicato in y, deve necessariamente attraversare l'arco (x, y) in direzione discendente (rigo 2 di TREE-SUCCESSOR). Quando la stampa dell'elemento massimo del sottoalbero radicato in y è completa, TREE-SUCCESSOR risalirà l'arco (x, y) in direzione da y a x; una volta fatto ciò, l'arco (x, y) non sarà mai più attraversato, in quanto tutti i nodi che consente di raggiungere sono già stati visitati.

2. La procedura BSTTOQUEUE utilizza TRUESUCCESSOR per scandire il BST in una singola passata, dall'elemento più piccolo al più grande, accodando gli elementi.

```
BSTTOQUEUE(T)
1  if T.root = NIL
2        return NIL
3  i := TREE-MINIMUM(T.root)
4  ENQUEUE(Q, i)
5  i := TRUESUCCESSOR(i)
6  while i ≠ NIL
7        ENQUEUE(Q, i)
8        i := TRUESUCCESSOR(i)
9  return Q
```

TREE-MINIMUM, come mostrato sopra, individua il minimo in un BST, ovvero, a partire dalla radice, segue la spina sinistra dell'albero fino ad arrivare alla foglia più a sinistra, che dunque contiene la chiave più piccola. L'attraversamento indotto da BSTTOQUEUE corrisponde ad un attraversamento simmetrico, che visita i nodi in ordine crescente del valore di chiave e richiede pertanto tempo $\Theta(n)$.

Esercizio 5.24. Si consideri un albero binario di ricerca (BST).

- Si descriva un algoritmo che, ricevendo in ingresso un BST e una coppia di chiavi appartenenti all'universo delle possibili chiavi usate nel BST, stampi l'elenco di tutte le chiavi presenti nell'albero il cui valore sia compreso tra le due chiavi fornite in ingresso. Si valuti la complessità asintotica dell'algoritmo fornito.

- È possibile modificare l'algoritmo suddetto o costruirne uno diverso in modo da ottenere una migliore complessità, nel caso l'albero in ingresso sia un albero rosso-nero? Motivare brevemente la risposta.

- È possibile modificare l'algoritmo suddetto o costruirne uno diverso in modo da ottenere una migliore complessità asintotica, nel caso l'albero in ingresso sia un albero rosso-nero e la coppia di chiavi individui un intervallo di possibili valori molto piccolo rispetto alla dimensione dell'albero (ad esempio decine di chiavi per alberi che contengano migliaia di nodi)? Motivare brevemente la risposta.

SOLUZIONE

- Un semplice algoritmo visita l'albero mediante un attraversamento simmetrico, stampando le chiavi comprese nell'intervallo $[k_{min}, k_{max}]$.

  La complessità è $\mathcal{O}(n)$, né è possibile ottenerne una più bassa, dato che, nel caso pessimo, potrebbe essere necessario stampare tutte le chiavi.

- Il fatto di usare un albero rosso-nero invece di un BST non cambia sostanzialmente le cose perché resta comunque necessario visitare tutto l'albero nel caso pessimo.

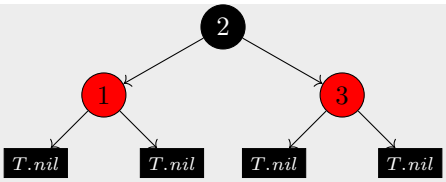
- In questo caso è possibile procedere cercando il valore più piccolo maggiore o uguale a k_{min} all'interno dell'albero, utilizzando la consueta procedura di ricerca per k_{min} (descritta in [2], capitolo 12). Questa operazione ha costo $\mathcal{O}(\log(n))$. Individuato l'elemento di cui sopra, è possibile scandire in ordine gli elementi dell'albero utilizzando una sequenza di chiamate alla procedura TREE-SUCCESSOR, sino a quando l'elemento restituito non è maggiore di k_{max}. La complessità di questo approccio è esattamente $\mathcal{O}(n')$ dove n' è il numero di elementi con chiavi all'interno dell'intervallo $[k_{min}, k_{max}]$ (è possibile ottenere questo risultato con un ragionamento analogo a quello della soluzione dell'esercizio 5.23).

Esercizio 5.25. Si considerino gli algoritmi RB-INSERT e RB-DELETE per l'inserimento e la cancellazione di nodi in e da alberi rosso-neri. È possibile, usando solo tali algoritmi, costruire un albero rosso-nero fatto di esattamente 3 nodi, tutti neri? Se sì, scrivere una sequenza di invocazioni di RB-INSERT e RB-DELETE che crea l'albero desiderato, motivando il perché la sequenza scritta produce la struttura richiesta.

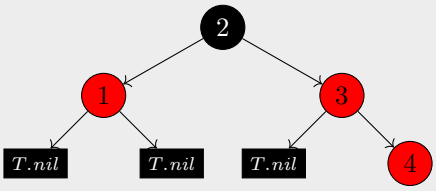
SOLUZIONE

Sì, è possibile, ad esempio inserendo $2, 1, 3, 4$

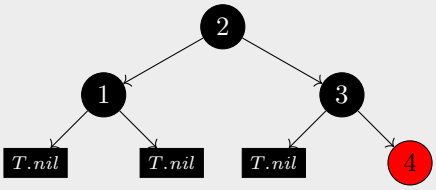
Si osservi che l'inserimento di tre valori, di cui il primo ha valore compreso tra il secondo ed il terzo, creerà sempre un albero a due livelli della seguente forma:



L'aggiunta del quarto valore causa la seguente violazione (figlio rosso di padre rosso)



che viene corretta ricolorando di nero padre e zio del nodo appena inserito:



Per ottenere l'albero desiderato, è a questo punto sufficiente cancellare, mediante RB-DELETE, il nodo rosso 4.

Esercizio 5.26. Si stabilisca quali sono i rapporti minimo e massimo tra il numero di nodi (interni) rossi e neri in un generico albero rosso-nero. Si giustifichi brevemente la risposta.

SOLUZIONE

Un albero può essere totalmente nero: un semplicissimo esempio è costituito da un albero con la sola radice. Si noti che normalmente un albero non banale di nodi unicamente neri si crea solo attraverso qualche cancellazione che trasformi un nodo rosso in uno nero, perché ogni nuova inserzione in un albero totalmente nero inserisce un nodo rosso. Quindi l'unico modo per avere un albero totalmente nero senza operare cancellazioni è che esso consista della sola radice. In tutti questi casi caso il rapporto minimo è evidentemente 0.

Il massimo numero di nodi rossi in un albero rosso-nero è vincolato dall'invariante che proibisce ad un nodo rosso di avere figli rossi (gli invarianti sull'altezza nera non hanno effetto). Questo invariante fa sì che i nodi rossi

dell'albero debbano essere presenti con padri e figli neri: di conseguenza, il massimo numero di nodi rossi inseribili può essere ottenuto considerando un albero colorato per "strati" (nodi alla stessa altezza). Ricordando che un albero rosso-nero deve avere radice nera e foglie vuote abbiamo che, per ogni strato nero, il successivo, che contiene il doppio dei nodi, può essere completamente rosso. Il rapporto massimo tra numero di nodi rossi e numero di nodi neri è quindi 2 (ignorando le foglie nere $T.nil$ nel conteggio).

Esercizio 5.27. Dato un albero binario di ricerca T, i cui nodi memorizzano una variabile che ne rappresenta il colore, si vuole determinare se T è un albero rosso-nero. Si definisca un algoritmo che risolva questo problema e se ne discuta la complessità.

SOLUZIONE

L'algoritmo seguente verifica che le proprietà degli alberi rosso-neri siano rispettate: verifica che la radice sia nera, quindi richiama una funzione che ricorsivamente analizza i sottoalberi per calcolarne l'altezza nera e controllare che i figli dei nodi rossi siano neri.

```
CONTROLLARB(T)
1  if (T.root = T.nil)
2        return true
3  if (T.root.color = RED)
4        return false
5  (bh, is_rb) := CONTROLLASOTTOAL(T.root, 0)
6  return is_rb

CONTROLLASOTTOAL(T, bh)
 1  if (T = T.nil)
 2        return (bh, true)
 3  if ((T.color = RED) and (T.left.color = RED or T.right.color = RED))
 4        return (bh, false)
 5  if (T.color = BLACK)
 6        bh := bh + 1
 7  (bh_l, is_rb_left) := CONTROLLASOTTOAL(T.left, bh)
 8  (bh_r, is_rb_right) := CONTROLLASOTTOAL(T.right, bh)
 9  if (is_rb_left ≠ true) or (is_rb_right ≠ true) or (bh_l ≠ bh_r)
10        return (bh, false)
11  return (bh_l, true)
```

La soluzione visita al più tutti i nodi dell'albero una sola volta, la complessità è quindi $\mathcal{O}(n)$.

Esercizio 5.28. Si descriva un algoritmo che, dato un BST (Binary Search Tree) stabilisca se esso possa essere colorato in modo tale da diventare un albero rosso-nero (RB). Si osservi che è unicamente richiesto di stabilire se è possibile colorare il BST ricevuto, non di effettuare la colorazione. Si ricordi inoltre che per convenzione in un albero RB T tutte le foglie sono $T.nil$ e nere. Si valuti la complessità temporale dell'algoritmo.

SOLUZIONE

Una condizione necessaria e sufficiente perché un BST possa essere colorato in modo da diventare un albero rosso-nero è che, nell'intero albero e in ogni suo sottoalbero il rapporto tra le lunghezze minime e massime dei cammini che dalla radice raggiungono le foglie non superi mai 2. Infatti in tal caso sarà sempre possibile colorare i nodi in modo da evitare due nodi rossi consecutivi sullo stesso cammino e l'altezza di ogni sottoalbero sarà comunque $\Theta(\log(n))$.

Un semplice algoritmo ricorsivo potrebbe quindi visitare l'albero, ad esempio con un attraversamento anticipato, e calcolare, per ogni nodo, la lunghezza dei cammini minimo e massimo da esso alle foglie incrementando di 1 i valori minimi e massimi ottenuti dai figli. Non appena il rapporto tra lunghezza del cammino massimo e minimo eccede 2 l'algoritmo ritorna $false$; se giunge fino alla radice con max/min ≤ 2, ritorna $true$.

```
COLORABILE(T)
1  (min, max, ok) := CALCOLOALTEZZE(T.root)
2  return ok

CALCOLOALTEZZE(T)
1  if T = T.nil
2        return (0, 0, true)
3  (min_l, max_l, ok_l) := CALCOLOALTEZZE(T.left)
4  (min_r, max_r, ok_r) := CALCOLOALTEZZE(T.right)
5  min := MIN(min_l, min_r) + 1
6  max := MAX(max_l, max_r) + 1
7  if max > 2 · min
8        return (min, max, false)
9  return (min, max, true)
```

Si noti però che, rispetto a un normale BST, in un albero rosso-nero le foglie sono tutte NIL e hanno convenzionalmente altezza nera nulla; ogni nodo interno ha quindi entrambi i figli. L'algoritmo ha evidentemente complessità $\Theta(n)$.

Esercizio 5.29. Si consideri un albero rosso-nero di altezza nera bh e costituito da soli nodi neri.

1. Quali sono, rispettivamente, il numero minimo e massimo di nodi contenuti nell'albero? (si precisi se nel conto viene incluso il nodo $T.nil$ o no).
2. È possibile ottenere da esso un nuovo albero rosso-nero costituito da soli nodi neri applicandovi esclusivamente operazioni di inserimento di nuovi nodi? In caso positivo in che modo? (eventualmente illustrandolo mediante un semplice esempio).
3. Qual è il numero massimo di nuovi nodi che si possono inserire in esso senza effettuare cancellazioni e senza alterare l'altezza nera?

SOLUZIONE

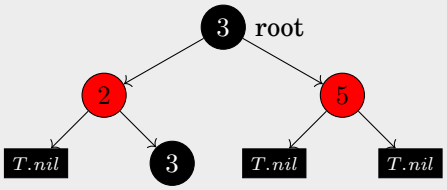
1. Si ricordi che che l'altezza nera si calcola contando il numero di nodi neri tra la radice, esclusa, e la foglia $T.nil$, compresa. Inoltre, ogni "cammino nero" dalla radice a $T.nil$ deve avere la stessa lunghezza, quindi un albero rosso-nero di soli nodi neri è necessariamente completo (e dunque perfettamente bilanciato). Il numero di nodi interni è quindi dato da $\sum_{h=0}^{bh-1} 2^h$, a cui va aggiunto 1 per tener conto dell'unico $T.nil$ che agisce in vece delle foglie. Abbiamo quindi che il numero, sia minimo che massimo, di nodi è 2^{bh}.
2. Ogni nodo nuovo viene inserito con colore rosso; successivamente la procedura di RB-INSERT-FIXUP (descritta in [2], capitolo 13) lascia in ogni caso almeno un nodo rosso. Quindi non è possibile ottenere alberi interamente neri esclusivamente mediante inserimenti (a meno che l'albero non consista della sola radice).
3. Se l'albero è inizialmente costituito da soli nodi neri si possono inserire nuovi nodi senza alterarne la bh, quindi necessariamente rossi, fino al massimo a raddoppiarne l'altezza a causa del vincolo di non avere nodi rossi con padri rossi: abbiamo dunque che $h = 2bh$. Quindi il numero totale di nodi (compreso $T.nil$) del nuovo albero di altezza 2^{bh} è 2^{2bh} e il numero massimo di nodi inseribili è $2^{2bh} - 2^{bh}$.

Esercizio 5.30. Si risponda alle seguenti domande, motivando opportunamente le risposte.

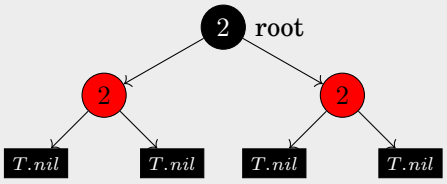
1. Si considerino alberi rosso-neri che ammettono chiavi duplicate:
   (a) è possibile che ci siano due elementi con la stessa chiave che non sono uno il padre dell'altro?
   (b) è possibile che ci siano due elementi con la stessa chiave che non sono uno antenato dell'altro?
2. Modificare gli algoritmi di RB-INSERT e RB-DELETE riportati nel capitolo 13 di [2] in modo da impedire che ci possano essere in un albero chiavi duplicate. Dire come cambia (se cambia) la complessità degli algoritmi modificati rispetto alle versioni originali.

SOLUZIONE

1.a È possibile, per esempio inserendo le seguenti chiavi: $3, 2, 5, 3$



1.b È possibile, per esempio inserendo 3 volte la stessa chiave: due delle tre chiavi avranno solo foglie nere $T.nil$ come figli:



2 Allo scopo di evitare l'inserimento di chiavi duplicate, è sufficiente andare a modificare unicamente la procedura di RB-INSERT; è infatti l'unica responsabile di potenziali violazioni dell'invariante. La modifica della procedura è molto semplice: si tratta infatti, prima di inserire un nodo, di controllare se il futuro padre dello stesso non contenga la chiave che si sta per inserire. È infatti noto che la procedura di inserimento in un BST (e dunque anche in un albero rosso-nero) inizia con il procedimento di ricerca della chiave che si vorrebbe inserire nell'albero.

Di seguito, la procedura di inserimento modificata; la modifica consiste nell'aggiunta delle righe 8 e 9, che non muta la complessità dell'algoritmo.

INSERISCINODUP(T, x)

```
 1  pre := T.nil
 2  cur := T.root
 3  while cur ≠ T.nil
 4        pre := cur
 5        if x.key < cur.key
 6              cur := cur.left
 7        else cur := cur.right
 8  if x.key = pre.key
 9        return
10  ...
```

5.5 Grafi

Esercizio 5.31. Si consideri il seguente problema: dati in ingresso un automa a stati finiti e uno dei suoi stati finali, sia esso q, si deve dire se q è raggiungibile dallo stato iniziale. Si descriva un algoritmo per il problema e se ne valuti la complessità temporale.

SOLUZIONE

Si memorizza l'automa come grafo e si fa una visita in ampiezza a partire dallo stato iniziale: appena q viene raggiunto, si risponde positivamente. Se alla fine della visita q non viene raggiunto, si risponde negativamente. La valutazione della complessità è esattamente la stessa di una visita in ampiezza: $\mathcal{O}(|V| + |E|)$.

Esercizio 5.32. Si consideri la rete stradale di una città.

1. Si definisca una struttura dati per rappresentare la rete stadale di una città. La rete stradale è fatta di incroci, collegati da strade, che possono essere a senso unico o a doppio senso di marcia; una strada si dice tratto di strada elementare se collega due incroci senza che tra essi ve ne siano altri; è anche possibile che ci siano dei cavalcavia, ossia strade che intersecano i propri tragitti senza per questo dare origine a un incrocio.
2. Si progetti un algoritmo per la struttura dati definita al punto 1 che riceve in ingresso un incrocio della città da cui partire e determina un percorso che ritorna all'incrocio di partenza senza passare due volte da alcun altro incrocio o da una stessa strada. Il percorso trovato dovrebbe essere quello di lunghezza minore tra quelli che soddisfano i vincoli, intendendo come lunghezza il numero di tratti di strada elementari che lo compongono.
3. Si valuti la complessità (asintotica) dell'algoritmo progettato.

SOLUZIONE

1. La struttura è un tipico esempio di grafo orientato.

   Il grafo cittadino è un esempio di grafo scarsamente connesso: il numero di strade uscenti da un incrocio, nella pratica, raramente supera 10, mentre ci sono centinaia di incroci in una città. Abbiamo quindi che il numero di archi è dell'ordine delle migliaia a fronte di un numero di archi per un grafo completamente connesso con lo stesso numero di nodi nell'ordine delle decine di migliaia. In questo senso quindi la struttura più efficiente in termini di complessità spaziale per rappresentare il grafo è una lista di adiacenza.

2. Il problema è un classico problema di raggiungibilità. Si noti che è impossibile passare due volte per la stessa strada, in un senso qualsiasi, senza anche passare due volte da uno stesso incrocio. È quindi possibile risolvere il problema adattando la consueta ricerca in ampiezza, verificando se il nodo correntemente esaminato v coincide con il nodo di partenza s: non appena questo test dà esito positivo, si è anche trovato un percorso a distanza minima, grazie all'impostazione di ricerca in ampiezza dell'algoritmo di visita. Si noti che l'algoritmo di visita in ampiezza nella forma descritta nel capitolo 20 di [2] non tiene traccia del cammino effettuato, che va quindi memorizzato separatamente.

3. La complessità temporale è quindi quella dell'algoritmo di visita in ampiezza: nel caso in cui si rappresenti il grafo come lista di adiacenza, questa sarà $\mathcal{O}(|V| + |E|)$.

Esercizio 5.33. Qual è il numero massimo di possibili ordinamenti topologici compatibili con un grafo diretto aciclico di n nodi?

SOLUZIONE

Il numero di possibili ordinamenti topologici di un grafo è ristretto dalla struttura degli archi dello stesso. Ogni arco, infatti, impone un ordinamento tra i due nodi tra cui è presente. Di conseguenza, il numero massimo di ordinamenti possibili è quello di un grafo privo di archi. Se il grafo non ha archi, qualsiasi permutazione dei nodi è un ordinamento topologico compatibile con esso; quindi se ne possono ottenere $n!$.

Esercizio 5.34. Dati un elenco di attività e di vincoli di precedenza fra esse scrivere un algoritmo che rappresenta queste attività in un grafo orientato di precedenze, o fallisce se i vincoli di precedenza sono incoerenti (ad esempio, se A precede B e B precede A, sia direttamente che indirettamente, i vincoli sono incoerenti).

SOLUZIONE

Come suggerito nel testo dell'esercizio, si costruisca un grafo orientato che rappresenti le attività (nodi) e i relativi vincoli di precedenza (archi): i vincoli sono incoerenti se e solo se il grafo contiene cicli. Ciò può essere verificato mediante una opportuna modifica dell'algoritmo per ottenere un ordinamento topologico del grafo di partenza: in particolare, il grafo è aciclico se e solo se esiste un ordinamento topologico. La modifica, proposta di seguito, consiste quindi nell'ignorare la costruzione dell'ordinamento topologico (che non ci interessa in questo caso), restituendo semplicemente un valore booleano, che indica se un tale ordinamento è possibile. L'algoritmo proposto è quindi $\mathcal{O}(|V| + |E|)$.

```
TopSortCheck(G)
1  for each i ∈ G.V
2       i.colore := bianco // Tutti i nodi sono inizialmente bianchi
3  for each i ∈ G.V
4       if i.colore = bianco
5            if TopSortCheckVisita(G, i) = false
6                 return false
7  return true

TopSortCheckVisita(G, i)
 1  if i.colore = nero
 2       return true
 3  if i.colore = grigio
 4       return false
 5  i.colore := grigio
 6  for each v ∈ Adiacenti(G, i)
 7       if TopSortCheckVisita(G, v) = false
 8            return false
 9  i.colore := nero
10  return true
```

Osserviamo infine che esistono diverse soluzioni in letteratura per verificare la presenza di cicli nei grafi. Un metodo alternativo a quello presentato, ad esempio, è l'algoritmo di Tarjan per individuare le componenti fortemente connesse di un grafo (cioè un sottografo massimale in cui esiste un cammino orientato tra ogni coppia di nodi ad esso appartenenti): un grafo è aciclico se e solo se non vi sono componenti fortemente connesse che comprendano più di un nodo. Tuttavia la complessità asintotica è la stessa del semplice algoritmo presentato qui.

5.6 Altre strutture dati

Esercizio 5.35. Si consideri un sistema di assegnazione di pacchi a corrieri presente in un centro smistamento. Il sistema gestisce pacchi caratterizzati da una data e ora di spedizione (il momento in cui il cliente ha consegnato il pacco da spedire) e da un livello di urgenza rappresentato da un intero positivo compreso tra 1 (meno urgente) e 10 (più urgente) e da un identificativo numerico unico. I pacchi sono acquisiti dal sistema già contrassegnati da una data/ora di spedizione e da un livello di urgenza assegnato. Il sistema di smistamento deve selezionare quali pacchi devono essere assegnati ad un corriere nel modo quanto più efficiente possibile considerando i seguenti vincoli:

- nessun pacco con urgenza x deve essere assegnato ad un corriere se ne esiste un altro nel sistema con urgenza maggiore di x;

- nel caso siano presenti pacchi allo stesso livello di urgenza, viene deciso di assegnare prima al corriere il pacco con data di spedizione più vecchia tra quelli con lo stesso livello di urgenza tra pacchi;
- si assuma, per semplicità, che non vi siano due pacchi con data e ora di spedizione identiche;
- il sistema di gestione può acquisire un numero arbitrario di pacchi tra una decisione di assegnamento di un pacco al corriere e la successiva decisione (e, analogamente, può effettuare un numero arbitrario di decisioni di assegnamento tra l'acquisizione di un pacco e la successiva acquisizione).

Si descrivano le strutture dati necessarie e le procedure PROSSIMO_PACCO() e ACQUISISCI_PACCO(IDENTIFICATIVO, DATA, URGENZA), e si valuti la loro complessità asintotica in funzione del numero di pacchi attualmente in carico al sistema di gestione del magazzino.

SOLUZIONE

La struttura dati necessaria è un vettore di min-heap avente tante celle quanti sono i livelli di urgenza (10). Ogni min-heap contiene i pacchi, utilizzando come valore su cui operare i confronti la data di acquisizione del pacco stesso e considerando l'ordine cronologico tra le date.

La procedura PROSSIMO_PACCO scorre il vettore di heap in ordine decrescente di urgenza fino quando non ne individua uno non vuoto. Quando è stato trovato uno heap non vuoto, estrae il minimo da esso e riorganizza il min-heap in tempo $\mathcal{O}(\log(n))$, con n numero dei pacchi nello heap.

La procedura ACQUISISCI_PACCO, invece, inserisce il pacco da acquisire nello heap corrispondente alla urgenza dello stesso; anche essa ha costo $\mathcal{O}(\log(n))$ con n numero dei pacchi nello heap; nel caso pessimo il numero di pacchi in un singolo heap è proporzionale al totale dei pacchi, quindi la complessità di entrambe le procedure è $\mathcal{O}(\log(n))$ anche in funzione del numero totale di pacchi.

Esercizio 5.36. Si consideri la struttura dati *heap*, utilizzata dall'algoritmo HEAP-SORT, e se ne progetti una generalizzazione (si chiami essa *3-heap*) con heap ternari anziché binari. Si implementi BUILD-MAX-HEAP per *3-heap* e se ne valuti la complessità, confrontandola con la versione tradizionale, binaria, degli heap.

SOLUZIONE

Per comodità si usi un array con indicizzazione da 0: in questo caso i tre figli di un nodo i sono $3i+1$ (sinistro), $3i+2$ (centrale) e $3i+3$ (destro), mentre il padre è $\lfloor (i-1)/3 \rfloor$. MAX-HEAPIFY risulta quasi identica; va aggiunto $c :=$ CENTER(i) e confrontato con gli altri per calcolare il massimo, inoltre va tenuto conto dell'indice 0, quindi un indice è valido solo se è strettamente

minore di *heap-size*. La complessità risultante è $\mathcal{O}(\log_3(n))$, ovvero (asintoticamente) la stessa degli heap binari. In BUILD-MAX-HEAP si cambia il ciclo per i, che ha indice con valore iniziale $\frac{A.length-2}{3}$ e scorre fino a 0. La valutazione della complessità è molto simile: altezza $h = \Theta(\log_3(n))$; il numero di nodi ad altezza h è al più $\lceil 2n/3^{h+1} \rceil$. Si ottiene quindi $\mathcal{O}(n)$, con costanti moltiplicative più basse (si noti che $\sum_{h=0}^{\infty} 2h/3^h = 3/2$, e si faccia riferimento a [2], appendice A, formula A.8).

Esercizio 5.37. Si consideri un insieme di n elementi diversi tra loro appartenenti a un insieme A. È definita una relazione d'ordine totale $<$ su A. Dati due elementi qualunque e_1 e e_2, diciamo che e_1 è migliore di e_2, se vale $e_1 < e_2$.

1. Descrivere in modo succinto ma preciso un algoritmo che stampi i migliori 10 elementi dell'insieme (si assuma $n > 10$).
2. Valutare la complessità asintotica, in funzione di n, dell'algoritmo descritto al punto 1.

Si consideri ora, in aggiunta, un intero positivo k.

3. Descrivere in modo succinto ma preciso un algoritmo che stampi i migliori k elementi dell'insieme (si assuma $n > k$).
4. Valutare la complessità asintotica, in funzione di n e k, dell'algoritmo descritto al punto 3.

SOLUZIONE

Una prima semplice soluzione consiste nell'ordinare l'insieme degli n elementi secondo l'ordinamento indotto da $<$. Una volta ordinato l'insieme, basta quindi stampare i primi 10 elementi dell'insieme ordinato. L'ordinamento ha un costo $\Theta(n \log(n))$ e le altre operazioni sono a costo costante. Complessivamente l'algoritmo si può fare in tempo $\Theta(n \log(n))$, ad esempio mediante HEAP-SORT.

Una soluzione più efficiente consiste nel mantenere in una struttura dati i 10 elementi migliori trovati, a mano a mano che si analizza l'insieme di elementi. È quindi possibile confrontare un nuovo elemento dell'insieme ancora non esaminato unicamente con il peggiore tra quelli contenuti nella struttura dati. Se l'elemento corrente è migliore almeno del peggiore dei 10 della struttura dati, allora lo si inserisce in essa, rimuovendo il peggiore. Nel caso di una struttura dati banale, ad esempio un array ordinato, trovare il minimo ha costo costante, mentre l'inserimento ha costo lineare in 10, ovvero costo costante. L'azione di confronto ed eventuale inserimento va ripetuta tante volte quanti sono gli elementi dell'insieme; la complessità temporale totale è quindi $\Theta(n)$.

Lo stesso schema di soluzione può essere adottato per i punti 3 e 4, avendo cura però di osservare che, essendo il numero di elementi migliori da stampare k variabile, la scelta della struttura dati per mantenere i migliori

diventa rilevante. Un approccio possibile è quello di utilizzare un max-heap dove vengono salvati i k migliori valori. È quindi possibile controllare se un nuovo valore va messo tra i k migliori presenti in $\mathcal{O}(1)$, ed eventualmente rimuovere il peggiore in $\mathcal{O}(1)$ ed inserire il nuovo elemento in $\mathcal{O}(\log(k))$. Questo consente di ottenere una complessità temporale di $\mathcal{O}(n\log(k))$. Anche in questo caso, l'approccio è migliore della soluzione ingenua che prevede di ordinare l'intero vettore (costo $\Theta(n\log(n))$), per poi selezionare k elementi (costo $\Theta(k)$), dato che il costo dell'ordinamento dell'intero vettore domina la complessità della detta soluzione ingenua.

Una soluzione alternativa all'uso del max-heap consiste nel mantenere un BST di al più k elementi. Si leggono, in sequenza, i vari elementi dell'insieme dato. I primi k elementi vengono inseriti nel BST. Dal $(k+1)$-esimo in poi, un elemento viene inserito nel BST solo se è migliore del peggiore tra gli elementi del BST, e in tale caso, prima dell'inserimento, l'elemento peggiore viene cancellato. Alla fine della lettura degli elementi dell'insieme, il BST conterrà i k elementi migliori. La complessità di questa soluzione è determinata dalla scansione dell'insieme di n elementi e dal costo dell'inserimento e della cancellazione di un elemento nel BST. Come noto, tali operazioni sui BST hanno una complessità logaritmica nella dimensione del BST, se si usano alberi rosso-neri. Il costo complessivo per il punto 4 è quindi $\Theta(n\log(k))$, se si fa uso di un albero rosso-nero, poiché le operazioni di inserimento e cancellazione hanno costo $\Theta(\log(k))$, dal momento che l'albero ha k elementi.

Esercizio 5.38. Si consideri un suggeritore per cercare parole che rimano con una parola data all'interno di un dizionario. In particolare, è possibile interagire con il suggeritore tramite due funzioni.

- AGGIUNGI(p): viene fornita al suggeritore una parola p da aggiungere al dizionario. Si denoti con n lunghezza della parola.
- TROVARIME(p, r): il suggeritore stampa tutte le parole che rimano con la parola p data (di lunghezza n) per i loro ultimi r caratteri.

Si descrivano, in opportuno dettaglio, le due funzioni, AGGIUNGI e TROVARIME. AGGIUNGI deve avere complessità temporale al più lineare in n, mentre TROVARIME deve avere complessità temporale al più lineare nella somma dei caratteri delle parole che rimano per r caratteri con quella data.

SOLUZIONE

È possibile memorizzare il dizionario come un albero con branching factor pari al numero di lettere dell'alfabeto. I nodi dello stesso hanno quindi un numero di puntatori pari alle lettere dell'alfabeto. Ogni nodo contiene anche un campo a valore booleano che indica se il nodo è corrispondente all'inizio di una parola. Ogni parola viene memorizzata, letta dalla fine all'inizio, come un percorso all'interno dell'albero con il seguente procedimento:

1. si imposta la radice come nodo corrente e l'ultima lettera della parola come lettera corrente;
2. si aggiunge, se non già presente, un nodo come i-esimo figlio del nodo corrente, dove i è il numero d'ordine della lettera corrente;
3. si imposta il nodo corrente a quello appena aggiunto (o a quello che si sarebbe dovuto aggiungere ed era già presente);
4. se la lettera corrente non è la prima della parola si torna al punto 2, altrimenti si imposta il valore booleano a `vero`.

Il costo di una chiamata a AGGIUNGI è quindi lineare nella lunghezza della parola da aggiungere.

La funzione TROVARIME(p, r), è realizzata con il seguente procedimento:

1. partendo dalla radice dell'albero come nodo corrente, e dall'ultima lettera della parola come lettera corrente, per r volte
   (a) controlla se è presente l'i-esimo figlio del nodo corrente, dove i è il numero d'ordine della lettera corrente;
   (b) se non è presente, la funzione termina la sua esecuzione senza stampare nulla, altrimenti imposta il figlio come nodo corrente e la lettera precedente nella parola a quella corrente;
2. effettua una visita in profondità dell'albero, a partire dal nodo raggiunto stampando ogni percorso fino ad ogni nodo con valore booleano impostato a `vero` in ordine inverso, seguito dalle ultime r lettere della parola p.

Le parole stampate sono tutte e sole quelle che hanno un suffisso da r caratteri comune alla parola data. Il costo della funzione è proporzionale ai nodi dell'albero visitati, dunque lineare nella somma dei caratteri delle parole che rimano con quella data.

Esercizio 5.39. Si scriva un algoritmo che ordini una coda di interi positivi, supposta già in memoria, assumendo che essa sia utilizzabile come una struttura astratta di cui sia nota solo l'interfaccia, a sua volta costituita esclusivamente dalle primitive canoniche ENQUEUE e DEQUEUE. Si valuti la complessità asintotica dell'algoritmo.

Si richiede di usare il minor numero possibile di celle ausiliarie, in ogni caso in quantità indipendente dal numero di elementi in coda.

SOLUZIONE

L'impossibilità di usare memoria ausiliaria in quantità arbitraria e la necessità di accedere alla coda solo mediante le primitive canoniche obbliga a

far scorrere la coda diverse volte fino ad ottenerne una trasformazione ordinata. Un semplice algoritmo che agisce con questo vincolo esegue i seguenti passi:

- trova l'elemento minimo nella coda e contestualmente conta il numero di elementi n, facendo DEQUEUE e ENQUEUE per tutti gli elementi, accodando preventivamente un marcatore per riconoscere l'inizio della coda;
- accoda il minimo trovato e ripete il procedimento cercando il minimo tra tutti gli altri $n-1$ elementi per poi accodarlo, poi tra gli altri $n-2$, ecc., fino ad aver collocato tutti gli elementi in modo ordinato.

```
ORDINACODA(C)
 1  if C.head = C.tail
 2       return // la coda è vuota
 3  x := DEQUEUE(C)
 4  min := x
 5  n := 1
 6  ENQUEUE(C, −1) // marcatore per contare numero elementi
 7  x := DEQUEUE(C)
 8  while x ≠ −1 // contiamo elementi e troviamo il minimo
 9       if x < min
10            ENQUEUE(C, min)
11            min := x
12       else
13            ENQUEUE(C, x)
14       x := DEQUEUE(C)
15       n := n + 1
16  ENQUEUE(C, min) // il minimo è in posizione; ora collochiamo gli altri
17  for i := n − 1 downto 1 // lunghezza sequenze elementi non collocati
18       x := DEQUEUE(C)
19       min := x
20       for j := 1 to i − 1 // si trova il minimo della sequenza
21            x := DEQUEUE(C)
22            if x < min
23                 ENQUEUE(C, min)
24                 min := x
25            else
26                 ENQUEUE(C, x)
27       for j := i to n − 1 // si trascrivono gli elementi già collocati
28            x := DEQUEUE(C)
29            ENQUEUE(C, x)
30       ENQUEUE(C, min) // si accoda il minimo
```

Questo algoritmo non aumenta la lunghezza originaria della coda (se non per un singolo marcatore aggiunto temporaneamente), usa le variabili ausiliarie min, x, n oltre ai contatori i, j, e ha complessità $\Theta(n^2)$ in quanto il ciclo alla riga 17 effettua n iterazioni e ne contiene a sua volta due che ne effettuano, complessivamente n.

Bibliografia

[1] Alan A. Bertossi e Alberto Montresor. *Algoritmi e strutture di dati, terza edizione*. Città Studi Edizioni, 2014.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, e Clifford Stein. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, e Clifford Stein. *Introduzione agli Algoritmi e Strutture Dati, terza edizione*. McGraw-Hill Education, 2010.

[4] Dino Mandrioli, Davide Martinenghi, Angelo Morzenti, Matteo Pradella e Matteo Rossi. *Lecture Notes on Monadic First- and Second-Order Logic on Strings*. Disponibile all'indirizzo `https://arxiv.org/abs/2301.10519`, doi: 10.48550/ARXIV.2301.10519. arXiv, 2023.

[5] Dino Mandrioli e Paola Spoletini. *Informatica Teorica, seconda edizione*. Città Studi Edizioni, 2011.

Il presente volume è una raccolta di più di 160 esercizi riguardanti argomenti propri dell'informatica teorica e dello studio di algoritmi e strutture dati. In particolare, la raccolta è suddivisa in cinque aree tematiche: modelli formali per la descrizione di linguaggi, problemi relativi alla computabilità di funzioni e decidibilità di insiemi, soluzione di equazioni di ricorrenza, valutazione della complessità algoritmica e sintesi di algoritmi. Gli esercizi sono corredati di soluzioni commentate, volte a descrivere il procedimento risolutivo degli stessi.
Le soluzioni presentate sono autocontenute e corredate di riferimenti alle fonti di nozioni utili alla loro comprensione, facendo riferimento a testi largamente adottati nei corsi di informatica teorica e algoritmi e strutture dati.

ALESSANDRO BARENGHI è professore associato presso il Dipartimento di Elettronica, Informazione e Bioingegneria del Politecnico di Milano, dove è attualmente titolare dei corsi di Algoritmi e Principi dell'Informatica e Computer Security. I suoi interessi di ricerca riguardano principalmente la sicurezza informatica, con particolare attenzione alla crittografia applicata, ed il calcolo quantistico.

DAVIDE MARTINENGHI è professore associato presso il Dipartimento di Elettronica, Informazione e Bioingegneria del Politecnico di Milano, dove è attualmente titolare dei corsi di Algoritmi e Principi dell'Informatica e Basi di Dati 2. I suoi interessi di ricerca riguardano principalmente le problematiche legate ai dati, tra cui il ranking, l'ottimizzazione delle interrogazioni e la rappresentazione dei dati.

MATTEO PRADELLA è professore associato presso il Dipartimento di Elettronica, Informazione e Bioingegneria del Politecnico di Milano, dove è attualmente titolare dei corsi di Algoritmi e Principi dell'Informatica e Principi dei Linguaggi di Programmazione. I suoi interessi di ricerca riguardano principalmente lo sviluppo di metodi formali per sistemi critici, i linguaggi formali ed i linguaggi i programmazione.

MATTEO ROSSI è professore associato presso il Dipartimento di Meccanica del Politecnico di Milano, dove è attualmente titolare dei corsi di Algoritmi e Principi dell'Informatica e Ingegneria del Software. I suoi interessi di ricerca riguardano principalmente lo sviluppo di metodi formali per la modellazione e l'analisi di sistemi critici e il loro impiego in applicazioni reali.

ISBN 978-88-9385-347-7

Feedback

Euro 20,00

Società Editrice ESCULAPIO
www.editrice-esculapio.it